

FAST-FORWARD FALCON (SPEED)

- Dynamic symbol of offensive speed & evasion
- Combines aggressive systems performance with agile networking

# n for Durin

*finitive guide to the Zen programming lan*

Zen Language Project

Zen 3.0 · ISBN 978-0-00000-000-0

# Contents


**1 Zen for Dummies: Complete Edition** **21**
1.1 Table of Contents . . . 21
1.2 About This Book . . . 23

**2 Chapter 1: Getting Started** **24**
2.1 1.1 What is Zen? . . . 24
2.2 1.2 Installation . . . 24
2.3 1.3 Your first program . . . 25
2.4 1.4 The REPL . . . 25
2.5 1.5 `zen explain` . . . 26
2.6 1.6 Type-checking without running . . . 26
2.7 1.7 Linting . . . 26
2.8 1.8 Formatting . . . 27
2.9 1.9 Building standalone executables . . . 27
2.10 1.10 Running tests . . . 27
2.11 1.11 Project layout . . . 27
2.12 1.12 Next steps . . . 27
2.13 1.5 Command-line interface reference (deep) . . . 28
2.14 1.6 Project layout conventions . . . 28
2.15 1.7 Running as script vs module . . . 29
2.16 1.8 Debugging workflow . . . 29
2.17 1.9 Common first-run issues . . . 29
2.18 1.10 Exercises (extended) . . . 29

**3 Chapter 2: Data Types** **30**
3.1 2.1 The type system at a glance . . . 30
3.2 2.2 Literals . . . 31
3.3 2.3 Type coercion matrix . . . 34
3.4 2.4 Truthiness . . . 35
3.5 2.5 Equality vs identity . . . 35
3.6 2.6 Numeric edge cases . . . 36
3.7 2.7 Type conversion reference . . . 36
3.8 2.8 Common pitfalls . . . 37
3.9 2.9 Quick reference . . . 37
3.10 2.10 Exercises . . . 38
3.11 2.11 Type coercion matrix (extended) . . . 38
3.12 2.12 Bytes and buffers (deep dive) . . . 39
3.13 2.13 Tuples (immutable sequences) . . . 39
3.14 2.14 Sets . . . 39
3.15 2.15 Numeric edge cases in depth . . . 40
3.16 2.16 Type checking patterns . . . 40
3.17 2.17 Identity vs equality . . . 40
3.18 2.18 Common pitfalls . . . 41
3.19 2.19 Deep exercises . . . 41

**4 Chapter 3: Operators and Expressions** **42**

4.1 3.1 Operator precedence (high to low) . . . 42
4.2 3.2 Arithmetic . . . 42
4.3 3.3 String concatenation and repetition . . . 43
4.4 3.4 Comparison . . . 43
4.5 3.5 Logic . . . 44
4.6 3.6 Safe member access: `?.` (optional chaining) . . . 44
4.7 3.7 Nullish coalescing: `??` . . . 45
4.8 3.8 Conditional expression (ternary) . . . 45
4.9 3.9 Assignment and augmented assignment . . . 45
4.10 3.10 `is` — type membership . . . 46
4.11 3.11 `in` / `!in` — membership . . . 46
4.12 3.12 Quick reference . . . 46
4.13 3.13 Exercises . . . 47
4.14 3.12 Complete precedence table (canonical) . . . 48
4.15 3.13 Truthiness rules . . . 48
4.16 3.14 Equality: `==`, `===`, `is` . . . 49
4.17 3.15 Short-circuit evaluation . . . 50
4.18 3.16 Chained comparisons . . . 51
4.19 3.17 Bitwise operators . . . 51
4.20 3.18 Range operators . . . 52
4.21 3.19 Compound assignment operators . . . 52
4.22 3.20 Type coercion rules in operators . . . 53
4.23 3.21 Optional chaining and null safety . . . 53
4.24 3.22 Membership: `in` and `not in` . . . 54
4.25 3.23 Interaction of `and`/`or` with non-booleans . . . 54
4.26 3.24 Common operator mistakes . . . 54
4.27 3.25 Operator overloading in classes . . . 54
4.28 3.26 Exercises . . . 55

**5 Chapter 4: Control Flow** **56**
5.1 4.1 Conditionals: `if` / `elif` / `else` . . . 56
5.2 4.2 Loops: `while` and `for` . . . 56
5.3 4.3 Pattern matching: `match` . . . 58
5.4 4.4 `when`: conditional expressions . . . 61
5.5 4.5 Error handling: `try` / `catch` / `throw` / `finally` . . . 61
5.6 4.6 Common pitfalls . . . 65
5.7 4.7 Quick reference . . . 65
5.8 4.8 Exercises . . . 65
5.9 4.12 `if` / `elif` / `else` — complete semantics . . . 66
5.10 4.13 `while` loops . . . 66
5.11 4.14 `for` loops . . . 67
5.12 4.15 `break`, `continue`, and `return` inside loops . . . 68
5.13 4.16 `match` expressions . . . 68
5.14 4.17 `when` expressions . . . 70
5.15 4.18 `try` / `catch` / `finally` as control flow . . . 70
5.16 4.19 Early-return style . . . 71
5.17 4.20 State machines with `match` . . . 71
5.18 4.21 Retry loops . . . 72

5.19 4.22 Infinite event loops and shutdown . . . 72
5.20 4.23 Loop performance notes . . . 72
5.21 4.24 Common control-flow mistakes . . . 73
5.22 4.25 Exercises . . . 73

**6 Chapter 5: Functions** **74**
6.1 5.1 Declaration and calling . . . 74
6.2 5.2 Return statements . . . 74
6.3 5.3 Default arguments . . . 75
6.4 5.4 Variadic behavior . . . 75
6.5 5.5 Functions as first-class values . . . 76
6.6 5.6 Closures . . . 76
6.7 5.7 Higher-order functions . . . 77
6.8 5.8 Anonymous functions (lambdas) . . . 77
6.9 5.9 `return` in closures . . . 78
6.10 5.10 Common pitfalls . . . 78
6.11 5.11 Quick reference . . . 78
6.12 5.12 Exercises . . . 79
6.13 5.11 Function declaration forms . . . 79
6.14 5.12 Parameters: defaults, variadic, and destructuring . . . 80
6.15 5.13 Return values . . . 81
6.16 5.14 First-class functions and higher-order functions . . . 81
6.17 5.15 Recursion . . . 82
6.18 5.16 Closures . . . 83
6.19 5.17 `self` in methods . . . 84
6.20 5.18 Function metadata and introspection . . . 84
6.21 5.19 Overloading and dispatch . . . 84
6.22 5.20 Default arguments and `nil` sentinels . . . 84
6.23 5.21 Pure functions and side effects . . . 85
6.24 5.22 Memoization . . . 85
6.25 5.23 Function composition . . . 86
6.26 5.24 Currying and partial application . . . 86
6.27 5.25 Error handling in functions . . . 86
6.28 5.26 Documentation comments . . . 86
6.29 5.27 Performance considerations . . . 87
6.30 5.28 Common function mistakes . . . 87
6.31 5.29 Exercises . . . 87

**7 Chapter 6: Lambdas and Closures** **88**
7.1 6.1 Lambda expressions . . . 88
7.2 6.2 Closure capture semantics . . . 88
7.3 6.3 The loop-variable capture trap . . . 89
7.4 6.4 `return` target in closures . . . 90
7.5 6.5 Closures and reference counting . . . 90
7.6 6.6 Closures in collections . . . 91
7.7 6.7 Closures and threads . . . 91
7.8 6.8 JIT lowering note . . . 91
7.9 6.9 Quick reference . . . 92

7.10 6.10 Exercises . . . 92
7.11 6.12 Lambda and anonymous-function syntax — complete reference . . . 92
7.12 6.13 First-class functions as values . . . 93
7.13 6.14 Closures and capture semantics . . . 94
7.14 6.15 Partial application and currying . . . 94
7.15 6.16 Function composition . . . 95
7.16 6.17 Memoization . . . 95
7.17 6.18 Callbacks and sorting with comparator functions . . . 96
7.18 6.19 Closures over mutable state . . . 96
7.19 6.20 Common closure pitfalls . . . 96
7.20 6.21 Exercises . . . 97

**8 Chapter 7: Strings Deep Dive 98**
8.1 7.1 String literals . . . 98
8.2 7.2 Indexing and slicing . . . 99
8.3 7.3 Concatenation and repetition . . . 100
8.4 7.4 String methods . . . 101
8.5 7.5 String formatting . . . 103
8.6 7.6 Character codes . . . 104
8.7 7.7 Immutability and performance . . . 104
8.8 7.7 Common pitfalls . . . 105
8.9 7.9 Quick reference . . . 106
8.10 7.10 Exercises . . . 106
8.11 7.12 String encoding and code points vs bytes . . . 107
8.12 7.13 Indexing, slicing, and negative indices . . . 107
8.13 7.14 String immutability . . . 107
8.14 7.15 String methods — complete reference . . . 108
8.15 7.16 Formatting and interpolation . . . 109
8.16 7.17 String comparison and ordering . . . 109
8.17 7.18 Common string pitfalls . . . 109
8.18 7.19 Exercises . . . 110

**9 Chapter 8: Lists and Arrays 111**
9.1 8.1 List literals . . . 111
9.2 8.2 Indexing and slicing . . . 111
9.3 8.3 Mutation methods . . . 112
9.4 8.4 List operations . . . 113
9.5 8.5 List comprehension . . . 114
9.6 8.6 Common patterns . . . 115
9.7 8.6 Sorting . . . 116
9.8 8.7 Common pitfalls . . . 117
9.9 8.8 Quick reference . . . 117
9.10 8.9 Exercises . . . 117

**10 Chapter 9: Dictionaries and Maps 119**
10.1 9.1 Dictionary literals . . . 119
10.2 9.2 Access and mutation . . . 119
10.3 9.3 Dictionary methods . . . 119

9.3 Iteration . . . 120
9.4 Common patterns . . . 121
9.5 Key rules . . . 123
9.6 Common pitfalls . . . 123
9.6 Quick reference . . . 123
9.7 Exercises . . . 123
9.12 Dictionaries — complete reference . . . 124
9.13 Access and mutation . . . 124
9.14 Dict methods . . . 124
9.15 Dict iteration . . . 125
9.16 Dict equality and hashing . . . 125
9.17 Common dict patterns . . . 125
9.18 Nested dicts . . . 126
9.19 Performance notes . . . 126
9.20 Common dict mistakes . . . 127
9.21 Exercises . . . 127

**11 Chapter 10: Modules and Imports** **128**
10.1 Importing modules . . . 128
10.2 Module resolution order . . . 129
10.3 Module tiers: pure-Zen vs native . . . 131
10.4 `requirements.txt` and `zen py-sync` . . . 132
10.5 `pymport` — Python interoperability . . . 133
10.6 Writing your own modules . . . 134
10.7 Module search path . . . 135
10.8 Circular imports . . . 135
10.9 `include` directive . . . 136
10.10 Common pitfalls . . . 136
10.10 Quick reference . . . 136
10.11 Exercises . . . 137
10.12 Modules and imports — complete reference . . . 137
10.13 Writing your own modules . . . 139
10.14 Import performance . . . 139
10.15 Common import mistakes . . . 139
10.16 Exercises . . . 139

**12 Chapter 11: Error Handling** **141**
11.1 Catching errors: `try` / `catch` . . . 141
11.2 Reading the real failure: `catch as e` . . . 141
11.3 Error fields: `e.msg`, `e.code`, `e.val` . . . 142
11.4 String + error concatenation . . . 142
11.5 Catching specific error types . . . 142
11.6 Raising errors: `throw` . . . 143
11.7 Cleanup: `finally` . . . 144
11.8 `exit()` and `finally` . . . 144
11.9 Nested handlers . . . 145
11.10 Errors from callees . . . 145
11.11 Error types . . . 145

12.12 11.12 Full error-code reference . . . 146
12.13 11.13 Common pitfalls . . . 146
12.14 11.14 Quick reference . . . 146
12.15 11.15 Exercises . . . 147
12.16 11.12 Error handling — complete reference . . . 147
12.17 11.13 Common runtime errors . . . 149
12.18 11.14 Defensive programming . . . 150
12.19 11.15 Debugging with `zen explain` . . . 150
12.20 11.16 Exercises . . . 150

**13 Chapter 12: Classes and Objects 151**
13.1 12.1 Defining a class . . . 151
13.2 12.2 Creating instances . . . 151
13.3 12.3 Fields and methods . . . 151
13.4 12.4 Inheritance: `extends` and `super` . . . 152
13.5 12.5 Method overriding . . . 153
13.6 12.6 `this` binding . . . 153
13.7 12.7 Common patterns . . . 154
13.8 12.8 Common pitfalls . . . 155
13.9 12.9 Quick reference . . . 155
13.10 12.10 Exercises . . . 155
13.11 12.12 Classes and objects — complete reference . . . 156
13.12 12.13 Common OOP mistakes . . . 159
13.13 12.14 Exercises . . . 159

**14 Chapter 13: File I/O 160**
14.1 13.1 The `fs` module . . . 160
14.2 13.2 The `os` module . . . 161
14.3 13.3 The `pathlib` module (pure-Zen) . . . 161
14.4 13.4 Reading and writing text files . . . 162
14.5 13.5 Binary files . . . 162
14.6 13.6 Temporary files . . . 162
14.7 13.7 The `with` statement (auto-scoping) . . . 163
14.8 13.8 Common patterns . . . 163
14.9 13.9 Common pitfalls . . . 164
14.10 13.10 Quick reference . . . 164
14.11 13.11 Exercises . . . 165
14.12 13.12 File I/O — complete reference . . . 165
14.13 13.13 Temporary files . . . 168
14.14 13.14 CSV-like parsing (manual) . . . 168
14.15 13.15 Exercises . . . 169

**15 Chapter 14: Memory and References 170**
15.1 14.1 Value representation . . . 170
15.2 14.2 Reference counting . . . 170
15.3 14.3 Ownership and aliasing . . . 170
15.4 14.4 Closures and captured variables . . . 171
15.5 14.5 Cycles and leaks . . . 171

14.6 Handles and resources . . . 172
14.7 `with` statement . . . 172
14.8 Step budget (for untrusted code) . . . 172
14.9 Common pitfalls . . . 172
14.10 Quick reference . . . 173
14.11 Exercises . . . 173
14.12 Values, references, and ownership — complete reference . . . 173
14.13 Exercises . . . 176

**15 Chapter 15: math Module** **177**
15.1 Constants . . . 177
15.2 Rounding . . . 177
15.3 Powers and logarithms . . . 178
15.4 Trigonometry . . . 178
15.5 Hyperbolic functions . . . 179
15.6 Other functions . . . 179
15.7 Using the native tier . . . 180
15.8 Quick reference . . . 181
15.9 Exercises . . . 181
15.12 math module — complete function reference . . . 181
15.13 Exercises . . . 183

**16 Chapter 16: random Module** **184**
16.1 Random numbers . . . 184
16.2 Random sequences . . . 185
16.3 Deterministic randomness (seed) . . . 186
16.4 Common patterns . . . 186
16.5 Quick reference . . . 188
16.6 Exercises . . . 188
16.12 random module — complete reference . . . 189
16.12 Worked example — dice and shuffle . . . 190
16.13 Troubleshooting randomness . . . 190
16.14 Exercises . . . 190

**17 Chapter 17: string Module** **191**
17.1 Case conversion . . . 191
17.2 Trimming . . . 191
17.3 Splitting and joining . . . 192
17.4 Padding and alignment . . . 193
17.5 Case checking . . . 194
17.6 Prefix/suffix . . . 195
17.7 Replacement and translation . . . 195
17.8 Quick reference . . . 196
17.9 Exercises . . . 196
17.12 string module — complete reference . . . 197

**18 Chapter 18: collections Module** **199**
18.1 deque — double-ended queue . . . 199

19.218.2 Counter — count hashable objects . . . 201
19.318.3 OrderedDict — dict that remembers order . . . 202
19.418.4 namedtuple — tuple with named fields . . . 202
19.518.5 defaultdict — dict with default factory . . . 203
19.618.6 Quick reference . . . 204
19.718.7 Exercises . . . 205
19.818.12 collections module — complete reference . . . 205

**20 Chapter 19: json Module 207**
20.119.1 Parsing JSON . . . 207
20.219.2 Serializing JSON . . . 207
20.319.3 Round-trip example . . . 208
20.419.4 Reading/writing JSON files . . . 208
20.519.5 Numbers . . . 209
20.619.6 Common pitfalls . . . 210
20.719.7 Quick reference . . . 210
20.819.8 Exercises . . . 210
20.919.12 json module — complete reference . . . 211
20.1019.12 Worked example — read, modify, write JSON . . . 212
20.1119.13 Troubleshooting JSON . . . 212
20.1219.14 Exercises . . . 212

**21 Chapter 20: datetime/time Module 213**
21.120.1 The `datetime` type . . . 213
21.220.2 Creating datetimes . . . 213
21.320.3 Formatting: `strftime` . . . 213
21.420.4 Datetime arithmetic . . . 214
21.520.5 The `time` module . . . 215
21.620.6 Common patterns . . . 216
21.720.7 Quick reference . . . 217
21.820.8 Exercises . . . 217
21.920.12 time/datetime module — complete reference . . . 217
21.1020.12 Worked example — current time and formatting . . . 218
21.1120.13 Troubleshooting time . . . 219
21.1220.14 Exercises . . . 219

**22 Chapter 21: pathlib & os Module 220**
22.121.1 pathlib.Path . . . 220
22.221.2 The `os` module . . . 222
22.321.3 Path manipulation: os.path . . . 223
22.421.4 Common patterns . . . 224
22.521.5 Quick reference . . . 225
22.621.9 Exercises . . . 225
22.721.12 pathlib & os module — complete reference . . . 226

**23 Chapter 22: re (Regex) Module 228**
23.122.1 Compiling patterns . . . 228
23.222.2 Match objects . . . 229

23.3 22.3 Find all matches . . . 230
23.4 22.4 Substitution . . . 231
23.5 22.5 Split . . . 231
23.6 22.6 Common patterns . . . 231
23.7 22.7 Compiling for reuse . . . 232
23.8 22.8 Common pitfalls . . . 233
23.9 22.9 Quick reference . . . 233
23.10 22.10 Exercises . . . 233
23.11 22.12 regex module — complete reference . . . 234

**24 Chapter 23: base64 & binascii** **235**
24.1 23.1 base64 . . . 235
24.2 23.2 binascii . . . 236
24.3 23.3 Common patterns . . . 236
24.4 23.4 Quick reference . . . 238
24.5 23.5 Exercises . . . 238
24.6 23.12 base64 & binascii — complete reference . . . 238
24.7 23.12 Worked example — encode/decode round trip . . . 239
24.8 23.13 Troubleshooting encoding . . . 239
24.9 23.14 Exercises . . . 239

**25 Chapter 24: hashlib Module** **240**
25.1 24.1 Hash functions . . . 240
25.2 24.2 File hashing . . . 241
25.3 24.3 Verifying file integrity . . . 241
25.4 24.4 Common pitfalls . . . 242
25.5 24.5 Quick reference . . . 242
25.6 24.6 Exercises . . . 243
25.7 24.12 hashlib module — complete reference . . . 243
25.8 24.12 Worked example — file integrity checker . . . 244
25.9 24.13 Troubleshooting hashing . . . 245
25.10 24.14 Exercises . . . 245

**26 Chapter 25: crypto Module — Complete Guide** **246**
26.1 25.1 Symmetric encryption . . . 246
26.2 25.2 Asymmetric encryption (RSA) . . . 247
26.3 25.3 Digital signatures . . . 248
26.4 25.4 Key agreement (X25519) . . . 249
26.5 25.5 Hashing and HMAC . . . 250
26.6 25.6 Key derivation (PBKDF2) . . . 251
26.7 25.7 JWT (JSON Web Tokens) . . . 251
26.8 25.8 OTP (TOTP/HOTP) . . . 252
26.9 25.9 X.509 certificates . . . 253
26.10 25.10 Constant-time compare and secure zero . . . 254
26.11 25.11 Algorithm policy (FIPS-ish mode) . . . 254
26.12 25.12 Quick reference . . . 255
26.13 25.13 Exercises . . . 256
26.14 25.12 crypto module — advanced reference . . . 256

**27 Chapter 26: Networking Fundamentals** **260**
27.1 26.1 Networking module map . . . 260
27.2 26.2 IP addressing . . . 260
27.3 26.3 Ports . . . 261
27.4 26.4 TCP vs UDP . . . 262
27.5 26.5 The OSI model and Zen . . . 262
27.6 26.6 Sockets: file descriptors . . . 263
27.7 26.6 Address families . . . 263
27.8 26.6 Timeouts . . . 264
27.9 26.6 Byte order . . . 264
27.10 26.7 Quick reference . . . 264
27.11 26.8 Exercises . . . 265
27.12 26.12 Socket address families and types (complete reference) . . . 265
27.13 26.13 Berkeley sockets API mapping . . . 266
27.14 26.14 TCP connection lifecycle (detailed) . . . 266
27.15 26.15 UDP communication . . . 266
27.16 26.16 Socket options (important) . . . 266
27.17 26.17 Blocking vs non-blocking . . . 267
27.18 26.18 Byte order (network order) . . . 267
27.19 26.19 Common ports . . . 267
27.20 26.20 Network troubleshooting checklist . . . 267
27.21 26.21 Lab exercises (comprehensive) . . . 268
27.22 26.22 Further reading . . . 268

**28 Chapter 27: socket Module** **269**
28.1 27.1 Socket creation . . . 269
28.2 27.2 Socket handle operations . . . 269
28.3 27.3 TCP client . . . 270
28.4 27.4 TCP server . . . 271
28.5 27.5 UDP . . . 272
28.6 27.6 Socket options . . . 273
28.7 27.7 Common patterns . . . 273
28.8 27.8 Common pitfalls . . . 275
28.9 27.8 Quick reference . . . 275
28.10 26.9 Exercises . . . 276
28.11 27.12 Advanced socket patterns . . . 276
28.12 27.13 select/poll multiplexing concept . . . 276
28.13 27.14 Unix domain sockets . . . 277
28.14 27.15 SSL/TLS over sockets (preview) . . . 277
28.15 27.16 Common errors . . . 277
28.16 27.17 Debugging tips . . . 277
28.17 27.18 Exercises . . . 277

**29 Chapter 28: HTTP Protocol** **278**
29.1 28.1 HTTP basics . . . 278
29.2 28.2 The `http` module . . . 278
29.3 28.3 Making requests with headers . . . 279
29.4 28.4 POST with body . . . 280

29.5 28.5 Status codes . . . 280
29.6 28.6 HTTP vs HTTPS . . . 280
29.7 28.7 Common patterns . . . 281
29.8 28.8 Common pitfalls . . . 282
29.9 28.9 Quick reference . . . 282
29.10 28.10 Exercises . . . 282
29.11 28.12 HTTP/1.1 detailed . . . 283
29.12 28.13 HTTP methods . . . 284
29.13 28.14 MIME types (common) . . . 284
29.14 28.15 Caching . . . 284
29.15 28.16 Authentication . . . 284
29.16 28.17 HTTPS overview . . . 284
29.17 28.18 Exercises . . . 284

**30 Chapter 29: requests Module 285**
30.1 29.1 Quick requests . . . 285
30.2 29.2 Options dictionary . . . 286
30.3 29.3 Sessions . . . 286
30.4 29.4 JSON helpers . . . 287
30.5 29.5 URL encoding utilities . . . 288
30.6 29.6 Common patterns . . . 288
30.7 29.7 Error handling . . . 289
30.8 29.8 Quick reference . . . 290
30.9 29.9 Exercises . . . 290
30.10 29.10 Advanced usage patterns . . . 291
30.11 29.11 Error handling . . . 292
30.12 29.12 Best practices . . . 292
30.13 29.13 Exercises . . . 292

**31 Chapter 30: http Server Module 294**
31.1 30.1 Starting a server . . . 294
31.2 30.2 Server options . . . 294
31.3 30.3 What it serves . . . 294
31.4 30.4 Example: serve a build directory . . . 294
31.5 30.5 Example: serve once and get the port . . . 295
31.6 30.6 MIME type guessing . . . 295
31.7 30.7 Common pitfalls . . . 296
31.8 30.8 Directory traversal protection . . . 296
31.9 30.9 Quick reference . . . 296
31.10 30.10 Exercises . . . 296
31.11 30.10 Building a more complete server . . . 296
31.12 30.11 Production considerations . . . 297
31.13 30.12 Exercises . . . 297
31.14 30.12 Worked example — a small HTTP server with routing . . . 297
31.15 30.13 Troubleshooting HTTP servers . . . 298
31.16 30.14 Exercises . . . 299

**32 Chapter 31: HTTPS & SSL/TLS Certificates 300**

32.1 31.1 The `tls` module . . . 300
32.2 31.2 Certificate pinning . . . 300
32.3 31.3 Plain TLS vs pinned TLS . . . 301
32.4 31.4 Complete HTTPS client example . . . 301
32.5 31.5 Error handling . . . 303
32.6 31.6 Quick reference . . . 303
32.7 31.7 Exercises . . . 303
32.8 31.12 HTTPS / SSL/TLS — deep reference . . . 304
32.9 31.12 Worked example — fetching HTTPS with verification . . . 305
32.10 31.13 Troubleshooting TLS/HTTPS . . . 305
32.11 31.14 Exercises . . . 306

**33 Chapter 32: dns Module** **307**
33.1 32.1 Resolve a hostname . . . 307
33.2 32.2 `gethostbyname` — single IPv4 . . . 307
33.3 32.3 `gethostbyname_ex` — extended info . . . 307
33.4 32.4 `gethostbyaddr` — reverse lookup . . . 307
33.5 32.5 `getfqdn` — fully qualified domain name . . . 308
33.6 32.6 Common patterns . . . 308
33.7 32.7 Quick reference . . . 309
33.8 32.8 Exercises . . . 309
33.9 32.12 DNS module — deep reference . . . 309
33.10 32.12 Worked example — DNS resolution . . . 311
33.11 32.13 Troubleshooting DNS . . . 311
33.12 32.14 Exercises . . . 312

**34 Chapter 33: url Module** **313**
34.1 33.1 Parsing URLs . . . 313
34.2 33.2 Building URLs . . . 314
34.3 33.3 Query strings . . . 314
34.4 33.4 Joining URLs . . . 315
34.5 33.5 Percent encoding . . . 315
34.6 33.6 Splitting and joining netloc . . . 316
34.7 33.6 Common patterns . . . 316
34.8 33.7 Quick reference . . . 317
34.9 33.8 Exercises . . . 317
34.10 33.12 URL module — deep reference . . . 318

**35 Chapter 34: Raw Sockets** **320**
35.1 34.1 TCP client . . . 320
35.2 34.2 TCP client . . . 321
35.3 34.3 TCP server . . . 321
35.4 34.4 UDP . . . 322
35.5 34.5 Raw sockets and packet crafting (net_raw) . . . 322
35.6 34.6 Common patterns . . . 324
35.7 34.7 Common pitfalls . . . 325
35.8 34.8 Quick reference . . . 325
35.9 34.8 Exercises . . . 325

35.10 34.12 Raw sockets — deep reference . . . 326

**36 Chapter 35: Threading Fundamentals 328**
36.1 35.1 The `thread` module . . . 328
36.2 35.2 Thread lifecycle . . . 329
36.3 35.3 Thread-local data . . . 329
36.4 35.3 Multiple threads . . . 330
36.5 35.4 Common pitfalls . . . 330
36.6 35.5 Quick reference . . . 330
36.7 35.6 Exercises . . . 330
36.8 Extended Content . . . 331
36.9 35.12 Worked examples — threading . . . 331
36.10 35.13 Common threading bugs . . . 333
36.11 35.14 Debugging threaded programs . . . 333
36.12 35.15 Exercises . . . 333

**37 Chapter 36: Channels 335**
37.1 36.1 The `channel` module . . . 335
37.2 36.2 Blocking semantics . . . 335
37.3 36.3 Producer-consumer . . . 336
37.4 36.4 Multiple producers and consumers . . . 337
37.5 36.5 Channels and shared values . . . 338
37.6 36.6 Common pitfalls . . . 338
37.7 36.7 Quick reference . . . 338
37.8 36.8 Exercises . . . 339
37.9 Extended Content . . . 339
37.10 36.12 Worked examples — channels . . . 339
37.11 36.13 Channel patterns . . . 341
37.12 36.14 Common channel bugs . . . 341
37.13 36.15 Exercises . . . 341

**38 Chapter 37: Mutexes & Synchronization 343**
38.1 37.1 The `shared` module — atomic shared values . . . 343
38.2 37.2 The `channel` module — message passing . . . 344
38.3 37.3 Threads: `thread` module . . . 345
38.4 37.4 Producer-consumer with channels . . . 345
38.5 37.5 Shared counter across threads . . . 346
38.6 37.6 Common patterns . . . 346
38.7 37.8 Common pitfalls . . . 348
38.8 37.9 Quick reference . . . 348
38.9 37.10 Exercises . . . 348
38.10 Extended Content . . . 349
38.11 37.12 Worked examples — mutexes and locks . . . 349
38.12 37.13 Locking guidelines . . . 351
38.13 37.14 Exercises . . . 351

**39 Chapter 38: Thread Safety & Patterns 352**
39.1 38.1 Worker pool pattern . . . 352

39.2 38.2 Pipeline pattern . . . 353
39.3 38.3 Reader-writer pattern with shared flag . . . 354
39.4 38.4 Avoiding deadlocks . . . 354
39.5 38.5 Avoiding race conditions . . . 354
39.6 38.6 Thread-local data . . . 355
39.7 38.7 Common patterns summary . . . 355
39.8 38.8 Quick reference . . . 356
39.9 38.9 Exercises . . . 356
39.10 Extended Content . . . 356
39.11 38.12 Worked examples — thread safety patterns . . . 357
39.12 38.13 Pattern catalog . . . 359
39.13 38.14 Exercises . . . 360

**40 Chapter 39: FFI Fundamentals 361**
40.1 39.1 Loading a library . . . 361
40.2 39.2 Finding symbols . . . 361
40.3 39.3 Calling functions . . . 361
40.4 39.4 Memory management . . . 362
40.5 39.5 Complete example: calling `qsort` . . . 364
40.6 39.6 Safety warnings . . . 364
40.7 39.7 Quick reference . . . 364
40.8 39.8 Exercises . . . 365
40.9 Advanced Topics and Exercises . . . 365
40.10 39.12 Worked examples — FFI fundamentals . . . 365
40.11 39.13 Data type mapping (typical) . . . 367
40.12 39.14 Safety considerations . . . 367
40.13 39.15 Debugging FFI . . . 367
40.14 39.16 Exercises . . . 367

**41 Chapter 40: FFI in Practice 369**
41.1 40.1 FFI capability gates . . . 369
41.2 40.2 Calling libc functions . . . 369
41.3 40.3 Working with strings . . . 370
41.4 40.4 Working with buffers . . . 370
41.5 40.5 Error handling . . . 370
41.6 40.6 Calling conventions and types . . . 371
41.7 40.7 Complete example: libc time . . . 371
41.8 40.8 Quick reference . . . 371
41.9 40.9 Exercises . . . 372
41.10 Advanced Topics and Exercises . . . 372
41.11 40.12 Worked examples — FFI in practice . . . 373
41.12 40.13 FFI best practices . . . 374
41.13 40.14 Exercises . . . 374
41.14 40.12 Worked example — wrap a C hash function . . . 375
41.15 40.13 Troubleshooting FFI . . . 375
41.16 40.14 Exercises . . . 375

**42 Chapter 41: sys Module 376**

42.141.1 File descriptor operations . . . 376
42.241.2 Process control . . . 377
42.341.3 Pipes . . . 377
42.441.4 Direct syscalls . . . 377
42.541.5 Signal handling (limited) . . . 378
42.641.6 Common patterns . . . 378
42.741.7 Quick reference . . . 379
42.841.8 Common pitfalls . . . 379
42.941.9 Exercises . . . 379
42.1041.10 Advanced Topics and Exercises . . . 380
42.1141.12 Worked examples — sys module . . . 380
42.1241.13 Platform differences . . . 381
42.1341.14 Exercises . . . 381
42.1441.12 Worked example — platform-aware script . . . 382
42.1541.13 Troubleshooting sys . . . 382
42.1641.14 Exercises . . . 382

**43 Chapter 42: pympor - Python Interoperability 383**
43.142.1 Why `pympor` (and not just `import`)? . . . 383
43.242.2 Syntax and Binding . . . 383
43.342.3 Building Zen with Python Support . . . 384
43.442.4 Environment Setup (venv + PYTHONPATH) . . . 384
43.542.5 Dependency Sync with `zen py-sync` and `requirements.txt` . . . 384
43.642.6 Using Python from Zen . . . 385
43.742.7 Error handling . . . 387
43.842.8 Limits and Non-Goals (v1) . . . 388
43.942.9 Working Examples (tested) . . . 388
43.1042.10 Troubleshooting . . . 388
43.1142.11 References . . . 389
43.1242.12 Quick reference . . . 389
43.1342.13 Exercises . . . 390
43.1442.10 Advanced Topics and Exercises . . . 390
43.1542.12 Worked examples — pympor . . . 391
43.1642.13 pympor best practices . . . 392
43.1742.14 Exercises . . . 392

**44 Chapter 50: Zen VM and Runtime Internals 394**
44.115.1 Runtime architecture overview . . . 394
44.215.2 Value representation . . . 394
44.315.3 Memory model . . . 395
44.415.4 Execution models . . . 396
44.515.5 Bytecode (conceptual) . . . 396
44.615.6 The call stack and frames . . . 397
44.715.7 Closures and upvalues . . . 397
44.815.8 Method dispatch . . . 397
44.915.9 FFI boundary (Zen ↔ C) . . . 397
44.1015.10 pympor boundary (Zen ↔ Python) . . . 398
44.1115.11 Garbage collection tuning (conceptual) . . . 398

44.12 25.12 Debugging the runtime . . . 398
44.13 25.13 Exercises . . . 398

**45 Chapter 51: Zen Type System In Depth 400**
45.1 16.1 The type taxonomy . . . 400
45.2 16.2 Type tags and immediate vs heap . . . 401
45.3 16.3 Type introspection . . . 401
45.4 16.4 Type coercion . . . 401
45.5 16.5 Equality: `==`, `===`, `is` . . . 402
45.6 16.6 Numeric details . . . 402
45.7 16.7 String type details . . . 403
45.8 16.8 Container type details . . . 403
45.9 16.9 Type stability and performance . . . 403
45.10 16.10 Common type mistakes . . . 404
45.11 16.11 Exercises . . . 404

**46 Chapter 52: Zen Bytecode Reference 405**
46.1 17.1 What bytecode is . . . 405
46.2 17.2 Instruction format (conceptual) . . . 405
46.3 17.3 Opcode categories . . . 405
46.4 17.4 Stack effect . . . 407
46.5 17.5 Constant pool . . . 407
46.6 17.6 How the interpreter executes . . . 408
46.7 17.7 How the JIT compiles . . . 408
46.8 17.8 Inline caches (conceptual) . . . 408
46.9 17.9 Reading bytecode (if your build exposes it) . . . 408
46.10 17.10 Exercises . . . 408

**47 Chapter 53: Garbage Collection and Memory Management 410**
47.1 18.1 Overview . . . 410
47.2 18.2 What gets collected . . . 410
47.3 18.3 Reachability . . . 410
47.4 18.4 Reference counting vs tracing . . . 410
47.5 18.5 Reference cycles (the classic leak) . . . 411
47.6 18.6 `shared` and thread-shared memory . . . 411
47.7 18.7 Writing memory-friendly code . . . 411
47.8 18.8 Memory leaks to watch for . . . 411
47.9 18.9 GC and performance . . . 412
47.10 18.10 Exercises . . . 412

**48 Chapter 54: The Cranelift JIT Compiler 413**
48.1 19.1 Overview . . . 413
48.2 19.2 Interpreter vs JIT . . . 413
48.3 19.3 Compilation process . . . 413
48.4 19.4 Inline caches . . . 413
48.5 19.5 On-stack replacement (OSR) . . . 414
48.6 19.6 Deoptimization . . . 414
48.7 19.7 When does JIT help? . . . 414

48.819.8 Observing JIT behavior . . . 414
48.919.9 JIT and FFI / pymport . . . 414
48.1019.10 Exercises . . . 415

**49 Chapter 43: CLI Tools 416**
49.143.1 Argument parsing: `argparser` . . . 416
49.243.2 Reading stdin . . . 417
49.343.3 Writing output . . . 417
49.443.4 Exit codes . . . 418
49.543.5 Complete example: a `cat` clone . . . 418
49.643.6 Complete example: a `grep` clone . . . 418
49.743.7 Complete example: a JSON pretty-printer . . . 419
49.843.8 Making scripts executable . . . 419
49.943.9 Common pitfalls . . . 420
49.1043.10 Quick reference . . . 420
49.1143.11 Exercises . . . 420
49.12Extended Project Ideas and Implementation Notes . . . 420
49.1343.12 Full tutorial — building a CLI todo manager . . . 421
49.1443.13 CLI design checklist . . . 423
49.1543.14 Exercises . . . 424

**50 Chapter 44: Web Scraping & Automation 425**
50.144.1 Setup . . . 425
50.244.2 Run the whole suite . . . 425
50.344.3 1) Scrape + Download with `requests` + `bs4` (pymport) . . . 426
50.444.4 2) Stateful Scraping + Download with `mechanicalsoup` (pymport) . . . 428
50.544.5 3) Browser Automation with Selenium (pymport) . . . 429
50.644.6 4) Low-Level Control: CDP over raw TCP + WebSocket sockets . . . 432
50.744.7 Tips and Gotchas . . . 436
50.844.8 Verified Artifacts . . . 437
50.944.9 Appendix: Tested Scripts . . . 437
50.10Extended Project Ideas and Implementation Notes . . . 437
50.1144.12 Full tutorial — simple web scraper . . . 438
50.1244.13 Requests usage recap . . . 439
50.1344.14 Exercises . . . 439

**51 Chapter 45: REST API Client 441**
51.145.1 Basic GET request . . . 441
51.245.2 POST with JSON body . . . 441
51.345.3 Reusable API client . . . 441
51.445.4 Authentication . . . 443
51.545.5 Pagination . . . 443
51.645.6 Rate limiting . . . 444
51.745.7 Error handling . . . 445
51.845.8 Complete CLI example . . . 446
51.945.9 Common pitfalls . . . 446
51.1045.10 Quick reference . . . 446
51.1145.11 Exercises . . . 447

51.12 Extended Project Ideas and Implementation Notes . . . 447
51.13 45.12 Full tutorial — REST API client . . . 448
51.14 45.13 REST client checklist . . . 449
51.15 45.14 Exercises . . . 450

**52 Chapter 46: File Processing Tools 451**
52.1 46.1 Reading text files . . . 451
52.2 46.2 Writing text files . . . 451
52.3 46.3 Binary files . . . 451
52.4 46.4 CSV processing (manual) . . . 452
52.5 46.5 JSON files . . . 452
52.6 46.6 Line processing pipeline . . . 453
52.7 46.7 Temporary files . . . 453
52.8 46.8 Directory walking . . . 453
52.9 46.9 File info . . . 454
52.10 46.10 Quick reference . . . 454
52.11 46.11 Exercises . . . 455
52.12 Extended Project Ideas and Implementation Notes . . . 455
52.13 46.12 Full tutorial — file processing pipeline . . . 455
52.14 46.13 File processing patterns . . . 457
52.15 46.14 Exercises . . . 457

**53 Chapter 47: Network Security Tools 458**
53.1 47.1 Port Scanner . . . 458
53.2 44. Subdomain Finder . . . 458
53.3 44. HTTP Header Checker . . . 459
53.4 44.5 Port Scan Report . . . 460
53.5 44.6 HTTP Header Analysis . . . 460
53.6 44.7 DNS Reconnaissance . . . 461
53.7 44.8 Common pitfalls . . . 462
53.8 44.9 Quick reference . . . 462
53.9 44.10 Exercises . . . 462
53.10 Extended Project Ideas and Implementation Notes . . . 463
53.11 47.12 Full tutorial — hash utility (ethical security tool) . . . 463
53.12 47.13 Security tool principles . . . 465
53.13 47.14 Exercises . . . 465

**54 Chapter 48: Capstone Projects 466**
54.1 48.1 Capstone 1: Web Metadata Archiver . . . 466
54.2 48.2 Capstone 2: Password Strength Checker . . . 467
54.3 48.3 Capstone 3: Log Analyzer . . . 469
54.4 48.4 Capstone 4: Site Uptime Monitor . . . 471
54.5 48.5 Running the Capstones . . . 473
54.6 48.6 Quick reference . . . 473
54.7 48.7 Exercises . . . 473
54.8 Extended Project Ideas and Implementation Notes . . . 473
54.9 48.12 Capstone — personal network toolkit . . . 474
54.10 48.13 Capstone lessons . . . 476

54.14 18.14 Exercises . . . 476

**55 Appendix A: Built-in Functions 477**
55.1 A.1 Type conversion . . . 477
55.2 A.2 Iteration . . . 477
55.3 A.3 Introspection . . . 478
55.4 A.4 I/O . . . 479
55.5 A.5 Truthiness . . . 479
55.6 A.6 Quick reference . . . 479
55.7 A.7 Exercises . . . 480
55.8 A.12 Complete built-in function reference . . . 480
55.9 A.13 Built-in constants . . . 482
55.10 A.14 Exercises . . . 482

**56 Appendix B: Error Codes Reference 483**
56.1 B.1 Lexer errors (Z01xx) . . . 483
56.2 B.2 Parser errors (Z02xx) . . . 483
56.3 B.3 Compile-time errors (Z03xx) . . . 483
56.4 B.4 Runtime errors (Z04xx) . . . 484
56.5 B.5 CLI/tool errors (Z05xx) . . . 484
56.6 Using `zen explain` . . . 484
56.7 Looking up an error . . . 485
56.8 Common runtime errors and fixes . . . 485
56.9 Exercises . . . 486
56.10 B.12 Error code reference Z01xx – Z05xx (exhaustive) . . . 486
56.11 B.13 How to read a Zen error . . . 491
56.12 B.14 Error handling patterns . . . 491
56.13 B.15 Exercises . . . 492

**57 Appendix C: Zen CLI Reference 493**
57.1 C.1 Commands . . . 493
57.2 C.2 Exit codes . . . 495
57.3 C.3 Environment variables . . . 495
57.4 C.4 Configuration files . . . 495
57.5 C.5 Common workflows . . . 496
57.6 C.6 REPL commands (if supported) . . . 496
57.7 C.7 Exercises . . . 496

**58 Appendix D: Standard Library Index (exhaustive) 498**
58.1 D.1 Module list . . . 498
58.2 D.2 math module . . . 499
58.3 D.3 random module . . . 499
58.4 D.4 string module . . . 499
58.5 D.5 collections module . . . 499
58.6 D.6 json module . . . 499
58.7 D.7 time / datetime module . . . 499
58.8 D.8 pathlib / os module . . . 500
58.9 D.9 re module . . . 500

58.10 D.10 base64 / binascii module . . . 500
58.11 D.11 hashlib module . . . 500
58.12 D.12 crypto module . . . 500
58.13 D.13 networking modules . . . 500
58.14 D.14 concurrency modules . . . 500
58.15 D.15 FFI / system / interop modules . . . 501
58.16 D.16 Module → chapter mapping . . . 501
58.17 D.17 Exercises . . . 501

**59 Appendix E: Quick Reference Card 503**
59.1 E.1 Syntax at a glance . . . 503
59.2 E.2 Literals . . . 503
59.3 E.3 Operators (precedence high→low) . . . 504
59.4 E.4 Control flow . . . 504
59.5 E.5 Built-ins (selected) . . . 505
59.6 E.6 String methods . . . 505
59.7 E.7 List methods . . . 505
59.8 E.8 Dict methods . . . 505
59.9 E.9 Socket basics . . . 505
59.10 E.10 Error codes (Z-ranges) . . . 505
59.11 E.11 CLI commands . . . 505
59.12 E.12 Truthiness cheat . . . 506
59.13 E.13 Equality cheat . . . 506
59.14 E.14 Common gotchas (one-liners) . . . 506
59.15 E.15 Exercises . . . 506

**60 Appendix F: Zen 3.0 Complete Language Reference 507**
60.1 F.1 Keywords and reserved words . . . 507
60.2 F.2 Literal forms . . . 507
60.3 F.3 Operator precedence (high → low) . . . 507
60.4 F.4 Truthiness . . . 508
60.5 F.5 Type coercion matrix . . . 508
60.6 F.6 Built-in functions (complete) . . . 508
60.7 F.7 Statement forms . . . 509
60.8 F.8 Zen CLI commands . . . 510
60.9 F.9 Environment variables . . . 510
60.10 F.10 Error codes (exhaustive Z01xx–Z05xx) . . . 510
60.11 F.11 Standard library — module → principal members . . . 512
60.12 F.12 Socket option reference . . . 513
60.13 F.13 Common ports . . . 514
60.14 F.14 HTTP status codes . . . 514
60.15 F.15 HTTP methods . . . 514
60.16 F.16 DNS record types . . . 514
60.17 F.17 URL components . . . 514
60.18 F.18 IPv4 header (20 bytes min) . . . 514
60.19 F.19 TCP connection state machine . . . 514
60.20 F.20 Coercion / truthiness quick examples . . . 515
60.21 F.21 Exercises (reference drill) . . . 515

# 1 Zen for Dummies: Complete Edition

*The Definitive Guide to the Zen Programming Language*

## 1.1 Table of Contents

This comprehensive guide is organized into detailed sections covering every aspect of Zen.


### 1.1.1 Part 1: Fundamentals

1. Getting Started - Installation, first program, REPL
2. Data Types - Primitives, collections, type system
3. Operators and Expressions - Arithmetic, logical, bitwise
4. Control Flow - Conditionals, loops, branching
5. Functions - Defining, calling, parameters, returns
6. Lambdas and Closures - Anonymous functions, higher-order functions

### 1.1.2 Part 2: Core Concepts

7. Strings Deep Dive - Manipulation, formatting, methods
8. Lists and Arrays - Operations, methods, iteration
9. Dictionaries and Maps - Key-value pairs, methods
10. Modules and Imports - Creating, importing, module system
11. Error Handling - Try/catch, throw, debugging

### 1.1.3 Part 3: Object-Oriented & Advanced Programming

12. Classes and Objects - OOP, inheritance, polymorphism
13. File I/O - Reading, writing, `with` statement
14. Memory and References - Values, references, ownership

### 1.1.4 Part 4: Standard Library Modules (Detailed)

15. math Module - Mathematical functions and constants
16. random Module - Random numbers and selection
17. string Module - String utilities
18. collections Module - Advanced data structures
19. json Module - JSON parsing and serialization
20. datetime/time Module - Time and date operations
21. pathlib & os Module - File system operations
22. re (Regex) Module - Pattern matching
23. base64 & binascii - Encoding/decoding
24. hashlib Module - Hashing algorithms
25. crypto Module - Complete Guide - Cryptography, encryption, JWT, X.509

#### 1.1.5 Part 5: Networking & Protocols

26. Networking Fundamentals - Sockets, TCP/IP basics
27. socket Module - Low-level socket programming
28. HTTP Protocol - HTTP/1.1, requests, headers
29. requests Module - High-level HTTP client
30. http Server Module - Building HTTP servers
31. HTTPS & SSL/TLS Certificates - Secure connections, certificates
32. DNS Module - Domain name resolution
33. url Module - URL parsing and manipulation
34. Raw Sockets - Packet-level networking

#### 1.1.6 Part 6: Concurrency & Threading

35. Threading Fundamentals - Threads, spawning, execution
36. Channels - Inter-thread communication
37. Mutexes & Locks - Synchronization primitives
38. Thread Safety & Patterns - Best practices

#### 1.1.7 Part 7: FFI, System & Interoperability

39. FFI Fundamentals - Foreign Function Interface basics
40. FFI in Practice - Calling C functions, handles
41. sys Module - System information and operations
42. pymport - Python Interoperability - Using Python from Zen (setup, sync, examples)

#### 1.1.8 Part 8: Practical Projects

43. CLI Tools - Building command-line applications
44. Web Scraping & Automation - Bots, scrapers, automation
45. REST API Client - API integration projects
46. File Processing Tools - Data processing pipelines
47. Network Security Tools - Ethical hacking utilities
48. Capstone Projects - Full applications

#### 1.1.9 Part 9: Runtime Internals

50. Zen VM and Runtime Internals - interpreter, JIT, GC, memory
51. Type System In Depth - types, coercion, equality
52. Bytecode Reference - bytecode opcodes and execution
53. Garbage Collection - memory management and cycles
54. Cranelift JIT Compiler - JIT, inline caches, OSR, deopt

#### 1.1.10 Appendices

- Appendix A: Built-in Functions
- Appendix B: Error Codes Reference
- Appendix C: Zen CLI Reference

- Appendix D: Standard Library Index
- Appendix E: Quick Reference Card
- Appendix F: Complete Language Reference

## 1.2 About This Book

This book is written for complete beginners but goes deep into every aspect of Zen. Each module gets its own detailed chapter with theory, examples, common pitfalls, and exercises.

# 2 Chapter 1: Getting Started

Welcome to *Zen for Dummies*. This chapter walks you through installing Zen, writing your first program, running it, and using the REPL. By the end you'll have a working Zen development environment and a solid grasp of the basic workflow.

## 2.1 1.1 What is Zen?

Zen 3.0 is a fast, native scripting language with a Python-flavoured syntax, a Cranelift JIT compiler, embedded-Python interoperability (`pymport`), Rust-grade coded diagnostics (`zen explain`), and a batteries-included standard library. It compiles to a single static binary.

Key capabilities:

- **JIT compilation** — Cranelift backend with monomorphic inline caches, OSR (on-stack replacement), and deoptimization.
- **`pymport`** — Import Python modules (stdlib, venv, project-local `.py`) with value marshalling and chained tracebacks.
- **Coded diagnostics** — Error codes `Z01xx`–`Z05xx`, `zen explain`, and Python tracebacks chained under Zen frames.
- **Stdlib** — HTTP, files, regex, JSON, threading, sockets, crypto, SQLite, and more.
- **REPL** — Interactive REPL with `zen` (no arguments).

## 2.2 1.2 Installation

### 2.2.1 From release binaries

Download the latest release from the Releases page.

**Linux (Debian/Ubuntu):**

```
sudo dpkg -i zen_3.0.0_amd64.deb
```

**Linux/macOS (tarball):**

```
tar xzf zen-linux-x86_64.tar.gz
cd zen-linux-x86_64
sudo ./install.sh
```

**Windows:**

Extract the zip and add `zen.exe` to your PATH.

### 2.2.2 From source

```
git clone https://github.com/3CN-O4D/zen.git
cd zen
cargo build --release --package zen-cli --bin zen --all-features
# Binary at: target/release/zen
```

Verify the install:

```
zen --version
# zen 3.0.0
```

## 2.3 1.3 Your first program

Create a file named `hello.z`:

```
print("Hello, Zen!")
```

Run it:

```
zen run hello.z
# Hello, Zen!
```

Zen scripts use `.z` by convention, but any extension works.

### 2.3.1 Variables and functions

Zen uses Python-flavoured syntax. Variables are declared with `let` (immutable by default), `var` (mutable), or `const` (compile-time constant). Functions use `fn`.

```
# hello2.z
func greet(name) {
    return "Hello, " + name + "!"
}

let recipient = "Zen programmer"
print(greet(recipient))
# Hello, Zen programmer!
```

Run it:

```
zen run hello2.z
```

### 2.3.2 Comments

Comments start with `#` and run to the end of the line:

```
# This is a comment
print("code")  # inline comment
```

## 2.4 1.4 The REPL

Start the interactive REPL by running `zen` with no arguments:

```
zen
# Zen 3.0 REPL
# >
```

In the REPL, type expressions and press Enter to evaluate them immediately:

```
> 1 + 2 * 3
7
> "Zen " + "3.0"
'Zen 3.0'
> let x = 42
> x * 2
84
> func double(n) { return n * 2 }
> double(21)
42
> :quit
```

REPL commands:

| Command | Effect |
|---|---|
| `:quit` / `:q` | Exit the REPL |
| `:help` | Show help |
| `:clear` | Clear the screen |

Multi-line statements are supported — the REPL detects when a block is incomplete (e.g. after `fn`, `if`, `for`) and keeps prompting until the block is closed.

## 2.5 1.5 `zen explain`

Every runtime error in Zen has a coded diagnostic (e.g. `Z0401`, `Z0325`). Use `zen explain` to see the long-form explanation:

```
zen explain Z0401
zen explain Z0325
zen explain Z0501
```

Output includes the error title, a detailed description, and often a `= help:` suggestion with a code example. This is your primary reference when something goes wrong.

## 2.6 1.6 Type-checking without running

`zen check` compiles a script and reports errors without executing it:

```
zen check app.z
# OK: app.z
```

This catches undefined names, structural errors, and unresolved imports before you run the program.

## 2.7 1.7 Linting

`zen lint` warns about suspicious patterns: unused variables, unused parameters, and dead (unreachable) code:

```
zen lint app.z
zen lint --strict app.z   # warnings become errors (Werror)
```

## 2.8 1.8 Formatting

`zen fmt` reformats source files to canonical Zen style:

```
zen fmt app.z         # format in place
zen fmt --check app.z # check formatting without changing files
```

## 2.9 1.9 Building standalone executables

`zen build` bundles a program into a standalone executable that carries its embedded bytecode payload:

```
zen build app.z -o app
./app
```

The resulting binary requires no Zen installation to run.

## 2.10 1.10 Running tests

`zen test` runs the project's test suite:

```
zen test
```

## 2.11 1.11 Project layout

A typical Zen project:

```
myproject/
  main.z          # entry point
  lib.z           # library module
  utils/
    helper.z
  requirements.txt # Python deps for pymport (optional)
  tests/
    test_main.z
```

Run from the project root:

```
zen run main.z
```

## 2.12 1.12 Next steps

- Chapter 2: Data Types — primitives, collections, and the type system.
- Chapter 3: Operators and Expressions — arithmetic, comparison, logic, `?.` safe access, `??` coalescing.
- Chapter 4: Control Flow — conditionals, loops, `match`, `when`, `try`/`catch`.

## 2.13 1.5 Command-line interface reference (deep)

The zen CLI has multiple modes. Complete reference:

| Command | Purpose | Examples |
|---|---|---|
| `zen` / `zen repl` | Interactive REPL | `zen` |
| `zen run <file>` | Execute script | `zen run app.z` |
| `zen -m <module>` | Run module as main | `zen -m http.server` |
| `zen -e <code>` | Inline execution | `zen -e 'print(1+1)'` |
| `zen check <file>` | Type/compile check without run | `zen check lib.z` |
| `zen lint <file>` | Static analysis warnings | `zen lint app.z` |
| `zen explain <code>` | Explain error code | `zen explain Z0308` |
| `zen build <file>` | Bundle to standalone executable | `zen build app.z -o app` |
| `zen test` | Run tests in project | `zen test` |
| `zen fmt` | Format sources | `zen fmt .` |
| `zen help` | Help | `zen help` |
| `zen --version` | Version | `zen --version` |

### 2.13.1 Environment variables

| Variable | Purpose | Default |
|---|---|---|
| `ZEN_PATH` | Module search paths (colon-separated) | platform dependent |
| `ZEN_DEBUG` | Enable debug logging | unset |
| `PYTHONPATH` | Used by pymport for Python module resolution | inherited |
| `VIRTUAL_ENV` | Python venv used by pymport | inherited |

## 2.14 1.6 Project layout conventions

A typical Zen project:

```
myproject/
├── main.z           # entry point
├── lib/             # local modules
│   ├── utils.z
│   └── config.z
├── tests/           # tests
└── README.md
```

Import local modules:

```
import lib.utils
from lib.config import get_config
```

## 2.15 1.7 Running as script vs module

When run as script (`zen run main.z`), `__name__` is typically `"__main__"` or script path. When imported, different. Check module behavior.

## 2.16 1.8 Debugging workflow

1. Use `print()` for quick debugging
2. Run with `ZEN_DEBUG=1 zen run app.z` for verbose output
3. Use `zen check` to catch compile errors
4. Use `zen explain ZXXXX` for detailed error explanations
5. Use REPL to experiment interactively

## 2.17 1.9 Common first-run issues

- **Command not found**: Add install dir to PATH
- **Permission denied**: `chmod +x zen`
- **Module not found**: Check ZEN_PATH and file locations
- **Import errors with pymport**: Ensure Python and deps installed in active env

## 2.18 1.10 Exercises (extended)

1. Install/locate `zen` binary and print version
2. Write `hello.z` that prints your name and run it
3. Use `zen -e` to compute `2**10`
4. Start REPL, define function, call it, exit
5. Create a multi-file project with local imports
6. Use `zen explain` on a made-up code? Or find real error
7. Write a script that reads args (see sys.argv patterns)
8. Compare `zen run` vs compiling concepts

# 3 Chapter 2: Data Types

Zen is a dynamically typed, strongly typed language. Every value carries a type tag at runtime, operations check operand types, and implicit conversions are minimal and explicit. This chapter is the definitive reference for every literal form, every type, every coercion, and every edge case in Zen's value system.

## 3.1 2.1 The type system at a glance

Zen values are 64-bit NaN-boxed tagged unions. There are **16 language types**:

| # | Type | Kind | Storage | `typeof()` returns |
|---|---|---|---|---|
| 1 | `nil` | primitive | inline | `"nil"` |
| 2 | `bool` | primitive | inline | `"bool"` |
| 3 | `int` | primitive | inline (i64) | `"int"` |
| 4 | `float` | primitive | inline (f64) | `"float"` |
| 5 | `string` | heap | `Arc<str>` | `"string"` |
| 6 | `bytes` | heap | `Arc<[u8]>` | `"bytes"` |
| 7 | `list` | heap | `Vec<Value>` | `"list"` |
| 8 | `dict` | heap | `IndexMap` | `"dict"` |
| 9 | `set` | heap | `IndexSet` | `"set"` |
| 10 | `tuple` | heap | `Vec<Value>` | `"tuple"` |
| 11 | `func` | heap | `FuncObj` | `"func"` |
| 12 | `class` | heap | `ClassObj` | `"class"` |
| 13 | `instance` | heap | `InstObj` | `"instance"` |
| 14 | `cell` | heap | `RefCell<Value>` | `"cell"` |
| 15 | `error` | heap | `ErrObj` | `"error"` |
| 16 | `foreign` | heap | `ForeignObj` | `"foreign"` |
| 17 | `channel` | heap | `ChanObj` | `"channel"` |
| 18 | `mutex` | heap | `MutexObj` | `"mutex"` |
| 19 | `handle` | heap | `HandleObj` | `"handle"` |

> Note: internally there are 16 machine tags over a NaN-boxed `u64`; small primitives (`nil`, `bool`, `int`, `float`) are stored inline in the payload bits. Larger values (strings, lists, dicts, etc.) are heap objects with manual reference counting (`retain`/`release`).

### 3.1.1 Type introspection

```
typeof(42)        # "int"
typeof(3.14)      # "float"
typeof("hello")   # "string"
typeof(true)      # "bool"
typeof(nil)       # "nil"
typeof([1, 2])    # "list"
typeof({"a": 1})  # "dict"
typeof(fn() {})   # "func"
```

`is` checks type membership:

```
42 is int         # true
"hello" is string # true
nil is nil        # true
42 is string      # false
```

`is` compares against a type *name* (a string literal or value), not against another value's type:

```
let x = 42
x is typeof(x)    # true — compares "int" is "int"
```

## 3.2 2.2 Literals

### 3.2.1 Nil and booleans

```
let nothing = nil
let yes = true
let no = false
```

`nil` is the absence of a value. It is falsy. `true`/`false` are the only boolean values.

### 3.2.2 Integers

Integer literals are 64-bit signed (`i64`). Decimal, hex (`0x`), octal (`0o`), and binary (`0b`) bases are supported. Underscores may be used as digit separators.

```
let dec = 42
let hex = 0x2A          # 42
let oct = 0o52          # 42
let bin = 0b101010      # 42
let big = 1_000_000_000 # 1000000000
let neg = -17
```

Integer range: `-9,223,372,036,854,775,808` to `9,223,372,036,854,775,807` (`i64::MIN` to `i64::MAX`). Overflow in checked contexts panics; in release builds overflow wraps (two's complement).

```
let max = 9223372036854775807   # i64::MAX
let min = -9223372036854775808  # i64::MIN
```

### 3.2.3 Floats

Float literals are 64-bit IEEE-754 (`f64`). A decimal point or exponent is required.

```
let pi = 3.14159
let big = 1.5e10      # 15000000000.0
```

```
let small = 2.5e-3    # 0.0025
let hex_f = 0x1.8p1   # 3.0 (hex float)
```

Special values:

```
let inf = 1e400       # overflow → infinity
let neg_inf = -1e400  # −infinity
let nan = 0.0 * inf   # not-a-number
```

NaN semantics: - `nan != nan` is `true` (IEEE-754) - `nan < anything` is `false`; `nan > anything` is `false` - `typeof(nan)` → `"float"` - Use `x != x` to test for NaN: `let is_nan = (x != x)`

```
let bad = 0.0 / 0.0   # NaN
print(bad == bad)     # false
print(bad != bad)     # true  ← NaN detection idiom
print(inf > 1e308)    # true
print(-inf < -1e308)  # true
```

### 3.2.4 Strings

String literals use double or single quotes. Escape sequences: `\n`, `\t`, `\\`, `\'`, `\"`, `\0`, `\xHH` (hex byte), `\u{HHHH}` (Unicode scalar).

```
let s1 = "hello"
let s2 = 'world'
let s3 = "it's a nice day"
let s4 = "say \"hello\""
let s5 = "line1\nline2\tline3"
let s6 = "byte: \x41"    # "byte: A"
let s7 = "emoji: \u{1F600}"  # "emoji: □"
```

Triple-quoted strings (multi-line, raw):

```
let poem = """Zen is fast,
native, and safe.
No escaping needed here."""
```

Strings are immutable (`Arc<str>`). Indexing `s[i]` returns the character at byte offset `i` as a one-character string; slicing `s[a:b]` returns a substring. Indexing out of bounds raises `IndexError`.

```
let s = "Zen"
print(s[0])   # "Z"
print(s[-1])  # "n"
print(s[0:2]) # "Ze"
```

### 3.2.5 Bytes

Byte literals use the `b""` prefix. Each element is a `u8` (0–255).

```
let data = b"\x48\x65\x6c\x6c\x6f"  # "Hello" as bytes
let empty = b""
```

Bytes support indexing (`data[0]` → `72` as int) and slicing. Bytes are *not* automatically convertible to strings; use the `string` module or explicit decoding.

### 3.2.6 Lists

List literals use square brackets. Heterogeneous contents are allowed.

```
let nums = [1, 2, 3]
let mixed = [1, "two", 3.0, true, nil]
let nested = [[1, 2], [3, 4]]
let empty = []
```

Lists are mutable, ordered, and allow duplicates. Indexing supports negative indices (from the end). Slicing `l[a:b]` returns a new list (copy, not a view).

```
let l = [10, 20, 30, 40, 50]
print(l[0])    # 10
print(l[-1])   # 50
print(l[1:4])  # [20, 30, 40]
l[2] = 99      # mutation
print(l)       # [10, 20, 99, 40, 50]
```

### 3.2.7 Dictionaries

Dict literals use `{key: value, ...}`. Keys must be strings. Values can be any type. Insertion order is preserved.

```
let person = {"name": "Zen", "version": 3, "active": true}
let nested = {"user": {"name": "Zen", "id": 1}, "tags": ["a", "b"]}
let empty = {}
```

Access `d["key"]`. Missing keys raise `KeyError`.

```
print(person["name"])   # Zen
person["version"] = 4   # update
person["lang"] = "Zen"  # add
print(person["missing"]) # KeyError
```

### 3.2.8 Sets

Set literals use `{value, value, ...}` with no colons. Elements must be hashable (strings, ints, floats, bools, tuples of hashables). Sets are unordered and deduplicate.

```
let s = {1, 2, 2, 3}
print(s)  # {1, 2, 3} — order not guaranteed, duplicates removed
print(2 in s)   # true
print(5 in s)   # false
```

Warning: `{}` alone is an empty *dict*, not an empty set. Use `set()` or `{1}` for a set.

### 3.2.9 Tuples

Tuple literals use `#(...)` with hash prefix. Immutable, ordered, heterogeneous.

```
let point = #(3, 4)
let pair = #("name", 42)
let single = #(5,)      # trailing comma required for 1-tuple
let empty = #()         # empty tuple
```

Access `t[0]`, `t[1]`. Tuples support destructuring in `match` patterns.

```
let p = #(10, 20)
print(p[0])   # 10
print(len(p)) # 2
```

## 3.3 2.3 Type coercion matrix

Zen does **not** implicitly coerce between most types. The `+` operator stringifies non-string operands when the other side is a string, but arithmetic and comparison require matching numeric types.

| From → To | int | float | string | bool |
|---|---|---|---|---|
| int | identity | `42 → 42.0` | `"42"` | `0 → false`, else true |
| float | `3.9 → 3` (trunc) | identity | `"3.9"` | `0.0 → false`, else true |
| string | `"42" → 42` (parse) | `"3.14" → 3.14` | identity | `"" → false`, else true |
| bool | `true → 1`, `false → 0` | `true → 1.0` | `"true"/"false"` | identity |

Explicit conversion functions:

```
int("42")       # 42
int(3.9)        # 3 (truncates toward zero)
float(42)       # 42.0
str(42)         # "42"
str(3.14)       # "3.14"
bool(0)         # false
bool(1)         # true
bool("")        # false
bool("hello")   # true
```

String-to-number parsing failures raise `ValueError`:

```
try {
    int("not a number")
} catch as e {
    print(e.msg)  # ValueError: invalid literal for int()
}
```

## 3.4 2.4 Truthiness

Zen truthiness: everything is truthy except `nil` and `false`.

| Value | Truthy? |
|---|---|
| `nil` | false |
| `false` | false |
| `0` | **true** (unlike Python) |
| `0.0` | **true** |
| `""` | **true** (unlike Python) |
| `[]` | **true** (unlike Python) |
| `{}` | **true** (unlike Python) |
| `42` | true |
| `"hello"` | true |
| `[1, 2]` | true |

Use `bool(x)` to coerce explicitly:

```
print(bool(0))    # true
print(bool(""))   # true
print(bool(nil))  # false
print(bool(false)) # false
```

## 3.5 2.5 Equality vs identity

`==` compares *value* (with numeric coercion: `1 == 1.0` is true). `is` compares *type membership* when the right side is a type name. There is no separate object-identity operator for heap objects beyond reference behavior; two equal lists are `==` but distinct objects.

```
print(1 == 1.0)   # true (numeric coercion)
print(1 == "1")   # false (type mismatch)
print([] == [])   # true (empty lists equal by value)
print({} == {})   # true (empty dicts equal by value)
```

Dict equality is order-insensitive:

```
print({"a": 1, "b": 2} == {"b": 2, "a": 1})  # true
```

## 3.6 2.6 Numeric edge cases

### 3.6.1 Infinity and NaN

```
let inf = 1e400          # +infinity
let neg_inf = -1e400     # −infinity
let nan = 0.0 * inf      # not-a-number

print(inf > 1e308)       # true
print(neg_inf < -1e308)  # true
print(nan == nan)        # false
print(nan != nan)        # true   ← NaN detection
print(inf + inf)         # inf
print(inf - inf)         # nan
print(0.0 * inf)         # nan
```

### 3.6.2 Integer overflow

```
let max = 9223372036854775807
print(max + 1)  # wraps to -9223372036854775808 (two's complement, release)
```

### 3.6.3 Division

```
print(7 / 2)    # 3.5 (float division)
print(7 // 2)   # 3 (floor division, integer result)
print(7 % 2)    # 1 (modulo, sign follows dividend)
print(-7 % 2)   # -1
print(-7 // 2)  # -4 (floor toward −inf)
```

Division by zero raises `DivideByZero`:

```
try {
    let x = 1 / 0
} catch as e {
    print(e.msg)  # DivideByZero: division by zero
}
```

## 3.7 2.7 Type conversion reference

```
# Number ↔ string
int("42")           # 42
int("3.9")          # 3 (truncates after float parse)
int(3.9)            # 3
float("3.14")       # 3.14
float(42)           # 42.0
str(42)             # "42"
```

```
str(3.14)           # "3.14"
str(true)           # "true"
str(nil)            # "nil"

# Boolean
bool(0)             # false
bool(1)             # true
bool(0.0)           # false
bool("")            # true
bool([])            # true
bool({})            # true
bool(nil)           # false
bool(false)         # false

# Collections
list("abc")         # ["a", "b", "c"]
tuple([1, 2])       # #(1, 2)
set([1, 1, 2])      # {1, 2}
dict([["a", 1]])    # {"a": 1}
```

## 3.8 2.8 Common pitfalls

- **`0` and `""` are truthy** in Zen (unlike Python). Use `x == 0` or `x == ""` for emptiness checks, not `if x`.
- **`nan != nan`**. Use `x != x` to detect NaN.
- **`{}` is an empty dict**, not a set. Use `{1}` or `set()` for sets.
- **Strings are immutable**. `s[0] = "X"` raises `TypeError`. Build new strings with concatenation.
- **Integer division `//` floors toward negative infinity**: `-7 // 2` is `-4`, not `-3`.
- **List slicing copies**: `l[1:3]` is a new list; mutating it doesn't affect the original.
- **Dict keys must be strings**: `d[42]` raises `TypeError`. Use `d["42"]`.

## 3.9 2.9 Quick reference

```
let x = 42
let f = 3.14
let s = "hello"
let b = true
let n = nil
let l = [1, 2, 3]
let d = {"a": 1}
let st = {1, 2}
let t = #(1, 2)

typeof(x)      # "int"
x is int       # true
```

```
x is string    # false
bool(x)        # true
str(x)         # "42"
int("42")      # 42
float(x)       # 42.0
len(l)         # 3
len(d)         # 1
len(s)         # 5
x == 42.0      # true
x == "42"      # false
```

## 3.10 2.10 Exercises

1. Write `is_nan(x)` that returns true only for NaN (hint: `x != x`).
2. Write `safe_int(s)` that parses a string to int, returning `nil` on failure instead of raising.
3. Demonstrate that `0` is truthy: print a message if `if 0 { ... }` executes.
4. Create a list, slice it, mutate the slice, and confirm the original is unchanged.
5. Show that `{"a": 1, "b": 2} == {"b": 2, "a": 1}` (dict order-insensitive equality).
6. Compute `-7 // 2` and `-7 % 2` and explain the results (floor division).
7. Write `type_name(x)` that returns a human-readable string for any value using `typeof()` and special-casing `nan`/`inf`.

## 3.11 2.11 Type coercion matrix (extended)

| From | To int | To float | To string | To bool | Notes |
|---|---|---|---|---|---|
| int | identity | widen | decimal | 0→false else true | |
| float | trunc toward zero? | identity | repr | 0.0/NaN? false if 0.0, else true (NaN is true) | check implementation |
| string | parse int (base 10) or error | parse float or error | identity | non-empty string is true | whitespace? |
| bool | true→1, false→0 | true→1.0,false→0.0 | “true”/“false” | identity | |
| nil | error | error | “nil” | false | |
| list | error | error | repr | empty false else true | |
| dict | error | error | repr | empty false else true | |
| set | error | error | repr | empty false else true | |

| From | To int | To float | To string | To bool | Notes |
|---|---|---|---|---|---|
| tuple | error | error | repr | empty false else true | |
| bytes | error | error | repr or utf8? | non-empty true | |

**Note:** In Zen, truthiness follows common rules: `false` and `nil` are falsy; everything else is truthy (including `0`, `0.0`, empty strings `""` are truthy in Python-like languages? Wait - check Zen behavior. In Python empty collections/strings/numbers-zero are falsy; in some languages 0 is falsy. Let us verify.)

## 3.12 2.12 Bytes and buffers (deep dive)

`bytes` is an immutable sequence of bytes (values 0-255).

```
let b = bytes([65, 66, 67])  # "ABC"
print(b[0])                  # 65
let s = b.decode("utf-8")    # "ABC"
let b2 = "ABC".encode("utf-8")
print(len(b))
```

### 3.12.1 bytes methods (common)

- `len(b)`, indexing/slicing
- `b.decode(encoding="utf-8", errors="strict")`
- `bytes([..])`, `b.hex()`, `b.fromhex(s)`

## 3.13 2.13 Tuples (immutable sequences)

```
let t = #(1, "a", true)
print(t[0], len(t))
# t[0] = 5  # error: immutable
```

Tuples are useful for fixed-size heterogeneous data and multiple return values.

## 3.14 2.14 Sets

```
let s = {1, 2, 2, 3}
print(s)      # {1,2,3} (order may vary)
print(2 in s) # true
s.add(4)
s.remove(2)
```

## 3.15 2.15 Numeric edge cases in depth

### 3.15.1 Integer precision

Integers are arbitrary precision (big ints) in Zen 3.0 - no fixed 64-bit overflow in normal operations.

```
let big = 10 ** 100
print(big + 1)
```

### 3.15.2 Float special values

```
let inf = 1.0 / 0.0      # inf
let ninf = -1.0 / 0.0    # -inf
let nan = 0.0 / 0.0      # nan
print(inf > 1000000)     # true
print(nan == nan)        # false (IEEE 754)
print(nan != nan)        # true
```

### 3.15.3 Division semantics

- `/` is true division returning float (or appropriate numeric type)
- `//` is floor division
- `%` follows floor division (sign of divisor matters in some languages; check)

## 3.16 2.16 Type checking patterns

```
func describe(x) {
    if typeof(x) == "int" { return "integer" }
    if typeof(x) == "float" { return "float" }
    if typeof(x) == "string" { return "string" }
    if typeof(x) == "bool" { return "boolean" }
    if x is nil { return "nil" }
    if typeof(x) == "list" { return "list" }
    if typeof(x) == "dict" { return "dict" }
    if typeof(x) == "set" { return "set" }
    if typeof(x) == "tuple" { return "tuple" }
    return "unknown"
}
```

`is` operator checks type identity/instance; `typeof()` returns type name string.

## 3.17 2.17 Identity vs equality

- `==` structural/value equality
- `is` identity (same object/reference)

```
let a = [1,2,3]
let b = [1,2,3]
print(a == b)  # true (value equal)
print(a is b)  # false (different objects)
let c = a
print(a is c)  # true
```

## 3.18 2.18 Common pitfalls

1. Modifying a list while iterating over it can skip elements
2. Using mutable default arguments (if supported) - better to use nil sentinel
3. Dict key type errors - only strings work as keys
4. Integer/string concatenation requires explicit conversion
5. Truthiness confusion with 0/“” depending on language - always test
6. Float comparison with == is dangerous; use epsilon

```
func approx_eq(a, b, eps) {
    if eps == nil { eps = 1e-9 }
    return abs(a-b) < eps
}
```

## 3.19 2.19 Deep exercises

1. Implement a type coercion helper that tries int, then float, then bool for strings.
2. Write a function to detect NaN using `x != x`.
3. Build a mini type registry using dict mapping type names to handlers.
4. Demonstrate integer big number arithmetic with powers.
5. Show the difference between `is` and == with lists, strings, ints.
6. Create a tuple and try to mutate - observe error.
7. Implement `flatten_once(lst)` for nested lists.
8. Write `deep_copy` conceptually (lists of lists) - note shallow vs deep.

# 4 Chapter 3: Operators and Expressions

Zen's operators cover arithmetic, comparison, logic, string handling, and safe member access. This chapter lists every operator, its precedence, associativity, and edge cases.

## 4.1 3.1 Operator precedence (high to low)

| Precedence | Operator(s) | Associativity | Description |
|---|---|---|---|
| 1 | `() [] . ?.` | left | Grouping, indexing, field access, safe access |
| 2 | `**` | right | Power |
| 3 | `-` (unary) `not !` | right | Negation, logical not |
| 4 | `* / // %` | left | Multiply, divide, floor-divide, modulo |
| 5 | `+ -` | left | Add, subtract, concatenate |
| 6 | `== != < <= > >=` | left | Comparison |
| 7 | `and` | left | Logical and (short-circuits) |
| 8 | `or` | left | Logical or (short-circuits) |
| 9 | `??` | left | Nullish coalescing |
| 10 | `if-else` (ternary) | right | Conditional expression |
| 11 | `= += -= …` | right | Assignment |

Use parentheses whenever precedence is unclear.

## 4.2 3.2 Arithmetic

| Operator | Meaning | Example | Result | Notes |
|---|---|---|---|---|
| `+` | add | `1 + 2` | `3` | also string concat |
| `-` | subtract | `5 - 3` | `2` | |
| `*` | multiply | `3 * 2` | `6` | also string repeat `"x"*3` |
| `/` | divide (float) | `7 / 2` | `3.5` | div by zero → `DivideByZero` |
| `//` | floor divide | `7 // 2` | `3` | floors toward −inf |

| Operator | Meaning | Example | Result | Notes |
|---|---|---|---|---|
| `%` | modulo | `7 % 3` | `1` | sign follows dividend |
| `**` | power | `2 ** 3` | `8` | right-associative |
| `-` | negate | `-x` | `-x` | unary |

### 4.2.1 Floor division and modulo edge cases

```
print(7 // 2)    # 3
print(-7 // 2)   # -4   (floor toward −inf, not truncation)
print(7 % 2)     # 1
print(-7 % 2)    # -1   (sign follows dividend)
print(7.5 // 2)  # 3.0  (floor works on floats too)
```

### 4.2.2 Division by zero

```
try {
    print(1 / 0)
} catch as e {
    print(e.msg)  # DivideByZero: division by zero
}
```

## 4.3 3.3 String concatenation and repetition

When either side of `+` is a string, the other side is stringified:

```
print("count: " + 42)       # "count: 42"
print("flag: " + true)      # "flag: true"
print(nil + "end")          # "nilend"
print("Zen " + "3.0")       # "Zen 3.0"
```

`42 + "count"` is a **type error** (number + string without string on the left). Use `str(42) + "count"`.

`*` repeats strings:

```
print("x" * 5)      # "xxxxx"
print("ab " * 3)    # "ab ab ab "
print(3 * "ab")     # also works: "abab"
```

## 4.4 3.4 Comparison

| Operator | Meaning | Notes |
|---|---|---|
| `==` | equal | numeric coercion: `1 == 1.0` → true |

| Operator | Meaning | Notes |
|---|---|---|
| `!=` | not equal | |
| `<` | less than | |
| `<=` | less than or equal | |
| `>` | greater than | |
| `>=` | greater than or equal | |

Numeric coercion in ==:

```
print(1 == 1.0)    # true
print(1 == "1")    # false (type mismatch, no coercion to string)
print([] == [])    # true (empty lists equal by value)
print({} == {})    # true (empty dicts equal, order-insensitive)
print({"a":1,"b":2} == {"b":2,"a":1})  # true
```

NaN: `nan == nan` is false; `nan != nan` is true.

## 4.5 3.5 Logic

Zen truthiness: everything is truthy except `nil` and `false`. Notably `0`, `0.0`, `""`, `[]`, `{}` are **truthy**.

| Operator | Meaning | Short-circuits? |
|---|---|---|
| `and` | logical and | yes — returns first falsy or last |
| `or` | logical or | yes — returns first truthy or last |
| `not` | logical not | prefix, returns bool |
| `!` | logical not | alias for `not` |

```
print(true and false)   # false
print(true or false)    # true
print(not true)         # false
print(!false)           # true

# Short-circuit: right side not evaluated if left decides
print(false and error())  # false — error() never called
print(true or error())    # true  — error() never called
```

`and`/`or` return the operand value, not necessarily a bool:

```
print("hello" and "world")   # "world"
print("" or "fallback")     # "fallback"  (note: "" is truthy in Zen!)
print(nil or "fallback")    # "fallback"
```

## 4.6 3.6 Safe member access: `?.` (optional chaining)

`?.` reads a field only if the object is non-nil; otherwise short-circuits to `nil`.

```
let user = nil
print(user?.name)           # nil  (no error)

let user = { name: "Zen" }
print(user?.name)           # "Zen"

let data = nil
print(data?.a?.b?.c)        # nil  (stops at first nil)
```

Compare with strict access, which raises on nil:

```
let user = nil
print(user.name)   # TypeError: cannot read field on nil
```

`?.` works on dicts, instances, and any object with fields. It does **not** work on lists for indexing — use `list[i]` directly or check bounds first.

## 4.7 3.7 Nullish coalescing: ??

`??` returns its left side if non-nil, otherwise the right side. It's a **nil-default**, not a falsy-default — `0`, `false`, and `""` are returned as-is.

```
print(nil ?? "default")     # "default"
print("set" ?? "default")   # "set"
print(0 ?? "default")       # 0      (0 is not nil)
print(false ?? "default")   # false
print(""  ?? "default")     # ""     (empty string is not nil)
print(nil ?? nil ?? "fb")   # "fb"   (chaining)
```

Common use: config fallback:

```
let timeout = config?.timeout ?? 30
```

## 4.8 3.8 Conditional expression (ternary)

```
let status = if score >= 60 { "pass" } else { "fail" }
```

The condition must be braced. The result is the value of the taken branch; both branches must be type-compatible (Zen is dynamically typed, so any values work, but mixing types may surprise downstream code).

## 4.9 3.9 Assignment and augmented assignment

```
let x = 10
x += 5      # x = 15
x -= 3      # x = 12
x *= 2      # x = 24
x /= 4      # x = 6.0  (float division)
```

```
x //= 2     # x = 3
x %= 2      # x = 1
```

Augmented assignment works on any lvalue (variable, field, index):

```
let d = {"count": 0}
d["count"] += 1   # d["count"] is now 1
```

## 4.10 3.10 `is` — type membership

`x is TypeName` checks whether `x`'s runtime type matches the named type:

```
print(42 is int)          # true
print(3.14 is float)      # true
print("hi" is string)     # true
print([] is list)         # true
print({} is dict)         # true
print(true is bool)       # true
print(nil is nil)         # true
print(42 is string)       # false
```

The right side must be a type keyword/literal, not a variable:

```
let t = "int"
print(42 is t)   # ParseError — type name required literally
```

Use `typeof(x) == t` instead for dynamic type-name comparison.

## 4.11 3.11 `in` / `!in` — membership

```
print(2 in [1, 2, 3])      # true
print(5 in [1, 2, 3])      # false
print(5 !in [1, 2, 3])     # true
print("a" in {"a": 1})     # true (dict key membership)
print("z" in "Zen")        # true (substring)
```

For strings, `in` checks substring membership. For lists/sets/dicts, it checks element/key membership.

## 4.12 3.12 Quick reference

```
# Arithmetic
1 + 2       # 3
7 / 2       # 3.5
7 // 2      # 3
7 % 3       # 1
2 ** 3      # 8
-5          # -5
```

```
# Strings
"a" + "b"   # "ab"
"x" * 3     # "xxx"
"count: " + 42   # "count: 42"

# Comparison
1 == 1.0    # true
"a" != "b"  # true

# Logic
true and false   # false
true or false    # true
not true         # false

# Safe access
nil?.name        # nil
{name: "Z"}?.name  # "Z"

# Coalescing
nil ?? "d"       # "d"
0 ?? "d"         # 0

# Ternary
if ok { "yes" } else { "no" }

# Type check
42 is int        # true

# Membership
2 in [1,2,3]     # true
"a" in "Zen"     # true
```

## 4.13 3.13 Exercises

1. Write `sign(x)` returning `1`, `-1`, or `0` using only comparison operators.
2. Demonstrate that `0` is truthy: `if 0 { print("zero is truthy") }` prints.
3. Write `safe_div(a, b)` returning `a // b` or `nil` if `b == 0`, using `try`/`catch`.
4. Show that `"0" ?? "default"` returns `"0"` (string “0” is not nil).
5. Prove `nan != nan` using `0.0 * inf`.
6. Write `in_range(x, lo, hi)` using chained comparisons (Zen does **not** chain `lo < x < hi` — use `lo < x and x < hi`).
7. Show the difference between `and` short-circuiting: `false and error()` does not evaluate `error()`.

## 4.14 3.12 Complete precedence table (canonical)

The parser uses the following levels, from lowest binding to highest. This matches `docs/03-syntax.md` exactly.

| Level | Operators | Associativity |
|---|---|---|
| 1 | `?:` ternary, `if expr else expr` | right |
| 2 | `??` nullish coalescing | left |
| 3 | `or`, `\|\|` | left |
| 4 | `and`, `&&` | left |
| 5 | `is`, `is not`, `===`, `!==`, `==`, `!=`, `<`, `>`, `<=`, `>=`, `in`, `not in` | left |
| 6 | `\|` bitwise or | left |
| 7 | `^` bitwise xor | left |
| 8 | `&` bitwise and | left |
| 9 | `<<`, `>>` shifts | left |
| 10 | `+`, `-` | left |
| 11 | `*`, `/`, `%` | left |
| 12 | unary `-`, `!`, `not`, `~`, `typeof` | right |
| 13 | `**` power | right |
| 14 | `.` `?.` `[]` call `()` | left (postfix) |
| 15 | literals, identifiers, grouping, containers, lambdas, `new` | — |

A common source of bugs is that `**` binds tighter than unary minus, so `-2 ** 2` parses as `-(2 ** 2) = -4`, not `(-2) ** 2 = 4`. Use parentheses when in doubt.

### 4.14.1 Worked examples

```
print(2 + 3 * 4)          # 14  (* before +)
print((2 + 3) * 4)        # 20
print(2 ** 3 ** 2)        # 512 (right associative: 3**2=9, 2**9=512)
print(-2 ** 2)            # -4
print((-2) ** 2)          # 4
print(1 < 2 and 3 < 4)    # true
print(1 < 2 ? "yes" : "no")  # yes
print(nil ?? "fallback")  # "fallback"
print(0 ?? "fallback")    # 0  (0 is not nullish)
print(false ?? true)      # false
```

## 4.15 3.13 Truthiness rules

Zen's `if`, `while`, `and`, `or`, `!`, and filter predicates all use a single "truthy" rule. The canonical rule in Zen 3.0 is: **`false`, `nil`, and `null` are falsy; every other value is**

**truthy.** In particular `0`, `0.0`, `""`, `[]`, `#{}`, and `%{}` are truthy. This differs from Python and surprises many beginners.

```
if 0 { print("0 is truthy") }        # prints
if "" { print("empty string truthy") } # prints
if [] { print("empty list truthy") }   # prints
if false { print("never") }            # does not print
if nil { print("never") }              # does not print
```

If you want Python-style emptiness checks, be explicit:

```
func is_empty(x) {
    if x == nil { return true }
    if typeof(x) == "string" || typeof(x) == "list" || typeof(x) == "dict" || typeof(x)
        return len(x) == 0
    }
    return false
}
```

## 4.16 3.14 Equality: ==, ===, is

Zen distinguishes three comparison operators that are easy to confuse.

| Operator | Name | Semantics |
|---|---|---|
| `==` | equality | structural/value equality with numeric promotion |
| `===` | strict equality | same type *and* same value |
| `is` | identity/plain equality | object identity for containers; value equality for scalars |
| `!=`, `!==`, `is not` | negations | — |

### 4.16.1 Numeric promotion in ==

`1 == 1.0` is `true`; `1 === 1.0` is `false` because the runtime types differ.

```
print(1 == 1.0)     # true
print(1 === 1.0)    # false
print("1" == 1)     # false (no implicit string→int)
print("1" === 1)    # false
print(true == 1)    # false (booleans are not integers)
```

### 4.16.2 Container equality

Lists, tuples, dicts, and sets compare structurally with ==. Dicts are order-insensitive; sets are order-insensitive; lists and tuples are order-sensitive.

```
print([1,2] == [1,2])                # true
print([1,2] == [2,1])                # false
```

```
print(#{a:1, b:2} == #{b:2, a:1})     # true
print(%{1,2} == %{2,1})               # true
print(#(1,2) == #(1,2))               # true
```

### 4.16.3 Identity with `is`

For heap values (lists, dicts, sets, instances, functions) `is` compares references. Two separately constructed equal lists are not `is`-identical.

```
let a = [1,2,3]
let b = [1,2,3]
let c = a
print(a == b)   # true
print(a is b)   # false
print(a is c)   # true
```

For scalars (int, float, bool, nil, string interning aside) `is` behaves like ==. Prefer == unless you specifically mean identity.

## 4.17 3.15 Short-circuit evaluation

`and`/`&&` and `or`/`||` short-circuit. The right operand is not evaluated when the result is already determined. This is the basis of guard idioms and also the reason side effects on the right may not occur.

```
func boom() { throw "boom" }

# safe: boom() never runs because the left side is false
if false and boom() { print("no") }

# safe: boom() never runs because the left side is true
if true or boom() { print("yes") }

# nil guarding
let name = nil
if name != nil and len(name) > 0 { print(name) }
```

`??` is also short-circuiting: it returns the left operand unless it is `nil`/`null`, in which case it evaluates and returns the right operand.

```
let port = config_port ?? 8080
```

`??=` assigns only when the target is currently nullish:

```
var count = nil
count ??= 0
count ??= 10
print(count)   # 0 — the second assignment is skipped
```

## 4.18 3.16 Chained comparisons

Comparisons chain Python-style: `a < b < c` means `(a < b) and (b < c)` and evaluates `b` only once.

```
let x = 5
print(1 < x < 10)     # true
print(1 < x > 3)      # true
print(10 < x < 20)    # false
```

## 4.19 3.17 Bitwise operators

Bitwise operators work on integers. Negative integers use two's complement semantics for shifts and masks.

| Operator | Meaning | Example | Result |
|---|---|---|---|
| & | and | `0b1100 & 0b1010` | `0b1000` = 8 |
| \| | or | `0b1100 \| 0b1010` | `0b1110` = 14 |
| ^ | xor | `0b1100 ^ 0b1010` | `0b0110` = 6 |
| ~ | not | `~0` | `-1` |
| << | left shift | `1 << 4` | 16 |
| >> | arithmetic right shift | `-8 >> 1` | `-4` |

### 4.19.1 Bit tricks

```
func is_even(n) { return (n & 1) == 0 }
func is_odd(n)  { return (n & 1) == 1 }

func set_bit(n, i)    { return n | (1 << i) }
func clear_bit(n, i)  { return n & ~(1 << i) }
func toggle_bit(n, i) { return n ^ (1 << i) }
func test_bit(n, i)   { return (n & (1 << i)) != 0 }

func low_mask(bits)   { return (1 << bits) - 1 }
func popcount(n) {
    let c = 0
    var x = n
    while x != 0 { c += (x & 1); x = x >> 1 }
    return c
}
```

### 4.19.2 Shifts and edge cases

- Shifting by a negative amount is an error.
- Large shift counts are defined by the runtime; do not rely on wraparound.
- `>>` is arithmetic for signed values (sign extension).

```
print(1 << 10)    # 1024
print(1024 >> 3)  # 128
print(-8 >> 1)    # -4
print(~5)         # -6
```

## 4.20 3.18 Range operators

Two range operators exist:

| Operator | Meaning | Example | Values |
|---|---|---|---|
| `..` | exclusive end | `0..5` | 0,1,2,3,4 |
| `->` | inclusive end | `0->5` | 0,1,2,3,4,5 |

`->` auto-descends when the start is greater than the end.

```
for i in 0..3 { print(i) }   # 0 1 2
for i in 0->3 { print(i) }   # 0 1 2 3
for i in 5->2 { print(i) }   # 5 4 3 2
```

Ranges used for slicing follow the same end-exclusivity as `..`; see the strings and lists chapters.

## 4.21 3.19 Compound assignment operators

All compound assignments evaluate the target once.

| Operator | Equivalent |
|---|---|
| `+=` | `x = x + y` |
| `-=` | `x = x - y` |
| `*=` | `x = x * y` |
| `/=` | `x = x / y` |
| `%=` | `x = x % y` |
| `??=` | `x = x ?? y` |
| `&=` | `x = x & y` |
| `\|=` | `x = x \| y` |
| `^=` | `x = x ^ y` |
| `<<=` | `x = x << y` |
| `>>=` | `x = x >> y` |

`++` and `--` increment/decrement by one, prefix or postfix.

```
var i = 0
print(i++)   # 0, then i becomes 1
print(++i)   # 2
print(i--)   # 2, then i becomes 1
print(--i)   # 0
```

## 4.22 3.20 Type coercion rules in operators

| Left | Op | Right | Result |
|---|---|---|---|
| int | `+ - * / % **` | int | int (except `/` → float when not exact? see note) |
| int | `+ - * / % **` | float | float |
| float | any arith | int/float | float |
| string | `+` | string | string |
| string | `+` | any | string (right stringified) |
| any | `+` | string | string (left stringified) |
| string | `*` | int | repeated string |
| list | `+` | list | concatenated list |
| list | `*` | int | repeated list |
| int | `& \| ^ << >> ~` | int | int |
| nil | `??` | any | any |
| any | `==` | any | bool |

**Note on `/`:** division always produces a float in Zen, even for exact integer division. `4 / 2` is `2.0`, while `4 // 2` is `2`.

```
print(4 / 2)    # 2.0
print(4 // 2)   # 2
print(4 / 2 == 2)  # true (numeric promotion)
print(typeof(4 / 2))  # "float"
```

## 4.23 3.21 Optional chaining and null safety

`?.` accesses a field or calls a method only if the receiver is not nil; otherwise the whole expression is nil.

```
let user = nil
print(user?.name)          # nil
print(user?.profile?.age)  # nil

let u = #{name: "Ada", profile: #{age: 36}}
print(u?.profile?.age)     # 36
print(u?.missing?.deep)    # nil
```

Optional chaining composes with `??` for defaulting:

```
let city = user?.address?.city ?? "Unknown"
```

## 4.24 3.22 Membership: `in` and `not in`

`in` tests membership for strings (substring), lists (element), sets (element), dicts (key), and tuples (element). Ranges support `in` by value.

```
print("ell" in "hello")     # true
print(3 in [1,2,3])         # true
print(4 not in [1,2,3])     # true
print("a" in #{a: 1})       # true
print(1 in %{1,2})          # true
print(2 in 0..5)            # true
print(9 in 0..5)            # false
```

## 4.25 3.23 Interaction of `and`/`or` with non-booleans

Unlike some languages, Zen's `and`/`or` return **booleans**, not the operands. If you want Python-style operand return values use the ternary operator or `??`.

```
print(1 and 2)     # true
print(nil or "x")  # true
# to get "x" when a is nil:
let a = nil
print(a ?? "x")    # "x"
```

## 4.26 3.24 Common operator mistakes

1. **Assuming `and` returns an operand.** It returns `bool`.
2. **Using `/` when integer division is intended.** Use `//`.
3. **Confusing `==` and `is`.** Containers compare by value with `==`.
4. **Forgetting that `0` and `""` are truthy.**
5. **Precedence of `??` vs `or`.** `??` binds tighter than `or`, looser than `and`.
6. **Chained assignment.** `a = b = 1` is not supported; assign separately.
7. **Modulo of negatives.** Check the sign convention for your use case.
8. **String repetition with huge counts.** Can exhaust memory; validate inputs.
9. **Relying on float equality.** Use an epsilon comparison.
10. **Bitwise ops on non-integers.** They raise a type error.

## 4.27 3.25 Operator overloading in classes

Classes may define dunder-style methods to control operators. The exact set supported depends on the runtime; the common ones are `__add__`, `__sub__`, `__mul__`, `__truediv__`, `__floordiv__`, `__mod__`, `__pow__`, `__eq__`, `__lt__`, `__le__`, `__gt__`, `__ge__`, `__str__`, `__len__`, `__getitem__`, `__setitem__`, `__contains__`, `__call__`.

```
class Vec2 {
    var x = 0.0
    var y = 0.0
    func init(x, y) { self.x = x; self.y = y }
    func __add__(other) { return new Vec2(self.x + other.x, self.y + other.y) }
```

```
    func __eq__(other) { return self.x == other.x and self.y == other.y }
    func __str__() { return "Vec2(${self.x}, ${self.y})" }
}

let a = new Vec2(1.0, 2.0)
let b = new Vec2(3.0, 4.0)
print(a + b)        # Vec2(4.0, 6.0)
print(a == b)       # false
```

Check the classes chapter for the authoritative list supported by the VM.

## 4.28 3.26 Exercises

1. Without running the code, compute `2 + 3 * 4 ** 2 // 5 % 3` and then verify.
2. Write a function that swaps two variables without a temporary using XOR.
3. Implement `sign(x)` returning -1, 0, or 1 using comparisons and ternary.
4. Use `??` to build a config lookup with nested defaults.
5. Explain why `if 0 { ... }` runs, and rewrite a Python-style check correctly.
6. Write `clamp(x, lo, hi)` using only comparisons and ternary.
7. Implement `is_power_of_two(n)` with a bit trick.
8. Show the difference between `==`, `===`, and `is` for `1`, `1.0`, and lists.
9. Use chained comparisons to validate a score in `[0, 100]`.
10. Use a range with `->` to print a countdown.
11. Implement a tiny bignum-less hash using `<<`, `^`, and `*` primes.
12. Write `safe_div(a, b)` returning `nil` on division by zero using `try`.

# 5 Chapter 4: Control Flow

Zen supports conditional execution, looping, pattern matching, and structured exception handling. This chapter covers `if`/`elif`/`else`, `while`, `for`, `break`/`continue`, `match`, `when`, and `try`/`catch`/`throw`/`finally`.

## 5.1 4.1 Conditionals: `if` / `elif` / `else`

```
let score = 87

if score >= 90 {
    print("A")
} elif score >= 80 {
    print("B")
} elif score >= 70 {
    print("C")
} else {
    print("Below C")
}
# prints: B
```

- `if` / `elif` / `else` chains are evaluated in order; the first true branch runs and the rest are skipped.
- The condition must be a truthy value. Zen truthiness: everything is truthy except `nil` and `false`.
- Braces `{ }` are required around bodies.

### 5.1.1 `if` as an expression

`if` can appear in expression position via a ternary form:

```
let status = if score >= 60 { "pass" } else { "fail" }
print(status)  # pass
```

## 5.2 4.2 Loops: `while` and `for`

### 5.2.1 `while`

```
let i = 0
let total = 0
while i < 100 {
    total = total + i
    i = i + 1
}
print(total)  # 4950
```

`while` loops continue while the condition is truthy.

### 5.2.2 `for`

`for` iterates over iterable values: lists, ranges, strings, dicts, and more.

```
# Iterate a list
for v in [1, 2, 3, 4] {
    print(v)
}

# Iterate a range
for i in 0..5 {
    print(i)  # 0, 1, 2, 3, 4
}

# Iterate a string
for ch in "Zen" {
    print(ch)
}

# Iterate dict items
let d = {"a": 1, "b": 2}
for k in d {
    print(k, d[k])
}
```

### 5.2.3 `break` and `continue`

```
# break: exit the loop early
for i in 0..10 {
    if i * i > 30 {
        break
    }
    print(i)
}
# prints: 0, 1, 2, 3, 4, 5

# continue: skip to the next iteration
let odd_sum = 0
for i in 1..10 {
    if i % 2 == 0 {
        continue
    }
    odd_sum = odd_sum + i
}
print(odd_sum)  # 25
```

### 5.2.4 Nested loops

```
let pairs = 0
for a in [1, 2, 3] {
    for b in [1, 2, 3] {
        if b > a {
            continue
        }
        pairs = pairs + 1
    }
}
print(pairs)  # 6
```

## 5.3 4.3 Pattern matching: `match`

`match` compares a subject against a series of patterns. The first matching arm determines the result.

```
let status = match "ready" {
    "loading": "spinner",
    "ready" => "check",
    _: "unknown"
}
print(status)  # check
```

### 5.3.1 Pattern forms

| Pattern | Matches |
|---|---|
| `"literal"` | Exact string equality |
| `42` | Exact numeric equality |
| `_` | Wildcard — matches anything |
| `n if cond` | Guard: binds `n`, then checks `cond` |
| `[a, b]` | List of exactly two elements |
| `[a, ...rest]` | List with head `a` and tail `rest` |
| `#(x, y)` | Tuple of exactly two elements |
| `pat as whole` | Binds `whole` to the entire matched value |
| `a \| b` | Either `a` or `b` |

### 5.3.2 Examples

Numeric patterns:

```
func label_of(n) {
    return match n {
        1: "one",
        2 => "two",
        _: "many"
```

```
    }
}
print(label_of(1))  # one
print(label_of(2))  # two
print(label_of(9))  # many
```

Guard patterns:

```
bound = match 42 {
    n if n > 100: "big",
    n if n == 42: "forty-two",
    _: "other"
}
print(bound)  # forty-two
```

Guard false falls through:

```
bound = match "err" {
    "err" if false: "guarded",
    "err": "plain",
    _: "other"
}
print(bound)  # plain
```

List destructuring:

```
func head_tail(xs) {
    return match xs {
        []: "empty",
        [only]: "one:" + str(only),
        [a, b]: "pair:" + str(a) + str(b),
        [a, ...rest]: "rest:" + str(a) + ":" + str(len(rest))
    }
}
print(head_tail([]))        # empty
print(head_tail([5]))       # one:5
print(head_tail([1, 2]))    # pair:12
print(head_tail([1, 2, 3])) # rest:1:2
```

Tuple destructuring:

```
func shape_of(x) {
    return match x {
        #(0, 0): "origin",
        #(x, y) if x == y: "on-diagonal",
        #(_, _): "some-pair"
    }
}
print(shape_of(#(0, 0)))    # origin
print(shape_of(#(4, 4)))    # on-diagonal
```

```
print(shape_of(#(4, 9)))    # some-pair
print(shape_of([1, 2]))     # some-pair (list matches tuple pattern)
print(shape_of(7))          # nil (non-iterable declines)
```

as pattern (bind whole):

```
func bind_whole(x) {
    return match x {
        [a, b] as whole: str(a) + str(b) + "/" + str(len(whole)),
        n as single: "single:" + str(single),
        _: "none"
    }
}
print(bind_whole([1, 2]))        # 12/2
print(bind_whole(7))             # single:7
print(bind_whole([1, 2, 3]))     # single:<list>
```

OR pattern (`a \| b`):

```
func classify(n) {
    return match n {
        1 \| 2: "small-prime",
        3 \| 5 \| 7: "other-prime",
        n if n % 2 == 0: "even",
        _: "odd"
    }
}
print(classify(1))  # small-prime
print(classify(4))  # even
print(classify(9))  # odd
```

Block bodies:

```
let printed = ""
let got = match 3 {
    1: { printed = "one" },
    2: { printed = "two" },
    3: { printed = "three" }
}
print(printed)  # three
print(got)      # nil (block bodies return nil unless they `return`)
```

Dict-literal bodies:

```
let usr = "bob"
let info = match usr {
    "bob": { name: "Bob", id: 7 },
    _: { name: "stranger", id: 0 }
}
print(info["name"])  # Bob
```

```
print(info["id"])    # 7
```

## 5.4 4.4 `when`: conditional expressions

`when` evaluates a series of condition arms and returns the value of the first true one. It's like `if`/`elif` but as an expression.

```
let score = 87
let grade = when {
    score >= 90: "A",
    score >= 85 => "A-",
    score >= 80: "B+",
    _: "C"
}
print(grade)  # A-
```

Wildcard fallback:

```
grade = when {
    score > 1000: "impossible",
    _: "fallback"
}
print(grade)  # fallback
```

Expression position:

```
let sz = when {
    score >= 85: "large",
    _: "small"
}
print(sz)  # large
```

## 5.5 4.5 Error handling: `try` / `catch` / `throw` / `finally`

Zen is "safe by default": runtime problems become catchable errors instead of crashes.

### 5.5.1 Bare `catch`

```
var caught = false
try {
    var bad = 1 / 0
    caught = bad == 0
} catch {
    caught = true
}
print(caught)  # true
```

A body that completes normally skips the catch:

```
var ok = true
try {
    var five = 10 / 2
    ok = five == 5
} catch {
    ok = false
}
print(ok)  # true
```

### 5.5.2 `catch as e` — bind the error

```
var em = ""
try {
    let boom = 1 / 0
} catch as e {
    em = str(e)
}
print(em)  # DivideByZero: division by zero
```

The caught error exposes `msg`, `code`, and `val`:

```
try {
    let y = 1 / 0
} catch as e {
    print(str(e))   # DivideByZero: division by zero
    print(e.msg)    # DivideByZero: division by zero
    print(e.code)   # 0
    print(e.val)    # nil
}
```

String + error concatenation works:

```
try {
    let y = 1 / 0
} catch as e {
    print("caught: " + e)  # caught: DivideByZero: division by zero
}
```

### 5.5.3 `throw`

`throw` raises your own errors. Strings become `UserError` messages:

```
var msg = ""
try {
    throw "custom failure"
} catch {
    msg = "handled"
}
```

```
print(msg)  # handled

var tm = ""
try {
    throw "custom boom"
} catch as e {
    tm = e.msg
}
print(tm)  # UserError: custom boom
```

`throw` accepts any value:

```
throw { code: 42, detail: "retry later" }
```

### 5.5.4 Typed catch clauses

You can add typed clauses to handle only certain errors. Unmatched clauses fall through:

```
try {
    risky()
} catch TypeError {
    print("bad type")
} catch UserError {
    print("user raised")
} catch {
    print("everything else")
}
```

A clause whose type does not match lets the error keep propagating outward.

### 5.5.5 `finally`

`finally` runs whether the `try` succeeded or failed, and whether or not `catch` handled the error. It must come **after** any `catch` clauses.

```
var log = ""
try {
    throw "boom"
} catch {
    log = log + "c"
} finally {
    log = log + "f"
}
print(log)  # cf

# finally also runs when nothing failed
var log2 = ""
try {
```

```
    log2 = "b"
} finally {
    log2 = log2 + "f"
}
print(log2)  # bf
```

### 5.5.6 Nested handlers

Inner handlers run before outer ones:

```
var order = ""
try {
    try { throw "inner" } catch { order = order + "i" }
    throw "outer"
} catch { order = order + "o" }
print(order)  # io
```

### 5.5.7 Errors from callees

Errors raised inside a function are catchable by the caller:

```
func deep() {
    return [1][5]
}
var hit9 = false
try {
    deep()
} catch IndexError {
    hit9 = true
}
print(hit9)  # true
```

### 5.5.8 Error types

Zen’s built-in error types:

- `TypeError` — wrong type for an operation
- `IndexError` — sequence index out of range
- `KeyError` — missing dict key
- `DivideByZero` — division by zero
- `ValueError` — inappropriate value
- `NativeError` — raised by a native module
- `ForeignAttribute` — failed foreign attribute access
- `PyImportError` — Python import failure
- `UserError` — raised by `throw`

## 5.6 4.6 Common pitfalls

- **`e` only exists inside its `catch` block.** Don't reference it outside.
- **`finally` goes after `catch`, not between `try` and `catch`.**
- **A bare `catch` swallows everything.** Use typed clauses when you want unexpected errors to propagate.
- **`str(e)` is the readable message.** Never rely on `typeof(e)` (it's just `"error"`) — use `e.msg` instead.

## 5.7 4.7 Quick reference

```
try {
    # risky code
} catch as e {
    print(str(e))   # real message
    print(e.msg)    # same as str(e)
    print(e.code)   # numeric code
    print(e.val)    # attached value
} catch TypeError {
    # only type errors
} finally {
    # always runs
}

throw "something went wrong"   # raise your own error

# match
let result = match value {
    pattern: expression,
    _: fallback
}

# when
let grade = when {
    cond1: "A",
    cond2: "B",
    _: "C"
}
```

## 5.8 4.8 Exercises

1. Write a `safe_divide(a, b)` function that returns `nil` on division by zero instead of crashing.
2. Use `match` to implement a simple router: map URL paths like `"/home"`, `"/about"`, `"/contact"` to page titles, with a 404 fallback.
3. Use `when` to implement letter-grade conversion from a numeric score.
4. Use `for` with `continue` to sum only the odd numbers in a list.

5. Write a function that uses `try`/`catch`/`finally` to guarantee a cleanup message prints regardless of success or failure.

## 5.9 4.12 `if` / `elif` / `else` — complete semantics

The condition is evaluated with the truthiness rule from Chapter 3: only `false`, `nil`, and `null` are falsy. `elif` is evaluated only if all previous conditions were falsy. `else` runs when no branch matched.

```
func classify(n) {
    if n < 0 {
        return "negative"
    } elif n == 0 {
        return "zero"
    } elif n < 10 {
        return "small"
    } else {
        return "large"
    }
}
```

Braces are mandatory; there is no significant indentation. Single-statement bodies may be written on one line with `;`.

```
if x > 0 { print("positive") }
if x > 0 { print("positive"); print("indeed") }
```

The ternary form is an expression, not a statement:

```
let label = x > 0 ? "pos" : "non-pos"
let label2 = "pos" if x > 0 else "non-pos"   # Python-style, also valid
```

`if` itself can also be used as an expression in many positions:

```
let msg = if x > 0 { "positive" } else { "non-positive" }
```

## 5.10 4.13 `while` loops

`while cond { ... }` re-evaluates `cond` before every iteration. `break` leaves the innermost loop; `continue` jumps to the next condition check.

```
var i = 0
while i < 5 {
    i += 1
    if i == 3 { continue }
    if i == 4 { break }
    print(i)     # 1, 2
}
```

### 5.10.1 Guard against infinite loops

Because `while true { ... }` is common for servers and REPLs, always ensure a reachable `break`, `return`, `throw`, or external stop condition.

```
var attempts = 0
while true {
    attempts += 1
    if attempts > 3 { break }
}
```

### 5.10.2 Loop with a post-condition

Zen has no `do/while`. Emulate it:

```
var n = 0
while true {
    n += 1
    if n >= 3 { break }
}
```

## 5.11 4.14 `for` loops

`for x in iterable { ... }` iterates over lists, tuples, sets, dict keys, strings (characters), ranges, and generators. The loop variable is a fresh binding per iteration for scalars.

```
for x in [10, 20, 30] { print(x) }
for ch in "abc" { print(ch) }
for k in #{a: 1, b: 2} { print(k) }
for i in 0..3 { print(i) }
```

### 5.11.1 Multi-variable iteration

Dict `.items()` yields `#(key, value)` tuples, but the loop can destructure them directly:

```
for k, v in #{a: 1, b: 2}.items() {
    print(k, v)
}
```

The same works for lists of pairs:

```
for a, b in [#(1, "x"), #(2, "y")] {
    print(a, b)
}
```

### 5.11.2 `enumerate`

`enumerate(iter)` returns a list of `[index, value]` pairs.

```
for i, v in enumerate(["a", "b"]) {
    print(i, v)   # 0 a / 1 b
}
```

### 5.11.3 Nested loops and labels

Zen has no loop labels. Use a flag or a helper function to break out of outer loops.

```
func find_pair(rows, target) {
    for i, row in enumerate(rows) {
        for j, v in enumerate(row) {
            if v == target { return #(i, j) }
        }
    }
    return nil
}
```

### 5.11.4 Mutating a collection while iterating

Do not add/remove elements of a list or dict while iterating it. Iterate over a copy instead:

```
for x in list(items) {        # copy
    if x < 0 { items.remove(x) }
}
```

For dicts, collect keys first:

```
for k in list(d.keys()) {
    if d[k] == nil { d.remove(k) }
}
```

## 5.12 4.15 `break`, `continue`, and `return` inside loops

- `break` exits the nearest enclosing loop.
- `continue` skips to the next iteration of the nearest loop.
- `return` exits the entire function immediately, even from nested loops.

```
func first_negative(xs) {
    for x in xs {
        if x < 0 { return x }
    }
    return nil
}
```

## 5.13 4.16 `match` expressions

`match` compares a value against patterns. Arms are separated by `=>`, `->`, or `:`; each arm yields an expression or a block.

```
func http_text(code) {
    return match code {
        200 => "OK",
        201 => "Created",
        301 | 302 | 307 | 308 => "Redirect",
        400 => "Bad Request",
        401 => "Unauthorized",
        403 => "Forbidden",
        404 => "Not Found",
        500 => "Internal Server Error",
        _ => "Unknown",
    }
}
```

### 5.13.1 Pattern kinds

| Pattern | Example | Matches |
|---|---|---|
| wildcard | `_` | anything |
| binding | `n` | anything, binds to `n` |
| literal int | `42` | exactly 42 |
| literal string | `"ok"` | exactly `"ok"` |
| literal bool/nil | `true`, `nil` | exactly that |
| list | `[a, b]` | two-element list, binds a,b |
| list with rest | `[head, ...tail]` | non-empty list |
| tuple | `#(x, y)` | two-tuple |
| dict | `#{type: t}` | dict containing key `type` |
| or-pattern | `1 \| 2 \| 3` | any of them |
| guard | `n if n > 0` | pattern plus condition |

### 5.13.2 List destructuring

```
func describe(xs) {
    return match xs {
        [] => "empty",
        [x] => "one: ${x}",
        [x, y] => "two: ${x}, ${y}",
        [head, ...rest] => "many starting with ${head}",
    }
}
```

### 5.13.3 Dict patterns

```
func handle_event(ev) {
    return match ev {
```

```
        #{type: "click", x, y} => "click at ${x},${y}",
        #{type: "key", key} => "key ${key}",
        #{type: t} => "unknown event ${t}",
        _ => "malformed",
    }
}
```

### 5.13.4 Guards

```
func sign(n) {
    return match n {
        x if x < 0 => -1,
        0 => 0,
        _ => 1,
    }
}
```

### 5.13.5 Exhaustiveness

The runtime does not require exhaustiveness, but an unmatched value raises an error. Always include _ unless you are certain the domain is closed.

## 5.14 4.17 `when` expressions

`when` is a cond-style chain: each arm has a boolean condition rather than a pattern.

```
let grade = when {
    score >= 90 => "A",
    score >= 80 => "B",
    score >= 70 => "C",
    score >= 60 => "D",
    _ => "F",
}
```

`when` is often clearer than a long `if/elif` chain when you want a value.

## 5.15 4.18 `try` / `catch` / `finally` as control flow

`try` is covered fully in Chapter 11, but control-flow interactions matter: `return` inside `try` still runs `finally`; `break`/`continue` inside `try` also run `finally`.

```
func f() {
    try {
        return 1
    } finally {
        print("cleanup")   # runs before returning
    }
}
```

## 5.16 4.19 Early-return style

Deeply nested `if`s are hard to read. Prefer guard clauses with early return.

```
# hard to read
func process(user) {
    if user != nil {
        if user.active {
            if user.email != nil {
                send(user.email)
            }
        }
    }
}

# easier
func process2(user) {
    if user == nil { return }
    if !user.active { return }
    if user.email == nil { return }
    send(user.email)
}
```

## 5.17 4.20 State machines with `match`

`match` on a state string is a clean way to write state machines.

```
class TrafficLight {
    var state = "red"

    func tick() {
        self.state = match self.state {
            "red" => "green",
            "green" => "yellow",
            "yellow" => "red",
            _ => "red",
        }
    }
}
```

For larger machines, define transition tables declaratively:

```
let transitions = #{
    red: #{green: "green"},
    green: #{yellow: "yellow"},
    yellow: #{red: "red"},
}
```

```
func next_state(state, event) {
    let row = transitions[state] ?? #{}
    return row[event] ?? state
}
```

## 5.18 4.21 Retry loops

A common pattern: retry an operation with backoff and a maximum attempt count.

```
func retry(fn, attempts, delay_ms) {
    var last_err = nil
    for i in 0..attempts {
        try {
            return fn()
        } catch as e {
            last_err = e
            sleep(delay_ms * (i + 1))
        }
    }
    throw last_err
}
```

## 5.19 4.22 Infinite event loops and shutdown

Servers and UIs run an event loop until a stop flag is set.

```
var running = true

func shutdown() { running = false }

while running {
    let event = poll_event()
    if event == nil { continue }
    handle(event)
}
```

For multi-threaded programs, use an atomic/`shared` flag or a channel close signal instead of a plain bool.

## 5.20 4.23 Loop performance notes

- Prefer `for x in list` over index loops when you do not need the index.
- Cache `len(xs)` in a local if the loop body cannot change it.
- Avoid building strings with `+=` in a hot loop; collect into a list and `join`.
- `break` early when searching.
- Avoid `try` inside tight loops when an error is expected frequently; test a condition first.

```
# slow
var s = ""
for w in words { s += w + " " }

# fast
let parts = []
for w in words { parts.push(w) }
let s2 = parts.join(" ")
```

## 5.21 4.24 Common control-flow mistakes

1. Using `=` instead of `==` in a condition (assignment is not an expression).
2. Forgetting braces.
3. Modifying a list while iterating it.
4. Assuming `0` is falsy.
5. Missing `_` in `match`, causing runtime errors on unhandled values.
6. Using `continue` inside a `finally` (avoid; confusing semantics).
7. Infinite loops without a reachable exit.
8. Off-by-one with `..` (exclusive) vs `->` (inclusive).
9. Deep recursion instead of iteration for large inputs.
10. Relying on dict iteration order across versions (use `.items()` if order matters).

## 5.22 4.25 Exercises

1. Write `fizzbuzz(n)` using `when` or `match`.
2. Implement a guessing game loop that exits on correct input or `quit`.
3. Write `sum_evens(xs)` using a `for` loop and `continue`.
4. Implement `binary_search(sorted, target)` with a `while` loop.
5. Rewrite a nested `if` pyramid using guard clauses.
6. Implement a simple calculator state machine with `match`.
7. Write `count_words(text)` using a dict and a loop.
8. Implement `flatten(nested)` with recursion and a loop.
9. Build a retry helper with exponential backoff.
10. Write `first_duplicate(xs)` returning the first repeated element or nil.
11. Implement `collatz_steps(n)` and guard against non-positive input.
12. Print a multiplication table using nested loops with aligned columns.

# 6 Chapter 5: Functions

Functions are first-class values in Zen. They can be declared with `fn`, assigned to variables, passed as arguments, returned from other functions, and stored in collections. This chapter covers declaration, calling, closures, default arguments, variadic behavior, and the `return` statement.

## 6.1 5.1 Declaration and calling

```
func add(a, b) {
    return a + b
}

print(add(2, 3))   # 5
```

- `func name(params) { body }` declares a function.
- The body is a block of statements; the last expression is **not** automatically returned — use `return`.
- Functions are values: `let f = add` then `f(2, 3)` works.

### 6.1.1 Calling with wrong argument count

```
try {
    add(1)            # missing argument
} catch as e {
    print(e.msg)      # ArgumentCount: expected 2, got 1
}

try {
    add(1, 2, 3)      # too many
} catch as e {
    print(e.msg)      # ArgumentCount: expected 2, got 3
}
```

Zen does **not** have keyword arguments. All calls are positional.

## 6.2 5.2 Return statements

`return` exits the function immediately with a value:

```
func abs(x) {
    if x < 0 {
        return -x
    }
    return x
}
```

```
print(abs(-5))  # 5
print(abs(3))   # 3
```

A function without `return` returns `nil`:

```
func nop() {
    print("side effect only")
}
print(nop())    # nil
```

`return` inside nested blocks exits the **enclosing function**, not just the block:

```
func find(items, target) {
    for i in 0..len(items) {
        if items[i] == target {
            return i          # exits find(), not just the if
        }
    }
    return -1
}
```

## 6.3 5.3 Default arguments

Zen supports default values for trailing parameters:

```
func greet(name, greeting = "Hello") {
    return greeting + ", " + name + "!"
}

print(greet("Zen"))            # "Hello, Zen!"
print(greet("Zen", "Hi"))      # "Hi, Zen!"
```

Rules: - Defaults must be on trailing parameters (no required parameter after a defaulted one). - Default expressions are evaluated at call time, not definition time: `zen  let counter = 0  func next(n = counter + 1) {  counter = counter + 1  return n  }  print(next())  # 1  print(next())  # 2` (counter incremented between calls)

## 6.4 5.4 Variadic behavior

Zen does **not** have `*args` or `**kwargs` syntax. All parameters are fixed-position. However, you can pass a list and unpack it manually:

```
func sum_all(nums) {
    let total = 0
    for n in nums {
        total = total + n
    }
    return total
}
```

```
let vals = [1, 2, 3, 4]
print(sum_all(vals))   # 10
```

If you need to forward arguments to another function, pass them explicitly or as a list.

## 6.5 5.5 Functions as first-class values

```
let f = add
print(f(10, 20))   # 30

let ops = [add, sub, mul]
print(ops[0](5, 3))  # 8
```

You can store functions in lists, dicts, and pass them to other functions:

```
func apply(f, x, y) {
    return f(x, y)
}

print(apply(add, 3, 4))   # 7
print(apply(sub, 3, 4))   # -1
```

## 6.6 5.6 Closures

Functions capture variables from their enclosing scope. Captured variables are shared (boxed) cells, so mutations are visible across calls:

```
func make_counter(start) {
    var count = start
    func inc() {
        count = count + 1
        return count
    }
    return inc
}

let c1 = make_counter(0)
let c2 = make_counter(100)

print(c1())   # 1
print(c1())   # 2
print(c2())   # 101
print(c1())   # 3   (c1 and c2 have independent count cells)
```

Each call to `make_counter` creates a **new** `count` cell. `c1` and `c2` capture different cells.

Closure over loop variables — be careful:

```
let funcs = []
for i in 1..3 {
    funcs.push(fn() { return i })
}
print(funcs[0]())   # 3 (all capture the same final i)
print(funcs[1]())   # 3
print(funcs[2]())   # 3
```

To capture per-iteration values, use a helper:

```
func make_fn(val) {
    return fn() { return val }
}

let funcs = []
for i in 1..3 {
    funcs.push(make_fn(i))
}
print(funcs[0]())   # 1
print(funcs[1]())   # 2
print(funcs[2]())   # 3
```

## 6.7 5.7 Higher-order functions

```
func twice(f, x) {
    return f(f(x))
}

func add_one(x) {
    return x + 1
}

print(twice(add_one, 5))   # 7

func map_list(f, items) {
    let result = []
    for x in items {
        result.push(f(x))
    }
    return result
}

print(map_list(add_one, [1, 2, 3]))   # [2, 3, 4]
```

## 6.8 5.8 Anonymous functions (lambdas)

Zen supports `lambda(params) { body }`:

```
let double = lambda(x) { return x * 2 }
print(double(21))   # 42
```

Lambdas are functions too — first-class, closures, passable. The difference from `fn` is syntactic: `lambda` is an expression, `fn` is a statement.

```
# lambda as expression (inline)
let result = (lambda(x, y) { return x + y })(3, 4)
print(result)   # 7
```

## 6.9 5.9 `return` in closures

`return` inside a lambda returns from the lambda, not the enclosing function:

```
func outer() {
    let f = lambda(x) {
        if x < 0 { return -1 }   # returns from the lambda
        return x * 2
    }
    return f
}

let f = outer()
print(f(-5))   # -1
print(f(5))    # 10
```

## 6.10 5.10 Common pitfalls

- **No keyword arguments.** All calls are positional. `add(a=1, b=2)` is a parse error.
- **Default args are evaluated at call time.** Mutable default expressions can surprise.
- **`return` exits the enclosing function**, not just the current block.
- **Closures capture by reference (shared cell)**, not by value. Mutating a captured var affects all closures over it.
- **Loop variable capture**: all closures in a loop share the loop var unless you use a helper function to bind per-iteration values.
- **Functions without `return` return `nil`.**

## 6.11 5.11 Quick reference

```
func name(a, b = default) { ... return value }
let f = name
f(1, 2)

lambda(x) { return x * 2 }
lambda(x, y) { return x + y }(3, 4)   # inline call
```

```
func make_counter(start) {
    var count = start
    return fn() { count = count + 1; return count }
}

func apply(f, x, y) { return f(x, y) }

return               # returns nil
return value         # returns value
```

## 6.12 5.12 Exercises

1. Write `compose(f, g)` that returns a function `h(x) = f(g(x))`.
2. Write `repeater(n)` that returns a function which, when called, returns the call count (1, 2, 3, ...).
3. Write `once(f)` that returns a wrapper calling `f` only on the first invocation and caching the result.
4. Write `curry2(f)` that takes a two-arg function and returns a function taking one arg that returns a function taking the second.
5. Write `memoize(f)` that caches results by argument (assume a single hashable arg).
6. Fix the loop-variable capture bug: create 3 functions returning 1, 2, 3 respectively using a helper.
7. Write `pipeline(funcs, x)` that applies each function in `funcs` to `x` in sequence.

## 6.13 5.11 Function declaration forms

Zen accepts several spellings for the same construct. They are aliases and behave identically:

```
func add(a, b) { return a + b }
def add2(a, b) { return a + b }
function add3(a, b) { return a + b }
```

Anonymous functions use `fn`:

```
let add = fn(a, b) { return a + b }
let add2 = fn(a, b) => a + b
let add3 = (a, b) => a + b
let legacy = lambda a, b : a + b
```

`fn(params) => expr` and `(params) => expr` are expression-bodied and return the expression without an explicit `return`. The parentheses around the parameter list are required for the `(a, b) => ...` form; `fn a => ...` is not valid.

## 6.14 5.12 Parameters: defaults, variadic, and destructuring

### 6.14.1 Default values

Defaults are evaluated at call time, in the scope of the function, left to right. A default may refer to an earlier parameter.

```
func greet(name, greeting = "Hello", punct = "!") {
    return "${greeting}, ${name}${punct}"
}

print(greet("Ada"))                 # Hello, Ada!
print(greet("Ada", "Hi"))           # Hi, Ada!
print(greet("Ada", "Hi", "."))      # Hi, Ada.

func box(w, h = w) { return w * h }
print(box(4))      # 16
print(box(4, 5))   # 20
```

Do not use a mutable container as a default when you expect a fresh one each call; use `nil` and create inside:

```
func append_item(x, acc = nil) {
    if acc == nil { acc = [] }
    acc.push(x)
    return acc
}
```

### 6.14.2 Variadic parameters

A trailing `...` parameter collects extra positional arguments into a list. Check the VM/parser for the exact syntax supported in your build; the common form is:

```
func sum_all(...nums) {
    var total = 0
    for n in nums { total += n }
    return total
}

print(sum_all(1, 2, 3, 4))   # 10
print(sum_all())             # 0
```

### 6.14.3 Destructuring parameters

List and tuple patterns can be destructured directly in the parameter list in newer builds:

```
func dist(#(x1, y1), #(x2, y2)) {
    let dx = x2 - x1
    let dy = y2 - y1
```

```
    return (dx*dx + dy*dy) ** 0.5
}
```

If your build does not support this, destructure inside the body:

```
func dist2(a, b) {
    let x1 = a[0]; let y1 = a[1]
    let x2 = b[0]; let y2 = b[1]
    let dx = x2 - x1
    let dy = y2 - y1
    return (dx*dx + dy*dy) ** 0.5
}
```

## 6.15 5.13 Return values

A function without an explicit `return` returns `nil`. `return` with no value also returns `nil`.

```
func log(msg) { print(msg) }          # returns nil
func nothing() { return }             # returns nil
```

Multiple return values are expressed with a tuple:

```
func divmod(a, b) {
    return #(a // b, a % b)
}

let q, r = divmod(17, 5)
print(q, r)   # 3 2
```

If tuple destructuring in assignment is unavailable, index the tuple:

```
let qr = divmod(17, 5)
print(qr[0], qr[1])
```

## 6.16 5.14 First-class functions and higher-order functions

Functions are values. They can be stored in variables, passed as arguments, returned, and placed in containers.

```
func apply(f, x) { return f(x) }
print(apply(fn(x) => x * 2, 21))   # 42

let ops = #{
    add: fn(a, b) => a + b,
    sub: fn(a, b) => a - b,
    mul: fn(a, b) => a * b,
}
print(ops["add"](2, 3))   # 5
```

### 6.16.1 Building pipelines

```
func pipe(value, fns) {
    var acc = value
    for f in fns { acc = f(acc) }
    return acc
}

let result = pipe(10, [
    fn(x) => x + 1,
    fn(x) => x * 2,
    fn(x) => "result=${x}",
])
print(result)   # result=22
```

## 6.17 5.15 Recursion

```
func fact(n) {
    if n <= 1 { return 1 }
    return n * fact(n - 1)
}
print(fact(10))   # 3628800
```

### 6.17.1 Tail recursion

The VM may or may not optimize tail calls. For large depths, rewrite as a loop.

```
func fact_iter(n) {
    var acc = 1
    var i = n
    while i > 1 {
        acc *= i
        i -= 1
    }
    return acc
}
```

### 6.17.2 Mutual recursion

```
func is_even(n) {
    if n == 0 { return true }
    return is_odd(n - 1)
}
func is_odd(n) {
    if n == 0 { return false }
    return is_even(n - 1)
```

```
}
```

Forward references are fine because resolution happens at call time.

### 6.17.3 Recursion depth

Deep recursion (tens of thousands of frames) can overflow the native stack. Add an explicit depth parameter or convert to iteration.

```
func walk(node, depth = 0) {
    if depth > 1000 { throw "too deep" }
    for child in node.children {
        walk(child, depth + 1)
    }
}
```

## 6.18 5.16 Closures

A closure captures variables from its defining scope. In Zen, captures are by reference to the binding (like Python nonlocal semantics); mutating a captured `var` from the closure affects the outer binding when the language permits it.

```
func make_counter() {
    var n = 0
    return fn() {
        n += 1
        return n
    }
}

let c = make_counter()
print(c())   # 1
print(c())   # 2
print(c())   # 3

let d = make_counter()
print(d())   # 1 — independent state
```

### 6.18.1 The late-binding loop variable trap

```
let fns = []
for i in 0..3 {
    fns.push(fn() => i)
}
for f in fns { print(f()) }
```

If all closures print the same value, the loop variable is shared. Fix by capturing a copy through a factory:

```
func make_constant(v) { return fn() => v }
let fns2 = []
for i in 0..3 { fns2.push(make_constant(i)) }
for f in fns2 { print(f()) }   # 0 1 2
```

## 6.19 5.17 `self` in methods

Inside a method, the receiver is bound to `self` (`this` is an accepted alias). Access fields with `self.field`; call sibling methods with `self.method()`.

```
class Counter {
    var n = 0
    func inc() { self.n += 1 }
    func inc_by(k) { self.n += k }
    func value() { return self.n }
}
```

## 6.20 5.18 Function metadata and introspection

Functions expose a type of `"func"` via `typeof`/`type`. Some builds expose `name`, `arity`, and parameter info; treat this as optional and avoid relying on it for portability.

```
let f = fn(x) => x
print(typeof(f))   # "func"
```

## 6.21 5.19 Overloading and dispatch

Zen does not support traditional static overloading by parameter types. Two functions with the same name replace one another in the same scope. Use runtime type checks or distinct names.

```
func area(shape) {
    if typeof(shape) == "dict" {
        if shape["kind"] == "circle" { return 3.14159 * shape["r"] ** 2 }
        if shape["kind"] == "rect" { return shape["w"] * shape["h"] }
    }
    throw "unsupported shape"
}
```

## 6.22 5.20 Default arguments and `nil` sentinels

Because defaults are evaluated per call, `nil` is a safe sentinel. Distinguish “not provided” from “explicitly nil” with a sentinel object if needed.

```
let MISSING = %{__missing__}

func get(d, key, default = MISSING) {
    if key in d { return d[key] }
```

```
    if default is MISSING { throw "missing key: ${key}" }
    return default
}
```

## 6.23 5.21 Pure functions and side effects

A pure function depends only on its arguments and returns a value without observable side effects. Pure functions are easier to test, cache, and parallelize.

```
# pure
func add(a, b) { return a + b }

# impure: reads global state and prints
var total = 0
func add_to_total(x) {
    total += x
    print("total=${total}")
}
```

Aim for a pure core and a thin impure shell around I/O.

## 6.24 5.22 Memoization

```
func memoize(f) {
    let cache = #{}
    return fn(n) {
        let key = str(n)
        if key in cache { return cache[key] }
        let v = f(n)
        cache[key] = v
        return v
    }
}

let fib = memoize(fn(n) {
    if n < 2 { return n }
    return fib(n - 1) + fib(n - 2)
})

print(fib(50))   # 12586269025
```

Note the self-reference: `fib` must be bound before the inner call runs, which it is by the time `fib(50)` executes.

## 6.25 5.23 Function composition

```
func compose(f, g) { return fn(x) => f(g(x)) }
func compose_all(fns) {
    return fn(x) {
        var acc = x
        for i in (len(fns) - 1)..-1 {
            acc = fns[i](acc)
        }
        return acc
    }
}
```

## 6.26 5.24 Currying and partial application

```
func curry(f) {
    return fn(a) => fn(b) => f(a, b)
}

let add = fn(a, b) => a + b
let add5 = curry(add)(5)
print(add5(10))   # 15
```

## 6.27 5.25 Error handling in functions

Decide on a contract: throw on invalid input, or return `nil`/a result type. Be consistent.

```
# throwing contract
func sqrt_strict(x) {
    if x < 0 { throw "sqrt of negative" }
    return x ** 0.5
}

# result contract
func sqrt_safe(x) {
    if x < 0 { return nil }
    return x ** 0.5
}
```

## 6.28 5.26 Documentation comments

Document parameters, return values, and failure modes. A consistent style:

```
# Computes the arithmetic mean of a non-empty list.
# Throws ValueError when xs is empty.
func mean(xs) {
    if len(xs) == 0 { throw "mean of empty list" }
```

```
    return sum(xs) / len(xs)
}
```

## 6.29 5.27 Performance considerations

- Avoid allocating closures in hot loops when a plain function will do.
- Prefer iterative solutions for large n.
- Pass big structures by reference (they are reference-counted handles).
- Avoid repeated global lookups; bind to a local for hot paths.
- Keep default argument expressions cheap; they run on every call.
- Do not use exceptions for ordinary control flow in hot loops.

```
# bind frequently used functions locally in tight loops
let push = fn(lst, v) { lst.push(v) }
```

## 6.30 5.28 Common function mistakes

1. Forgetting `return`, resulting in `nil`.
2. Mutating a default list/dict argument (use `nil` sentinel).
3. Assuming closures capture by value.
4. Deep recursion causing a stack overflow.
5. Reassigning a `let` binding inside a function.
6. Calling a method without `self.` inside the class.
7. Confusing a function value with its result (missing `()`).
8. Relying on argument names at call sites (Zen uses positional args).
9. Shadowing a builtin unintentionally.
10. Defining two functions with the same name in one scope.

## 6.31 5.29 Exercises

1. Write `is_prime(n)` and test it on a range.
2. Implement `gcd` and `lcm` recursively and iteratively; compare.
3. Write `map_list(xs, f)` without using `.map`.
4. Write `filter_list(xs, pred)` without using `.filter`.
5. Implement `reduce_list(xs, f, init)`.
6. Build `compose(f, g)` and verify with numeric functions.
7. Write `memoize(f)` and apply it to a slow function.
8. Implement `partial(f, a)` returning a unary function.
9. Write `make_adder(n)` and create several adders.
10. Implement `compose_all([...])` and test order.
11. Write a recursive `flatten` and an iterative one.
12. Implement `safe_div` with a result tuple and with exceptions; compare.
13. Write `debounce(f, ms)` conceptually using timestamps.
14. Implement `once(f)` that only runs `f` the first time.
15. Write `retry(f, n)` and test with a flaky function.

# 7 Chapter 6: Lambdas and Closures

This chapter is a deep dive into anonymous functions and closure semantics: how captures work, what is shared vs copied, the `lambda` expression vs `fn` statement, and how closures interact with the JIT and reference counting.

## 7.1 6.1 Lambda expressions

A lambda is an anonymous function defined as an expression:

```
let double = lambda(x) { return x * 2 }
print(double(21))   # 42
```

Syntax: `lambda(params) { body }`. The body is a block; use `return` to produce a value (omitting `return` yields `nil`).

### 7.1.1 Lambda as expression vs fn as statement

| | `lambda` | `fn` |
|---|---|---|
| Kind | Expression | Statement |
| Can appear inline | Yes (`(lambda(x){...})(1)`) | No |
| Assignable | Yes | Yes (`let f = fn() {...}`) |
| First-class | Yes | Yes |
| Closure capture | Yes | Yes |
| `return` target | The lambda itself | The enclosing function |

Because `lambda` is an expression, you can call it immediately:

```
let result = (lambda(x, y) { return x + y })(3, 4)
print(result)   # 7
```

And pass it directly:

```
print((lambda(x) { return x * 2 })(5))   # 10
```

## 7.2 6.2 Closure capture semantics

Zen functions (both `fn` and `lambda`) capture variables from their enclosing scope by **shared reference** (a boxed `cell`). Mutating a captured variable is visible to every closure that captured it, and persists across calls.

### 7.2.1 Basic capture

```
func make_counter(start) {
    var count = start
    func inc() {
        count = count + 1
```

```
        return count
    }
    return inc
}

let c1 = make_counter(0)
let c2 = make_counter(100)

print(c1())   # 1
print(c1())   # 2
print(c2())   # 101
print(c1())   # 3   (c1 and c2 are independent)
```

Each call to `make_counter` allocates a fresh `count` cell. `c1` and `c2` point to different cells, so they count independently.

### 7.2.2 Multiple closures share the same cell

```
func make_pair() {
    var value = 0
    func get() { return value }
    func set(v) { value = v }
    return #(get, set)
}

let #(get, set) = make_pair()
print(get())     # 0
set(42)
print(get())     # 42
```

`get` and `set` share the same `value` cell — mutation through one is visible through the other.

## 7.3 6.3 The loop-variable capture trap

This is the most common closure bug. A loop variable is a single cell; every closure created in the loop captures **that same cell**, so they all see the final value:

```
let funcs = []
for i in 1..3 {
    funcs.push(fn() { return i })
}
print(funcs[0]())   # 3   ← not 1!
print(funcs[1]())   # 3
print(funcs[2]())   # 3
```

All three closures return `3` because `i` ended at `3` (the loop exits when `i == 3`).

### 7.3.1 Fix: bind per-iteration with a helper function

```
func make_fn(val) {
    return fn() { return val }
}

let funcs = []
for i in 1..3 {
    funcs.push(make_fn(i))   # i is copied into `val` at each call
}
print(funcs[0]())   # 1
print(funcs[1]())   # 2
print(funcs[2]())   # 3
```

`make_fn(i)` copies the current `i` into a fresh `val` parameter cell, so each closure captures its own `val`.

## 7.4 6.4 `return` target in closures

`return` inside a lambda returns from the lambda, **not** from the enclosing function:

```
func outer() {
    let f = lambda(x) {
        if x < 0 { return -1 }   # returns from lambda
        return x * 2
    }
    return f
}

let f = outer()
print(f(-5))   # -1
print(f(5))    # 10
print(outer()(-3))  # -1
```

If you need the enclosing function to return from inside a closure, use an error or a sentinel value instead.

## 7.5 6.5 Closures and reference counting

A closure retains every variable it captures. As long as the closure is alive, its captured variables stay alive:

```
func make_big() {
    let big = [0 for _ in 1..1000000]   # large allocation
    return fn() { return len(big) }     # keeps `big` alive
}

let f = make_big()
print(f())   # 1000000   (big still in memory)
```

When `f` is dropped (goes out of scope or reassigned), the captured cell is released, and `big` can be collected.

```
f = nil
# now `big` can be freed
```

## 7.6 6.6 Closures in collections

Functions are first-class, so they can live in lists and dicts:

```
let ops = {
    "add": lambda(a, b) { return a + b },
    "sub": lambda(a, b) { return a - b },
}

print(ops["add"](5, 3))   # 8
print(ops["sub"](5, 3))   # 2
```

## 7.7 6.7 Closures and threads

Closures can be passed to `thread.spawn`, but captured shared variables become concurrently accessible — use `shared` for atomic counters:

```
import shared
import thread

let counter = shared.create(0)

for i in 1..3 {
    thread.spawn(fn() {
        shared.increment(counter, 1)
    })
}
# ... join threads ...
print(shared.get(counter))   # 3
```

Without `shared`, each thread would capture its own copy and updates would be lost.

## 7.8 6.8 JIT lowering note

Zen's Cranelift JIT lowers hot closures to native code. Closures that are called repeatedly in a loop get tiered up; the capture cells are accessed through a stable context pointer. You generally don't need to think about this, but it means:

- A closure called once stays interpreted.
- A closure called millions of times becomes native code with equivalent capture semantics.
- `ZEN_NO_JIT=1` disables tiering (all interpreted, same semantics).

## 7.9 6.9 Quick reference

```
# Lambda expression
let f = lambda(x) { return x * 2 }
print(f(5))           # 10

# Inline call
print((lambda(x) { return x + 1 })(41))   # 42

# Closure with shared cell
func make_counter(start) {
    var count = start
    return fn() { count = count + 1; return count }
}

# Fix loop capture
let funcs = []
for i in 1..3 {
    funcs.push((lambda(v) { return fn() { return v } })(i))
}
print(funcs[0]())   # 1

# return from lambda, not enclosing func
let f = fn(x) { if x < 0 { return -1 } return x * 2 }
```

## 7.10 6.10 Exercises

1. Write `make_adder(n)` returning a lambda that adds `n` to its argument.
2. Write `make_accumulator()` returning `fn(x)` that adds `x` to a running total and returns the total.
3. Write `repeater(n)` returning `fn()` that returns `n` on every call, but after `k` calls returns `nil` (capture a counter).
4. Write `compose(f, g)` returning `fn(x) { return f(g(x)) }`.
5. Demonstrate the loop-capture bug and fix it with `make_fn`.
6. Write `twice(f)` returning `fn(x) { return f(f(x)) }`.
7. Write `curry(f)` for a two-argument function: `curry(f)(a)(b)` equals `f(a, b)`.

## 7.11 6.12 Lambda and anonymous-function syntax — complete reference

Zen offers several spellings for anonymous functions. All are values and can be stored, passed, returned, and called immediately.

| Form | Example | Notes |
|---|---|---|
| `fn(params) { ... }` | `fn(x) { return x * 2 }` | block body, explicit return |
| `fn(params) => expr` | `fn(x) => x * 2` | expression body, implicit return |

| Form | Example | Notes |
|---|---|---|
| `(params) => expr` | `(x, y) => x + y` | parens required around params |
| `lambda params : expr` | `lambda x : x * 2` | legacy alias |

### 7.11.1 Expression-bodied vs block-bodied

```
# block body: explicit return, multiple statements allowed
let a = fn(x) { let y = x + 1; return y * 2 }

# expression body: implicit return
let b = fn(x) => (x + 1) * 2
let c = (x) => x * x
```

### 7.11.2 Arrow operator vs range operator

`->` in signature/range position is also the inclusive range operator; in the context `(x) => ...` it is parsed as the arrow, not a range. When you write ranges, `0 -> 5` means inclusive values 0..5; `0 .. 5` means exclusive 0..4.

## 7.12 6.13 First-class functions as values

Functions can be stored in variables, lists, dicts, returned from other functions, and passed as arguments.

```
let ops = #{
    add: fn(a, b) => a + b,
    sub: fn(a, b) => a - b,
    mul: fn(a, b) => a * b,
}
let pick = ops["add"]
print(pick(2, 3))   # 5

func apply(op, a, b) { return op(a, b) }
print(apply(ops["mul"], 4, 5))   # 20
```

A function value can itself return a function:

```
func multiplier(n) {
    return fn(x) => n * x
}

let double = multiplier(2)
let triple = multiplier(3)
print(double(7))   # 14
print(triple(7))   # 21
```

## 7.13 6.14 Closures and capture semantics

A closure "closes over" variables from its defining scope. The inner function remembers those bindings even after the outer function returns.

```
func make_counter(start = 0) {
    var n = start
    return fn() {
        n += 1
        return n
    }
}

let c = make_counter()
print(c())   # 1
print(c())   # 2
print(c())   # 3

let d = make_counter(10)
print(d())   # 11
print(d())   # 12
```

Each call to `make_counter` creates a fresh binding, so each counter is independent.

### 7.13.1 The late-binding loop variable trap

```
let fns = []
for i in 0..3 {
    fns.push(fn() => i)
}
for f in fns { print(f()) }
```

If all closures print the same final value, the loop variable is shared across iterations. Fix by capturing a copy through a factory function:

```
func make_constant(v) { return fn() => v }

let fns2 = []
for i in 0..3 { fns2.push(make_constant(i)) }
for f in fns2 { print(f()) }   # 0 1 2
```

## 7.14 6.15 Partial application and currying

```
func partial(f, a) { return fn(b) => f(a, b) }
func curry(f) { return fn(a) => fn(b) => f(a, b) }

let add = fn(a, b) => a + b
let add5 = partial(add, 5)
```

```
print(add5(10))   # 15

let curried = curry(add)
print(curried(3)(4))   # 7
```

## 7.15 6.16 Function composition

```
func compose(f, g) { return fn(x) => f(g(x)) }

let double = fn(x) => x * 2
let inc = fn(x) => x + 1
let inc_then_double = compose(double, inc)
print(inc_then_double(3))   # 8  (double(inc(3)) = double(4) = 8)

func compose_all(fns) {
    return fn(x) {
        var acc = x
        for i in (len(fns) - 1)..-1 {
            acc = fns[i](acc)
        }
        return acc
    }
}
```

## 7.16 6.17 Memoization

```
func memoize(f) {
    let cache = #{}
    return fn(n) {
        let key = str(n)
        if key in cache { return cache[key] }
        let v = f(n)
        cache[key] = v
        return v
    }
}

let fib = memoize(fn(n) {
    if n < 2 { return n }
    return fib(n - 1) + fib(n - 2)
})

print(fib(50))   # 12586269025
```

The self-reference works because `fib` is bound before any call executes.

## 7.17 6.18 Callbacks and sorting with comparator functions

```
func sort_by(xs, key_fn) {
    # simple insertion sort using the key function
    let out = list(xs)
    for i in 1..len(out) {
        let key = out[i]
        var j = i - 1
        while j >= 0 and key_fn(out[j]) > key_fn(key) {
            out[j + 1] = out[j]
            j -= 1
        }
        out[j + 1] = key
    }
    return out
}

let words = ["banana", "apple", "cherry", "kiwi"]
print(sort_by(words, fn(w) => len(w)))   # shorter first
```

Passing comparator/key functions is the standard way to customize behavior.

## 7.18 6.19 Closures over mutable state

Captured `var` bindings can be mutated from inside the closure, creating shared state between the closure and its enclosing scope.

```
func adder_factory(init) {
    var total = init
    return fn(x) {
        total += x
        return total
    }
}

let acc = adder_factory(0)
print(acc(5))   # 5
print(acc(7))   # 12
print(acc(3))   # 15
```

## 7.19 6.20 Common closure pitfalls

1. **Late binding**: loop variables are shared; use a factory to capture values.
2. **Stale closures**: a closure captures the binding, not a snapshot, so later mutations are visible inside the closure.
3. **Mutating captured variables**: surprising when multiple closures share one variable — prefer explicit state passed as arguments.

4. **Returning closures that reference `nil`-initialized variables**: initialize before returning.
5. **Deeply nested closures**: hard to read; name intermediate functions.

## 7.20 6.21 Exercises

1. Write `make_multiplier(n)` returning a function that multiplies by `n`.
2. Implement `compose(f, g)` and verify `compose(double, inc)(3) == 8`.
3. Write `repeater(f, n)` returning a function that calls `f` `n` times.
4. Build `memoize` and apply it to Fibonacci; time the difference.
5. Implement `curry2(f)` for two-argument functions.
6. Write `pipe(value, fns)` that threads a value through functions left-to-right.
7. Sort a list of dicts by a chosen key using a comparator closure.
8. Write `once(f)` that invokes `f` only on the first call.
9. Create `counter()` closures that can increment, decrement, and reset.
10. Demonstrate the late-binding trap and two ways to fix it.

# 8 Chapter 7: Strings Deep Dive

Strings in Zen are immutable sequences of Unicode characters. They are stored as `Arc<str>` (reference-counted, immutable), which makes copying a string cheap (pointer copy) but means every "modification" produces a new string. This chapter is the definitive reference for string literals, indexing, slicing, methods, formatting, immutability, and performance.

## 8.1 7.1 String literals

### 8.1.1 Single and double quotes

Double and single quotes are interchangeable:

```
let a = "hello"
let b = 'world'
```

Escape sequences work in both:

| Escape | Meaning |
|---|---|
| `\n` | newline |
| `\t` | tab |
| `\\` | backslash |
| `\'` | single quote |
| `\"` | double quote |
| `\0` | null byte |
| `\xHH` | hex byte (e.g. `\x41` = A) |
| `\u{HHHH}` | Unicode scalar (e.g. `\u{1F600}` = 😀) |

```
let s1 = "it's a nice day"   # single quotes inside double
let s2 = 'say "hello"'      # double quotes inside single
let s3 = "line1\nline2\tline3"
let s4 = "byte: \x41"       # "byte: A"
let s5 = "emoji: \u{1F600}" # "emoji: 😀"
```

### 8.1.2 Triple-quoted raw strings

Triple-quoted strings span multiple lines and do NOT process escape sequences:

```
let poem = """Zen is fast,
native, and safe.
No escaping needed here."""
```

```
let path = """C:\Users\zen\docs"""   # backslashes are literal
```

### 8.1.3 Immutability

Strings are immutable. Indexing `s[i]` reads a character; you cannot assign to `s[i]`. Every "modification" returns a new string:

```
let s = "hello"
# s[0] = "H"   # TypeError: strings are immutable

let s2 = "H" + s[1:]   # build a new string
print(s2)   # Hello
print(s)    # hello  (unchanged)
```

This matters for performance: building a string in a loop with + creates a new rope each iteration. Zen uses rope concatenation (O(1) per append at the rope level), but the final flatten is O(n). For heavy string building, collect parts in a list and `join` once:

```
# Slower: repeated rope concatenation
let result = ""
for i in 1..1000 {
    result = result + "x"
}

# Faster: collect then join
let parts = []
for i in 1..1000 {
    parts.push("x")
}
let result = "".join(parts)
```

## 8.2 7.2 Indexing and slicing

### 8.2.1 Indexing

`s[i]` returns the character at position `i` as a one-character string. Negative indices count from the end.

```
let s = "Zen"

print(s[0])   # "Z"
print(s[1])   # "e"
print(s[2])   # "n"
print(s[-1])  # "n"  (last)
print(s[-2])  # "e"  (second to last)
```

Indexing out of bounds raises `IndexError`:

```
try {
    print(s[10])
} catch as e {
    print(e.msg)  # IndexError: index 10 out of bounds for length 3
```

```
}
```

### 8.2.2 Slicing

`s[a:b]` returns a substring from `a` (inclusive) to `b` (exclusive). Omitting `a` starts at 0; omitting `b` goes to the end. Negative indices work.

```
let s = "Zen 3.0"

print(s[0:3])   # "Zen"
print(s[4:6])   # "3."
print(s[:3])    # "Zen"   (from start)
print(s[4:])    # "3.0"   (to end)
print(s[-2:])   # ".0"    (last two)
print(s[:-2])   # "Zen 3" (all but last two)
print(s[::])    # "Zen 3.0" (full copy)
```

Slice bounds are clamped: `s[0:100]` on a 3-char string returns the whole string; `s[-100:2]` returns from the start.

```
let s = "Zen"
print(s[0:100])   # "Zen"
print(s[-100:2])  # "Ze"
```

Slicing creates a **new string** (a copy of the substring, or a rope node referencing the original). Mutating the original never affects a slice, and vice versa, because strings are immutable.

## 8.3 7.3 Concatenation and repetition

### 8.3.1 Concatenation: +

When either side of `+` is a string, the other side is stringified via `str()`:

```
print("Zen " + "3.0")    # "Zen 3.0"
print("count: " + 42)    # "count: 42"
print("flag: " + true)   # "flag: true"
print(nil + "end")       # "nilend"
print(42 + "count")      # TypeError! (number + string, string not on left)
print(str(42) + "count") # "42count"  (correct)
```

`+` between two strings concatenates them. `+` between a string and anything else stringifies the non-string side via `str()`.

### 8.3.2 Repetition: *

`*` repeats a string. Either operand may be the string:

```
print("x" * 5)      # "xxxxx"
print("ab " * 3)    # "ab ab ab "
print(3 * "ab")     # "abab"   (left operand is int, still works)
```

Repeating by 0 or a negative number yields `""`:

```
print("abc" * 0)    # ""
print("abc" * -1)   # ""
```

## 8.4 7.4 String methods

Zen strings have the following methods. All return new strings (or lists/ints/bools) — none mutate the receiver.

| Method | Signature | Description | Example | Result |
|---|---|---|---|---|
| `upper()` | `str → str` | Uppercase | `"zen".upper()` | `"ZEN"` |
| `lower()` | `str → str` | Lowercase | `"ZEN".lower()` | `"zen"` |
| `title()` | `str → str` | Title case | `"zen language".title()` | `"Zen Language"` |
| `capitalize()` | `str → str` | Capitalize first char | `"zen".capitalize()` | `"Zen"` |
| `swapcase()` | `str → str` | Invert case | `"Zen".swapcase()` | `"zEN"` |
| `trim()` / `strip()` | `str → str` | Whitespace both ends | `"  zen ".trim()` | `"zen"` |
| `lstrip()` | `str → str` | Whitespace left | `"  zen".lstrip()` | `"zen"` |
| `rstrip()` | `str → str` | Whitespace right | `"zen  ".rstrip()` | `"zen"` |
| `replace(old, new)` | `str → str` | Replace all occurrences | `"aaa".replace("a", "b")` | `"bbb"` |
| `split(sep?)` | `str → list` | Split into list | `"a,b,c".split(",")` | `["a", "b", "c"]` |
| `split_whitespace()` | `str → list` | Split on whitespace | `"a b  c".split_whitespace()` | `["a", "b", "c"]` |
| `join(list)` | `list → str` | Join list with string | `", ".join(["a","b"])` | `"a, b"` |
| `starts_with(prefix)` | `str → bool` | Boolean | `"zen".starts_with("ze")` | `true` |
| `ends_with(suffix)` | `str → bool` | Boolean | `"zen".ends_with("en")` | `true` |
| `contains(sub)` | `str → bool` | Boolean | `"zen".contains("e")` | `true` |
| `find(sub)` | `str → int` | Index or -1 | `"zen".find("e")` | `1` |
| `find(sub, start)` | `str → int` | Index from start | `"zzzen".find("e", 2)` | `4` |
| `count(sub)` | `str → int` | Count occurrences | `"aaa".count("a")` | `3` |
| `len(s)` | builtin | Length (not a method) | `len("zen")` | `3` |
| `ord(ch)` | builtin | Character code | `ord("A")` | `65` |
| `chr(code)` | builtin | Character from code | `chr(65)` | `"A"` |

### 8.4.1 Case conversion examples

```
import string  # or use methods directly

let s = "Hello World"

print(s.upper())        # "HELLO WORLD"
print(s.lower())        # "hello world"
print(s.title())        # "Hello World"
print(s.capitalize())   # "Hello world"
print(s.swapcase())     # "hELLO wORLD"
```

### 8.4.2 Trimming examples

```
let s = "  hello  "

print(s.trim())    # "hello"
print(s.lstrip())  # "hello  "
print(s.rstrip())  # "  hello"

# Custom character trimming is NOT supported — trim only removes whitespace
```

### 8.4.3 Split examples

```
print("a,b,c".split(","))           # ["a", "b", "c"]
print("a b  c".split_whitespace())  # ["a", "b", "c"]
print("a,,b".split(","))           # ["a", "", "b"]  (empty strings preserved)
print("".split(","))               # [""]              (single empty string)
```

`split()` with no argument splits on any whitespace and discards empty strings:

```
print("  a   b  c  ".split())       # ["a", "b", "c"]
```

### 8.4.4 Join examples

```
let tags = ["zen", "lang", "script"]
print(", ".join(tags))   # "zen, lang, script"
print("#".join(tags))    # "zen#lang#script"
print("".join(tags))     # "zenlangscript"
```

`join` requires all elements to be strings:

```
try {
    print("-".join([1, 2, 3]))  # TypeError
} catch as e {
    print(e.msg)
}
```

### 8.4.5 find / count examples

```
let s = "banana"

print(s.find("an"))      # 1   (first occurrence)
print(s.find("an", 2))   # 3   (start at index 2)
print(s.find("zz"))      # -1  (not found)
print(s.count("a"))      # 3
print(s.count("ana"))    # 2   (overlapping: positions 0 and 2)
```

`find` returns -1 when not found (unlike `index` in some languages which raises).

### 8.4.6 starts_with / ends_with / contains

```
let s = "https://example.com/page"

print(s.starts_with("https"))   # true
print(s.ends_with(".com"))      # false
print(s.ends_with("/page"))     # true
print(s.contains("example"))    # true
print(s.contains("example.org")) # false
```

## 8.5 7.5 String formatting

Zen does **not** have f-strings or printf-style formatting built into the language. Use concatenation and `str()`:

```
let name = "Zen"
let version = 3.0

print(name + " version " + str(version))
# "Zen version 3.0"

let items = 5
print("You have " + str(items) + " items")
# "You have 5 items"
```

### 8.5.1 Padded output

For aligned output, use string repetition and slicing:

```
let name = "Zen"
let padded = name + " " * (10 - len(name))
print("[" + padded + "]")
# "[Zen      ]"
```

```
func pad_right(s, width) {
    if len(s) >= width { return s }
```

```
    return s + " " * (width - len(s))
}

func pad_left(s, width) {
    if len(s) >= width { return s }
    return " " * (width - len(s)) + s
}

print("[" + pad_right("Zen", 10) + "]")   # [Zen      ]
print("[" + pad_left("Zen", 10) + "]")    # [      Zen]
```

### 8.5.2 Number formatting

`str()` on floats uses a default representation. There is no built-in `format(2.5, ".2f")`. For fixed decimals, implement manually:

```
func format2(x) {
    let scaled = (x * 100).to_int()   # if to_int exists
    // ... manual decimal insertion
}
```

> Note: Zen's float-to-string conversion uses the default Rust `f64` display, which may produce varying precision. For financial/formatted output, implement your own formatter.

## 8.6 7.6 Character codes

```
print(ord("A"))   # 65
print(ord("z"))   # 122
print(ord("□"))  # 128512  (Unicode scalar value)
print(chr(65))    # "A"
print(chr(122))   # "z"
print(chr(128512)) # "□"
```

`ord()` takes a one-character string and returns its Unicode scalar value (u32). `chr()` takes a u32 and returns the corresponding one-character string. `chr()` with an invalid code point may panic or produce a replacement character.

```
print(ord("A") == 65)     # true
print(chr(ord("z")) == "z")  # true
```

## 8.7 7.7 Immutability and performance

### 8.7.1 Immutability

Strings are `Arc<str>` — reference-counted and immutable. You cannot modify a string in place:

```
let s = "hello"
# s[0] = "H"   # TypeError: strings are immutable
let s2 = "H" + s[1:]   # new string
```

This means: - Copying a string is cheap (pointer copy, refcount bump). - Concatenation allocates. - Repeated concatenation in a loop allocates each iteration.

### 8.7.2 Rope concatenation

Zen uses rope concatenation for `+` on strings. `a + b` creates a rope node referencing both sides without copying. The rope is flattened only when needed (e.g., when passed to `print`, `len`, or indexing). This makes `a + b + c + d` efficient (O(n) total for the final flatten, not O(n²)).

However:

```
let result = ""
for i in 1..10000 {
    result = result + "x"   # O(n²) rope nodes, then O(n) flatten
}
```

Prefer collecting in a list and joining:

```
let parts = []
for i in 1..10000 {
    parts.push("x")
}
let result = "".join(parts)   # O(n) total
```

## 8.8 7.7 Common pitfalls

- **Strings are immutable.** Use concatenation to build new strings; `s[i] = "X"` is a TypeError.
- **`+` auto-stringifies only when a string is present.** `"count: " + 42` works; `42 + "count"` is a TypeError. Use `str(42) + "count"`.
- **`len(s)` is a builtin**, not a method: `len(s)`, not `s.len()`.
- **`split()` with no argument** splits on whitespace and discards empty strings. `split(",")` preserves empty strings.
- **`join` requires all elements to be strings.** `[1, 2, 3]` must be stringified first: `"-".join([str(x) for x in [1,2,3]])`.
- **`find` returns -1** when not found (does not raise).
- **Slicing out of bounds clamps** rather than raising (except direct indexing `s[i]` which raises on out of bounds).
- **Triple-quoted strings do NOT process escapes** — `\n` stays as literal backslash-n.

## 8.9 7.9 Quick reference

```
let s = "Zen 3.0"

len(s)              # 7
s[0]                # "Z"
s[-1]               # "0"
s[0:3]              # "Zen"
s + "!"             # "Zen 3.0!"
"-" * 3             # "---"
s.upper()           # "ZEN 3.0"
s.lower()           # "zen 3.0"
s.title()           # "Zen 3.0"
s.capitalize()      # "Zen 3.0"
s.trim()            # "Zen 3.0"
s.split(" ")        # ["Zen", "3.0"]
", ".join(["a","b"])# "a, b"
s.contains("3")     # true
s.find("3")         # 4
s.count("3")        # 1
s.starts_with("Ze") # true
s.ends_with(".0")   # true
s.replace("3", "4") # "Zen 4.0"
ord("Z")            # 90
chr(90)             # "Z"
int("42")           # 42
str(42)             # "42"
ord("□")           # 128512
chr(128512)         # "□"
```

## 8.10 7.10 Exercises

1. Write `is_palindrome(s)` that returns true if `s` reads the same forwards and backwards (case-insensitive). Use `lower()` and slicing.
2. Write `acronym(words)` that takes a phrase and returns its acronym (first letter of each word, uppercase). Use `split_whitespace()` and `upper()`.
3. Write `mask_credit_card(number)` that shows only the last 4 digits, replacing the rest with `*`.
4. Use `split` and `join` to convert a comma-separated string into a pipe-separated string: `"a,b,c"` → `"a|b|c"`.
5. Write `repeat_pattern(pattern, count)` that builds a string by repeating `pattern` `count` times separated by `-`.
6. Write `word_count(text)` that returns the number of words (split on whitespace).
7. Write `is_pangram(text)` that checks if text contains every letter a-z at least once (case-insensitive).
8. Write `camel_to_snake(name)` that converts `camelCase` to `camel_case` by inserting `_` before uppercase letters and lowercasing.

9. Benchmark: build a 100,000-character string with `+` in a loop vs `join`, and compare.
10. Write `strip_quotes(s)` that removes matching surrounding quotes (`"` or `'`) if present.

## 8.11 7.12 String encoding and code points vs bytes

Zen strings are sequences of Unicode characters (code points). Indexing and slicing operate on *characters*, not raw bytes, unless you explicitly work with `bytes`.

```
let s = "Zen"           # 3 code points, 3 bytes
print(len(s))           # 3

let s2 = "Zën"          # ë is one code point, 2 bytes in UTF-8
print(len(s2))          # 3 (characters, not bytes)
```

Convert between strings and bytes explicitly:

```
let b = s2.encode("utf-8")   # bytes
let s3 = b.decode("utf-8")   # back to string
print(s2 == s3)              # true
```

### 8.11.1 Byte-oriented operations (when you need them)

Work on `bytes` if you need raw offsets, binary protocols, or fixed-width encodings:

```
let b = bytes([72, 101, 108, 108, 111])  # "Hello" in ASCII
print(len(b))   # 5
```

## 8.12 7.13 Indexing, slicing, and negative indices

Strings support `s[i]` and `s[start:end]` (end-exclusive, like lists). Negative indices count from the end.

```
let s = "Zen 3.0"
print(s[0])     # "Z"
print(s[-1])    # "0"
print(s[0:3])   # "Zen"
print(s[4:])    # "3.0"
print(s[:3])    # "Zen"
print(s[-3:])   # "3.0"
print(s[0:100]) # "Zen 3.0"  (out-of-range end clamps)
```

Slicing produces a *new* string; strings are immutable.

## 8.13 7.14 String immutability

You cannot assign to `s[i]`. Any “modification” creates a new string.

```
let s = "hello"
# s[0] = "H"   # error: strings are immutable
let s2 = "H" + s[1:]
print(s2)   # "Hello"
print(s)    # "hello"  (unchanged)
```

## 8.14 7.15 String methods — complete reference

Common methods. If a method is missing from your stdlib build, implement it manually; the behavior shown is the canonical semantics.

| Method | Purpose | Example | Result |
|---|---|---|---|
| `upper()` | to uppercase | `"abc".upper()` | `"ABC"` |
| `lower()` | to lowercase | `"ABC".lower()` | `"abc"` |
| `title()` | title case | `"hello world".title()` | `"Hello World"` |
| `capitalize()` | first char upper | `"hello".capitalize()` | `"Hello"` |
| `swapcase()` | flip case | `"AbC".swapcase()` | `"aBc"` |
| `strip()` | trim whitespace | `"  hi  ".strip()` | `"hi"` |
| `lstrip()` | trim left | `"  hi".lstrip()` | `"hi"` |
| `rstrip()` | trim right | `"hi  ".rstrip()` | `"hi"` |
| `split(sep)` | split on separator | `"a,b,c".split(",")` | `["a","b","c"]` |
| `splitlines()` | split on newlines | `"a\nb\nc".splitlines()` | `["a","b","c"]` |
| `join(iter)` | join with separator | `"-".join(["a","b"])` | `"a-b"` |
| `replace(old, new)` | replace occurrences | `"aaa".replace("a","b")` | `"bbb"` |
| `count(sub)` | count occurrences | `"banana".count("an")` | `2` |
| `find(sub)` | first index or -1 | `"hello".find("ll")` | `2` |
| `rfind(sub)` | last index or -1 | `"hello".rfind("l")` | `3` |
| `startswith(prefix)` | boolean | `"hello".startswith("he")` | `true` |
| `endswith(suffix)` | boolean | `"hello".endswith("lo")` | `true` |
| `contains(sub)` | boolean | `"hello".contains("ell")` | `true` |
| `ljust(width)` | pad left | `"hi".ljust(5)` | `"hi   "` |
| `rjust(width)` | pad right | `"hi".rjust(5)` | `"   hi"` |
| `center(width)` | center pad | `"hi".center(5)` | `" hi "` |
| `zfill(width)` | zero pad | `"42".zfill(5)` | `"00042"` |
| `isdigit()` | all digits | `"123".isdigit()` | `true` |
| `isalpha()` | all letters | `"abc".isalpha()` | `true` |
| `isalnum()` | letters/digits | `"abc1".isalnum()` | `true` |
| `isspace()` | all whitespace | `"   ".isspace()` | `true` |
| `islower()` | all lowercase | `"abc".islower()` | `true` |
| `isupper()` | all uppercase | `"ABC".isupper()` | `true` |

```
print("hello world".replace("world", "Zen"))   # hello Zen
print("-".join(["2024", "01", "15"]))          # 2024-01-15
print("  spaced  ".strip())                    # spaced
print("PYTHON".lower())                        # python
print("hello".count("l"))                      # 2
```

## 8.15 7.16 Formatting and interpolation

Double-quoted and triple-quoted strings support `${expr}` interpolation:

```
let name = "Ada"
let age = 36
print("Name: ${name}, Age: ${age}")   # Name: Ada, Age: 36
print("${name} is ${age + 1} next year")   # Ada is 37 next year
```

Escapes work inside interpolation strings: `\n \t \\ \" \' \xNN \u{NNNN}`. Single-quoted and backtick strings do not interpolate:

```
print('Cost: ${price}')   # literal: Cost: ${price}
print(`Cost: ${price}`)   # literal, raw
```

Inside `${...}`, the expression is parsed normally; escapes in the surrounding string are not processed inside the interpolation.

For older-style formatting without interpolation, concatenate or use `str()`:

```
print("Value: " + str(42))   # Value: 42
```

## 8.16 7.17 String comparison and ordering

Strings compare lexicographically by code point. Uppercase sorts before lowercase in ASCII/Unicode order.

```
print("apple" < "banana")   # true
print("Z" < "a")            # true  (uppercase before lowercase)
print("a" == "A")           # false
print("hello" == "hello")   # true
```

For case-insensitive comparison, normalize:

```
func ieq(a, b) { return a.lower() == b.lower() }
print(ieq("Zen", "ZEN"))   # true
```

## 8.17 7.18 Common string pitfalls

1. **Assuming `len` is byte length.** It counts characters/code points.
2. **Indexing multi-byte characters.** Indexing is by character, so `s[1]` of `"Zën"` is ë, not a raw byte.
3. **Strings are immutable.** `s[i] = x` is an error; build a new string.

4. **Case folding for Unicode.** `lower()`/`upper()` handle ASCII well; full Unicode case folding may need explicit handling.
5. **Trailing newline in `splitlines`.** A string ending in `\n` may or may not produce a trailing empty element depending on the build.
6. **`find` returns -1 when not found**, not `nil` — check for `-1`.
7. **Interpolation inside `${...}` does not process surrounding escapes.**

## 8.18 7.19 Exercises

1. Write `is_palindrome(s)` ignoring case and spaces.
2. Implement `count_vowels(s)` returning a dict of vowel counts.
3. Write `mask_card(number)` showing only the last 4 digits.
4. Implement `slugify(text)` → lowercase, spaces to hyphens, strip.
5. Write `format_bytes(n)` producing human-readable sizes (KB, MB).
6. Implement `wrap_text(text, width)` inserting newlines.
7. Write `extract_emails(text)` using `find`/slicing manually.
8. Implement `caesar_cipher(text, shift)` for letters only.
9. Write `validate_password(pw)` checking length, digit, upper, lower.
10. Implement `abbreviate(sentence)` taking first letters of words.

# 9 Chapter 8: Lists and Arrays

Lists are Zen's ordered, mutable, heterogeneous sequences. They support indexing, slicing, appending, insertion, removal, sorting, comprehensions, and more. This chapter covers lists in depth, including aliasing, mutation, and performance.

## 9.1 8.1 List literals

```
let empty = []
let nums = [1, 2, 3, 4, 5]
let mixed = [1, "two", 3.0, true, nil]
let nested = [[1, 2], [3, 4], [5, 6]]
let single = [42]
```

Lists can contain any type, including other lists, dicts, functions, and foreign handles. An empty list `[]` is distinct from an empty dict `{}`.

## 9.2 8.2 Indexing and slicing

### 9.2.1 Indexing

```
let nums = [10, 20, 30, 40, 50]

print(nums[0])    # 10
print(nums[2])    # 30
print(nums[-1])   # 50  (last)
print(nums[-2])   # 40  (second to last)
```

Indexing out of bounds raises `IndexError`:

```
try {
    print(nums[10])
} catch as e {
    print(e.msg)  # IndexError: index 10 out of bounds for length 5
}
```

### 9.2.2 Slicing

`s[a:b]` returns a new list from `a` (inclusive) to `b` (exclusive). Omitting `a` starts at 0; omitting `b` goes to the end. Negative indices work. Bounds are clamped.

```
let nums = [10, 20, 30, 40, 50]

print(nums[1:4])   # [20, 30, 40]
print(nums[:3])    # [10, 20, 30]
print(nums[3:])    # [40, 50]
print(nums[-2:])   # [40, 50]
print(nums[:-2])   # [10, 20, 30]
```

```
print(nums[0:100]) # [10, 20, 30, 40, 50]  (clamped)
print(nums[-100:2])# [10, 20]             (clamped)
```

Slicing creates a **new list** (a copy). Mutating the slice never affects the original, and vice versa:

```
let a = [1, 2, 3]
let b = a[0:2]
b[0] = 99
print(a)  # [1, 2, 3]  (unchanged)
print(b)  # [99, 2]
```

## 9.3 8.3 Mutation methods

| Method | Signature | Description | Example |
|---|---|---|---|
| `push(x)` | `list → nil` | Append to end | `nums.push(6)` |
| `pop()` | `list → value` | Remove and return last | `nums.pop()` |
| `pop(i)` | `list → value` | Remove and return index i | `nums.pop(0)` |
| `insert(i, x)` | `list → nil` | Insert at index | `nums.insert(0, 0)` |
| `remove(x)` | `list → nil` | Remove first equal to x | `nums.remove(20)` |
| `clear()` | `list → nil` | Remove all | `nums.clear()` |
| `sort()` | `list → nil` | Sort in place | `nums.sort()` |
| `reverse()` | `list → nil` | Reverse in place | `nums.reverse()` |
| `copy()` | `list → list` | Shallow copy | `nums.copy()` |
| `index(x)` | `list → int` | Index of first x | `nums.index(30)` |
| `count(x)` | `list → int` | Count occurrences of x | `nums.count(20)` |
| `extend(list)` | `list → nil` | Extend with another list | `nums.extend([6, 7])` |
| `len(list)` | builtin | Length (not a method) | `len(nums)` |

### 9.3.1 Examples

```
let nums = [3, 1, 4, 1, 5]

nums.push(9)
print(nums)   # [3, 1, 4, 1, 5, 9]

let last = nums.pop()
print(last)   # 9
print(nums)   # [3, 1, 4, 1, 5]

nums.insert(0, 0)
print(nums)   # [0, 3, 1, 4, 1, 5]
```

```
nums.remove(4)
print(nums)   # [0, 3, 1, 1, 5]

nums.sort()
print(nums)   # [0, 1, 1, 3, 5]

nums.reverse()
print(nums)   # [5, 3, 1, 1, 0]

print(nums.index(3))  # 1
print(nums.count(1))  # 2
print(len(nums))      # 5

nums.clear()
print(nums)   # []
```

### 9.3.2 `pop` behavior

```
let nums = [10, 20, 30]

print(nums.pop())   # 30  (removes last)
print(nums)         # [10, 20]

print(nums.pop(0))  # 10  (removes first)
print(nums)         # [20]
```

`pop()` with no index removes the last element (O(1)). `pop(i)` removes at index i (O(n) because elements after i shift).

### 9.3.3 `remove` removes only the first match

```
let nums = [1, 2, 1, 2, 1]
nums.remove(1)
print(nums)   # [2, 1, 2, 1]  (only first 1 removed)
```

If the value is not present, `remove` raises `ValueError`.

## 9.4 8.4 List operations

### 9.4.1 Concatenation and repetition

```
let a = [1, 2]
let b = [3, 4]

print(a + b)    # [1, 2, 3, 4]
```

```
print([0] * 3)  # [0, 0, 0]
print(3 * [0])  # [0, 0, 0]
```

+ between lists concatenates (creates a new list). * with an integer repeats. * is commutative for lists: `[0] * 3` and `3 * [0]` both work.

### 9.4.2 Membership

```
let nums = [1, 2, 3]

print(2 in nums)   # true
print(5 in nums)   # false
print(5 !in nums)  # true
```

Membership uses == comparison. For lists of lists, == is order-insensitive for dicts but order-sensitive for lists:

```
print([1, 2] in [[1, 2], [3, 4]])   # true
print([2, 1] in [[1, 2], [3, 4]])   # false
```

## 9.5 8.5 List comprehension

Zen supports list comprehensions with `for` and optional `if`:

```
let squares = [x * x for x in 1..5]
print(squares)  # [1, 4, 9, 16, 25]

let evens = [x for x in 1..10 if x % 2 == 0]
print(evens)    # [2, 4, 6, 8, 10]

let words = ["Zen", "lang", "script"]
let lengths = [len(w) for w in words]
print(lengths)  # [3, 4, 6]
```

Syntax: `[expr for var in iterable]` or `[expr for var in iterable if condition]`.

The iterable can be a list, range, string, dict (iterates keys), or any iterable value.

### 9.5.1 Nested comprehensions

```
let nested = [[1, 2], [3, 4], [5, 6]]
let flat = [x for sub in nested for x in sub]
print(flat)  # [1, 2, 3, 4, 5, 6]
```

The order of `for` clauses matches nesting order: outer `for` first, inner `for` second.

## 9.6 8.6 Common patterns

### 9.6.1 Sum, min, max

```
let nums = [5, 2, 8, 1, 9]

let total = 0
for n in nums {
    total = total + n
}
print(total)  # 25

# min / max via sort
let sorted = nums.copy()
sorted.sort()
print(sorted[0])   # 1 (min)
print(sorted[-1])  # 9 (max)
```

### 9.6.2 Filter and map

```
let nums = [1, 2, 3, 4, 5, 6]

# Filter
let evens = [n for n in nums if n % 2 == 0]
print(evens)  # [2, 4, 6]

# Map
let doubled = [n * 2 for n in nums]
print(doubled)  # [2, 4, 6, 8, 10, 12]

# Filter + map
let even_doubled = [n * 2 for n in nums if n % 2 == 0]
print(even_doubled)  # [4, 8, 12]
```

### 9.6.3 Flatten nested lists

```
let nested = [[1, 2], [3, 4], [5, 6]]
let flat = [x for sub in nested for x in sub]
print(flat)  # [1, 2, 3, 4, 5, 6]
```

### 9.6.4 Find first match

```
let words = ["apple", "banana", "cherry"]
var found = nil
for w in words {
    if w.starts_with("b") {
```

```
        found = w
        break
    }
}
print(found)  # banana
```

### 9.6.5 Remove all matching elements

```
let nums = [1, 2, 3, 2, 4, 2, 5]
let filtered = [n for n in nums if n != 2]
print(filtered)  # [1, 3, 4, 5]
```

### 9.6.6 Zip (manual)

Zen has no built-in `zip`. Implement manually:

```
let keys = ["a", "b", "c"]
let vals = [1, 2, 3]

let pairs = []
for i in 0..len(keys) {
    pairs.push(#(keys[i], vals[i]))
}
print(pairs)  # [#("a", 1), #("b", 2), #("c", 3)]
```

## 9.7 8.6 Sorting

```
let nums = [3, 1, 4, 1, 5]

nums.sort()
print(nums)   # [1, 1, 3, 4, 5]

nums.reverse()
print(nums)   # [5, 4, 3, 1, 1]
```

`sort()` sorts in place using the elements' ==/< ordering. For mixed-type lists, sort behavior is undefined (may raise TypeError). `sort()` is stable.

Sorting strings:

```
let words = ["banana", "apple", "cherry"]
words.sort()
print(words)   # ["apple", "banana", "cherry"]
```

There is no `sort(key=...)` or `reverse=` parameter. To sort descending, sort then `reverse()`. To sort by a key, precompute keys or use a decorator-sort-undecorate pattern.

## 9.8 8.7 Common pitfalls

- **Lists are mutable.** `b = a` makes `b` an alias — changes to `a` affect `b`. Use `a.copy()` for an independent copy.
- **`pop()` with no index** removes the last element (fast). `pop(0)` removes the first (slow, O(n) shift).
- **`remove(x)` removes only the first** occurrence of `x`. Raises `ValueError` if not found.
- **Slicing creates a new list.** `nums[1:3]` is a copy, not a view.
- **List comprehensions create new lists.** They don't modify the original.
- **`sort()` and `reverse()` mutate in place** and return `nil`. Don't write `let s = nums.sort()` expecting `s` to be the sorted list — `s` will be `nil`.
- **`index(x)` raises `ValueError`** if x is not present.
- **Mixed-type lists** may not sort or compare meaningfully.

## 9.9 8.8 Quick reference

```
let nums = [3, 1, 4, 1, 5]

len(nums)           # 5
nums[0]             # 3
nums[-1]            # 5
nums[1:3]           # [1, 4]
nums.push(9)        # [3, 1, 4, 1, 5, 9]
nums.pop()          # 9
nums.pop(0)         # 3
nums.insert(0, 0)   # [0, 3, 1, 4, 1, 5]
nums.remove(4)      # [0, 3, 1, 1, 5]
nums.sort()         # [0, 1, 1, 3, 5]
nums.reverse()      # [5, 3, 1, 1, 0]
nums.clear()        # []
nums.copy()         # shallow copy
nums.index(3)       # position of 3
nums.count(1)       # 2
nums.extend([6, 7]) # [.., 6, 7]
2 in nums           # true
[x * x for x in nums]  # list comprehension
[x for x in nums if x > 2]  # filtered comprehension
```

## 9.10 8.9 Exercises

1. Write `remove_duplicates(lst)` that returns a list with duplicates removed while preserving order.
2. Write `flatten(nested)` that flattens a list of lists one level deep.
3. Write `rotate(lst, k)` that rotates a list right by `k` positions.
4. Write `interleave(a, b)` that alternates elements from two lists.
5. Write `chunk(lst, size)` that splits a list into chunks of `size`.

6. Write `running_total(lst)` that returns a list of running sums.
7. Write `is_sorted(lst)` that checks if a list is in non-decreasing order.
8. Write `second_largest(lst)` that returns the second-largest distinct value.
9. Write `merge_sorted(a, b)` that merges two sorted lists into one sorted list.
10. Demonstrate aliasing: create `a = [1,2,3]`, `b = a`, mutate `a`, show `b` changed; then do the same with `.copy()` and show `b` unchanged.

# 10 Chapter 9: Dictionaries and Maps

Dictionaries are Zen's key-value mappings. They use string keys, preserve insertion order, and support fast lookup, insertion, and deletion. This chapter covers dictionaries in depth, including key rules, iteration, and common patterns.

## 10.1 9.1 Dictionary literals

```
let empty = {}
let person = {"name": "Zen", "version": 3, "active": true}
let nested = {"user": {"name": "Zen", "id": 1}, "tags": ["a", "b"]}
```

Keys must be strings. Values can be any type. `{}` alone is an empty **dict**, not a set (use `{1}` or `set()` for a set).

## 10.2 9.2 Access and mutation

```
let person = {"name": "Zen", "version": 3}

print(person["name"])    # Zen
print(person["version"]) # 3

person["version"] = 4    # update
person["lang"] = "Zen"   # add

print(person)            # {"name": "Zen", "version": 4, "lang": "Zen"}
```

Missing keys raise `KeyError`:

```
try {
    print(person["missing"])
} catch as e {
    print(e.msg)  # KeyError: missing
}
```

## 10.3 9.3 Dictionary methods

| Method | Signature | Description | Example |
|---|---|---|---|
| `keys()` | `dict → list` | List of keys | `d.keys()` |
| `values()` | `dict → list` | List of values | `d.values()` |
| `items()` | `dict → list` | List of [key, value] pairs | `d.items()` |
| `get(key, default?)` | `dict → value` | Value or default | `d.get("missing", 0)` |
| `has(key)` | `dict → bool` | Key exists | `d.has("name")` |
| `len(d)` | builtin | Number of entries | `len(d)` |
| `copy()` | `dict → dict` | Shallow copy | `d.copy()` |

| Method | Signature | Description | Example |
|---|---|---|---|
| `clear()` | `dict → nil` | Remove all | `d.clear()` |
| `pop(key)` | `dict → value` | Remove and return value | `d.pop("name")` |
| `update(other)` | `dict → nil` | Merge another dict | `d.update({"a": 1})` |

### 10.3.1 Examples

```
let d = {"a": 1, "b": 2, "c": 3}

print(d.keys())      # ["a", "b", "c"]
print(d.values())    # [1, 2, 3]
print(d.items())     # [["a", 1], ["b", 2], ["c", 3]]

print(d.get("a"))           # 1
print(d.get("z", 0))        # 0 (default)
print(d.has("b"))           # true
print(d.has("z"))           # false
print(len(d))               # 3

let d2 = d.copy()
d2.update({"c": 30, "d": 4})
print(d2)  # {"a": 1, "b": 2, "c": 30, "d": 4}

let popped = d.pop("b")
print(popped)  # 2
print(d)       # {"a": 1, "c": 3}
```

`pop(key)` raises `KeyError` if the key is missing:

```
try {
    d.pop("nonexistent")
} catch as e {
    print(e.msg)  # KeyError: nonexistent
}
```

`get(key)` returns `nil` if the key is missing and no default is given:

```
print(d.get("nonexistent"))  # nil
print(d.get("nonexistent", 42))  # 42
```

## 10.4 9.3 Iteration

```
let d = {"a": 1, "b": 2, "c": 3}

# Iterate keys
for k in d {
```

```
    print(k)
}

# Iterate keys explicitly
for k in d.keys() {
    print(k)
}

# Iterate values
for v in d.values() {
    print(v)
}

# Iterate items
for kv in d.items() {
    print(kv[0], kv[1])
}
```

Iteration order is **insertion order**, not sorted:

```
let d = {"z": 1, "a": 2, "m": 3}
for k in d {
    print(k)   # z, a, m  (insertion order)
}
```

## 10.5 9.4 Common patterns

### 10.5.1 Counting frequencies

```
let words = ["zen", "lang", "zen", "script", "lang", "zen"]
var counts = {}

for w in words {
    if d.has(w) {
        d[w] = d[w] + 1
    } else {
        d[w] = 1
    }
}
print(d)  # {"zen": 3, "lang": 2, "script": 1}
```

More concisely with `get`:

```
var counts = {}
for w in words {
    counts[w] = counts.get(w, 0) + 1
}
```

### 10.5.2 Grouping by key

```
let items = [
    {"category": "fruit", "name": "apple"},
    {"category": "fruit", "name": "banana"},
    {"category": "veg", "name": "carrot"}
]
var groups = {}

for item in items {
    let cat = item["category"]
    if !groups.has(cat) {
        groups[cat] = []
    }
    groups[cat].push(item["name"])
}
print(groups)  # {"fruit": ["apple", "banana"], "veg": ["carrot"]}
```

### 10.5.3 Merging dictionaries

```
let a = {"x": 1, "y": 2}
let b = {"y": 3, "z": 4}

let merged = a.copy()
merged.update(b)
print(merged)  # {"x": 1, "y": 3, "z": 4}  (b wins on conflict)
```

`update` overwrites existing keys and adds new ones.

### 10.5.4 Default values with `get`

```
let config = {"host": "localhost", "port": 8080}
let host = config.get("host", "0.0.0.0")
let timeout = config.get("timeout", 30)
print(host)    # localhost
print(timeout) # 30
```

### 10.5.5 Inverting a dictionary

```
let d = {"a": 1, "b": 2, "c": 3}
var inverted = {}
for k in d {
    inverted[d[k]] = k
}
print(inverted)  # {1: "a", 2: "b", 3: "c"}
```

Warning: if values are not unique, later keys overwrite earlier ones.

## 10.6 9.5 Key rules

- **Keys must be strings.** `d[42]` raises `TypeError`. Use `d["42"]`.
- **Keys are compared by string content**, not identity.
- **Missing keys raise `KeyError`** on direct access `d["missing"]`. Use `get(key, default)` or `has(key)` to avoid.
- **Insertion order is preserved** and is the iteration order.

## 10.7 9.6 Common pitfalls

- **Dicts are mutable.** `b = a` aliases; use `a.copy()` for independence.
- **`len(d)` is a builtin**, not a method.
- **Iteration order** is insertion order, not sorted.
- **`pop(key)` raises `KeyError`** if missing. Use `get` with a default if you want `nil` instead.
- **Non-string keys are rejected.** `d[42]` → TypeError. Even `d[true]` → TypeError.

## 10.8 9.6 Quick reference

```
let d = {"a": 1, "b": 2}

d["a"]             # 1
d["c"] = 3         # add/update
d.get("z", 0)      # 0
d.has("a")         # true
d.keys()           # ["a", "b", "c"]
d.values()         # [1, 2, 3]
d.items()          # [["a", 1], ...]
len(d)             # 3
d.copy()           # shallow copy
d.pop("a")         # 1 (removes "a")
d.update({"z": 9}) # merge (later wins)
for k in d { ... } # iterate keys (insertion order)
```

## 10.9 9.7 Exercises

1. Write `invert_dict(d)` that swaps keys and values (assumes values are unique strings).
2. Write `sub_dict(d, keys)` that returns a new dict with only the given keys.
3. Write `deep_get(d, path)` that navigates a nested dict using a list of keys, returning `nil` if any key is missing.
4. Write `word_frequencies(text)` that counts words in a string.
5. Write `merge_dicts(*dicts)` that merges multiple dicts (later ones win).
6. Write `dict_intersection(d1, d2)` that returns a dict with keys present in both, using values from d1.

7. Write `dict_diff(d1, d2)` that returns keys in d1 but not d2.
8. Demonstrate that dict equality is order-insensitive: `{"a":1,"b":2} == {"b":2,"a":1}`.

## 10.10 9.12 Dictionaries — complete reference

A dict is an unordered collection of key/value pairs. Keys must be strings.

```
let empty = #{}
let person = #{name: "Ada", age: 36, city: "London"}
let mixed = #{a: 1, b: "two", c: true, d: [1, 2]}
```

### 10.10.1 Creation

| Form | Result |
|---|---|
| `#{}` | empty dict |
| `#{k: v, ...}` | literal with string keys |
| `dict(pairs)` | from list of `[k, v]` pairs |
| `list(d)` | list of keys |

Note: Legacy `{k: v}` still parses with a deprecation warning; prefer `#{k: v}`.

## 10.11 9.13 Access and mutation

```
let d = #{name: "Ada", age: 36}
print(d["name"])    # Ada
d["age"] = 37       # update
d["city"] = "London"  # add
print(d)            # #{name: "Ada", age: 37, city: "London"}
```

Missing key access raises an error. Use `in` or `??` to be safe:

```
print("name" in d)          # true
print("country" in d)       # false
print(d["country"] ?? "UK") # UK  (default when missing)
```

## 10.12 9.14 Dict methods

| Method | Purpose | Example |
|---|---|---|
| `keys()` | list of keys | `d.keys()` |
| `values()` | list of values | `d.values()` |
| `items()` | list of `[k, v]` pairs | `d.items()` |
| `get(k, default)` | value or default | `d.get("x", 0)` |
| `pop(k)` | remove and return value | `d.pop("age")` |
| `clear()` | remove all entries | `d.clear()` |

| Method | Purpose | Example |
|---|---|---|
| `update(other)` | merge another dict | `d.update(#{x: 1})` |

```
for k, v in d.items() {
    print(k, v)
}
```

## 10.13 9.15 Dict iteration

Iterating a dict yields keys. Use `.items()` for key/value, `.values()` for values.

```
for k in d { print(k) }              # keys
for k, v in d.items() { print(k, v) } # pairs
for v in d.values() { print(v) }     # values
```

Iteration order is not guaranteed across versions — do not rely on it.

## 10.14 9.16 Dict equality and hashing

Dicts compare order-insensitively with ==:

```
print(#{a: 1, b: 2} == #{b: 2, a: 1})   # true
```

Keys must be strings; non-string keys raise a TypeError. There is no hashing of arbitrary keys in Zen 3.0.

## 10.15 9.17 Common dict patterns

### 10.15.1 Counting frequencies

```
func count_words(words) {
    let freqs = #{}
    for w in words {
        freqs[w] = (freqs[w] ?? 0) + 1
    }
    return freqs
}
print(count_words(["a", "b", "a", "c", "b", "a"]))
# #{a: 3, b: 2, c: 1}
```

### 10.15.2 Grouping by key

```
func group_by(xs, key_fn) {
    let groups = #{}
    for x in xs {
        let k = key_fn(x)
```

```
        if k in groups {
            groups[k].push(x)
        } else {
            groups[k] = [x]
        }
    }
    return groups
}
```

### 10.15.3 Merging dicts

```
let base = #{a: 1, b: 2}
let overrides = #{b: 20, c: 3}
let merged = #{}
for k in base { merged[k] = base[k] }
for k in overrides { merged[k] = overrides[k] }
print(merged)   # #{a: 1, b: 20, c: 3}
```

### 10.15.4 Default-dict pattern

```
func get_or_default(d, k, default) {
    if k in d { return d[k] }
    return default
}
```

## 10.16 9.18 Nested dicts

```
let config = #{database: #{host: "localhost", port: 5432}}
print(config["database"]["host"])   # localhost
```

Safely navigate nested dicts:

```
func deep_get(d, path, default = nil) {
    var cur = d
    for p in path {
        if typeof(cur) != "dict" || !(p in cur) { return default }
        cur = cur[p]
    }
    return cur
}
print(deep_get(config, ["database", "host"]))   # localhost
print(deep_get(config, ["database", "user"], "n/a"))   # n/a
```

## 10.17 9.19 Performance notes

- Key lookup is O(1) on average.

- Dicts use more memory than lists for small collections.
- Avoid rebuilding dicts in hot loops; mutate in place.
- `in` on a dict checks keys, not values.

## 10.18 9.20 Common dict mistakes

1. **Accessing a missing key** without `??` or `in` raises an error.
2. **Assuming iteration order.** Sort keys if order matters.
3. **Non-string keys** are not allowed.
4. **Aliasing:** `b = a` shares the dict; copy manually for independence.
5. **Mutating while iterating** over keys.
6. **Confusing `.get(k, default)` with `d[k] ?? default`** — equivalent but `??` also triggers on explicit `nil` values stored in the dict.

## 10.19 9.21 Exercises

1. Write `invert_dict(d)` swapping keys and values (values must be strings).
2. Implement `subtract_dicts(a, b)` removing b's keys from a.
3. Write `deep_merge(a, b)` recursively merging nested dicts.
4. Implement `word_frequencies(text)` using split + counting.
5. Write `filter_dict(d, pred)` keeping entries where pred(k, v) is true.
6. Implement `dict_from_pairs(pairs)` from `[k, v]` list.
7. Write `common_keys(a, b)` returning keys present in both.
8. Implement `rename_keys(d, mapping)` renaming per a mapping dict.
9. Write `nested_set(d, path, value)` setting a value at a dotted path.
10. Implement `dict_diff(a, b)` showing added/removed/changed keys.

# 11 Chapter 10: Modules and Imports

Zen's module system lets you organize code into reusable files and import from both pure-Zen modules (`.z`) and native Rust modules. This chapter covers `import`, `from ... import`, module resolution, `requirements.txt`, `zen py-sync`, and the `native` tier in exhaustive detail.

## 11.1 10.1 Importing modules

### 11.1.1 Bare import

```
import fs
import os
import json
import socket
import requests
import crypto
import tls
import ffi
import sys
import process
import thread
import channel
import shared
import net
import dns
import http
import urllib
import html
import lxml
import sqlite
import qr
import wa
import ffmpeg
import hydra
import raw
```

After importing, access members with dot notation:

```
import fs

let exists = fs.exists("hello.z")
let content = fs.read("hello.z")
```

### 11.1.2 `from ... import`

Import specific names into the current scope:

```
from fs import read, write
from json import loads, dumps

print(loads('{"a": 1}'))
write("out.txt", "hello")
```

Multiple names:

```
from fs import read, write, exists, mkdirs
```

### 11.1.3 `from ... import as`

Rename on import:

```
import requests as req
import os.path as path
import json as j

let r = req.get("https://example.com")
print(path.join("a", "b"))
print(j.loads('{"a": 1}'))
```

### 11.1.4 Importing submodules

```
import urllib.parse
import urllib.request
import lxml.etree
import net.dns
import crypto.symmetric
import crypto.asymmetric
```

Submodule access uses dot notation:

```
let parsed = urllib.parse.urlparse("https://example.com")
let dns_addrs = net.dns.resolve("example.com")
```

## 11.2 10.2 Module resolution order

When you `import foo`, Zen searches for `foo` in this order:

### 11.2.1 Step 1: Native (Rust) modules

Built-in modules compiled into the Zen binary. These have no `.z` counterpart and are always available:

| Module | Purpose |
|---|---|
| `fs` | File system operations |

| Module | Purpose |
|---|---|
| `os` | OS operations, env, process info |
| `time` | Time operations |
| `json` | JSON (also has `std/json.z` pure tier) |
| `socket` | TCP/UDP/raw sockets |
| `ffi` | Foreign function interface (C libraries) |
| `sys` | Low-level syscalls |
| `process` | Process spawn/control |
| `thread` | Threading |
| `channel` | Message-passing channels |
| `shared` | Atomic shared values |
| `net` | TCP/UDP helpers |
| `net.dns` | DNS resolution |
| `requests` | HTTP client |
| `http` | HTTP client |
| `http.server` | HTTP static server |
| `urllib` | URL utilities |
| `urllib.parse` | URL parsing |
| `urllib.request` | URL fetching |
| `html` | HTML parsing |
| `html.parser` | HTML parser |
| `lxml` | lxml compatibility |
| `lxml.etree` | XML element tree |
| `lxml.html` | HTML parsing |
| `raw` | Raw socket/packet crafting |
| `crypto` | Cryptography (also has `std/crypto/` pure tier) |
| `tls` | TLS with certificate pinning |
| `sqlite` | SQLite database |
| `qr` | QR code generation |
| `wa` | WebAssembly |
| `ffmpeg` | FFmpeg |
| `hydra` | Hydra |
| `raw` | Raw sockets |

If `foo` is a native module, it is used directly and the search stops.

### 11.2.2 Step 2: Pure-Zen `.z` modules

Files in `std/` (the standard library) or in your project directory:

```
std/argparser.z    std/base64.z    std/binascii.z   std/bytes.z
std/collections.z  std/crunch.z    std/crypto/      std/ffmpeg.z
std/functools.z    std/glob.z      std/hashlib.z    std/itertools.z
std/json.z         std/logging.z   std/math.z       std/nmap.z
std/operator.z     std/pathlib.z   std/random.z     std/shutil.z
std/statistics.z   std/string.z    std/struct.z     std/tempfile.z
std/uuid.z
```

### 11.2.3 Step 3: Project-relative modules

Files next to your entry script or in subdirectories relative to the entry script's directory.

### 11.2.4 Step 4: `ZEN_MODULE_PATH`

Directories in the `ZEN_MODULE_PATH` environment variable (colon-separated on Linux/macOS, semicolon-separated on Windows).

```
export ZEN_MODULE_PATH="/home/user/mymodules:/opt/zen/modules"
zen run app.z
```

### 11.2.5 Complete resolution order

```
1. Native (Rust) module
2. Pure-Zen .z module (std/ or project dir)
3. Project-relative subdirectory
4. ZEN_MODULE_PATH directories
```

If none match, `Z0324: module 'foo' not found` is raised.

### 11.2.6 Subdirectory imports

```
import utils.helper
from utils.helper import format_date
```

Zen looks for `utils/helper.z` relative to the entry script's directory and the search path. Nested subdirectories work to any depth:

```
import a.b.c.d   # looks for a/b/c/d.z
```

## 11.3 10.3 Module tiers: pure-Zen vs native

Modules that ship both a pure-Zen `.z` implementation and a native Rust implementation expose the native tier under `<name>.native`:

```
import math

print(math.pi)           # pure-Zen tier
print(math.native.pi)    # native Rust tier
```

### 11.3.1 Forcing tiers

**Force native tier** (for all modules that have one):

```
ZEN_NATIVE_ONLY=math zen run app.z
```

Multiple modules: `ZEN_NATIVE_ONLY=math,json,socket`

**Force pure-Zen tier:** Set `ZEN_PURE = true` at the top of a `.z` module:

```
// std/math.z
ZEN_PURE = true
```

This makes the pure-Zen tier win over the native tier for that module. The native members remain reachable as `math.native.*`.

### 11.3.2 Tier selection logic

The linker decides per-module:

| Condition | Result |
|---|---|
| `ZEN_NATIVE_ONLY` names module | Native tier (no `.z` lookup) |
| `.z` exists AND `ZEN_PURE = true` | Pure-Zen tier, native registered as `.native` |
| `.z` exists, no `ZEN_PURE` | Pure-Zen tier wins; native registered as `.native` |
| No `.z`, native exists | Native tier directly |
| Neither exists | `Z0324: module not found` |

### 11.3.3 Example: comparing tiers

```
import math

let pure_pi = math.pi
let native_pi = math.native.pi

print(pure_pi == native_pi)  # true (same value)
```

```
# Compare behavior under each tier
zen run app.z                    # pure-Zen math tier
ZEN_NATIVE_ONLY=math zen run app.z  # native math tier
```

## 11.4 10.4 `requirements.txt` and `zen py-sync`

Zen can scan your `.z` files for `pymport` statements and ensure a Python virtual environment with the needed packages.

### 11.4.1 Create `requirements.txt`

Put it in your project root (or nearest to the entry). Map modules to pip package names where they differ:

```
beautifulsoup4
mechanicalsoup
selenium
requests
lxml
```

A module name is not always the pip name (e.g., `bs4` comes from `beautifulsoup4`). The scanner reports what was found; Zen does **not** guess package names.

### 11.4.2 Run sync

```
zen py-sync
```

### 11.4.3 Behavior

1. **Scan**: recursively parses entry + transitive `.z` imports to find all `pymport` statements.
2. **Plan**: determines state (`Clean`, `NoRequirements`, `MissingVenv`, `MissingPackages`).
3. **Ensure**: creates `.zen_venv` in project root, runs `pip install -r requirements.txt` in the foreground, writes a stamp (requirements length + hash).
4. **Point**: `zen run` prepends `.zen_venv/lib/python*/site-packages` to `PYTHONPATH`.

### 11.4.4 Control flags

| Flag | Meaning |
|---|---|
| `ZEN_NO_PY_BOOTSTRAP=1` | Skip bootstrap entirely (for tests/hermetic runs). Runtime will still fail if packages are missing. |
| (none) | Normal bootstrap; if `requirements.txt` is missing and `pymport` is used, Zen reports an error (it will not auto-install). |

### 11.4.5 Example end-to-end

```
cat > requirements.txt << 'EOF'
paramiko
EOF
zen py-sync
zen run my_script.z  # uses .zen_venv
```

## 11.5 10.5 `pymport` — Python interoperability

`pymport` imports Python modules into Zen. Covered in depth in Chapter 42. Quick summary:

```
pymport json
pymport os
pymport requests
pymport bs4
pymport selenium.webdriver as webdriver
pymport urllib.request
pymport builtins
```

Calls are **positional-only** (no keyword arguments):

```
pymport json
let data = json.loads('{"a": 1}')
print(data["a"])

pymport os
print(os.getcwd())
print(os.path.join("a", "b"))
```

Foreign objects (e.g. `datetime.datetime`) are opaque handles — attributes and methods work, scalar results marshal back to Zen values:

```
pymport datetime
let now = datetime.datetime.now()
print(now.year)
print(now.isoformat())
print(builtins.len([1, 2, 3]))
```

Python exceptions become catchable Zen errors:

```
try {
    json.loads("{bad}")
} catch as e {
    print("caught: " + e.msg)
}
```

## 11.6 10.6 Writing your own modules

Create `mymod.z`:

```
// mymod.z
let VERSION = "1.0.0"

func add(a, b) {
    return a + b
}

func greet(name) {
    return "Hello, " + name
}
```

Import it:

```
import mymod

print(mymod.VERSION)   # 1.0.0
print(mymod.add(2, 3)) # 5
print(mymod.greet("Zen"))  # Hello, Zen
```

Or import specific names:

```
from mymod import add, greet
print(add(2, 3))
print(greet("Zen"))
```

### 11.6.1 Module execution on import

When a module is imported, its top-level code runs once. The module's exports become its dictionary. Subsequent imports of the same module return the cached module (no re-execution):

```
// counter.z
let count = 0
func inc() { count = count + 1; return count }
```

```
import counter
import counter   // does NOT re-run; count is still 0
print(counter.inc())  # 1
print(counter.inc())  # 2
```

## 11.7 10.7 Module search path

Zen searches these directories for modules:

1. **The directory of the entry script.**
2. **`std/`** (the standard library).
3. **Project-relative subdirectories** (relative to entry script).
4. **Directories in `ZEN_MODULE_PATH`** (colon-separated on Linux/macOS, semicolon-separated on Windows).

```
export ZEN_MODULE_PATH="/home/user/mymodules:/opt/zen/modules"
zen run app.z
```

### 11.7.1 Example: custom module path

```
# Create a module in a custom directory
mkdir -p /home/user/mymodules
echo 'func hello() { return "hi from custom" }' > /home/user/mymodules/custom.z

ZEN_MODULE_PATH="/home/user/mymodules" zen run -e 'import custom; print(custom.hello())'
# hi from custom
```

## 11.8 10.8 Circular imports

Zen handles circular imports by returning partially-initialized modules. If module A imports B and B imports A, B will see A's bindings defined so far. Avoid circular imports where possible.

```
// a.z
import b
let a_val = 1
func a_func() { return b.b_val }

// b.z
import a
let b_val = 2
func b_func() { return a.a_val }  // a.a_val may be nil if a hasn't finished
```

## 11.9 10.9 `include` directive

Zen has an `include` keyword (token `KwInclude`) that inlines another file's contents at the point of the directive. This is different from `import` — it's textual inclusion, not module scoping.

```
include "utils.z"
```

Use `include` for code generation or when you want to literally splice file contents. Prefer `import` for normal modularity.

## 11.10 10.10 Common pitfalls

- **`import` is not hoisted.** Import at the top of the file before use.
- **Module names must be valid identifiers.** `import foo.bar` resolves `foo/bar.z`.
- **`pymport` is positional-only.** No keyword arguments.
- **`requirements.txt` maps module names to pip names.** `bs4` → `beautifulsoup4`.
- **Native modules have no `.z` source.** `import fs` uses the Rust tier directly.
- **Circular imports** return partially-initialized modules; avoid them.
- **`ZEN_MODULE_PATH`** must be set before running `zen`, not inside the script.

## 11.11 10.10 Quick reference

```
import fs
import os.path as path
from json import loads, dumps
pymport requests
import utils.helper

fs.exists("file.z")
path.join("a", "b")
loads('{"a": 1}')
requests.get("https://example.com")

zen py-sync           # install Python deps
ZEN_NO_PY_BOOTSTRAP=1 # skip bootstrap
```

```
math.native.pi        # native tier
ZEN_NATIVE_ONLY=math  # force native tier
```

## 11.12 10.11 Exercises

1. Create a `utils.z` module with `format_date`, `slugify`, and `truncate` functions. Import and use them from `main.z`.
2. Create `requirements.txt` with `beautifulsoup4`, run `zen py-sync`, and write a script that imports `bs4` and parses HTML.
3. Write a script that uses both the pure-Zen `math` tier and `math.native` and compares results.
4. Set `ZEN_MODULE_PATH` to a custom directory, place a module there, and import it.
5. Use `zen check` and `zen lint` on a project with intentional errors and unused variables.
6. Create two modules that import each other (circular) and observe the behavior.
7. Use `include` to splice a file's contents and compare with `import`.

## 11.13 10.12 Modules and imports — complete reference

Zen's module system maps `import a.b.c` to `a/b/c.z` or `a/b/c/mod.z`. The `mod.z` file is the canonical package entry point; `main.z` is deprecated.

### 11.13.1 Import forms

| Form | Meaning |
|---|---|
| `import module` | bind module namespace |
| `import module as alias` | shorten the name |
| `import mod.sub.mod as alias` | nested module |
| `from module import name, name as alias` | bind specific members |
| `from module import *` | import all public members |

### 11.13.2 Visibility

`pub` prefix on `func`, `var`, `const`, `class` exports the symbol. Non-`pub` is private to the file.

```
# utils.z
pub func add(a, b) { return a + b }   # exported
func helper() { return 42 }            # private

# main.z
import utils
print(utils.add(2, 3))   # 5
# utils.helper()  # error: not exported
```

### 11.13.3 Module resolution order

1. Standard library modules (e.g. `math`, `json`, `socket`)
2. Local files relative to the importing file / project root
3. `ZEN_PATH` entries (colon-separated)
4. Site-packages / installed modules

### 11.13.4 Circular imports

Circular imports can cause partially-initialized modules. Design modules as DAGs; put shared definitions in a third module imported by both.

```
# a.z imports b.z; b.z imports a.z  -> avoid
```

### 11.13.5 Reloading and caching

Modules are typically loaded once and cached. Re-importing returns the cached module. To pick up changes, restart the process.

### 11.13.6 `ZEN_PATH`

Colon-separated list of directories searched for user modules:

```
export ZEN_PATH="$HOME/zen/lib:/opt/zen/modules"
zen run app.z
```

### 11.13.7 `__name__`

A module's `__name__` (if exposed) is `"__main__"` when run directly and the module path when imported. Use it for guard clauses:

```
# entry.z
func main() { print("running directly") }
if __name__ == "__main__" { main() }
```

### 11.13.8 Packages

A directory with `mod.z` is a package:

```
mypkg/
    mod.z        # package entry point
    sub/
        mod.z    # mypkg.sub
```

Import: `import mypkg`, `import mypkg.sub`.

### 11.13.9 stdlib module list (selected)

- math, random, string, collections, json, datetime/time
- pathlib, os, re, base64, binascii, hashlib, crypto

- socket, http, requests, dns, url
- threading, channels, mutexes
- ffi, sys, pymport

See Appendix D for the full index.

## 11.14 10.13 Writing your own modules

```
# geometry.z
pub const PI = 3.141592653589793

pub func circle_area(r) {
    return PI * r * r
}

pub func circle_circumference(r) {
    return 2 * PI * r
}

func _internal() { return "not exported" }
```

```
import geometry
print(geometry.circle_area(5))   # 78.5398...
```

### 11.14.1 Module docstrings / documentation

Place a comment at the top of each `.z` file describing purpose, exports, and usage. Some tooling may extract leading comments as docs.

## 11.15 10.14 Import performance

- Import once at the top of the file.
- Avoid imports inside hot loops.
- Local (relative) imports resolve faster than deep stdlib lookups.

## 11.16 10.15 Common import mistakes

1. **Missing `pub`** means the symbol is unavailable to importers.
2. **Wrong path:** `import foo.bar` looks for `foo/bar.z` or `foo/bar/mod.z`.
3. **Name shadowing:** naming a local file `math.z` shadows stdlib `math`.
4. **Circular imports** lead to partially initialized modules.
5. **Assuming re-import re-runs code.** Modules are cached.
6. **`from module import *`** brings only `pub` members; be explicit.

## 11.17 10.16 Exercises

1. Create a `utils.z` with `pub func` helpers and import it in `main.z`.
2. Build a `geometry` package with `mod.z` and `sub/area.z`.

3. Demonstrate `import module as alias` shortening a long name.
4. Use `from module import name` to bind `math.sin` locally.
5. Show that two files importing the same module share state.
6. Write a module with a `pub const` and read it from another file.
7. Demonstrate `if __name__ == "__main__"` guard behavior.
8. Set `ZEN_PATH` and import a module from a non-default directory.
9. Create a name-shadowing file `json.z` and observe stdlib being hidden.
10. Write a module that exports both functions and constants; document each.

# 12 Chapter 11: Error Handling

Zen is “safe by default”: runtime problems become catchable errors instead of crashes. This chapter covers `try`/`catch`/`throw`/`finally` exhaustively, error fields, typed catch clauses, error types, and the interaction between `exit()` and `finally`.

## 12.1 11.1 Catching errors: `try` / `catch`

Wrap risky code in `try`. If anything inside raises an error, execution jumps to the matching `catch` block.

```
try {
    let x = 1 / 0        # raises DivideByZero
    print("never reached")
} catch {
    print("division failed")
}
# prints: division failed
```

### 12.1.1 Bare `catch`

A bare `catch` handles **any** error. If the `try` body completes normally, `catch` is skipped:

```
var caught = false
try {
    var bad = 1 / 0
    caught = bad == 0
} catch {
    caught = true
}
print(caught)  # true

var ok = true
try {
    var five = 10 / 2
    ok = five == 5
} catch {
    ok = false
}
print(ok)  # true
```

## 12.2 11.2 Reading the real failure: `catch as e`

Without a binding, you only know *that* something failed, not *what*. Bind the error with `catch as e` to inspect it:

```
try {
    let y = 1 / 0
} catch as e {
```

```
    print(str(e))   # DivideByZero: division by zero
}
```

`str(e)` renders the real message. You can also concatenate a string with the error directly:

```
try {
    let y = 1 / 0
} catch as e {
    print("caught: " + e)   # caught: DivideByZero: division by zero
}
```

## 12.3 11.3 Error fields: `e.msg`, `e.code`, `e.val`

A caught error is an object with three readable fields:

| Field | Meaning | Example |
|---|---|---|
| `e.msg` | Human-readable message | `"DivideByZero: division by zero"` |
| `e.code` | Numeric error code | `0` |
| `e.val` | Extra attached value (often `nil`) | `nil` |

```
try {
    let a = [1][5]   # IndexError
} catch as e {
    print(e.msg)   # IndexError: index 5 out of bounds for length 1
    print(e.code)  # 0
    print(e.val)   # nil
}
```

## 12.4 11.4 String + error concatenation

```
try {
    let y = 1 / 0
} catch as e {
    print("caught: " + e)   # caught: DivideByZero: division by zero
}
```

## 12.5 11.5 Catching specific error types

You can add typed clauses to handle only certain errors. Unmatched clauses fall through to the next one:

```
try {
    risky()
```

```
} catch TypeError {
    print("bad type")
} catch UserError {
    print("user raised")
} catch {
    print("everything else")
}
```

A clause whose type does not match lets the error keep propagating outward:

```
try {
    throw "z"
} catch TypeError {   # doesn't match UserError
    print("t")        # skipped
} catch UserError {
    print("u")        # catches it
}
print("u")  # u
```

## 12.6 11.6 Raising errors: `throw`

Use `throw` to raise your own errors. Strings become `UserError` messages:

```
var msg = ""
try {
    throw "custom failure"
} catch {
    msg = "handled"
}
print(msg)  # handled

var tm = ""
try {
    throw "custom boom"
} catch as e {
    tm = e.msg
}
print(tm)  # UserError: custom boom
```

`throw` accepts any value:

```
throw { code: 42, detail: "retry later" }
```

The value is stringified into `e.msg`; `e.val` is `nil` for `throw`:

```
try {
    throw { code: 42 }
} catch as e {
    print(e.msg)   # UserError: { code: 42, detail: ... }
```

```
    print(e.val)   # nil
}
```

## 12.7 11.7 Cleanup: `finally`

A `finally` block runs whether the `try` succeeded or failed, and whether or not `catch` handled the error. It must come **after** any `catch` clauses.

```
var log = ""
try {
    throw "boom"
} catch {
    log = log + "c"
} finally {
    log = log + "f"
}
print(log)  # cf
```

```
# finally also runs when nothing failed
var log2 = ""
try {
    log2 = "b"
} finally {
    log2 = log2 + "f"
}
print(log2)  # bf
```

## 12.8 11.8 `exit()` and `finally`

`exit(code)` terminates the process immediately with the given code. Because `exit` terminates the process, `finally` blocks do **not** run after `exit()`:

```
try {
    exit(3)
} finally {
    print("this will NOT print")   # process already exited
}
```

```
# exit inside catch does not run finally
try {
    throw "boom"
} catch {
    exit(1)
} finally {
    print("this will NOT print")   # process exited in catch
}
```

If you need cleanup before exiting, do it before calling `exit()`:

```
try {
    risky()
} catch as e {
    cleanup_resources()   # explicit cleanup before exit
    exit(1)
}
```

## 12.9 11.9 Nested handlers

Inner handlers run before outer ones:

```
var order = ""
try {
    try { throw "inner" } catch { order = order + "i" }
    throw "outer"
} catch { order = order + "o" }
print(order)  # io
```

## 12.10 11.10 Errors from callees

Errors raised inside a function are catchable by the caller:

```
func deep() {
    return [1][5]
}
var hit9 = false
try {
    deep()
} catch IndexError {
    hit9 = true
}
print(hit9)  # true
```

## 12.11 11.11 Error types

Zen's built-in error types:

| Type | Meaning | Typical cause |
|---|---|---|
| `TypeError` | Wrong type for an operation | `"a" + 1`, `nil.field` |
| `IndexError` | Sequence index out of bounds | `[1][5]`, `"ab"[5]` |
| `KeyError` | Missing dict key | `d["missing"]` |
| `DivideByZero` | Division by zero | `1 / 0`, `1 % 0` |
| `ValueError` | Inappropriate value | `int("not a number")` |
| `NativeError` | Raised by a native module | fs, socket, etc. |

| Type | Meaning | Typical cause |
|---|---|---|
| `ForeignAttribute` | Failed foreign attribute access | `py_obj.bad_attr` |
| `PyImportError` | Python import failure | `pymport nonexistent` |
| `UserError` | Raised by `throw` | `throw "msg"` |

## 12.12 11.12 Full error-code reference

Every error carries a code `Z####`. See Appendix B for the complete list. Common runtime codes:

| Code | Type | Meaning |
|---|---|---|
| `Z0401` | TypeError | Type mismatch |
| `Z0402` | IndexError | Index out of bounds |
| `Z0403` | KeyError | Key not found |
| `Z0404` | DivideByZero | Division by zero |
| `Z0405` | ValueError | Value error |
| `Z0406` | Attribute error | Attribute access |
| `Z0407` | NativeError | Native module error |
| `Z0408` | ArgumentCount | Wrong arg count |
| `Z0409` | HandleError | Handle error |
| `Z0410` | IOMismatch | I/O mismatch |
| `Z0411` | NativeError | Native module error (general) |
| `Z0414` | PyImportError | Python import failed |

Use `zen explain Z0401` to see the full explanation for any code.

## 12.13 11.13 Common pitfalls

- **`e` only exists inside its `catch` block.** Don't reference it outside.
- **`finally` goes after `catch`, not between `try` and `catch`.**
- **A bare `catch` swallows everything.** Use typed clauses when you want unexpected errors to propagate.
- **`str(e)` is the readable message.** Never rely on `typeof(e)` (it's just `"error"`) — use `e.msg` instead.
- **`exit()` skips `finally`**. Do cleanup before calling `exit()`.
- **`throw` with a string** creates `UserError: <message>`; `e.val` is `nil`.
- **`throw` with an object** stringifies it into `e.msg`; `e.val` is `nil`.

## 12.14 11.14 Quick reference

```
try {
    # risky code
} catch as e {
    print(str(e))   # real message
```

```
    print(e.msg)    # same as str(e)
    print(e.code)   # numeric code
    print(e.val)    # attached value
} catch TypeError {
    # only type errors
} catch UserError {
    # only user-thrown errors
} catch {
    # everything else
} finally {
    # always runs (unless exit() was called)
}

throw "something went wrong"   # raise your own error
throw { code: 42 }             # throw any value

try {
    exit(1)   # process exits; finally does NOT run
}
```

## 12.15 11.15 Exercises

1. Write `safe_divide(a, b)` that returns `nil` on division by zero instead of crashing.
2. Use `try`/`catch` to read a file that might not exist, returning a default value.
3. Write a function that uses `try`/`catch`/`finally` to guarantee a cleanup message prints regardless of success or failure.
4. Demonstrate that `exit()` skips `finally`.
5. Write `parse_int_or_default(s, default)` that uses `try`/`catch` around `int(s)`.
6. Create a custom error type by throwing an object and inspecting `e.msg`.
7. Write a nested try/catch where the inner catch handles one error type and the outer catch handles another.

## 12.16 11.12 Error handling — complete reference

Zen uses `try`/`catch`/`finally` and `throw`. Errors carry coded diagnostics `Z01xx`–`Z05xx` and `zen explain` shows details.

### 12.16.1 Syntax

```
try {
    # code that may fail
} catch [Type] as name {
    # handle specific error type
} catch as name {
    # handle any error
} finally {
```

```
    # always runs (cleanup)
}
```

Multiple `catch` clauses are checked in order; the first matching type wins.

### 12.16.2 Throwing

```
throw "something went wrong"
throw Error("detail", code: "Z0308")   # if structured errors supported
throw ValueError("bad input")
```

Catching:

```
try {
    int("not a number")
} catch as e {
    print(e.msg)      # error message
    print(e.code)     # e.g. Z0308, if available
}
```

### 12.16.3 Error codes Z01xx–Z05xx

| Range | Category | Examples |
|---|---|---|
| Z01xx | Lexer / syntax | invalid token, bad number literal |
| Z02xx | Parser | unexpected token, unbalanced braces |
| Z03xx | Runtime | type mismatch, division by zero, index out of range |
| Z04xx | I/O / system | file not found, network, permission |
| Z05xx | FFI / interop | symbol not found, pymport failure, handle error |

Use `zen explain Z0308` to see the full explanation and suggested fix.

```
zen explain Z0308
```

### 12.16.4 `finally` semantics

`finally` runs whether the `try` block completes normally, throws, returns, breaks, or continues. It is the place for resource cleanup.

```
func read_file(path) {
    let f = open(path)
    try {
```

```
        return f.read()
    } catch as e {
        print("read failed: " + e.msg)
        return nil
    } finally {
        f.close()
    }
}
```

Even `return` inside `try` still runs `finally` before returning.

### 12.16.5 Best practices

1. **Catch specific errors first**, broad `catch as e` last.
2. **Don't swallow errors silently** — log or rethrow.
3. **Use `finally` for cleanup** (close files, sockets, locks).
4. **Throw early, catch late** — validate at boundaries, handle at policy level.
5. **Don't use exceptions for control flow** in hot loops.
6. **Include context** when rethrowing: `throw "load_config failed: " + e.msg`.
7. **Document which errors a function throws.**

### 12.16.6 Result-style pattern (no exceptions)

For fallible operations, return `nil` or a `#{ok, value}` / `#{ok, error}` dict:

```
func safe_div(a, b) {
    if b == 0 { return #{ok: false, error: "division by zero"} }
    return #{ok: true, value: a / b}
}

let result = safe_div(10, 0)
if !result["ok"] { print(result["error"]) }
```

### 12.16.7 Error chaining (pymport)

Python errors raised under `pymport` are chained under Zen frames, preserving the original traceback. See Chapter 42.

## 12.17 11.13 Common runtime errors

| Error | Cause | Fix |
|---|---|---|
| Z0301 TypeError | wrong type for operation | check types before operating |
| Z0302 IndexError | index out of range | check `len` first |
| Z0303 KeyError | missing dict key | use `in` or `??` |
| Z0304 DivideByZero | dividing by zero | check divisor != 0 |
| Z0305 ValueError | bad value (e.g. `int("x")`) | validate input |

| Error | Cause | Fix |
|---|---|---|
| Z0401 FileNotFound | file doesn’t exist | check before open |
| Z0402 PermissionDenied | insufficient rights | fix permissions |

## 12.18 11.14 Defensive programming

Validate inputs at function boundaries:

```
func process_user(user) {
    if typeof(user) != "dict" { throw "user must be a dict" }
    if !("name" in user) { throw "user missing 'name'" }
    if typeof(user["name"]) != "string" { throw "name must be string" }
    # ... proceed safely
}
```

Use assertions for internal invariants:

```
assert len(xs) > 0, "xs must not be empty"
```

## 12.19 11.15 Debugging with `zen explain`

```
zen explain Z0308
zen explain Z0401
```

Each code prints: meaning, common causes, and a suggested fix.

## 12.20 11.16 Exercises

1. Write `safe_div(a, b)` returning `nil` on division by zero.
2. Write `safe_int(s)` returning `nil` on parse failure.
3. Implement `read_first_line(path)` handling FileNotFound and empty file.
4. Build a `validate_age(age)` throwing if age < 0 or > 150.
5. Demonstrate `finally` running after `return`.
6. Write `retry(fn, n)` catching errors and retrying up to `n` times.
7. Implement `partition(xs, pred)` without exceptions.
8. Create a function that throws different error types based on input.
9. Show `zen explain` output for Z0301 and Z0401.
10. Write a batch processor that logs errors per item but continues.

# 13 Chapter 12: Classes and Objects

Zen supports class-based object-oriented programming with fields, methods, constructors (`init`), inheritance (`extends`), `super`, `this`, and bound methods. This chapter covers OOP in depth.

## 13.1 12.1 Defining a class

```
class Person {
    var name
    var age

    func init(name, age) {
        this.name = name
        this.age = age
    }

    func greet() {
        return "Hi, I'm " + this.name
    }

    func birthday() {
        this.age = this.age + 1
    }
}
```

- `class Name { ... }` defines a class.
- `var name` / `var age` declare fields (mutable).
- `func init(...)` is the constructor, called by `new Person(...)`.
- `this` refers to the current instance.

## 13.2 12.2 Creating instances

```
let p = new Person("Zen", 30)
print(p.name)    # Zen
print(p.age)     # 30
print(p.greet()) # Hi, I'm Zen

p.birthday()
print(p.age)     # 31
```

## 13.3 12.3 Fields and methods

Fields are accessed with `this.field` inside methods and `instance.field` outside. Methods are called with `instance.method()`.

```
class Counter {
    var count

    func init() {
        this.count = 0
    }

    func inc() {
        this.count = this.count + 1
    }

    func get() {
        return this.count
    }
}

let c = new Counter()
c.inc()
c.inc()
print(c.get())  # 2
```

## 13.4 12.4 Inheritance: `extends` and `super`

```
class Employee extends Person {
    var salary

    func init(name, age, salary) {
        super.init(name, age)
        this.salary = salary
    }

    func greet() {
        return super.greet() + " and I earn " + str(this.salary)
    }
}

let e = new Employee("Zen", 30, 100000)
print(e.name)       # Zen
print(e.age)        # 30
print(e.salary)     # 100000
print(e.greet())    # Hi, I'm Zen and I earn 100000
```

- `extends Parent` establishes inheritance.
- `super.init(...)` calls the parent constructor.
- `super.method()` calls the parent's overridden method.
- Child classes inherit all parent fields and methods.

## 13.5 12.5 Method overriding

```
class Shape {
    func area() {
        return 0
    }
}

class Circle extends Shape {
    var radius

    func init(radius) {
        this.radius = radius
    }

    func area() {
        return 3.14159 * this.radius * this.radius
    }
}

let c = new Circle(5)
print(c.area())  # 78.53975
```

## 13.6 12.6 `this` binding

`this` refers to the instance on which a method was called. When a method is extracted as a bound method, `this` is preserved:

```
class Counter {
    var count

    func init() {
        this.count = 0
    }

    func inc() {
        this.count = this.count + 1
    }
}

let c = new Counter()
let inc = c.inc   # bound method
inc()
inc()
print(c.count)   # 2
```

## 13.7 12.7 Common patterns

### 13.7.1 Data class

```
class Point {
    var x
    var y

    func init(x, y) {
        this.x = x
        this.y = y
    }

    func distance_to(other) {
        let dx = this.x - other.x
        let dy = this.y - other.y
        return (dx * dx + dy * dy) ** 0.5
    }

    func __str__() {
        return "Point(" + str(this.x) + ", " + str(this.y) + ")"
    }
}

let p1 = new Point(0, 0)
let p2 = new Point(3, 4)
print(p1.distance_to(p2))  # 5.0
print(p1)                  # Point(0, 0)
```

### 13.7.2 Singleton

```
class Config {
    var instance = nil

    func init() {
        if Config.instance != nil {
            return Config.instance
        }
        Config.instance = this
        this.settings = {}
    }

    func get(key) {
        return this.settings[key]
    }

    func set(key, value) {
```

```
        this.settings[key] = value
    }
}

let c1 = new Config()
let c2 = new Config()
c1.set("theme", "dark")
print(c2.get("theme"))  # dark
print(c1 == c2)         # true (same instance)
```

## 13.8 12.8 Common pitfalls

- **`this` is required** to access fields and other methods inside a class. Bare `name` refers to a local variable, not a field.
- **`super.init` must be called** in a child constructor if the parent has required fields, or those fields won't be initialized.
- **Fields are per-instance.** Class-level (static) data is not directly supported; use a module-level variable instead.
- **Methods are bound** when extracted — `this` follows the instance, not the call site.

## 13.9 12.9 Quick reference

```
class Name extends Parent {
    var field

    func init(args) {
        super.init(args)
        this.field = value
    }

    func method(arg) {
        return this.field + arg
    }
}

let obj = new Name(args)
obj.field
obj.method(arg)
super.method()   # from within a child method
```

## 13.10 12.10 Exercises

1. Create a `BankAccount` class with `deposit`, `withdraw`, and `get_balance` methods. `withdraw` should reject amounts exceeding the balance.

2. Create a `Shape` base class with `area()` and `perimeter()` methods, then `Rectangle` and `Circle` subclasses.
3. Implement a `Stack` class using a list internally, with `push`, `pop`, `peek`, and `is_empty`.
4. Implement a `Queue` class with `enqueue` and `dequeue`.
5. Create an `Animal` base class with `speak()`, then `Dog` and `Cat` subclasses that override `speak()`.

## 13.11 12.12 Classes and objects — complete reference

Zen supports classes with single inheritance, constructors, methods, instance fields, and `pub` visibility.

```
class Point {
    var x = 0.0
    var y = 0.0

    func init(x, y) {
        self.x = x
        self.y = y
    }

    func distance_to(other) {
        let dx = self.x - other.x
        let dy = self.y - other.y
        return (dx * dx + dy * dy) ** 0.5
    }

    func __str__() {
        return "Point(${self.x}, ${self.y})"
    }
}

let p = new Point(3.0, 4.0)
print(p.x, p.y)          # 3.0 4.0
print(p.distance_to(new Point(0.0, 0.0)))   # 5.0
print(p)                 # Point(3.0, 4.0)
```

### 13.11.1 Class members

| Member | Declaration | Purpose |
|---|---|---|
| instance field | `var name = default` | per-object state |
| method | `func name(params) { ... }` | behavior on self |
| constructor | `func init(params) { ... }` | runs on `new` |
| static/class method | `pub func name(...) { ... }` | namespace member (no self) |

### 13.11.2 `self` / `this`

Inside a method, `self` is the receiver. `this` is an accepted alias. Use `self.field` to read/write fields and `self.method()` to call siblings.

```
class Counter {
    var n = 0
    func inc() { self.n += 1 }
    func value() { return self.n }
}
```

### 13.11.3 Instantiation

`new ClassName(args)` calls `init`. Fields default to their declared values when not provided in `init`.

```
let c = new Counter()
print(c.value())   # 0
c.inc()
print(c.value())   # 1
```

### 13.11.4 Single inheritance

```
class Animal {
    var name = ""
    func init(name) { self.name = name }
    func speak() { return "..." }
}

class Dog extends Animal {
    func speak() { return self.name + " says woof" }
}

let d = new Dog("Rex")
print(d.speak())   # Rex says woof
```

Call the parent constructor explicitly if needed (`super` where supported; otherwise re-initialize parent fields in `init`).

### 13.11.5 Method overriding

Subclasses override methods by redefining them with the same name.

```
class Cat extends Animal {
    func speak() { return self.name + " says meow" }
}
```

### 13.11.6 Visibility (pub)

pub members are accessible outside the module; non-pub are private to the file.

```
class BankAccount {
    var balance = 0.0
    pub func deposit(amount) { self.balance += amount }
    func _apply_fee() { self.balance -= 1.0 }   # private
}
```

### 13.11.7 __str__ and representation

Define __str__() for a human-readable string form used by print/string interpolation. If absent, a default representation is used.

### 13.11.8 Object equality

By default, == on instances compares identity (same object). Override __eq__ for value equality if desired.

```
func __eq__(other) {
    return self.x == other.x and self.y == other.y
}
```

### 13.11.9 Instance vs class state

Fields declared with var at class body level become instance fields (each new gets its own). Class-level shared state (if supported) uses module-level variables or shared.

### 13.11.10 Common patterns

```
class Money {
    var amount = 0.0
    var currency = "USD"
    func init(amount, currency) { self.amount = amount; self.currency = currency }
    func add(other) {
        if self.currency != other.currency { throw "currency mismatch" }
        return new Money(self.amount + other.amount, self.currency)
    }
}
```

#### 13.11.10.1 Value object (immutable-ish)

```
class Emailer {
    var host = ""
    func init(host) { self.host = host }
```

```
    func send(to, body) { print("sending to ${to} via ${self.host}") }
}
```

#### 13.11.10.2 Service object

## 13.12 12.13 Common OOP mistakes

1. **Forgetting `self.`** on field access inside methods.
2. **Not calling parent init** when overriding (fields stay at defaults).
3. **Mutating shared mutable default** across instances (e.g. `var items = []` shared — be careful; each `new` should initialize its own).
4. **Relying on == for value equality** without overriding `__eq__`.
5. **Overusing inheritance** when composition is clearer.
6. **Exposing internal state** via public fields; prefer methods.

## 13.13 12.14 Exercises

1. Create a `Rectangle` class with `width`, `height`, `area()`, `perimeter()`.
2. Create a `BankAccount` class with `deposit`, `withdraw`, `balance`, and overdraft protection.
3. Build an `Animal` hierarchy: `Dog`, `Cat`, `Bird` with `speak()`.
4. Implement a `Stack` class using a list (`push`, `pop`, `peek`, `is_empty`).
5. Implement a `Queue` class (`enqueue`, `dequeue`, `is_empty`).
6. Create a `Fraction` class with `__eq__` and `__str__`.
7. Build a `Shape` base class with `area()`; subclasses `Circle`, `Rectangle`.
8. Implement a `Point` class with `__add__` vector addition if supported.
9. Write a `Logger` class with `info`, `warn`, `error` methods and a level.
10. Create a `Config` class wrapping a dict with typed getters.

# 14 Chapter 13: File I/O

Zen's `fs` module provides file system operations: reading, writing, creating directories, checking existence, removing, copying, moving, and more. The `os` module adds path manipulation. This chapter covers file I/O in depth.

## 14.1 13.1 The `fs` module

```
import fs

# Read a file
let content = fs.read("hello.txt")

# Write a file
fs.write("out.txt", "Hello, file!")

# Check existence
if fs.exists("data.json") {
    print("exists")
}

# Create directories
fs.mkdirs("a/b/c")

# Remove
fs.remove("temp.txt")
fs.rmtree("a/b/c")

# Check type
print(fs.is_dir("folder"))
print(fs.is_file("file.txt"))

# Path operations
print(fs.join("a", "b", "c"))   # a/b/c
print(fs.basename("/a/b/c.txt")) # c.txt
print(fs.dirname("/a/b/c.txt"))  # /a/b

# Copy and move
fs.copy("src.txt", "dst.txt")
fs.move("old.txt", "new.txt")

# File size
print(fs.size("file.txt"))   # bytes

# List directory
for name in fs.list(".") {
    print(name)
```

```
}
```

### 14.1.1 `fs.read` vs `fs.read_binary`

```
# Text read (returns string)
let text = fs.read("README.md")

# Binary read (returns bytes)
let data = fs.read_binary("image.png")
```

### 14.1.2 `fs.write` vs `fs.write_binary`

```
# Text write
fs.write("note.txt", "hello")

# Binary write
fs.write_binary("out.bin", bytes_data)
```

## 14.2 13.2 The `os` module

```
import os

print(os.getcwd())          # current working directory
print(os.getenv("HOME"))    # environment variable
print(os.getpid())          # process ID

# Path manipulation
print(os.path.join("a", "b"))          # a/b
print(os.path.basename("/a/b/c.txt"))  # c.txt
print(os.path.dirname("/a/b/c.txt"))   # /a/b
print(os.path.exists("/tmp/foo"))      # true/false
```

## 14.3 13.3 The `pathlib` module (pure-Zen)

```
import pathlib

let p = pathlib.Path("/usr/bin/zen")
print(p.name)        # zen
print(p.stem)        # zen
print(p.suffix)      # (empty)
print(p.parent)      # /usr/bin
print(p.exists())    # true/false
print(p.is_file())   # true/false
print(p.is_dir())    # true/false
```

```
# Read / write
let content = p.read_text()
p.write_text("new content")

# Iterate directory
for child in pathlib.Path(".").iterdir() {
    print(child.name)
}
```

## 14.4 13.4 Reading and writing text files

```
import fs

# Read entire file
let content = fs.read("input.txt")
print(content)

# Read line by line
let lines = fs.read("input.txt").split("\n")
for line in lines {
    print(line)
}

# Write entire file
fs.write("output.txt", "Line 1\nLine 2\nLine 3\n")

# Append (read + write)
let existing = fs.exists("log.txt") ? fs.read("log.txt") : ""
fs.write("log.txt", existing + "new entry\n")
```

## 14.5 13.5 Binary files

```
import fs

# Read binary
let data = fs.read_binary("image.png")
print(len(data))   # byte count

# Write binary
fs.write_binary("copy.png", data)
```

## 14.6 13.6 Temporary files

```
import tempfile

let tmp = tempfile.mktemp()
fs.write(tmp, "temporary data")
let content = fs.read(tmp)
fs.remove(tmp)

# Or use a context
let dir = tempfile.mkdtemp()
# ... use dir ...
fs.rmtree(dir)
```

## 14.7 13.7 The `with` statement (auto-scoping)

Zen's `with` statement provides automatic resource management:

```
import fs

with fs.open("data.txt", "r") as f {
    let content = f.read()
    print(content)
}
# f is automatically closed here
```

```
with fs.open("out.txt", "w") as f {
    f.write("hello from with")
}
```

## 14.8 13.8 Common patterns

### 14.8.1 Copy a file

```
import fs

fs.copy("source.txt", "destination.txt")
```

### 14.8.2 Move a file

```
fs.move("old_name.txt", "new_name.txt")
```

### 14.8.3 Delete a file safely

```
if fs.exists("temp.txt") {
    fs.remove("temp.txt")
}
```

### 14.8.4 Walk a directory tree

```
import fs

func walk(dir) {
    for name in fs.list(dir) {
        let path = fs.join(dir, name)
        if fs.is_dir(path) {
            walk(path)
        } else {
            print(path)
        }
    }
}
walk(".")
```

### 14.8.5 Read CSV (simple)

```
import fs

let lines = fs.read("data.csv").split("\n")
let header = lines[0].split(",")
for line in lines[1:] {
    if line == "" { continue }
    let values = line.split(",")
    let row = {}
    for i in 0..len(header) {
        row[header[i]] = values[i]
    }
    print(row)
}
```

## 14.9 13.9 Common pitfalls

- **`fs.read` returns a string.** For binary data, use `fs.read_binary`.
- **`fs.write` overwrites.** It does not append. Read-modify-write to append.
- **Directories must exist before writing.** Use `fs.mkdirs` first.
- **Paths use `/`** even on Windows (Zen normalizes them).
- **`fs.exists` returns `false` for broken symlinks.**

## 14.10 13.10 Quick reference

```
import fs
import os
import pathlib
```

```
fs.read("file.txt")           # string
fs.read_binary("file.bin")    # bytes
fs.write("file.txt", "data")
fs.write_binary("file.bin", bytes)
fs.exists("file.txt")
fs.is_dir("folder")
fs.is_file("file.txt")
fs.mkdirs("a/b/c")
fs.remove("file.txt")
fs.rmtree("dir")
fs.size("file.txt")
fs.join("a", "b")
fs.basename("/a/b/c.txt")
fs.dirname("/a/b/c.txt")
fs.copy("src", "dst")
fs.move("old", "new")
fs.list(".")                  # directory entries

os.getcwd()
os.getenv("HOME")
os.getpid()
os.path.join("a", "b")

import pathlib
pathlib.Path("file.txt").read_text()
pathlib.Path("file.txt").write_text("data")
```

## 14.11 13.11 Exercises

1. Write a function `file_size_human(path)` that returns a human-readable file size (e.g. “1.5 MB”).
2. Write a script that copies all `.txt` files from one directory to another.
3. Write `grep_file(pattern, path)` that prints lines containing `pattern`.
4. Write a file tailer that reads the last `n` lines of a file.
5. Use `with fs.open(...)` to read a file and count its words.

## 14.12 13.12 File I/O — complete reference

Zen reads/writes files with `open(path, mode)` and the `with` statement for automatic closing.

### 14.12.1 Modes

| Mode | Meaning |
|---|---|
| `"r"` | read (text) — default |
| `"w"` | write (text), truncate |

| Mode | Meaning |
|---|---|
| `"a"` | append (text) |
| `"rb"` | read binary |
| `"wb"` | write binary |
| `"ab"` | append binary |
| `"r+"` | read and write |

### 14.12.2 Reading

```
# read whole file
let f = open("data.txt", "r")
let text = f.read()
f.close()

# with statement (recommended)
with open("data.txt", "r") as f {
    let text = f.read()
}   # f closed automatically
```

### 14.12.3 Reading line by line

```
with open("data.txt", "r") as f {
    for line in f {
        print(line)
    }
}
```

### 14.12.4 Writing

```
with open("out.txt", "w") as f {
    f.write("hello\n")
    f.write("world\n")
}

# append
with open("out.txt", "a") as f {
    f.write("more\n")
}
```

### 14.12.5 Binary I/O

```
with open("image.bin", "rb") as f {
    let data = f.read()   # bytes
}
```

```
with open("copy.bin", "wb") as f {
    f.write(data)
}
```

### 14.12.6 `os/pathlib` helpers

Use `pathlib.Path` for path manipulation and `os` for filesystem operations:

```
import pathlib
import os

let p = pathlib.Path("data.txt")
print(p.exists())
print(p.is_file())
print(p.name)
print(p.stem)
print(p.suffix)

let files = os.listdir(".")   # list directory
print(os.getcwd())
os.makedirs("a/b/c", exist_ok=true)
os.remove("temp.txt")
```

### 14.12.7 File methods

| Method | Purpose |
|---|---|
| `read()` | read all contents |
| `read(n)` | read n bytes/chars |
| `readline()` | read one line |
| `readlines()` | read all lines into a list |
| `write(s)` | write string/bytes |
| `flush()` | flush buffers |
| `close()` | close the file |
| `seek(offset)` | move position |
| `tell()` | current position |

### 14.12.8 Working directory and paths

```
print(os.getcwd())          # current dir
os.chdir("/tmp")            # change dir
let abspath = pathlib.Path("data.txt").resolve()
```

### 14.12.9 Error handling

File operations raise errors (Z04xx range) on failure:

```
try {
    with open("missing.txt", "r") as f {
        print(f.read())
    }
} catch as e {
    print("cannot open: " + e.msg)
}
```

### 14.12.10 Best practices

1. Always use `with` so files close even on error.
2. Prefer binary mode for non-text data.
3. Specify encoding when reading/writing text across platforms.
4. Flush before reading what another process wrote.
5. Use absolute paths or resolve relative to a known base.
6. Close network/resources promptly.

## 14.13 13.13 Temporary files

If `tempfile` is available:

```
import tempfile
let path = tempfile.mkstemp()
# ... use path ...
os.remove(path)
```

Otherwise, create files in `os.getcwd()` or a known temp dir and clean up.

## 14.14 13.14 CSV-like parsing (manual)

```
func parse_csv(text) {
    let lines = text.splitlines()
    let header = lines[0].split(",")
    let rows = []
    for i in 1..len(lines) {
        let vals = lines[i].split(",")
        let row = #{}
        for j in 0..len(header) {
            row[header[j]] = vals[j]
        }
        rows.push(row)
    }
    return rows
}
```

## 14.15 13.15 Exercises

1. Write `read_all(path)` returning file contents or nil on error.
2. Write `write_all(path, text)` creating/overwriting a file.
3. Implement `copy_file(src, dst)` reading and writing in chunks.
4. Write `line_count(path)` counting lines without loading all.
5. Implement `grep_file(path, pattern)` printing matching lines.
6. Write `append_log(path, message)` appending a timestamped line.
7. Create `list_files(dir)` returning files (not subdirs).
8. Write `ensure_dir(path)` creating parent dirs if missing.
9. Implement `read_csv(path)` returning a list of dicts.
10. Write `find_files(dir, suffix)` recursively listing files by suffix.

# 15 Chapter 14: Memory and References

Zen uses manual-ish memory management with explicit reference counting (`retain`/`release`) rather than a garbage collector. This chapter explains how values are stored, how references work, and how to avoid leaks.

## 15.1 14.1 Value representation

Zen values are 64-bit NaN-boxed tagged unions. There are 16 language types:

- `nil`, `bool`, `int`, `float`, `string`, `bytes`
- `list`, `dict`, `set`, `tuple`
- `func`, `class`, `instance`, `cell`
- `error`, `channel`, `mutex`, `handle`
- `foreign` (Python interop)

Small values (`nil`, `bool`, `int`, `float`) are stored inline. Larger values (`string`, `list`, `dict`, etc.) are heap objects with a manual refcount.

## 15.2 14.2 Reference counting

Every heap object carries a reference count in its header:

- `retain(obj)` — increment the refcount.
- `release(obj)` — decrement the refcount; free when it reaches zero.

Zen's VM inserts `retain`/`release` automatically at assignment, function calls, and scope exit. You generally don't call them directly unless you're writing native code.

## 15.3 14.3 Ownership and aliasing

Assignment copies the reference, not the value:

```
let a = [1, 2, 3]
let b = a          # b aliases the same list
b.push(4)
print(a)           # [1, 2, 3, 4] (a sees the change!)
```

For an independent copy:

```
let b = a.copy()
b.push(4)
print(a)           # [1, 2, 3] (unchanged)
print(b)           # [1, 2, 3, 4]
```

Dicts behave the same way:

```
let a = {"x": 1}
let b = a
b["y"] = 2
print(a)           # {"x": 1, "y": 2}
```

Strings and numbers are immutable, so aliasing them is safe:

```
let a = "hello"
let b = a
# can't modify strings anyway
```

## 15.4 14.4 Closures and captured variables

Functions that capture variables from their enclosing scope create shared `cell` objects:

```
func make_counter() {
    var count = 0
    func inc() {
        count = count + 1
        return count
    }
    return inc
}

let c1 = make_counter()
let c2 = make_counter()
print(c1())  # 1
print(c1())  # 2
print(c2())  # 1 (separate capture)
```

Each `make_counter()` call creates a new `count` cell. `c1` and `c2` capture different cells.

## 15.5 14.5 Cycles and leaks

Reference counting cannot collect reference cycles. If object A references B and B references A, neither will ever be freed:

```
# Potential cycle (conceptual)
class Node {
    var next
}
let a = new Node()
let b = new Node()
a.next = b
b.next = a   # cycle: a -> b -> a
```

To avoid cycles:

- Break the cycle explicitly: `a.next = nil` when done.
- Use weak references where supported.
- Prefer tree-like (parent-owns-child) structures over graph-like ones.

## 15.6 14.6 Handles and resources

Some native resources (files, sockets, channels, mutexes) use handles — integer IDs into a resource table. When the Zen value is dropped, the handle should be closed explicitly:

```
import fs

let fd = fs.open("data.txt", "r")
let content = fd.read()
fd.close()   # explicit close
```

Or use `with` for automatic cleanup:

```
import fs

with fs.open("data.txt", "r") as f {
    let content = f.read()
}
# f is closed automatically
```

## 15.7 14.7 `with` statement

`with` guarantees cleanup when exiting a block, whether normally or via error:

```
with fs.open("out.txt", "w") as f {
    f.write("data")
    throw "oops"   # f still closes
}
```

## 15.8 14.8 Step budget (for untrusted code)

Zen supports an instruction budget to guarantee termination:

```
// From Rust embedding:
vm.set_step_budget(Some(1000000))
```

If the budget is exhausted, the run aborts with a `NativeError`. This lets you run untrusted Zen scripts safely.

## 15.9 14.9 Common pitfalls

- **Aliasing is the default.** `b = a` shares the object. Use `.copy()`.
- **Cycles leak.** Break reference cycles when done with graph structures.
- **Handles must be closed.** Files, sockets, and channels don’t close themselves unless you use `with`.
- **Strings are immutable.** Building strings in a loop with + creates a new rope each time — still O(1) per append due to rope structure, but be aware.

## 15.10 14.10 Quick reference

```
let a = [1, 2, 3]
let b = a              # alias (same object)
let c = a.copy()       # independent copy

a.push(4)              # b sees it, c doesn't

let s = "hello"        # immutable, safe to alias
let t = s

with fs.open("f.txt", "r") as f {
    let data = f.read()   # auto-close
}

vm.set_step_budget(Some(1000000))  # Rust embedding only
```

## 15.11 14.11 Exercises

1. Create two lists that alias each other, modify one, and observe the other.
2. Use `.copy()` to make an independent list and confirm the original is unchanged.
3. Write a function that takes a list and returns a new list with the same elements (without aliasing).
4. Use `with fs.open(...)` to write to a file, then throw an error inside the block — confirm the file is still closed.
5. Create a simple linked list class and manually break the cycle when disposing of it.

## 15.12 14.12 Values, references, and ownership — complete reference

Zen is a garbage-collected / reference-counted language. Values such as ints, floats, bools, and nil are small and copied by value. Lists, dicts, sets, strings, bytes, instances, and functions are heap objects referenced by handles.

### 15.12.1 Copying semantics

```
let a = [1, 2, 3]
let b = a          # b refers to the SAME list (no copy)
b.push(4)
print(a)           # [1, 2, 3, 4]  (a changed too!)
print(a is b)      # true
```

To copy a list, use `list(a)` or slice `a[:]` (shallow copy):

```
let c = list(a)
c.push(5)
print(a)           # [1, 2, 3, 4]  (unchanged)
```

```
print(a is c)      # false
```

Note: shallow copy shares nested mutable objects.

### 15.12.2 Strings are immutable

```
let s = "hello"
let t = s          # same string handle
t = "world"        # rebinding t, not mutating s
print(s)           # "hello"
```

Because strings are immutable, sharing is safe.

### 15.12.3 `is` vs `==`

- `==` : structural/value equality
- `is` : identity (same handle/object)

```
let x = [1, 2]
let y = [1, 2]
let z = x
print(x == y)   # true
print(x is y)   # false
print(x is z)   # true
```

### 15.12.4 Reference counting and lifetime

Heap objects are freed when no references remain. Cycles (e.g. two objects referencing each other) may leak unless the GC collects cycles. Avoid reference cycles in long-running programs, or break them explicitly.

```
class Node {
    var value = 0
    var parent = nil
    var child = nil
}

let a = new Node()
let b = new Node()
a.child = b
b.parent = a    # cycle: a -> b -> a
```

To break: set `a.child = nil` or `b.parent = nil` when done.

### 15.12.5 `shared` — thread-shared mutable state

`shared name = expr` creates an `Arc<Mutex<_>>`-like handle safe to share across threads. Access requires locking (often automatic via methods).

```
shared counter = 0
# threads can safely read/write counter
```

See Chapter 35–38 for concurrency details.

### 15.12.6 `let`, `var`, `const`

- `let` : immutable binding (rebind not allowed)
- `var` : mutable binding
- `const` : compile-time/module constant

```
let x = 1          # can't reassign x
var y = 2          # y = 3 OK
const MAX = 100    # fixed
```

`let` on a heap object prevents rebinding the variable but NOT mutating the object:

```
let lst = [1, 2]
lst.push(3)   # OK — mutating the object, not rebinding
# lst = [4]   # error — can't rebind let
```

### 15.12.7 Move semantics (conceptual)

Some operations "move" ownership (invalidating the source). In Zen 3.0 most heap values are copied-by-reference, so no explicit move is needed unless a function consumes the argument.

### 15.12.8 Memory leaks to avoid

1. **Reference cycles** — break them when tearing down graphs.
2. **Global caches growing without bound** — use size-limited caches/eviction.
3. **Event listeners/callbacks not removed** — they hold references.
4. **Closures capturing large objects** — the closure keeps the object alive.
5. **Open files/sockets not closed** — use `with` / `finally`.

### 15.12.9 Inspecting memory

Some builds expose `sys.getsizeof` or similar; otherwise reason about references. Use `typeof` to check if a value is a container.

```
print(typeof([1,2,3]))   # "list"
print(typeof(42))        # "int"
```

### 15.12.10 Performance tips

- Prefer `let` for bindings that don't change (signals intent, may help optimizer).
- Pass large containers by reference (they're handles — cheap to pass).
- Avoid copying large lists/dicts in hot loops.
- Reuse buffers instead of allocating repeatedly.

- Null out references to large structures when done (helps the collector).

## 15.13 14.13 Exercises

1. Demonstrate that `b = a` shares a list, and `list(a)` copies it.
2. Show that nested lists are shared after a shallow copy.
3. Write `deep_copy` for lists and dicts (manual recursion).
4. Create a reference cycle between two objects and break it.
5. Show `let` prevents rebinding but not mutation.
6. Compare == and `is` for two equal lists and two aliases.
7. Write a function that accepts a list and does NOT mutate the caller's list.
8. Demonstrate `shared` usage across two threads (concurrency chapter).
9. Write `clear_all(lst)` emptying a list in place vs rebinding.
10. Explain why `"a" is "a"` may be true (string interning) but `[] is []` is false.

# 16 Chapter 15: math Module

The `math` module provides mathematical functions and constants. It ships as a pure-Zen implementation (`std/math.z`) with a native Rust tier reachable via `math.native`. Set `ZEN_PURE = true` to prefer the pure tier; set `ZEN_NATIVE_ONLY=math` to force the native tier.

## 16.1 15.1 Constants

```
import math

print(math.pi)     # 3.141592653589793
print(math.e)      # 2.718281828459045
print(math.tau)    # 6.283185307179586
```

| Constant | Value | Meaning |
|---|---|---|
| `math.pi` | 3.141592653589793 | π, ratio of circumference to diameter |
| `math.e` | 2.718281828459045 | Euler's number, base of natural logarithm |
| `math.tau` | 6.283185307179586 | τ = 2π |

## 16.2 15.2 Rounding

| Function | Description | Example | Result |
|---|---|---|---|
| `floor(x)` | Largest int ≤ x | `math.floor(3.9)` | `3` |
| `ceil(x)` | Smallest int ≥ x | `math.ceil(3.1)` | `4` |
| `trunc(x)` | Truncate toward zero | `math.trunc(-3.9)` | `-3` |
| `round(x)` | Round to nearest | `math.round(3.5)` | `4` |

```
import math

print(math.floor(3.9))   # 3
print(math.ceil(3.1))    # 4
print(math.trunc(-3.9))  # -3
print(math.round(3.5))   # 4
print(math.round(2.5))   # 2 (banker's rounding)
print(math.floor(-3.9))  # -4 (floor toward −inf)
print(math.ceil(-3.9))   # -3 (ceil toward +inf)
```

`floor` and `ceil` always return an integer-valued float:

```
print(typeof(math.floor(3.9)))  # "float"
print(math.floor(3.9) == 3)     # true
```

## 16.3 15.3 Powers and logarithms

| Function | Description | Example | Result |
|---|---|---|---|
| `pow(x, y)` | x raised to y | `math.pow(2, 3)` | `8` |
| `sqrt(x)` | Square root | `math.sqrt(16)` | `4` |
| `exp(x)` | e raised to x | `math.exp(1)` | `2.718...` |
| `log(x)` | Natural log | `math.log(math.e)` | `1` |
| `log10(x)` | Base-10 log | `math.log10(100)` | `2` |
| `log2(x)` | Base-2 log | `math.log2(8)` | `3` |

```
import math

print(math.pow(2, 10))   # 1024
print(math.sqrt(81))     # 9
print(math.exp(0))       # 1
print(math.log(1))       # 0
print(math.log10(1000))  # 3
print(math.log2(1024))   # 10
print(math.exp(1))       # 2.718281828459045
print(math.pow(4, 0.5))  # 2
```

Edge cases: - `math.sqrt(-1)` → ValueError (negative sqrt) - `math.log(0)` → −infinity - `math.log(-1)` → nan - `math.pow(0, 0)` → 1 - `math.pow(-1, 0.5)` → nan (fractional power of negative)

## 16.4 15.4 Trigonometry

All angles in radians:

| Function | Description |
|---|---|
| `sin(x)` | Sine |
| `cos(x)` | Cosine |
| `tan(x)` | Tangent |
| `asin(x)` | Arcsine (returns radians) |
| `acos(x)` | Arccosine (returns radians) |
| `atan(x)` | Arctangent (returns radians) |
| `atan2(y, x)` | Arctangent of y/x (correct quadrant) |
| `degrees(x)` | Radians → degrees |
| `radians(x)` | Degrees → radians |

```
import math

print(math.sin(math.pi / 2))   # 1
print(math.cos(0))             # 1
print(math.tan(math.pi / 4))   # 1
```

```
print(math.degrees(math.pi))   # 180
print(math.radians(180))       # 3.141592653589793
print(math.asin(1))            # 1.5707963267948966 (π/2)
print(math.acos(1))            # 0
print(math.atan(1))            # 0.7853981633974483 (π/4)
print(math.atan2(1, 1))        # 0.7853981633974483 (π/4)
print(math.atan2(1, -1))       # 2.356194490192345 (3π/4, correct quadrant)
```

Domain: - `asin(x)`, `acos(x)` require $-1 \le x \le 1$, else `nan` - `atan2(0, 0)` → 0 (implementation-defined)

## 16.5 15.5 Hyperbolic functions

| Function | Description |
|---|---|
| `sinh(x)` | Hyperbolic sine |
| `cosh(x)` | Hyperbolic cosine |
| `tanh(x)` | Hyperbolic tangent |
| `asinh(x)` | Inverse hyperbolic sine |
| `acosh(x)` | Inverse hyperbolic cosine |
| `atanh(x)` | Inverse hyperbolic tangent |

```
import math

print(math.sinh(0))     # 0
print(math.cosh(0))     # 1
print(math.tanh(0))     # 0
print(math.asinh(0))    # 0
print(math.acosh(1))    # 0
print(math.atanh(0))    # 0
```

`acosh(x)` requires $x \ge 1$; `atanh(x)` requires $-1 < x < 1$.

## 16.6 15.6 Other functions

| Function | Description | Example | Result |
|---|---|---|---|
| `abs(x)` | Absolute value | `math.abs(-5)` | `5` |
| `fsum(items)` | Accurate float sum | `math.fsum([0.1, 0.2, 0.3])` | `0.6` |
| `prod(items)` | Product of items | `math.prod([1, 2, 3, 4])` | `24` |
| `min(a, b)` | Minimum of two | `math.min(3, 7)` | `3` |
| `max(a, b)` | Maximum of two | `math.max(3, 7)` | `7` |
| `hypot(x, y)` | Euclidean distance | `math.hypot(3, 4)` | `5` |

| Function | Description | Example | Result |
|---|---|---|---|
| `gcd(a, b)` | Greatest common divisor | `math.gcd(48, 18)` | `6` |
| `lcm(a, b)` | Least common multiple | `math.lcm(12, 8)` | `24` |
| `isclose(a, b)` | Approximate equality | `math.isclose(0.1+0.2, 0.3)` | `true` |

```
import math

print(math.abs(-5))       # 5
print(math.fsum([0.1, 0.2, 0.3]))  # 0.6
print(math.prod([1, 2, 3, 4]))     # 24
print(math.min(3, 7))              # 3
print(math.max(3, 7))              # 7
print(math.hypot(3, 4))            # 5
print(math.gcd(48, 18))            # 6
print(math.lcm(12, 8))             # 24
print(math.isclose(0.1 + 0.2, 0.3)) # true
print(math.gcd(0, 5))              # 5
print(math.lcm(0, 5))              # 0
```

> Note: In the pure-Zen tier, `min`/`max`/`fsum`/`prod`/`hypot` accept up to two arguments (each may be a number or a list). Calls like `math.min(1, 2, 3)` are unsupported in the pure tier.

```
# Pure tier: two args only
print(math.min([1, 2, 3]))   # 1 (list as single arg)
print(math.max([1, 2, 3]))   # 3

# But this fails in pure tier:
# math.min(1, 2, 3)  # ArgumentCount error

# Native tier may accept variadic:
ZEN_NATIVE_ONLY=math zen run app.z
```

## 16.7 15.7 Using the native tier

```
import math

# Pure-Zen tier
print(math.pi)

# Native Rust tier
print(math.native.pi)
```

Force native tier:

```
ZEN_NATIVE_ONLY=math zen run app.z
```

Force pure-Zen tier (set at top of std/math.z):

```
ZEN_PURE = true
```

## 16.8 15.8 Quick reference

```
import math

math.pi          # 3.14159...
math.e           # 2.71828...
math.tau         # 6.28318...

math.floor(3.9)  # 3
math.ceil(3.1)   # 4
math.sqrt(16)    # 4
math.pow(2, 10)  # 1024
math.sin(math.pi / 2)  # 1
math.gcd(48, 18) # 6
math.hypot(3, 4) # 5
math.fsum([0.1, 0.2, 0.3])  # 0.6
```

## 16.9 15.9 Exercises

1. Write `distance(x1, y1, x2, y2)` using `math.hypot`.
2. Write `celsius_to_fahrenheit(c)` and `fahrenheit_to_celsius(f)`.
3. Use `math.gcd` to simplify a fraction `a/b` to lowest terms.
4. Compute the hypotenuse of a right triangle given legs `a` and `b`.
5. Use `math.log10` to compute the number of digits in an integer `n`.
6. Write `degrees_to_dms(degrees)` that converts decimal degrees to degrees-minutes-seconds.
7. Write `compound_interest(principal, rate, years)` using `math.pow`.
8. Demonstrate `math.fsum` is more accurate than naive sum for `[0.1, 0.2, 0.3]`.
9. Write `is_perfect_square(n)` using `math.sqrt`.
10. Write `greatest_common_divisor_list(nums)` using `math.gcd` repeatedly.

## 16.10 15.12 math module — complete function reference

Constants: `pi`, `e`, `tau`, `inf`, `nan`.

| Function | Purpose | Example | Result |
|---|---|---|---|
| `abs(x)` | absolute value | `abs(-5)` | `5` |
| `sqrt(x)` | square root | `sqrt(16)` | `4.0` |
| `pow(a, b)` | power | `pow(2, 3)` | `8.0` |
| `exp(x)` | e^x | `exp(1)` | `2.718...` |

| Function | Purpose | Example | Result |
|---|---|---|---|
| `log(x, base=None)` | logarithm | `log(100, 10)` | `2.0` |
| `log2(x)` | base-2 log | `log2(8)` | `3.0` |
| `log10(x)` | base-10 log | `log10(100)` | `2.0` |
| `floor(x)` | floor | `floor(3.7)` | `3.0` |
| `ceil(x)` | ceiling | `ceil(3.2)` | `4.0` |
| `trunc(x)` | truncate toward zero | `trunc(-3.7)` | `-3.0` |
| `round(x)` | round nearest | `round(3.5)` | `4.0` |
| `sin(x)` | sine (radians) | `sin(0)` | `0.0` |
| `cos(x)` | cosine | `cos(0)` | `1.0` |
| `tan(x)` | tangent | `tan(0)` | `0.0` |
| `asin(x)` | arcsine | `asin(1)` | `1.570...` |
| `acos(x)` | arccosine | `acos(1)` | `0.0` |
| `atan(x)` | arctangent | `atan(1)` | `0.785...` |
| `atan2(y, x)` | atan of y/x | `atan2(1, 1)` | `0.785...` |
| `degrees(x)` | radians to degrees | `degrees(pi)` | `180.0` |
| `radians(x)` | degrees to radians | `radians(180)` | `3.141...` |
| `gcd(a, b)` | greatest common divisor | `gcd(12, 8)` | `4` |
| `lcm(a, b)` | least common multiple | `lcm(4, 6)` | `12` |
| `factorial(n)` | factorial | `factorial(5)` | `120` |
| `comb(n, k)` | combinations | `comb(5, 2)` | `10` |
| `perm(n, k=None)` | permutations | `perm(5, 2)` | `20` |
| `remainder(x, y)` | remainder | `remainder(7, 3)` | `1` |
| `copysign(x, y)` | sign of y to x | `copysign(1, -2)` | `-1.0` |
| `min(a, b=None)` | minimum | `min(3, 1)` | `1` |
| `max(a, b=None)` | maximum | `max(3, 1)` | `3` |
| `fsum(a, b=None)` | precise sum | `fsum([0.1,0.2])` | `0.3` |
| `prod(a, b=None)` | product | `prod([2,3,4])` | `24` |
| `hypot(a, b=None)` | hypotenuse | `hypot(3, 4)` | `5.0` |
| `isnan(x)` | is NaN | `isnan(nan)` | `true` |
| `isfinite(x)` | is finite | `isfinite(inf)` | `false` |
| `isinf(x)` | is infinite | `isinf(inf)` | `true` |
| `modf(x)` | fractional+integral | `modf(3.5)` | `#(0.5, 3.0)` |
| `frexp(x)` | mantissa/exponent | `frexp(8.0)` | `#(0.5, 4)` |
| `ldexp(x, exp)` | x * 2^exp | `ldexp(0.5, 3)` | `4.0` |

### 16.10.1 Usage

```
import math
print(math.pi)           # 3.141592653589793
print(math.sqrt(25))     # 5.0
print(math.hypot(3, 4))  # 5.0
print(math.factorial(5)) # 120
print(math.gcd(48, 18))  # 6
print(math.isnan(0/0))   # true (if 0/0 yields nan)
```

### 16.10.2 Edge cases

- `sqrt(-1)` raises a domain error.
- `log(0)` or `log(negative)` raises a domain error.
- `factorial` of negative raises a ValueError.
- `inf` and `nan` propagate through most operations per IEEE 754.
- `nan != nan` is true.

## 16.11 15.13 Exercises

1. Compute the area of a circle: `pi * r * r`.
2. Implement `distance(x1, y1, x2, y2)` using `hypot`.
3. Write `degrees_to_radians` and `radians_to_degrees` wrappers.
4. Verify `sin(x)**2 + cos(x)**2 == 1` for several x.
5. Implement `nth_root(x, n)` using `pow(x, 1/n)`.
6. Write `is_perfect_square(n)` using `sqrt`.
7. Compute compound interest: `P * (1 + r/n)**(n*t)`.
8. Implement `greatest_common_divisor` via `math.gcd`.
9. Write `safe_log(x)` returning nil for x <= 0.
10. Create a trigonometric table for 0, 30, 45, 60, 90 degrees.

# 17 Chapter 16: random Module

The `random` module provides pseudo-random number generation and random selection utilities.

## 17.1 16.1 Random numbers

| Function | Description | Example |
|---|---|---|
| `random()` | Random float [0.0, 1.0) | `random.random()` |
| `randint(a, b)` | Random int [a, b] inclusive | `random.randint(1, 6)` |
| `randbelow(n)` | Random int [0, n) | `random.randbelow(10)` |
| `uniform(a, b)` | Random float [a, b] | `random.uniform(1.0, 10.0)` |
| `gauss(mu, sigma)` | Gaussian random | `math.gauss(0, 1)` |
| `seed(n)` | Seed the RNG | `random.seed(42)` |

```
import random

print(random.random())   # 0.374... (varies each run)
print(random.randint(1, 6))  # 1-6
print(random.randbelow(10))  # 0-9
print(random.uniform(1.0, 10.0))  # 1.0-10.0
print(random.gauss(0, 1))   # bell curve
```

### 17.1.1 Result ranges

| Function | Range | Inclusive? |
|---|---|---|
| `random()` | [0.0, 1.0) | lower inclusive, upper exclusive |
| `randint(a, b)` | [a, b] | both inclusive |
| `randbelow(n)` | [0, n) | lower inclusive, upper exclusive |
| `uniform(a, b)` | [a, b] | both inclusive (may include endpoint) |

```
import random

# randint is inclusive on both ends
for i in 1..1000 {
    let r = random.randint(1, 6)
    if r < 1 or r > 6 {
        print("out of range!")
        break
    }
}

# randbelow is [0, n)
```

```
for i in 1..1000 {
    let r = random.randbelow(10)
    if r < 0 or r >= 10 {
        print("out of range!")
        break
    }
}
```

## 17.2 16.2 Random sequences

| Function | Description | Example |
|---|---|---|
| `choice(seq)` | Random element | `random.choice(["a","b","c"])` |
| `choices(seq, k)` | k elements with replacement | `random.choices("abcd", 3)` |
| `sample(seq, k)` | k unique elements | `random.sample(1..10, 3)` |
| `shuffle(seq)` | Shuffle in place | `random.shuffle(my_list)` |

```
import random

let colors = ["red", "green", "blue"]
print(random.choice(colors))   # "green"

print(random.choices(colors, 3))  # ["red", "red", "blue"]

print(random.sample(1..10, 3))   # [3, 7, 1] (unique)

let deck = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
random.shuffle(deck)
print(deck)  # shuffled
```

### 17.2.1 choice

Picks one random element from a non-empty sequence. Raises if the sequence is empty.

```
try {
    random.choice([])
} catch as e {
    print(e.msg)  # IndexError or ValueError
}
```

### 17.2.2 choices (with replacement)

Returns `k` elements, each independently chosen. Duplicates are possible.

```
print(random.choices(["a", "b"], 5))   # ["a", "b", "a", "a", "b"]
```

### 17.2.3 sample (without replacement)

Returns `k` unique elements. Raises if `k > len(seq)`.

```
try {
    random.sample([1, 2], 5)
} catch as e {
    print(e.msg)  # sample larger than population
}
```

### 17.2.4 shuffle (in place)

Shuffles the list in place using Fisher-Yates. Returns `nil`.

```
let deck = [1, 2, 3, 4, 5]
random.shuffle(deck)
print(deck)  # e.g. [3, 1, 5, 2, 4]
print(len(deck))  # 5 (same length, reordered)
```

## 17.3 16.3 Deterministic randomness (seed)

```
import random

random.seed(42)
print(random.randint(1, 100))  # same every run with seed 42
random.seed(42)
print(random.randint(1, 100))  # same as above
```

Use `seed` for reproducibility in tests, simulations, and games.

```
import random

random.seed(12345)
let a = random.randint(1, 1000)
random.seed(12345)
let b = random.randint(1, 1000)
print(a == b)  # true
```

## 17.4 16.4 Common patterns

### 17.4.1 Roll dice

```
import random

func roll_dice(sides, count) {
```

```
    let total = 0
    for i in 1..count {
        total = total + random.randint(1, sides)
    }
    return total
}

print(roll_dice(6, 2))   # 2-12
```

### 17.4.2 Pick a random element

```
func pick_winner(entries) {
    if len(entries) == 0 {
        return nil
    }
    return random.choice(entries)
}

print(pick_winner(["Alice", "Bob", "Carol"]))
```

### 17.4.3 Shuffle and deal

```
func deal(deck, hand_size) {
    random.shuffle(deck)
    let hand = deck[0:hand_size]
    let remaining = deck[hand_size:]
    return #(hand, remaining)
}

let deck = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13] * 4
let #(hand, rest) = deal(deck, 5)
print(hand)   # 5 cards
print(len(rest))  # 47
```

### 17.4.4 Generate a random password

```
import random

let chars = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#$%"

func random_password(length) {
    let pw = []
    for i in 1..length {
        pw.push(random.choice(chars))
    }
```

```
    return "".join(pw)
}

print(random_password(12))
```

### 17.4.5 Simulate coin flips

```
import random

func flip_coin() {
    return random.randint(0, 1) == 0 ? "heads" : "tails"
}

let heads = 0
let tails = 0
for i in 1..1000 {
    if flip_coin() == "heads" {
        heads = heads + 1
    } else {
        tails = tails + 1
    }
}
print("heads: " + str(heads) + ", tails: " + str(tails))
```

## 17.5 16.5 Quick reference

```
import random

random.random()        # float [0,1)
random.randint(1, 6)   # int [1,6]
random.randbelow(10)   # int [0,10)
random.uniform(1.0, 10.0)  # float [1,10]
random.gauss(0, 1)     # normal distribution
random.choice(["a","b","c"])   # random element
random.choices("abcd", 3)  # with replacement
random.sample(1..10, 3)      # unique sample
random.shuffle(my_list)  # in-place shuffle
random.seed(42)        # deterministic
```

## 17.6 16.6 Exercises

1. Simulate rolling two dice 1000 times and count the frequency of each sum.
2. Write a function `pick_winner(entries)` that picks a random entry.
3. Shuffle a deck of cards and deal 5 cards.
4. Generate a random password of 12 characters from letters and digits.

5. Use `random.seed` to make a simulation reproducible.
6. Write `weighted_choice(items, weights)` that picks an item with probability proportional to its weight.
7. Write `bootstrap_sample(data, k)` that samples `k` elements with replacement from `data`.

## 17.7 16.12 random module — complete reference

| Function | Purpose | Example |
|---|---|---|
| `seed(n)` | seed the generator | `random.seed(42)` |
| `random()` | float in [0, 1) | `random.random()` |
| `randint(a, b)` | int in [a, b] inclusive | `random.randint(1, 6)` |
| `randrange(start, stop=None, step=1)` | int from range | `random.randrange(0, 10, 2)` |
| `choice(seq)` | random element | `random.choice(["a","b"])` |
| `choices(seq, k=1)` | k elements with replacement | `random.choices([1,2], k=3)` |
| `sample(seq, k)` | k unique elements | `random.sample([1,2,3], 2)` |
| `shuffle(seq)` | shuffle in place | `random.shuffle(xs)` |
| `uniform(a, b)` | float in [a, b] | `random.uniform(1.0, 2.0)` |
| `hex(n)` | n random hex chars | `random.hex(16)` |

### 17.7.1 Usage

```
import random
random.seed(42)
print(random.random())     # deterministic given seed
print(random.randint(1, 100))
print(random.choice(["rock","paper","scissors"]))
let deck = list(range(1, 53))
random.shuffle(deck)
print(deck[:5])
```

### 17.7.2 Cryptographic randomness

For security-sensitive tokens, do NOT use `random`. Use `crypto` module functions (`crypto.generate_secret`, `crypto.secure_token`, `crypto.bytes`).

### 17.7.3 Exercises

1. Roll a die 1000 times and count frequencies.
2. Shuffle a list and verify it's a permutation.
3. Generate a random password of length 12 from letters+digits.

4. Pick a random element weighted by a list of weights (manual).
5. Simulate coin flips until 3 heads in a row; count flips.
6. Generate 6 unique lottery numbers from 1..49.
7. Compare `random()` distribution (bucket into 10 bins).
8. Seed twice with same value; confirm identical sequences.
9. Implement `dice(sides, count)` returning sum.
10. Use `crypto.secure_token()` for a session token instead of `random`.

## 17.8 16.12 Worked example — dice and shuffle

```
import random

random.seed(42)
print(random.randint(1, 6))       # die roll
let deck = list(range(1, 53))
random.shuffle(deck)
print(deck[:5])                   # first 5 shuffled cards
print(random.choice(["rock","paper","scissors"]))
print(random.uniform(1.0, 2.0))   # random float
```

## 17.9 16.13 Troubleshooting randomness

| Symptom | Cause | Fix |
|---|---|---|
| same sequence every run | seed fixed | use different seed or no seed |
| not secure | `random` is not CSPRNG | use `crypto.secure_token` for secrets |
| out of range | bad args to randint | check bounds |

Never use `random` for passwords/tokens; use `crypto`.

## 17.10 16.14 Exercises

1. Roll a die 1000 times; count frequencies.
2. Shuffle a list; verify it's a permutation.
3. Generate a 12-char password (letters+digits).
4. Pick 6 unique lottery numbers from 1..49.
5. Simulate coin flips until 3 heads in a row.

# 18 Chapter 17: string Module

The `string` module provides string utilities: case conversion, trimming, splitting, joining, padding, alignment, and character classification. These operate on Zen strings (immutable `Arc<str>`) and return new strings.

## 18.1 17.1 Case conversion

| Function | Description | Example |
|---|---|---|
| `upper(s)` | Uppercase | `string.upper("zen")` → `"ZEN"` |
| `lower(s)` | Lowercase | `string.lower("ZEN")` → `"zen"` |
| `title(s)` | Title case | `string.title("zen lang")` → `"Zen Lang"` |
| `capitalize(s)` | Capitalize first | `string.capitalize("zen")` → `"Zen"` |
| `swapcase(s)` | Invert case | `string.swapcase("ZEN")` → `"zen"` |

```
import string

print(string.upper("hello"))      # HELLO
print(string.lower("WORLD"))      # world
print(string.title("hello world"))# Hello World
print(string.capitalize("hello")) # Hello
print(string.swapcase("HeLLo"))   # hEllO
```

### 18.1.1 Edge cases

```
print(string.upper(""))           # "" (empty stays empty)
print(string.lower("123ABC"))     # "123abc" (digits unaffected)
print(string.title(""))           # ""
print(string.capitalize(""))      # ""
print(string.swapcase("123"))     # "123" (digits unaffected)
```

`title()` capitalizes the first character of each word and lowercases the rest:

```
print(string.title("HELLO WORLD"))  # "Hello World"
print(string.title("hello   world"))  # "Hello   World" (multiple spaces preserved)
```

## 18.2 17.2 Trimming

| Function | Description | Example |
|---|---|---|
| `trim(s)` | Whitespace both ends | `string.trim("  x  ")` → `"x"` |
| `lstrip(s)` | Whitespace left | `string.lstrip("  x")` → `"x"` |
| `rstrip(s)` | Whitespace right | `string.rstrip("x  ")` → `"x"` |

Whitespace includes space, tab, newline, carriage return, form feed.

```
import string

print(string.trim("  hello  "))   # hello
print(string.lstrip("  hello"))   # hello
print(string.rstrip("hello  "))   # hello
print(string.trim("\n\t hello \n"))  # hello
```

Custom character trimming is **not** supported — `trim` only removes whitespace.

```
print(string.trim("xxhelloxx"))   # "xxhelloxx" (x is not whitespace)
```

## 18.3 17.3 Splitting and joining

| Function | Description | Example |
|---|---|---|
| `split(s, sep?)` | Split into list | `string.split("a,b,c", ",")` |
| `split_whitespace(s)` | Split on whitespace | `string.split_whitespace("a b")` |
| `join(sep, list)` | Join list with sep | `string.join(", ", ["a","b"])` |
| `partition(s, sep)` | Split into 3 parts | `string.partition("a=1", "=")` |
| `rpartition(s, sep)` | Split from right | `string.rpartition("a=1=b", "=")` |

```
import string

print(string.split("a,b,c", ","))         # ["a", "b", "c"]
print(string.split_whitespace("a  b c"))  # ["a", "b", "c"]
print(string.join(", ", ["a", "b", "c"])) # a, b, c
print(string.partition("a=1", "="))       # ["a", "=", "1"]
print(string.rpartition("a=1=b", "="))    # ["a=1", "=", "b"]
```

### 18.3.1 split behavior

```
print(string.split("a,,b", ","))           # ["a", "", "b"]  (empty strings preserved)
print(string.split("", ","))               # [""]              (single empty string)
```

```
print(string.split("a,b,", ","))            # ["a", "b", ""]
print(string.split_whitespace("  a   b  c  "))  # ["a", "b", "c"]  (empties discarded)
```

`split_whitespace` with no argument splits on any whitespace and discards empty strings:

```
print(string.split_whitespace("  a   b  c  "))       # ["a", "b", "c"]
```

### 18.3.2 join behavior

```
let tags = ["zen", "lang", "script"]
print(string.join(", ", tags))   # "zen, lang, script"
print(string.join("#", tags))    # "zen#lang#script"
print(string.join("", tags))     # "zenlangscript"
```

`join` requires all elements to be strings:

```
try {
    print(string.join("-", [1, 2, 3]))  # TypeError
} catch as e {
    print(e.msg)
}
```

### 18.3.3 partition / rpartition

`partition` splits on the first occurrence of `sep`, returning `[before, sep, after]`. If `sep` is not found, returns `[s, "", ""]`.

```
print(string.partition("a=1=b", "="))   # ["a", "=", "1=b"]
print(string.partition("no_eq", "="))   # ["no_eq", "", ""]
```

`rpartition` splits on the **last** occurrence:

```
print(string.rpartition("a=1=b", "="))   # ["a=1", "=", "b"]
print(string.rpartition("no_eq", "="))   # ["no_eq", "", ""]
```

## 18.4 17.4 Padding and alignment

| Function | Description | Example |
|---|---|---|
| `ljust(s, width, fill?)` | Left-justify | `string.ljust("x", 5, "-")` → `"x----"` |
| `rjust(s, width, fill?)` | Right-justify | `string.rjust("x", 5, "-")` → `"----x"` |
| `center(s, width, fill?)` | Center | `string.center("x", 5, "-")` → `"--x--"` |
| `zfill(s, width)` | Zero-pad left | `string.zfill("42", 5)` → `"00042"` |

```
import string

print(string.ljust("x", 5, "-"))   # x----
print(string.rjust("x", 5, "-"))   # ----x
print(string.center("x", 5, "-"))  # --x--
print(string.zfill("42", 5))       # 00042
```

If `width <= len(s)`, the original string is returned unchanged:

```
print(string.ljust("hello", 3, "-"))   # "hello"
print(string.rjust("hello", 3, "-"))   # "hello"
print(string.center("hello", 3, "-"))  # "hello"
print(string.zfill("hello", 3))        # "hello"
```

`zfill` only pads with `0` and treats the string as potentially signed:

```
print(string.zfill("-42", 5))   # "-0042" (sign preserved, zeros after sign)
print(string.zfill("+42", 5))   # "+0042"
```

## 18.5 17.5 Case checking

| Function | Description | Example |
|---|---|---|
| `isupper(s)` | All uppercase | `string.isupper("ZEN")` → `true` |
| `islower(s)` | All lowercase | `string.islower("zen")` → `true` |
| `istitle(s)` | Title case | `string.istitle("Zen Lang")` → `true` |
| `isspace(s)` | All whitespace | `string.isspace("  ")` → `true` |
| `isdigit(s)` | All digits | `string.isdigit("123")` → `true` |
| `isalnum(s)` | Alphanumeric | `string.isalnum("abc123")` → `true` |
| `isalpha(s)` | All letters | `string.isalpha("abc")` → `true` |

```
import string

print(string.isupper("HELLO"))  # true
print(string.isupper("Hello"))  # false
print(string.islower("hello"))  # true
print(string.istitle("Zen Lang"))  # true
print(string.istitle("Zen lang"))  # false
print(string.isspace("   "))    # true
print(string.isspace(""))       # false (empty is not whitespace)
print(string.isdigit("123"))    # true
print(string.isdigit("12.3"))   # false
print(string.isalnum("abc123")) # true
print(string.isalnum("abc_123")) # false
print(string.isalpha("abc"))    # true
print(string.isalpha("abc123")) # false
```

Edge cases: - `isupper("")` → false (empty has no cased characters) - `islower("")` →

false - `istitle("")` → false - `isdigit("")` → false - `isalnum("")` → false - `isalpha("")` → false

## 18.6 17.6 Prefix/suffix

| Function | Description | Example |
|---|---|---|
| `starts_with(s, prefix)` | Boolean | `string.starts_with("zen", "ze")` → `true` |
| `ends_with(s, suffix)` | Boolean | `string.ends_with("zen", "en")` → `true` |

```
import string

print(string.starts_with("https://", "https"))  # true
print(string.ends_with("file.z", ".z"))         # true
print(string.starts_with("Zen", "zen"))         # false (case-sensitive)
print(string.ends_with("file.Z", ".z"))         # false (case-sensitive)
print(string.starts_with("", ""))               # true (empty prefix matches)
print(string.ends_with("", ""))                 # true
```

## 18.7 17.7 Replacement and translation

| Function | Description | Example |
|---|---|---|
| `replace(s, old, new)` | Replace all | `string.replace("aaa", "a", "b")` → `"bbb"` |
| `count(s, sub)` | Count occurrences | `string.count("banana", "a")` → `3` |
| `find(s, sub)` | Index or -1 | `string.find("zen", "e")` |

```
import string

print(string.replace("hello world", "world", "zen"))  # hello zen
print(string.count("banana", "a"))                    # 3
print(string.find("zen 3.0", "3"))                    # 4
```

### 18.7.1 replace replaces ALL occurrences

```
print(string.replace("aaa", "a", "b"))   # "bbb"
print(string.replace("ababab", "ab", "xy"))  # "xyxyxy"
print(string.replace("hello", "x", "y"))  # "hello" (not found, unchanged)
```

`replace` with empty `old`:

```
print(string.replace("abc", "", "x"))   # behavior: inserts x between every char and at
```

### 18.7.2 count counts non-overlapping occurrences

```
print(string.count("banana", "a"))      # 3
print(string.count("aaaa", "aa"))       # 2  (non-overlapping: positions 0-1 and 2-3)
print(string.count("aaaa", "aaa"))      # 1  (non-overlapping: positions 0-2)
print(string.count("hello", "z"))       # 0
```

### 18.7.3 find returns -1 when not found

```
print(string.find("zen 3.0", "3"))      # 4
print(string.find("hello", "x"))        # -1
print(string.find("banana", "an", 2))   # 3  (note: string.find in Zen does NOT take a
```

> Note: Zen's `string.find(s, sub)` takes only two arguments (no start position). To search from a position, slice first: `string.find(s[2:], "an")`.

## 18.8 17.8 Quick reference

```
import string

string.upper("zen")           # ZEN
string.lower("ZEN")           # zen
string.title("zen lang")      # Zen Lang
string.capitalize("zen")      # Zen
string.swapcase("HeLLo")      # hEllO
string.trim("  x  ")          # x
string.split("a,b,c", ",")    # ["a", "b", "c"]
string.join(", ", ["a","b"])  # a, b
string.ljust("x", 5, "-")     # x----
string.zfill("42", 5)         # 00042
string.isdigit("123")         # true
string.replace("aaa", "a", "b") # bbb
string.count("banana", "a")   # 3
string.find("zen", "e")       # 1
string.starts_with("zen", "ze") # true
string.ends_with("zen", "en") # true
```

## 18.9 17.9 Exercises

1. Write `snake_case(name)` that converts "Hello World" to "hello_world".
2. Write `wrap_text(text, width)` that wraps text at `width` characters.
3. Write `is_palindrome(s)` using `string.lower` and slicing.
4. Write `mask_email(email)` that hides the username part.

5. Write `columnate(rows, width)` that prints columns aligned.
6. Write `to_snake_case(text)` handling spaces and hyphens.
7. Write `strip_html_tags(text)` that removes `<...>` tags using repeated `find` and slicing.
8. Write `format_number_with_commas(n)` that inserts commas every 3 digits.

## 18.10 17.12 string module — complete reference

Constants: `digits`, `hexdigits`, `octdigits`, `ascii_letters`, `ascii_lowercase`, `ascii_uppercase`, `punctuation`, `whitespace`, `printable`.

| Function | Purpose | Example |
|---|---|---|
| `upper(s)` | uppercase | `string.upper("abc")` |
| `lower(s)` | lowercase | `string.lower("ABC")` |
| `title(s)` | title case | `string.title("hello world")` |
| `capitalize(s)` | first char upper | `string.capitalize("hello")` |
| `swapcase(s)` | flip case | `string.swapcase("AbC")` |
| `strip(s)` | trim whitespace | `string.strip("  hi  ")` |
| `lstrip(s)` / `rstrip(s)` | trim one side | `string.lstrip("  hi")` |
| `split(s, sep=None)` | split | `string.split("a,b,c", ",")` |
| `join(sep, parts)` | join | `string.join("-", ["a","b"])` |
| `replace(s, old, new)` | replace | `string.replace("aaa","a","b")` |
| `count(s, sub)` | count occurrences | `string.count("banana","an")` |
| `find(s, sub)` | first index or -1 | `string.find("hello","ll")` |
| `startswith(s, prefix)` | boolean | `string.startswith("hello","he")` |
| `endswith(s, suffix)` | boolean | `string.endswith("hello","lo")` |
| `isdigit(s)` | all digits | `string.isdigit("123")` |
| `isalpha(s)` | all letters | `string.isalpha("abc")` |
| `isalnum(s)` | letters/digits | `string.isalnum("abc1")` |
| `isspace(s)` | all whitespace | `string.isspace("   ")` |

### 18.10.1 Usage

```
import string
print(string.upper("zen"))        # ZEN
print(string.digits)              # "0123456789"
print(string.join(":", ["2024","01","15"]))  # 2024:01:15
print(string.isdigit("42"))       # true
```

### 18.10.2 Exercises

1. Write `is_palindrome(s)` using string module case helpers.
2. Implement `mask_all_but_last(s, n)` masking characters.

3. Write `extract_digits(s)` keeping only digits.
4. Implement `title_case(s)` manually and compare to `string.title`.
5. Write `validate_username(s)` (alphanumeric, 3-16 chars).
6. Implement `word_count(text)` using split.
7. Write `remove_punctuation(s)` using `string.punctuation`.
8. Implement `caesar_shift(s, k)` using `ascii_letters`.
9. Write `is_pangram(s)` checking all letters present.
10. Implement `wrap(s, width)` inserting newlines.

# 19 Chapter 18: collections Module

The `collections` module provides advanced data structures beyond the built-in list and dict: `deque`, `Counter`, `OrderedDict`, `namedtuple`, and `defaultdict`-style helpers.

## 19.1 18.1 deque — double-ended queue

```
import collections

let d = collections.deque([1, 2, 3])
d.append(4)           # right append
d.appendleft(0)       # left append
print(d.pop())        # 4 (right pop)
print(d.popleft())    # 0 (left pop)
print(d)              # [1, 2, 3]
```

`deque` supports O(1) appends and pops from both ends, making it ideal for queues and sliding windows.

### 19.1.1 deque methods

| Method | Description |
|---|---|
| `append(x)` | Add x to the right end |
| `appendleft(x)` | Add x to the left end |
| `pop()` | Remove and return rightmost element |
| `popleft()` | Remove and return leftmost element |
| `len(d)` | Number of elements (builtin) |

```
import collections

let d = collections.deque()
d.append(1)
d.append(2)
d.appendleft(0)
print(d)          # [0, 1, 2]
print(d.pop())    # 2
print(d.popleft()) # 0
print(d)          # [1]
```

`pop()` and `popleft()` raise `IndexError` on an empty deque.

```
let d = collections.deque()
try {
    d.pop()
} catch as e {
    print(e.msg)  # IndexError: pop from empty deque
}
```

### 19.1.2 Use cases

**Queue (FIFO):**

```
import collections

let q = collections.deque()
q.append("task1")
q.append("task2")
print(q.popleft())  # "task1"
print(q.popleft())  # "task2"
```

**Stack (LIFO) using right end:**

```
let s = collections.deque()
s.append("a")
s.append("b")
print(s.pop())  # "b"
print(s.pop())  # "a"
```

**Sliding window:**

```
import collections

func sliding_window_max(nums, k) {
    let dq = collections.deque()
    let result = []
    for i in 0..len(nums) {
        # Remove indices out of window
        if len(dq) > 0 and dq[0] < i - k + 1 {
            dq.popleft()
        }
        # Remove smaller values from right
        while len(dq) > 0 and nums[dq[len(dq)-1]] < nums[i] {
            dq.pop()
        }
        dq.append(i)
        if i >= k - 1 {
            result.push(nums[dq[0]])
        }
    }
    return result
}

print(sliding_window_max([1, 3, -1, -3, 5, 3, 6, 7], 3))
# [3, 3, 5, 5, 6, 7]
```

## 19.2 18.2 Counter — count hashable objects

```
import collections

let c = collections.Counter(["zen", "lang", "zen", "script", "lang", "zen"])
print(c["zen"])   # 3
print(c["lang"])  # 2
print(c.most_common(2))  # [["zen", 3], ["lang", 2]]
print(list(c.elements()))  # all elements with repetition
```

Counter is useful for frequency counting, voting, and histograms.

### 19.2.1 Counter methods

| Method | Description |
|---|---|
| `c[key]` | Count of key (0 if absent) |
| `most_common(n)` | Top n [key, count] pairs |
| `elements()` | Iterator over elements repeating by count |
| `len(c)` | Number of distinct keys |

```
import collections

let c = collections.Counter(["a", "b", "a", "c", "b", "a"])
print(c["a"])       # 3
print(c["z"])       # 0 (missing key returns 0, not KeyError)
print(c.most_common(2))  # [["a", 3], ["b", 2]]
print(c.most_common())  # all, sorted by count desc
print(len(c))         # 3 (distinct keys)

# elements()
for e in c.elements() {
    print(e)   # a, a, a, b, b, c (order arbitrary)
}
```

Counter returns 0 for missing keys (unlike dict which raises KeyError):

```
let c = collections.Counter(["a"])
print(c["missing"])  # 0
```

### 19.2.2 Common patterns

**Word frequency:**

```
import collections

let text = "the quick brown fox jumps over the lazy dog"
let words = text.split(" ")
```

```
let c = collections.Counter(words)
print(c.most_common(3))  # [["the", 2], ...]
```

**Voting:**

```
let votes = ["alice", "bob", "alice", "carol", "alice", "bob"]
let c = collections.Counter(votes)
print(c.most_common(1))  # [["alice", 3]]
print(c.most_common(1)[0][0])  # "alice" (winner)
```

## 19.3 18.3 OrderedDict — dict that remembers order

Zen dicts already preserve insertion order, so `OrderedDict` is mostly for API compatibility:

```
import collections

let od = collections.OrderedDict()
od["a"] = 1
od["b"] = 2
od["c"] = 3
print(od.keys())  # ["a", "b", "c"]
```

### 19.3.1 OrderedDict methods

Same as dict: `keys()`, `values()`, `items()`, `get()`, `has()`, `pop()`, `update()`, `len()`.

```
import collections

let od = collections.OrderedDict()
od["first"] = 1
od["second"] = 2
od["third"] = 3

for k in od.keys() {
    print(k)  # first, second, third (insertion order)
}
```

## 19.4 18.4 namedtuple — tuple with named fields

```
import collections

let Point = collections.namedtuple("Point", ["x", "y"])
let p = Point(3, 4)
print(p.x)   # 3
print(p.y)   # 4
print(p[0])  # 3 (tuple access)
```

```
print(p[1])  # 4
print(len(p)) # 2
```

Namedtuples are lightweight, immutable records.

### 19.4.1 namedtuple behavior

```
import collections

let Person = collections.namedtuple("Person", ["name", "age"])
let p = Person("Zen", 30)

print(p.name)   # "Zen"
print(p.age)    # 30
print(p[0])     # "Zen"  (tuple index access)
print(p[1])     # 30
print(len(p))   # 2

# Namedtuples are immutable
try {
    p.age = 31
} catch as e {
    print(e.msg)  # TypeError or similar (immutable)
}
```

```
import collections

let Point = collections.namedtuple("Point", ["x", "y", "z"])
let p = Point(1, 2, 3)
print(p.x, p.y, p.z)  # 1 2 3
print(p[0], p[1], p[2])  # 1 2 3
```

## 19.5 18.5 defaultdict — dict with default factory

```
import collections

let d = collections.defaultdict(0)  # default int → 0
d["a"] = d["a"] + 1
print(d["a"])  # 1
print(d["b"])  # 0 (default, auto-created)
```

### 19.5.1 defaultdict behavior

```
import collections

let d = collections.defaultdict(0)
```

```
d["a"] = 1
print(d["a"])  # 1
print(d["b"])  # 0  (auto-created on access)
print(d["c"])  # 0
print(d)       # {"a": 1, "b": 0, "c": 0}  (accessing missing keys creates them!)
```

> Warning: accessing a missing key with `d["key"]` **auto-creates** it with the default value. Use `has()` to check without creating:

```
import collections

let d = collections.defaultdict(0)
print(d.has("missing"))  # false (doesn't create)
print(d)                 # {}  (still empty)
d["missing"]             # creates it with 0
print(d.has("missing"))  # true
```

The default factory is called with no arguments:

```
import collections

let d = collections.defaultdict(list)  # default → []
d["a"].push(1)
print(d["a"])  # [1]
print(d["b"])  # []  (auto-created empty list)
```

```
let d = collections.defaultdict(lambda: 0)  # if lambda supported
# or use a constant
let d = collections.defaultdict(0)
```

## 19.6 18.6 Quick reference

```
import collections

let d = collections.deque([1, 2, 3])
d.append(4)
d.appendleft(0)
d.pop()
d.popleft()

let c = collections.Counter(["a", "b", "a"])
c["a"]   # 2
c.most_common(1)  # [["a", 2]]

let od = collections.OrderedDict()
od["a"] = 1

let Point = collections.namedtuple("Point", ["x", "y"])
```

```
let p = Point(1, 2)
p.x  # 1

let dd = collections.defaultdict(0)
dd["missing"]   # 0
```

## 19.7 18.7 Exercises

1. Use `collections.deque` to implement a sliding-window average.
2. Use `collections.Counter` to find the most common word in a text.
3. Use `collections.namedtuple` to represent a `Person` with `name` and `age`.
4. Use `collections.defaultdict(list)` to group items by category.
5. Compare `collections.deque` vs list for a queue (push/pop both ends).
6. Write `top_k_words(text, k)` using `collections.Counter`.
7. Write `group_by(items, key_func)` using `collections.defaultdict(list)`.
8. Write a `RingBuffer` class using `collections.deque` with a fixed max size.

## 19.8 18.12 collections module — complete reference

| Function | Purpose | Example |
|---|---|---|
| `Counter(xs)` | count frequencies | `collections.Counter(["a","b","a"])` |
| `chain(*iters)` | chain iterables | `collections.chain([1,2], [3,4])` |
| `flatten(xs)` | flatten one level | `collections.flatten([[1,2],[3]])` |

### 19.8.1 Counter

```
import collections
let c = collections.Counter(["a","b","a","c","b","a"])
print(c)   # Counter: a->3, b->2, c->1
print(c["a"])   # 3
print(c.most_common(1))   # if supported
```

### 19.8.2 chain

```
for x in collections.chain([1,2], [3,4]) {
    print(x)   # 1 2 3 4
}
```

### 19.8.3 flatten

```
print(collections.flatten([[1,2],[3,4],[5]]))   # [1,2,3,4,5]
```

### 19.8.4 Exercises

1. Implement `Counter` manually using a dict.
2. Write `most_common(freqs, n)` returning top n by count.
3. Implement `chain` manually with nested loops.
4. Write `deep_flatten(xs)` recursively flattening all levels.
5. Implement `group_by(xs, key_fn)` returning dict of lists.
6. Write `merge_counters(a, b)` adding counts.
7. Implement `intersection_counter(a, b)` min counts.
8. Write `frequency_sort(xs)` sorting by frequency desc.
9. Implement `chain_from_iterable(iters)` chaining many.
10. Write `sliding_window(xs, n)` returning n-tuples.

# 20 Chapter 19: json Module

The `json` module provides JSON parsing and serialization. Zen ships a pure-Zen implementation (`std/json.z`) with numbers stringified via `str()`.

## 20.1 19.1 Parsing JSON

```
import json

let data = json.loads('{"name": "Zen", "version": 3, "tags": ["ffi", "pymport"]}')
print(data["name"])     # Zen
print(data["version"])  # 3
print(data["tags"][1])  # pymport
```

`json.loads(text)` parses a JSON string into Zen values:

| JSON type | Zen type |
|---|---|
| object | Zen dict (string keys only) |
| array | Zen list |
| string | Zen string |
| number | Zen int or float |
| true/false | Zen bool |
| null | Zen nil |

```
import json

let obj = json.loads('{"name": "Zen", "version": 3, "tags": ["ffi", "pymport"]}')
print(obj["name"])     # Zen
print(obj["version"])  # 3
print(obj["tags"][1])  # pymport

let arr = json.loads('[1, 2, 3]')
print(arr[0])   # 1

let bool_val = json.loads('true')
print(bool_val)   # true

let null_val = json.loads('null')
print(null_val)   # nil
```

## 20.2 19.2 Serializing JSON

```
import json

let obj = {"name": "Zen", "version": 3, "tags": ["ffi", "pymport"]}
```

```
let text = json.dumps(obj)
print(text)  # {"name": "Zen", "version": 3, "tags": ["ffi", "pymport"]}
```

`json.dumps(value)` serializes Zen values to a JSON string:

| Zen type | JSON type |
|---|---|
| dict (string keys) | object |
| list | array |
| string | string |
| int/float | number |
| bool | true/false |
| nil | null |

```
import json

let obj = {"players": 4, "active": true, "scores": [10, 20, 30], "name": null}
let text = json.dumps(obj)
print(text)  # {"players": 4, "active": true, "scores": [10, 20, 30], "name": null}
```

## 20.3 19.3 Round-trip example

```
import json

let original = {"players": 4, "active": true, "scores": [10, 20, 30]}
let text = json.dumps(original)
let parsed = json.loads(text)

print(parsed["players"])   # 4
print(parsed["active"])    # true
print(parsed["scores"][1]) # 20
```

## 20.4 19.4 Reading/writing JSON files

```
import json
import fs

# Write JSON to a file
let data = {"config": {"theme": "dark", "font": 12}}
fs.write("config.json", json.dumps(data))

# Read JSON from a file
let text = fs.read("config.json")
let config = json.loads(text)
print(config["config"]["theme"])  # dark
```

### 20.4.1 Complete config example

```
import json
import fs

func load_config(path, defaults) {
    if !fs.exists(path) {
        return defaults
    }
    let text = fs.read(path)
    return json.loads(text)
}

func save_config(path, config) {
    fs.write(path, json.dumps(config))
}

let defaults = {"theme": "light", "font": 12, "debug": false}
let config = load_config("config.json", defaults)
print(config["theme"])

config["debug"] = true
save_config("config.json", config)
```

## 20.5 19.5 Numbers

The pure-Zen `json.z` stringifies numbers via `str()`. This means: - `json.dumps(42)` → `"42"` - `json.dumps(3.14)` → `"3.14"` - Nested dict access uses explicit keys: `data["a"]["b"]`

```
import json

print(json.dumps(42))       # "42"
print(json.dumps(3.14))     # "3.14"
print(json.dumps(0))        # "0"
print(json.dumps(-1))       # "-1"
print(json.dumps(1e10))     # "10000000000"
```

Numbers round-trip correctly:

```
import json

let original = 3.14159
let text = json.dumps(original)
let parsed = json.loads(text)
print(parsed == original)  # true
```

## 20.6 19.6 Common pitfalls

- **Dict keys must be strings.** Non-string keys are rejected.
- **`json.loads` raises `ValueError`** on malformed JSON. Use `try`/`catch`.
- **Numbers round-trip as strings** in some edge cases — verify critical numeric payloads.
- **`nil` serializes as `null`.**
- **`json.dumps` does not support indentation/pretty-printing** in this version. Output is compact.

```
import json

try {
    json.loads("{not json}")
} catch as e {
    print(e.msg)  # ValueError: ...
}
```

```
import json

print(json.dumps({"a": 1, "b": nil}))  # {"a": 1, "b": null}
print(json.dumps({"a": true, "b": false}))  # {"a": true, "b": false}
```

## 20.7 19.7 Quick reference

```
import json

json.loads('{"a": 1}')     # {"a": 1}
json.dumps({"a": 1})       # '{"a": 1}'
json.loads("[1, 2, 3]")    # [1, 2, 3]
json.dumps([1, 2, 3])      # '[1, 2, 3]'
json.loads('true')         # true
json.loads('null')         # nil

import fs
fs.write("data.json", json.dumps(obj))
json.loads(fs.read("data.json"))
```

## 20.8 19.8 Exercises

1. Write `load_config(path)` that reads a JSON config file with `get` defaults for missing keys.
2. Write `save_scores(path, scores)` that appends a score to a JSON array file.
3. Write `merge_json(a, b)` that merges two JSON objects (b wins on conflict).
4. Write a script that fetches JSON from an HTTP endpoint and parses it.
5. Write `json_to_csv(data)` that converts a list of flat dicts to CSV text.
6. Write `validate_json(s)` that returns true if `s` is valid JSON, false otherwise (using try/catch).

7. Write `extract_field(obj, path)` that navigates a nested dict using a dot-separated path like “a.b.c”.

## 20.9 19.12 json module — complete reference

| Function | Purpose | Example |
|---|---|---|
| `parse(s)` | parse JSON string | `json.parse('{"a":1}')` |
| `stringify(x)` / `encode(x)` | serialize to JSON | `json.stringify(#{a:1})` |
| `pretty(x)` | pretty-printed JSON | `json.pretty(#{a:1})` |
| `load(path)` | parse JSON from file | `json.load("data.json")` |
| `save(path, x)` | write JSON to file | `json.save("out.json", d)` |

### 20.9.1 Usage

```
import json
let d = #{name: "Ada", age: 36, tags: ["a","b"]}
let s = json.stringify(d)
print(s)   # {"name":"Ada","age":36,"tags":["a","b"]}
let back = json.parse(s)
print(back["name"])   # Ada
let pretty = json.pretty(d)
print(pretty)
```

### 20.9.2 JSON types mapping

| JSON | Zen |
|---|---|
| object | dict |
| array | list |
| string | string |
| number | int/float |
| boolean | bool |
| null | nil |

### 20.9.3 Exercises

1. Write `safe_parse(s)` returning nil on invalid JSON.
2. Implement `json_get(d, path, default)` for dotted paths.
3. Write `merge_json(a, b)` merging two JSON objects.
4. Implement `filter_json(d, pred)` keeping matching entries.
5. Write `json_to_csv(data)` converting list of dicts to CSV.
6. Implement `deep_compare(a, b)` for JSON values.
7. Write `strip_nulls(d)` removing nil values recursively.
8. Implement `sort_keys(d)` returning a sorted dict.

9. Write `validate_json_schema(data, schema)` conceptually.
10. Implement `merge_json_files(paths)` merging multiple files.

## 20.10 19.12 Worked example — read, modify, write JSON

```
import json
import pathlib

let p = pathlib.Path("config.json")

# read
let cfg = json.load(str(p))
print("theme=" + str(cfg["theme"]))

# modify
cfg["theme"] = "dark"
cfg["version"] = 2

# write
json.save(str(p), cfg)
print("saved")

# parse a JSON string
let s = '{"a":1,"b":[2,3]}'
let d = json.parse(s)
print(d["a"])   # 1
```

## 20.11 19.13 Troubleshooting JSON

| Symptom | Cause | Fix |
|---|---|---|
| parse error | malformed JSON (trailing comma, single quotes) | fix the JSON |
| missing key | key absent | use `in` or `??` |
| wrong type | value isn't what you expect | check with typeof |

## 20.12 19.14 Exercises

1. Write `safe_parse(s)` returning nil on invalid JSON.
2. Implement `json_get(d, path, default)` for dotted paths.
3. Merge two JSON objects.
4. Convert a list of dicts to CSV.
5. Strip nil values from a dict recursively.

# 21 Chapter 20: datetime/time Module

The `datetime/time` module provides date and time manipulation: parsing, formatting, arithmetic, and timestamps. Zen ships a pure-Zen implementation with a native tier.

## 21.1 20.1 The `datetime` type

```
import datetime

let now = datetime.datetime.now()
print(now.year)       # 2026
print(now.month)      # 10
print(now.day)        # 27
print(now.hour)       # 14
print(now.minute)     # 30
print(now.second)     # 45
print(now.isoformat())# 2026-10-27T14:30:45
print(now.date())     # date object
```

## 21.2 20.2 Creating datetimes

```
import datetime

let epoch = datetime.datetime.fromtimestamp(0)
print(epoch.isoformat())  # 1970-01-01T00:00:00

let specific = datetime.datetime(2030, 1, 2, 3, 4, 5)
print(specific.strftime("%Y-%m-%d %H:%M:%S"))  # 2030-01-02 03:04:05

let today = datetime.datetime.now()
print(today.date().isoformat())  # 2026-10-27
```

### 21.2.1 Constructor signatures

```
datetime.datetime.now()           # current local datetime
datetime.datetime.fromtimestamp(ts)  # from Unix timestamp (float)
datetime.datetime(y, m, d)        # date only, midnight
datetime.datetime(y, m, d, h, mi, s)  # full datetime
```

## 21.3 20.3 Formatting: `strftime`

```
import datetime

let now = datetime.datetime.now()
print(now.strftime("%Y-%m-%d"))   # 2026-10-27
```

```
print(now.strftime("%H:%M:%S"))   # 14:30:45
print(now.strftime("%A, %B %d"))  # Tuesday, October 27
print(now.strftime("%Y-%m-%d %H:%M:%S"))  # full
```

Format codes:

| Code | Meaning | Example |
|---|---|---|
| `%Y` | 4-digit year | `2026` |
| `%m` | Month (01-12) | `10` |
| `%d` | Day (01-31) | `27` |
| `%H` | Hour (00-23) | `14` |
| `%M` | Minute (00-59) | `30` |
| `%S` | Second (00-59) | `45` |
| `%A` | Weekday name | `Tuesday` |
| `%a` | Abbrev weekday | `Tue` |
| `%B` | Month name | `October` |
| `%b` | Abbrev month | `Oct` |
| `%y` | 2-digit year | `26` |
| `%%` | Literal % | `%` |

```
import datetime

let d = datetime.datetime(2026, 10, 27, 14, 30, 45)
print(d.strftime("%Y-%m-%d"))          # 2026-10-27
print(d.strftime("%H:%M:%S"))          # 14:30:45
print(d.strftime("%A, %B %d, %Y"))     # Tuesday, October 27, 2026
print(d.strftime("%I:%M %p"))          # 02:30 PM
print(d.strftime("%Y-%m-%dT%H:%M:%S")) # ISO-like
```

## 21.4 20.4 Datetime arithmetic

```
import datetime

let now = datetime.datetime.now()
let later = datetime.datetime(2030, 1, 1)

# Difference
let delta = later - now
print(delta.days)   # days between

# Add a timedelta
let one_week = datetime.timedelta(days=7)
let next_week = now + one_week
print(next_week.isoformat())
```

Note: `datetime` objects are foreign handles from Python. Arithmetic (`+`, `-`) works when the Python side supports it; otherwise use explicit methods.

### 21.4.1 timedelta

```
import datetime

let one_day = datetime.timedelta(days=1)
let one_hour = datetime.timedelta(hours=1)
let one_minute = datetime.timedelta(minutes=30)

let now = datetime.datetime.now()
print((now + one_day).isoformat())      # tomorrow
print((now - one_hour).isoformat())     # 1 hour ago
print((now + one_minute).isoformat())   # 30 min from now

let delta = datetime.timedelta(days=7, hours=3, minutes=30)
print(delta.days)   # 7
print(delta.seconds)  # seconds component (not total)
print(delta.total_seconds())  # total seconds if available
```

## 21.5 20.5 The `time` module

```
import time

print(time.time())        # current timestamp (float)
print(time.sleep(1))      # sleep 1 second
print(time.strftime("%H:%M:%S"))  # formatted local time

let t = time.time()
# ... do work ...
print(time.time() - t)    # elapsed seconds
```

### 21.5.1 time functions

| Function | Description |
|---|---|
| `time.time()` | Current Unix timestamp (float seconds) |
| `time.sleep(secs)` | Sleep for secs seconds |
| `time.strftime(fmt)` | Format current local time |

```
import time

let start = time.time()
# ... do work ...
```

```
let elapsed = time.time() - start
print("elapsed: " + str(elapsed) + "s")

time.sleep(1)   # sleep 1 second
time.sleep(0.5) # sleep 500ms
```

## 21.6 20.6 Common patterns

### 21.6.1 Timestamp → datetime

```
import datetime

let ts = 1785309045.0
let dt = datetime.datetime.fromtimestamp(ts)
print(dt.isoformat())
```

### 21.6.2 datetime → timestamp

```
import datetime
pymport datetime as pydt

let dt = datetime.datetime.now()
let ts = pydt.datetime.now().timestamp()
print(ts)
```

### 21.6.3 Measure execution time

```
import time

let start = time.time()
# ... some work ...
let elapsed = time.time() - start
print("elapsed: " + str(elapsed) + "s")
```

### 21.6.4 Sleep

```
import time

time.sleep(1)   # sleep 1 second
time.sleep(0.5) # sleep 500ms
```

### 21.6.5 Countdown timer

```
import time

func countdown(seconds) {
    while seconds > 0 {
        print(str(seconds))
        time.sleep(1)
        seconds = seconds - 1
    }
    print("Done!")
}

countdown(5)
```

## 21.7 20.7 Quick reference

```
import datetime
import time

datetime.datetime.now()           # current datetime
datetime.datetime(2030, 1, 2, 3, 4, 5)
datetime.datetime.fromtimestamp(0)
now.strftime("%Y-%m-%d %H:%M:%S")
now.date().isoformat()
datetime.timedelta(days=7)
now + datetime.timedelta(days=7)

time.time()                       # timestamp
time.sleep(1)                     # sleep
time.strftime("%H:%M:%S")
```

## 21.8 20.8 Exercises

1. Write `age(birthdate)` that computes age in years.
2. Write `days_until(target_date)` that returns days remaining.
3. Use `time.time()` to measure how long a sorting operation takes.
4. Write a countdown timer that prints seconds remaining.
5. Format a datetime as “Monday, October 26, 2026 at 2:30 PM”.
6. Write `is_weekend(dt)` that returns true for Saturday/Sunday.
7. Write `add_business_days(dt, n)` that adds n business days (skipping weekends).

## 21.9 20.12 time/datetime module — complete reference

| Function | Purpose | Example |
|---|---|---|
| `time()` | current epoch seconds | `time.time()` |
| `sleep(ms)` | sleep milliseconds | `time.sleep(1000)` |

| Function | Purpose | Example |
|---|---|---|
| `now()` | current datetime | `datetime.now()` |
| `today()` | current date | `datetime.today()` |
| `strftime(fmt, t)` | format time | `datetime.strftime("%Y-%m-%d", t)` |
| `strptime(s, fmt)` | parse time | `datetime.strptime("2024-01-15", "%Y-%m-%d")` |
| `timestamp(dt)` | datetime to epoch | `datetime.timestamp(dt)` |
| `fromtimestamp(ts)` | epoch to datetime | `datetime.fromtimestamp(ts)` |
| `timedelta(...)` | duration | `datetime.timedelta(days=1)` |

### 21.9.1 Usage

```
import time
import datetime
print(time.time())              # epoch seconds
time.sleep(500)                 # sleep 0.5s
let now = datetime.now()
print(datetime.strftime("%Y-%m-%d %H:%M:%S", now))
let parsed = datetime.strptime("2024-01-15", "%Y-%m-%d")
let later = now + datetime.timedelta(days=7)
```

### 21.9.2 Exercises

1. Write `is_leap_year(year)`.
2. Implement `days_between(d1, d2)`.
3. Write `age(birthdate)` in years.
4. Implement `format_duration(seconds)` → "Xh Ym Zs".
5. Write `is_weekend(dt)`.
6. Implement `add_business_days(dt, n)`.
7. Write `countdown(target)` printing seconds remaining.
8. Implement `timer(fn)` measuring execution time.
9. Write `cron_match(expr, dt)` conceptually.
10. Implement `timeit(fn, n)` running fn n times and reporting avg.

## 21.10 20.12 Worked example — current time and formatting

```
import time
import datetime

print("epoch: " + str(time.time()))
let now = datetime.now()
print(datetime.strftime("%Y-%m-%d %H:%M:%S", now))
let parsed = datetime.strptime("2024-01-15", "%Y-%m-%d")
print("parsed: " + str(parsed))
```

```
time.sleep(500)   # sleep 0.5 s
print("awake")
```

## 21.11 20.13 Troubleshooting time

| Symptom | Cause | Fix |
|---|---|---|
| wrong format | format string mismatch | check `%Y-%m-%d` vs input |
| sleep too long | ms vs seconds confusion | verify units |
| parse fails | invalid date string | validate input first |

## 21.12 20.14 Exercises

1. `is_leap_year(year)`.
2. `days_between(d1, d2)`.
3. `age(birthdate)` in years.
4. `format_duration(seconds)` → “Xh Ym Zs”.
5. `is_weekend(dt)`.

# 22 Chapter 21: pathlib & os Module

The `pathlib` module provides an object-oriented filesystem path API, and the `os` module provides low-level OS operations: environment variables, process info, and path manipulation.

## 22.1 21.1 pathlib.Path

```
import pathlib

let p = pathlib.Path("/usr/bin/zen")

print(p.name)        # zen
print(p.stem)        # zen
print(p.suffix)      # (empty for no extension)
print(p.parent)      # /usr/bin
print(p.parts)       # ["", "usr", "bin", "zen"]
print(p.exists())    # true/false
print(p.is_file())   # true/false
print(p.is_dir())    # true/false

# Read / write text
let content = p.read_text()
p.write_text("new content")

# Iterate directory
for child in pathlib.Path(".").iterdir() {
    print(child.name)
}

# Join paths
let p2 = pathlib.Path("/usr").joinpath("bin", "zen")
print(p2)  # /usr/bin/zen
```

### 22.1.1 Path construction

```
import pathlib

let p1 = pathlib.Path("/usr/bin")
let p2 = pathlib.Path("zen")
let p3 = p1 / p2   # if / operator supported, or use joinpath
print(p3)  # /usr/bin/zen

let p4 = pathlib.Path("/usr").joinpath("bin", "zen")
print(p4)  # /usr/bin/zen
```

### 22.1.2 Path properties

| Property | Description | Example |
|---|---|---|
| `name` | Final component | `Path("/a/b/c.txt").name` → `"c.txt"` |
| `stem` | Final component without suffix | `Path("/a/b/c.txt").stem` → `"c"` |
| `suffix` | Final suffix | `Path("/a/b/c.txt").suffix` → `".txt"` |
| `parent` | Parent directory | `Path("/a/b/c.txt").parent` → `"/a/b"` |
| `parts` | All components | `Path("/a/b/c.txt").parts` → `["", "a", "b", "c.txt"]` |

```
import pathlib

let p = pathlib.Path("/usr/bin/zen.tar.gz")

print(p.name)        # "zen.tar.gz"
print(p.stem)        # "zen.tar" (only last suffix removed)
print(p.suffix)      # ".gz"
print(p.parent)      # "/usr/bin"
print(p.parts)       # ["", "usr", "bin", "zen.tar.gz"]
```

### 22.1.3 Path operations

```
import pathlib

let p = pathlib.Path("docs/guide.z")

print(p.with_name("index.z"))   # docs/index.z
print(p.with_suffix(".md"))     # docs/guide.md
print(p.absolute())             # /cwd/docs/guide.z
print(p.relative_to("docs"))    # guide.z
```

### 22.1.4 Read/write

```
import pathlib

let p = pathlib.Path("notes.txt")

# Read
let content = p.read_text()
print(content)
```

```
# Write
p.write_text("new content")
let back = p.read_text()
print(back)  # new content
```

### 22.1.5 Directory iteration

```
import pathlib

for child in pathlib.Path(".").iterdir() {
    print(child.name)
    print(child.is_file())
    print(child.is_dir())
}
```

## 22.2 21.2 The os module

```
import os

# Process info
print(os.getpid())          # process ID
print(os.getppid())         # parent process ID
print(os.getuid())          # user ID
print(os.geteuid())         # effective user ID
print(os.getgid())          # group ID
print(os.getegid())         # effective group ID

# Environment
print(os.getenv("HOME"))    # /home/user
print(os.getenv("PATH", "/usr/bin"))  # with default

# Working directory
print(os.getcwd())          # /current/dir
```

### 22.2.1 Environment variables

```
import os

print(os.getenv("HOME"))           # /home/user (or nil if unset)
print(os.getenv("PATH", "/usr/bin"))  # with default

let port = os.getenv("PORT", "8080")
let host = os.getenv("HOST", "localhost")
let debug = os.getenv("DEBUG", "false") == "true"
```

```
print("Listening on " + host + ":" + port)
```

### 22.2.2 Process IDs

```
import os

print(os.getpid())     # 12345
print(os.getppid())    # 1234
print(os.getuid())     # 1000
print(os.geteuid())    # 1000
print(os.getgid())     # 1000
print(os.getegid())    # 1000
```

### 22.2.3 Working directory

```
import os

print(os.getcwd())   # /home/user/project
```

## 22.3 21.3 Path manipulation: os.path

```
import os

print(os.path.join("a", "b", "c"))        # a/b/c
print(os.path.basename("/a/b/c.txt"))     # c.txt
print(os.path.dirname("/a/b/c.txt"))      # /a/b
print(os.path.exists("/tmp/foo"))         # true/false
print(os.path.isfile("/tmp/foo.txt"))     # true/false
print(os.path.isdir("/tmp"))              # true/false
print(os.path.abspath("relative.z"))      # /abs/relative.z
print(os.path.realpath("/a/b/c.txt"))     # resolved path
```

### 22.3.1 os.path functions

| Function | Description | Example |
|---|---|---|
| `os.path.join(*parts)` | Join path components | `os.path.join("a", "b", "c")` → `"a/b/c"` |
| `os.path.basename(path)` | Final component | `os.path.basename("/a/b/c.txt")` → `"c.txt"` |
| `os.path.dirname(path)` | Directory portion | `os.path.dirname("/a/b/c.txt")` → `"/a/b"` |
| `os.path.exists(path)` | Exists | `os.path.exists("/tmp/foo")` |
| `os.path.isfile(path)` | Is regular file | `os.path.isfile("/tmp/foo.txt")` |

| Function | Description | Example |
|---|---|---|
| `os.path.isdir(path)` | Is directory | `os.path.isdir("/tmp")` |
| `os.path.abspath(path)` | Absolute path | `os.path.abspath("relative.z")` |
| `os.path.realpath(path)` | Resolved path | `os.path.realpath("/a/b/c.txt")` |

## 22.4 21.4 Common patterns

### 22.4.1 Walk a directory tree

```
import fs

func walk(dir) {
    for name in fs.list(dir) {
        let path = fs.join(dir, name)
        if fs.is_dir(path) {
            walk(path)
        } else {
            print(path)
        }
    }
}
walk(".")
```

### 22.4.2 Check if path exists before writing

```
import os
import fs

let path = "output/result.txt"
if !os.path.exists("output") {
    fs.mkdirs("output")
}
fs.write(path, "result")
```

### 22.4.3 Read environment with default

```
import os

let port = os.getenv("PORT", "8080")
let host = os.getenv("HOST", "localhost")
let debug = os.getenv("DEBUG", "false") == "true"

print("Listening on " + host + ":" + port)
```

### 22.4.4 Expand user home

```
import os

let home = os.getenv("HOME")
let config = os.path.join(home, ".config", "zen", "config.z")
print(config)
```

### 22.4.5 Build paths portably

```
import os

let config_dir = os.path.join(os.getenv("HOME"), ".config", "zen")
let config_file = os.path.join(config_dir, "config.z")
print(config_file)
```

## 22.5 21.5 Quick reference

```
import pathlib
import os

pathlib.Path("/usr/bin/zen")
pathlib.Path("file.txt").read_text()
pathlib.Path("file.txt").write_text("data")
pathlib.Path(".").iterdir()

os.getcwd()
os.getenv("HOME", "/default")
os.getpid()
os.path.join("a", "b")
os.path.basename("/a/b/c.txt")
os.path.dirname("/a/b/c.txt")
os.path.exists(path)
os.path.isfile(path)
os.path.isdir(path)
```

## 22.6 21.9 Exercises

1. Write a function `find_files(dir, extension)` that returns all files with a given extension under `dir` (recursive).
2. Write `backup_file(path)` that copies `path` to `path.bak`.
3. Write `ensure_dir(path)` that creates parent directories if missing.
4. Write a script that prints all environment variables containing “ZEN”.
5. Write `human_path(path)` that replaces `$HOME` with ~.

## 22.7 21.12 pathlib & os module — complete reference

### 22.7.1 pathlib.Path

| Method | Purpose | Example |
|---|---|---|
| `Path(s)` | construct path | `pathlib.Path("a/b.txt")` |
| `/` operator | join paths | `p / "sub" / "file.txt"` |
| `exists()` | exists | `p.exists()` |
| `is_file()` | is file | `p.is_file()` |
| `is_dir()` | is dir | `p.is_dir()` |
| `name` | last component | `p.name` → “file.txt” |
| `stem` | name without suffix | `p.stem` → “file” |
| `suffix` | extension | `p.suffix` → “.txt” |
| `parent` | parent path | `p.parent` |
| `resolve()` | absolute path | `p.resolve()` |
| `mkdir()` | create dir | `p.mkdir()` |
| `glob(pattern)` | match files | `p.glob("*.txt")` |

### 22.7.2 os

| Function | Purpose | Example |
|---|---|---|
| `getcwd()` | current dir | `os.getcwd()` |
| `chdir(path)` | change dir | `os.chdir("/tmp")` |
| `listdir(path)` | list dir | `os.listdir(".")` |
| `makedirs(path)` | create dirs | `os.makedirs("a/b")` |
| `remove(path)` | delete file | `os.remove("tmp.txt")` |
| `rmdir(path)` | remove empty dir | `os.rmdir("dir")` |
| `rename(src, dst)` | rename | `os.rename("a","b")` |
| `stat(path)` | file stats | `os.stat("f.txt")` |
| `environ` | env vars | `os.environ["PATH"]` |

### 22.7.3 Usage

```
import pathlib
import os
let p = pathlib.Path("data") / "config.txt"
print(p.exists())
for f in pathlib.Path(".").glob("*.z") {
    print(f.name)
}
print(os.getcwd())
```

### 22.7.4 Exercises

1. Write `find_files(dir, suffix)` recursively.

2. Implement `file_size(path)` using os.stat.
3. Write `ensure_dir(path)` creating parents.
4. Implement `copy_tree(src, dst)` copying a directory.
5. Write `recent_files(dir, n)` listing n most recently modified.
6. Implement `backup(path)` copying to `path.bak`.
7. Write `list_env(prefix)` showing env vars starting with prefix.
8. Implement `temp_name(dir, prefix)` generating a temp path.
9. Write `walk(dir)` yielding all files recursively.
10. Implement `safe_delete(path)` removing file if it exists.

# 23 Chapter 22: re (Regex) Module

The `re` module provides regular expression matching, searching, and replacement. Zen's regex engine supports the common Perl-compatible syntax.

## 23.1 22.1 Compiling patterns

```
import re

let pattern = re.compile("hello")
let m = pattern.search("hello world")
print(m.group())  # hello
```

If you only need one match, use the module-level functions:

```
import re

let m = re.search("world", "hello world")
print(m.group())  # world

let m2 = re.match("hello", "hello world")
print(m2.group()) # hello

let m3 = re.fullmatch("hello", "hello")
print(m3.group()) # hello
```

### 23.1.1 compile(pattern)

Compiles a regex pattern for reuse. Returns a regex object with `search`, `match`, `fullmatch`, `findall`, `sub` methods.

```
import re

let email_re = re.compile("^[\\w.-]+@[\\w.-]+\\.\\w+$")

for addr in addresses {
    if email_re.match(addr) {
        print("valid: " + addr)
    }
}
```

### 23.1.2 search(pattern, text)

Searches for the first match anywhere in the text.

```
import re
```

```
let m = re.search("world", "hello world")
print(m.group())  # world
```

### 23.1.3 match(pattern, text)

Matches at the **start** of the string only (anchored at beginning, not end).

```
import re

let m = re.match("hello", "hello world")
print(m.group())  # hello

let m2 = re.match("world", "hello world")
print(m2)         # nil (not at start)
```

### 23.1.4 fullmatch(pattern, text)

Matches the **entire** string.

```
import re

let m = re.fullmatch("hello", "hello")
print(m.group())  # hello

let m2 = re.fullmatch("hello", "hello world")
print(m2)         # nil (extra characters)
```

## 23.2 22.2 Match objects

A successful match returns a match object with:

- `group()` — the full match
- `group(i)` — capture group `i`
- `groups()` — all capture groups
- `start()` / `end()` — match positions
- `span()` — `[start, end]`

```
import re

let m = re.search("([a-z]+) ([0-9]+)", "items 42")
print(m.group())      # items 42
print(m.group(1))     # items
print(m.group(2))     # 42
print(m.groups())     # ["items", "42"]
print(m.start(1))     # 7
print(m.end(1))       # 12
print(m.span(1))      # [7, 12]
```

### 23.2.1 group(0) is the full match

```
import re

let m = re.search("([a-z]+) ([0-9]+)", "items 42")
print(m.group(0))   # items 42 (same as m.group())
print(m.group(1))   # items
print(m.group(2))   # 42
```

### 23.2.2 groups() returns all capture groups

```
import re

let m = re.search("([a-z]+) ([0-9]+)", "items 42")
print(m.groups())   # ["items", "42"]

let m2 = re.search("([0-9]+)", "abc")
print(m2.groups())  # ["42"]
```

### 23.2.3 start/end/span

```
import re

let m = re.search("([a-z]+) ([0-9]+)", "items 42")
print(m.start(0))   # 0
print(m.end(0))     # 9
print(m.span(0))    # [0, 9]
print(m.start(1))   # 7
print(m.end(1))     # 10
print(m.start(2))   # 11
print(m.end(2))     # 13
```

## 23.3 22.3 Find all matches

```
import re

let matches = re.findall("[0-9]+", "a1b23c456")
print(matches)  # ["1", "23", "456"]
```

`findall` returns a list of strings. If the pattern has capture groups, returns a list of tuples (or lists) of groups:

```
import re

let matches = re.findall("([a-z]+)([0-9]+)", "a1b23")
print(matches)  # [["a", "1"], ["b", "23"]]
```

## 23.4 22.4 Substitution

```
import re

let result = re.sub("[0-9]+", "#", "a1b23c456")
print(result)  # a#b#c#

# With count limit
let result2 = re.sub("[0-9]+", "#", "a1b23c456", 1)
print(result2) # a#b23c456
```

`re.sub(pattern, repl, text, count?)` replaces all (or first `count`) occurrences.

```
import re

print(re.sub("[aeiou]", "*", "hello world"))  # h*ll* w*rld
print(re.sub(" +", " ", "a   b    c"))        # "a b c"
print(re.sub("[0-9]+", "N", "a1b23c", 1))     # "aNb23c" (only first)
```

## 23.5 22.5 Split

```
import re

let parts = re.split("[,;]", "a,b;c,d")
print(parts)  # ["a", "b", "c", "d"]
```

`re.split(pattern, text)` splits on the regex pattern. Capture groups in the pattern are included in the result.

```
import re

print(re.split("[,;]", "a,b;c,d"))      # ["a", "b", "c", "d"]
print(re.split("([,;])", "a,b;c,d"))    # ["a", ",", "b", ";", "c", "d"]
```

## 23.6 22.6 Common patterns

| Pattern | Meaning |
|---|---|
| `.` | Any character except newline |
| `\d` | Digit |
| `\D` | Non-digit |
| `\w` | Word character (alnum + underscore) |
| `\W` | Non-word character |
| `\s` | Whitespace |
| `\S` | Non-whitespace |
| `^` | Start of string |
| `$` | End of string |
| `*` | Zero or more |

| Pattern | Meaning |
|---|---|
| `+` | One or more |
| `?` | Zero or one |
| `{n}` | Exactly n |
| `{n,}` | n or more |
| `{n,m}` | Between n and m |
| `\|` | Alternation (OR) |
| `(...)` | Capture group |
| `(?:...)` | Non-capturing group |
| `[...]` | Character class |
| `[^...]` | Negated character class |

### 23.6.1 Examples

```
import re

# Validate email
let email = re.match("^[\\w.-]+@[\\w.-]+\\.\\w+$", "user@example.com")
print(email != nil)  # true

# Extract hashtags
let tags = re.findall("#\\w+", "hello #zen #lang #script")
print(tags)  # ["#zen", "#lang", "#script"]

# Replace whitespace with single space
let cleaned = re.sub("\\s+", "hello   world", " ")
print(cleaned) # hello world

# Validate phone number
let phone = re.match("^\\d{3}-\\d{3}-\\d{4}$", "555-123-4567")
print(phone != nil)  # true
```

### 23.6.2 Escape sequences in Zen strings

In Zen strings, `\\d` becomes the regex `\d`. Use double backslashes:

```
import re

let pattern = "\\d{3}-\\d{3}-\\d{4}"  # matches "555-123-4567"
let m = re.match(pattern, "555-123-4567")
print(m != nil)  # true
```

## 23.7 22.7 Compiling for reuse

If you use the same pattern many times, compile it once:

```
import re

let email_re = re.compile("^[\\w.-]+@[\\w.-]+\\.\\w+$")

for addr in addresses {
    if email_re.match(addr) {
        print("valid: " + addr)
    }
}
```

## 23.8 22.8 Common pitfalls

- **Escape backslashes.** In Zen strings, `\\d` becomes the regex `\d`. Use raw strings if available, or double-escape: `"\\\\d"` for a literal backslash-d in the regex.
- **`re.match` anchors at the start** but not the end. Use `re.fullmatch` or add `$` for full-string matching.
- **`re.findall` with groups** returns tuples of groups, not full matches.
- **`re.sub` replaces ALL occurrences** by default; pass `count` to limit.

## 23.9 22.9 Quick reference

```
import re

re.match(pattern, text)      # match at start
re.search(pattern, text)     # search anywhere
re.fullmatch(pattern, text)  # match entire string
re.findall(pattern, text)    # all matches
re.sub(pattern, repl, text)  # replace
re.split(pattern, text)      # split on pattern
re.compile(pattern)          # compile for reuse

m.group()        # full match
m.group(1)       # capture group 1
m.groups()       # all groups
m.start(1) / m.end(1)  # positions
m.span(1)         # [start, end]
```

## 23.10 22.10 Exercises

1. Write `is_valid_email(email)` using a regex.
2. Write `extract_urls(text)` that finds all `http://` or `https://` URLs.
3. Write `mask_phone(phone)` that replaces digits with `X` except the last 4.
4. Write `camel_to_snake(name)` that converts `camelCase` to `camel_case`.
5. Write `highlight(text, word)` that wraps `word` occurrences in **.
6. Write `extract_hashtags(text)` that finds all `#word` patterns.
7. Write `validate_hex_color(s)` that checks for `#RRGGBB` format.

## 23.11 22.12 regex module — complete reference

| Function | Purpose | Example |
|---|---|---|
| `match(pattern, s)` | match at start | `re.match(r"\d+", "123abc")` |
| `search(pattern, s)` | search anywhere | `re.search(r"\d+", "abc123")` |
| `findall(pattern, s)` | all matches | `re.findall(r"\d+", "a1b2c3")` |
| `sub(pattern, repl, s)` | substitute | `re.sub(r"\d+", "#", "a1b2")` |
| `split(pattern, s)` | split on pattern | `re.split(r"\s+", "a b c")` |
| `compile(pattern)` | compile pattern | `re.compile(r"\d+")` |

### 23.11.1 Usage

```
import re
print(re.findall(r"\d+", "order 123, item 456"))   # ["123","456"]
print(re.sub(r"\s+", " ", "a   b    c"))           # "a b c"
m = re.match(r"(\d{4})-(\d{2})-(\d{2})", "2024-01-15")
print(m[0], m[1], m[2], m[3])   # if match object supports indexing
```

### 23.11.2 Common patterns

- `\d` digit, `\D` non-digit
- `\w` word char, `\W` non-word
- `\s` whitespace, `\S` non-whitespace
- `.` any char, `^` start, `$` end
- * zero+, `+` one+, `?` optional, `{n,m}` range
- `(...)` group, `|` alternation
- `\b` word boundary

### 23.11.3 Exercises

1. Write `is_email(s)` validating a simple email.
2. Implement `extract_urls(text)` finding http/https URLs.
3. Write `validate_phone(s)` for a phone format.
4. Implement `hashtags(text)` extracting #tags.
5. Write `mask_ssn(s)` masking all but last 4 digits.
6. Implement `word_split(text)` splitting on non-words.
7. Write `is_strong_password(s)` with length, digit, upper, lower, symbol.
8. Implement `template_fill(tmpl, vars)` replacing `${key}`.
9. Write `count_words(text)` ignoring punctuation.
10. Implement `highlight(text, pattern)` wrapping matches in **.

# 24 Chapter 23: base64 & binascii

The `base64` and `binascii` modules provide binary-to-text encoding: Base64, hex, and other binary representations.

## 24.1 23.1 base64

```
import base64

# Encode
let encoded = base64.b64encode("hello")
print(encoded)  # aGVsbG8=

# Decode
let decoded = base64.b64decode("aGVsbG8=")
print(decoded)  # hello

# URL-safe variant
let encoded_url = base64.b64encode_url("hello?x=1&y=2")
print(encoded_url)
let decoded_url = base64.b64decode_url(encoded_url)
print(decoded_url)
```

### 24.1.1 Functions

| Function | Description |
|---|---|
| `b64encode(data)` | Base64 encode |
| `b64decode(data)` | Base64 decode |
| `b64encode_url(data)` | URL-safe Base64 encode |
| `b64decode_url(data)` | URL-safe Base64 decode |

```
import base64

print(base64.b64encode("hello"))     # aGVsbG8=
print(base64.b64decode("aGVsbG8="))  # hello
print(base64.b64encode(""))          # ""
print(base64.b64decode(""))          # ""
```

Base64 encoding of binary data:

```
import base64
import fs

let data = fs.read_binary("image.png")
let encoded = base64.b64encode(data)
print(encoded)  # long base64 string
```

```
let decoded = base64.b64decode(encoded)
print(len(decoded) == len(data))  # true
```

## 24.2 23.2 binascii

```
import binascii

# Hex encode / decode
let hexed = binascii.hexlify("hello")
print(hexed)  # 68656c6c6f

let unhexed = binascii.unhexlify("68656c6c6f")
print(unhexed)  # hello
```

### 24.2.1 Functions

| Function | Description |
|---|---|
| `hexlify(data)` | Hex encode |
| `unhexlify(data)` | Hex decode |

```
import binascii

print(binascii.hexlify("hello"))      # 68656c6c6f
print(binascii.hexlify(""))           # ""
print(binascii.unhexlify("68656c6c6f"))  # hello
print(binascii.unhexlify(""))         # ""
```

Hex encoding produces lowercase hex digits:

```
print(binascii.hexlify("AB"))         # 4142
print(binascii.hexlify("ab"))         # 6162
```

`unhexlify` requires an even number of hex digits:

```
try {
    binascii.unhexlify("abc")  # odd length
} catch as e {
    print(e.msg)  # odd-length string
}
```

## 24.3 23.3 Common patterns

### 24.3.1 Encode binary for safe transport

```
import base64
import fs

let data = fs.read_binary("image.png")
let encoded = base64.b64encode(data)
# Send `encoded` over text-based protocols (email, JSON, URLs)
```

### 24.3.2 Decode received base64

```
import base64
import fs

let encoded = "iVBORw0KGgoAAAANSUhEUg..."  # received from network
let data = base64.b64decode(encoded)
fs.write_binary("out.png", data)
```

### 24.3.3 Hex dump

```
import binascii

let data = fs.read_binary("file.bin")
let hexed = binascii.hexlify(data)
# Print in 16-byte lines
for i in 0..len(hexed) step 32 {
    print(hexed[i:i+32])
}
```

### 24.3.4 Round-trip verification

```
import base64

let original = "binary data \x00\x01\x02"
let encoded = base64.b64encode(original)
let decoded = base64.b64decode(encoded)
print(decoded == original)  # true
```

```
import binascii

let original = "binary data"
let hexed = binascii.hexlify(original)
let decoded = binascii.unhexlify(hexed)
print(decoded == original)  # true
```

## 24.4 23.4 Quick reference

```
import base64
import binascii

base64.b64encode("hello")     # aGVsbG8=
base64.b64decode("aGVsbG8=")  # hello
base64.b64encode_url("a/b")   # use - and _ instead of + and /

binascii.hexlify("hello")     # 68656c6c6f
binascii.unhexlify("68656c6c6f")  # hello
```

## 24.5 23.5 Exercises

1. Write a function `encode_file(path)` that base64-encodes a file's contents.
2. Write `decode_file(encoded, path)` that decodes base64 and writes to a file.
3. Write `hex_dump(data)` that prints a hex dump with ASCII side column.
4. Write `safe_transfer(data)` that base64-encodes for URL-safe transport.
5. Write `round_trip(data)` that base64-encodes then decodes and verifies.
6. Write `is_valid_hex(s)` that checks if a string is valid hexadecimal.
7. Write `hex_to_bytes(s)` that converts hex to a list of byte values.

## 24.6 23.12 base64 & binascii — complete reference

| Function | Purpose | Example |
|---|---|---|
| `b64encode(data)` | base64 encode | `base64.b64encode("hello")` |
| `b64decode(s)` | base64 decode | `base64.b64encode(...)` → back |
| `url_encode(s)` / `url_decode(s)` | URL-safe base64 | `base64.url_encode(data)` |
| `hexlify(b)` | bytes to hex | `binascii.hexlify(bytes)` |
| `unhexlify(s)` | hex to bytes | `binascii.unhexlify("4865")` |

### 24.6.1 Usage

```
import base64
import binascii
let enc = base64.b64encode("hello")
print(enc)               # aGVsbG8=
print(base64.b64decode(enc))   # hello
let hx = binascii.hexlify(bytes([72,101,108,108,111]))
print(hx)                # 48656c6c6f
print(binascii.unhexlify(hx))
```

### 24.6.2 Exercises

1. Encode a string to base64 and decode back; verify round-trip.
2. Encode an image/file's bytes to base64.
3. Implement `decode_b64url(s)` using url_decode.
4. Write `hex_dump(b)` printing bytes as hex rows.
5. Implement `compare_hex(a, b)` case-insensitive hex compare.
6. Write `encode_file(path)` reading and base64-encoding contents.
7. Implement `is_valid_base64(s)`.
8. Write `rot13` conceptually (not base64, but a transform).
9. Implement `hex_to_bytes(s)` manually.
10. Write `bytes_to_hex(b)` manually.

## 24.7 23.12 Worked example — encode/decode round trip

```
import base64
import binascii

let text = "Hello, Zen!"
let enc = base64.b64encode(text)
print(enc)                          # base64 string
let dec = base64.b64decode(enc)
print(dec == text)                  # true

let raw = bytes([72, 101, 108, 108, 111])  # "Hello"
let hx = binascii.hexlify(raw)
print(hx)                           # 48656c6c6f
let back = binascii.unhexlify(hx)
print(back == raw)                  # true
```

## 24.8 23.13 Troubleshooting encoding

| Symptom | Cause | Fix |
|---|---|---|
| decode error | input isn't valid base64 | check padding / alphabet |
| mismatch round-trip | encoding altered data | use bytes, not string |
| wrong hex case | upper vs lower | normalize case before compare |

## 24.9 23.14 Exercises

1. Base64-encode a string and decode back; verify round-trip.
2. Hex-encode `bytes([1,2,3])` and decode back.
3. Encode a file's contents to base64.
4. Implement `is_valid_base64(s)`.
5. Write `hex_dump(b)` printing bytes as hex rows.

# 25 Chapter 24: hashlib Module

The `hashlib` module provides cryptographic hash functions: MD5, SHA-1, SHA-224/256/384/512, SHA-3, and BLAKE2/3.

> ⚠ Warning: MD5 and SHA-1 are cryptographically broken. Use them only for legacy compatibility or non-security purposes (e.g. checksums). For security-sensitive work, use SHA-256 or better.

## 25.1 24.1 Hash functions

| Function | Description | Example |
|---|---|---|
| `md5(data)` | MD5 digest | `hashlib.md5("hello")` |
| `sha1(data)` | SHA-1 digest | `hashlib.sha1("hello")` |
| `sha224(data)` | SHA-224 digest | `hashlib.sha224("hello")` |
| `sha256(data)` | SHA-256 digest | `hashlib.sha256("hello")` |
| `sha384(data)` | SHA-384 digest | `hashlib.sha384("hello")` |
| `sha512(data)` | SHA-512 digest | `hashlib.sha512("hello")` |
| `sha3_256(data)` | SHA3-256 digest | `hashlib.sha3_256("hello")` |
| `sha3_512(data)` | SHA3-512 digest | `hashlib.sha3_512("hello")` |
| `blake2b(data)` | BLAKE2b digest | `hashlib.blake2b("hello")` |
| `blake2s(data)` | BLAKE2s digest | `hashlib.blake2s("hello")` |

All return lowercase hex strings.

```
import hashlib

print(hashlib.md5("hello"))       # 5d41402abc4b2a76b9719d911017c592
print(hashlib.sha256("hello"))    # 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e730
print(hashlib.sha3_256("hello"))  # SHA3-256 hex
print(hashlib.blake2b("hello"))   # BLAKE2b hex
```

### 25.1.1 Output lengths

| Algorithm | Hex length | Bytes |
|---|---|---|
| MD5 | 32 | 16 |
| SHA-1 | 40 | 20 |
| SHA-224 | 56 | 28 |
| SHA-256 | 64 | 32 |
| SHA-384 | 96 | 48 |
| SHA-512 | 128 | 64 |
| SHA3-256 | 64 | 32 |
| SHA3-512 | 128 | 64 |
| BLAKE2b | 128 | 64 |
| BLAKE2s | 64 | 32 |

```
import hashlib

print(len(hashlib.md5("hello")))       # 32
print(len(hashlib.sha1("hello")))      # 40
print(len(hashlib.sha256("hello")))    # 64
print(len(hashlib.sha512("hello")))    # 128
```

## 25.2 24.2 File hashing

```
import hashlib
import fs

func hash_file(path) {
    let data = fs.read_binary(path)
    return hashlib.sha256(data)
}

let digest = hash_file("document.pdf")
print(digest)
```

### 25.2.1 Hash a large file in chunks

```
import hashlib
import fs

func hash_file_chunks(path, chunk_size = 8192) {
    let data = fs.read_binary(path)
    return hashlib.sha256(data)
    # For very large files, read in chunks and update incrementally
    # (if hashlib supports incremental hashing)
}
```

## 25.3 24.3 Verifying file integrity

```
import hashlib
import fs

let expected = "2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824"
let actual = hashlib.sha256(fs.read_binary("file.txt"))

if actual == expected {
    print("file intact")
} else {
    print("file corrupted or tampered")
}
```

### 25.3.1 Verify downloaded file

```
import hashlib
import fs
import requests

let url = "https://example.com/file.zip"
let expected_sha256 = "abc123..."

let resp = requests.get(url)
fs.write_binary("file.zip", resp.content)

let actual = hashlib.sha256(fs.read_binary("file.zip"))
if actual == expected_sha256 {
    print("download verified")
} else {
    print("download corrupted - delete and retry")
    fs.remove("file.zip")
}
```

## 25.4 24.4 Common pitfalls

- **Hashes are one-way.** You cannot recover the original data from a hash.
- **MD5/SHA-1 are broken** for collision resistance. Don't use them for signatures or security.
- **Hashes return hex strings**, not raw bytes. Use `binascii.unhexlify` if you need raw bytes.
- **Empty input has a defined hash.** `hashlib.sha256("")` is a well-known value:

  `e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855`

```
import hashlib

print(hashlib.sha256(""))  # e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b78
print(hashlib.md5(""))     # d41d8cd98f00b204e9800998ecf8427e
```

## 25.5 24.5 Quick reference

```
import hashlib

hashlib.md5("hello")        # md5 hex
hashlib.sha1("hello")       # sha1 hex
hashlib.sha256("hello")     # sha256 hex
hashlib.sha384("hello")     # sha384 hex
hashlib.sha512("hello")     # sha512 hex
hashlib.sha3_256("hello")   # sha3-256 hex
```

```
hashlib.sha3_512("hello")   # sha3-512 hex
hashlib.blake2b("hello")    # blake2b hex
hashlib.blake2s("hello")    # blake2s hex
```

## 25.6 24.6 Exercises

1. Write `file_fingerprint(path)` that returns the SHA-256 of a file.
2. Write `verify_checksum(path, expected)` that compares a file's hash.
3. Write `find_duplicates(files)` that groups files by identical hash.
4. Write a script that hashes all files in a directory and writes a manifest.
5. Compare `hashlib.md5` vs `hashlib.sha256` output lengths.
6. Write `verify_download(url, expected_hash)` that downloads and verifies.
7. Write `hash_compare(h1, h2)` that compares two hashes case-insensitively.

## 25.7 24.12 hashlib module — complete reference

| Function | Purpose | Example |
|---|---|---|
| `md5(data)` | MD5 digest | `hashlib.md5("hello")` |
| `sha1(data)` | SHA-1 digest | `hashlib.sha1("hello")` |
| `sha224(data)` | SHA-224 | `hashlib.sha224("hello")` |
| `sha256(data)` | SHA-256 | `hashlib.sha256("hello")` |
| `sha384(data)` | SHA-384 | `hashlib.sha384("hello")` |
| `sha512(data)` | SHA-512 | `hashlib.sha512("hello")` |
| `sha3_256(data)` | SHA3-256 | `hashlib.sha3_256("hello")` |
| `sha3_512(data)` | SHA3-512 | `hashlib.sha3_512("hello")` |
| `blake2b(data)` | BLAKE2b | `hashlib.blake2b("hello")` |
| `blake2s(data)` | BLAKE2s | `hashlib.blake2s("hello")` |
| `pbkdf2_hmac(pass, salt, iters)` | key derivation | `hashlib.pbkdf2_hmac(...)` |

### 25.7.1 Usage

```
import hashlib
print(hashlib.sha256("hello"))   # hex digest
print(hashlib.md5("hello"))      # 32 hex chars
print(hashlib.sha256("hello") == hashlib.sha256("hello"))  # true
print(hashlib.sha256("Hello") != hashlib.sha256("hello"))  # true (case sensitive)
```

### 25.7.2 Properties

- Deterministic: same input → same output.
- One-way: cannot reverse to input.
- Avalanche: tiny input change → very different output.
- Collision-resistant (for secure hashes like SHA-256).

### 25.7.3 Exercises

1. Compute SHA-256 of a string and verify it matches a known value.
2. Write `file_hash(path)` hashing a file's contents.
3. Implement `verify_checksum(data, expected)`.
4. Write `hash_password(password, salt)` combining salt + hash.
5. Implement `find_collision_prefix(prefix)` brute-force conceptually.
6. Compare MD5 vs SHA-256 output lengths.
7. Write `dedupe_files(paths)` grouping identical files by hash.
8. Implement `merkle_root(hashes)` conceptually.
9. Write `constant_time_compare(a, b)` conceptually.
10. Implement `pbkdf2` key derivation using hashlib.pbkdf2_hmac.

## 25.8 24.12 Worked example — file integrity checker

```
import hashlib
import sys
import pathlib

func file_sha256(path) {
    let b = pathlib.Path(path)
    if !b.exists() { throw "not found: " + path }
    let data = b.read_bytes()   # if supported; otherwise read chunks
    return hashlib.sha256(data)
}

func main(argv) {
    if len(argv) < 2 {
        print("usage: hashutil <file> [expected]")
        return 1
    }
    let path = argv[1]
    let expected = argv[2] ?? nil
    try {
        let actual = file_sha256(path)
        print("SHA-256(" + path + ") = " + actual)
        if expected != nil {
            if actual == expected { print("MATCH") }
            else { print("MISMATCH"); return 1 }
        }
    } catch as e {
        print("error: " + e.msg)
        return 1
    }
    return 0
}
```

```
if __name__ == "__main__" { sys.exit(main(sys.argv)) }
```

Sample:

```
$ zen run hashutil.z document.pdf
SHA-256(document.pdf) = a3f5...c9
$ zen run hashutil.z document.pdf a3f5...c9
SHA-256(document.pdf) = a3f5...c9
MATCH
```

## 25.9 24.13 Troubleshooting hashing

| Symptom | Cause | Fix |
|---|---|---|
| different hash for same file | file changed or encoding differs | re-hash exact bytes |
| hash mismatch expected vs actual | wrong algorithm or corrupted file | verify algorithm & source |
| slow on large files | reading whole file at once | hash in chunks |

Use SHA-256 or SHA-512 for integrity; avoid MD5/SHA-1 for security.

## 25.10 24.14 Exercises

1. SHA-256 a string and compare to a known value.
2. Hash a file and save the digest to a sidecar `.sha256` file.
3. Verify a file against a digest file.
4. Hash all files in a folder into a manifest.
5. Compare MD5 vs SHA-256 digest lengths.

# 26 Chapter 25: crypto Module — Complete Guide

The `crypto` module provides symmetric encryption, asymmetric keys, hashing, HMAC, key derivation, JWT, OTP, and X.509. This chapter covers the complete crypto API.

 Warning: Cryptography is easy to get wrong. Use well-established algorithms and modes. Never roll your own crypto for production security.

## 26.1 25.1 Symmetric encryption

```
import crypto

# Generate a key
let key = crypto.generate_key("aes-256-gcm")

# Encrypt
let ciphertext = crypto.encrypt_aes_gcm(key, "secret message")

# Decrypt
let plaintext = crypto.decrypt_aes_gcm(key, ciphertext)
print(plaintext)  # secret message
```

### 26.1.1 Symmetric functions

| Function | Description |
|---|---|
| `generate_key(alg?)` | Generate a key (“aes-256-gcm” or “chacha20-poly1305”) |
| `encrypt(alg, key, plaintext, aad?)` | Encrypt with given algorithm |
| `decrypt(alg, key, ciphertext, aad?)` | Decrypt with given algorithm |
| `encrypt_aes_gcm(key, plaintext, aad?)` | AES-256-GCM encrypt |
| `decrypt_aes_gcm(key, ciphertext, aad?)` | AES-256-GCM decrypt |
| `encrypt_chacha20_poly1305(key, plaintext, aad?)` | ChaCha20-Poly1305 encrypt |
| `decrypt_chacha20_poly1305(key, ciphertext, aad?)` | ChaCha20-Poly1305 decrypt |

```
import crypto

let key = crypto.generate_key("aes-256-gcm")
let ct = crypto.encrypt_aes_gcm(key, "hello world")
let pt = crypto.decrypt_aes_gcm(key, ct)
print(pt)  # hello world
```

```
let key2 = crypto.generate_key("chacha20-poly1305")
let ct2 = crypto.encrypt_chacha20_poly1305(key2, "secret")
let pt2 = crypto.decrypt_chacha20_poly1305(key2, ct2)
print(pt2)  # secret
```

### 26.1.2 With associated data (AAD)

AAD is authenticated but not encrypted. If you tamper with the ciphertext or AAD, decryption raises an error:

```
import crypto

let key = crypto.generate_key("aes-256-gcm")
let aad = "header-data"

let ct = crypto.encrypt_aes_gcm(key, "secret", aad)
let pt = crypto.decrypt_aes_gcm(key, ct, aad)
print(pt)  # secret

# Tampered AAD fails
try {
    crypto.decrypt_aes_gcm(key, ct, "tampered-aad")
} catch as e {
    print(e.msg)  # authentication failed
}
```

### 26.1.3 Generic encrypt/decrypt

```
import crypto

let key = crypto.generate_key("aes-256-gcm")

let ct = crypto.encrypt("aes-256-gcm", key, "plaintext")
let pt = crypto.decrypt("aes-256-gcm", key, ct)
print(pt)  # plaintext
```

## 26.2 25.2 Asymmetric encryption (RSA)

```
import crypto

let keys = crypto.generate_rsa(2048)
let public_key = keys.public_key
let private_key = keys.private_key

let ciphertext = crypto.rsa_encrypt(public_key, "secret")
```

```
let plaintext = crypto.rsa_decrypt(private_key, ciphertext)
print(plaintext)  # secret
```

### 26.2.1 Asymmetric functions

| Function | Description |
|---|---|
| `generate_rsa(bits?)` | Generate RSA keypair (default 2048) |
| `rsa_encrypt(public_key, plaintext)` | RSA encrypt |
| `rsa_decrypt(private_key, ciphertext)` | RSA decrypt |
| `rsa_sign(private_key, message)` | RSA sign |
| `rsa_verify(public_key, message, sig)` | RSA verify → bool |

```
import crypto

let keys = crypto.generate_rsa(2048)

print(keys.public_key)   # PEM public key
print(keys.private_key)  # PEM private key

let ct = crypto.rsa_encrypt(keys.public_key, "top secret")
let pt = crypto.rsa_decrypt(keys.private_key, ct)
print(pt)  # top secret
```

RSA encryption has size limits (key size / padding overhead). For large data, use hybrid encryption: encrypt data with a random symmetric key, encrypt the key with RSA.

## 26.3 25.3 Digital signatures

### 26.3.1 RSA sign/verify

```
import crypto

let keys = crypto.generate_rsa(2048)
let message = "hello world"

let sig = crypto.rsa_sign(keys.private_key, message)
let valid = crypto.rsa_verify(keys.public_key, message, sig)
print(valid)  # true

# Tampered message fails
let invalid = crypto.rsa_verify(keys.public_key, "tampered", sig)
print(invalid)  # false
```

### 26.3.2 Ed25519 sign/verify

```
import crypto

let keys = crypto.generate_ed25519()
let message = "hello world"

let sig = crypto.ed25519_sign(keys.private_key, message)
let valid = crypto.ed25519_verify(keys.public_key, message, sig)
print(valid)  # true

# Tampered fails
let invalid = crypto.ed25519_verify(keys.public_key, "tampered", sig)
print(invalid)  # false
```

### 26.3.3 Signature functions

| Function | Description |
|---|---|
| `rsa_sign(private_key, message)` | RSA sign → signature |
| `rsa_verify(public_key, message, sig)` | RSA verify → bool |
| `ed25519_sign(private_key, message)` | Ed25519 sign → signature |
| `ed25519_verify(public_key, message, sig)` | Ed25519 verify → bool |

```
import crypto

let keys = crypto.generate_ed25519()
let msg = "important document"

let sig = crypto.ed25519_sign(keys.private_key, msg)

# Anyone with the public key can verify
let ok1 = crypto.ed25519_verify(keys.public_key, msg, sig)
print(ok1)  # true

let ok2 = crypto.ed25519_verify(keys.public_key, "tampered", sig)
print(ok2)  # false
```

## 26.4 25.4 Key agreement (X25519)

```
import crypto

let alice = crypto.generate_x25519()
let bob = crypto.generate_x25519()

let alice_secret = crypto.x25519_shared_secret(alice.private_key, bob.public_key)
```

```
let bob_secret = crypto.x25519_shared_secret(bob.private_key, alice.public_key)

print(alice_secret == bob_secret)  # true (shared secret matches)
```

### 26.4.1 X25519 functions

| Function | Description |
|---|---|
| `generate_x25519()` | Generate X25519 keypair |
| `x25519_shared_secret(private_key, public_key)` | Compute shared secret |

```
import crypto

# Two parties establish a shared secret without transmitting it
let alice = crypto.generate_x25519()
let bob = crypto.generate_x25519()

// Alice computes: her private key + Bob's public key
let alice_shared = crypto.x25519_shared_secret(alice.private_key, bob.public_key)
// Bob computes: his private key + Alice's public key
let bob_shared = crypto.x25519_shared_secret(bob.private_key, alice.public_key)

print(alice_shared == bob_shared)  # true
```

## 26.5 25.5 Hashing and HMAC

```
import crypto

print(crypto.hash("sha256", "hello"))
print(crypto.hmac("sha256", "key", "message"))
```

Hash algorithms: `md5`, `sha1`, `sha224`, `sha256`, `sha384`, `sha512`, `sha3_256`, `sha3_512`, `blake2b`, `blake2s`.

HMAC algorithms: `sha256`, `sha512`.

```
import crypto

print(crypto.hash("sha256", "hello"))     # 2cf24dba...
print(crypto.hash("sha512", "hello"))     # longer hex
print(crypto.hash("blake2b", "hello"))    # BLAKE2b
print(crypto.hash("md5", "hello"))        # MD5 (broken, don't use for security)

let mac = crypto.hmac("sha256", "secret-key", "message")
print(mac)  # HMAC-SHA256 hex
```

```
let mac2 = crypto.hmac("sha512", "secret-key", "message")
print(mac2)  # HMAC-SHA512 hex
```

HMAC for message authentication:

```
import crypto

let key = "shared-secret"
let message = "authenticate this"

let mac = crypto.hmac("sha256", key, message)

# Receiver with same key verifies
let expected = crypto.hmac("sha256", key, message)
print(mac == expected)  # true

let wrong = crypto.hmac("sha256", "wrong-key", message)
print(mac == wrong)  # false
```

## 26.6 25.6 Key derivation (PBKDF2)

```
import crypto

let key = crypto.pbkdf2("password", "salt", 100000, 32)
print(key)  # derived key (hex)
```

### 26.6.1 PBKDF2

```
import crypto

# Derive a key from a password
let derived = crypto.pbkdf2("my-password", "random-salt", 100000, 32)
print(derived)  # 64 hex chars (32 bytes)

# More iterations = slower but more secure
let slow = crypto.pbkdf2("password", "salt", 1000000, 32)
```

Use PBKDF2 for password hashing (though Argon2/scrypt are preferred in modern systems). Always use a unique random salt per password.

## 26.7 25.7 JWT (JSON Web Tokens)

```
import crypto

let payload = {"sub": "123", "name": "Zen"}
let secret = "my-secret"
```

```
let token = crypto.jwt_encode(payload, secret)
print(token)  # eyJ...

let decoded = crypto.jwt_decode(token, secret)
print(decoded["sub"])  # 123
```

### 26.7.1 JWT functions

| Function | Description |
|---|---|
| `jwt_encode(payload, secret)` | Create JWT |
| `jwt_decode(token, secret)` | Verify and decode JWT |

```
import crypto

let payload = {"sub": "user-42", "role": "admin", "exp": 9999999999}
let secret = "super-secret"

let token = crypto.jwt_encode(payload, secret)
print(token)  # eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

let decoded = crypto.jwt_decode(token, secret)
print(decoded["sub"])    # user-42
print(decoded["role"])   # admin

# Tampered token fails
try {
    crypto.jwt_decode(token, "wrong-secret")
} catch as e {
    print(e.msg)  # signature verification failed
}
```

## 26.8 25.8 OTP (TOTP/HOTP)

```
import crypto

let totp = crypto.otp_totp("secret")
print(totp)  # 6-digit code

let valid = crypto.otp_verify("secret", "123456")
print(valid)  # true/false
```

### 26.8.1 OTP functions

| Function | Description |
|---|---|
| `otp_totp(secret)` | Generate TOTP code |
| `otp_verify(secret, code)` | Verify TOTP code → bool |

```
import crypto

let secret = "JBSWY3DPEHPK3PXP"

let code = crypto.otp_totp(secret)
print(code)  # 6-digit time-based code

let valid = crypto.otp_verify(secret, code)
print(valid)  # true (if within time window)

let invalid = crypto.otp_verify(secret, "000000")
print(invalid)  # false
```

TOTP codes change every 30 seconds. `otp_verify` accepts codes within a small time window (±1 step) to account for clock skew.

## 26.9 25.9 X.509 certificates

```
import crypto

let cert = crypto.x509_parse(cert_pem)
print(cert.subject)
print(cert.issuer)
print(cert.not_after)
```

### 26.9.1 X.509 functions

| Function | Description |
|---|---|
| `x509_parse(pem)` | Parse PEM certificate |

```
import crypto

let pem = "-----BEGIN CERTIFICATE-----\n...\n-----END CERTIFICATE-----"

let cert = crypto.x509_parse(pem)
print(cert.subject)    # subject DN
print(cert.issuer)     # issuer DN
print(cert.not_after)  # expiry
print(cert.not_before) # validity start
```

## 26.10 25.10 Constant-time compare and secure zero

```
import crypto

# Timing-safe comparison (for HMACs, tokens, passwords)
let ok = crypto.constant_time_eq(expected, received)
print(ok)  # true/false

# Zeroize a secret after use
let pass = read_secret()
# ... use pass ...
crypto.secure_zero(pass)
pass = nil
```

```
import crypto

// Constant-time comparison prevents timing attacks
let expected = compute_hmac(key, message)
let received = hmac_from_request()

if crypto.constant_time_eq(expected, received) {
    print("authenticated")
} else {
    print("rejected")
}

// Secure zero - overwrite secret in memory
let password = read_user_password()
authenticate(password)
crypto.secure_zero(password)
password = nil
```

## 26.11 25.11 Algorithm policy (FIPS-ish mode)

```
import crypto

crypto.policy("strict")   # block md5, sha1, weak ciphers
crypto.policy("default")  # allow all
```

Or set `ZEN_FIPS=1` in the environment.

```
import crypto

// Check current policy behavior
try {
    crypto.hash("md5", "data")  // raises if strict
} catch as e {
    print(e.msg)  // blocked by crypto policy
```

```
}

crypto.policy("strict")
try {
    crypto.hash("md5", "data")  // still blocked
} catch as e {
    print("blocked as expected")
}

crypto.policy("default")
print(crypto.hash("md5", "data"))  # works again
```

### 26.11.1 Blocked algorithms in strict mode

| Algorithm | Blocked in strict? |
|---|---|
| md5 | yes |
| sha1 | yes |
| sha224 | yes |
| sha256 | no |
| sha384 | no |
| sha512 | no |
| sha3_256 | no |
| sha3_512 | no |
| blake2b | no |
| blake2s | no |
| des | yes |
| rc4 | yes |

## 26.12 25.12 Quick reference

```
import crypto

# Symmetric
let key = crypto.generate_key("aes-256-gcm")
let ct = crypto.encrypt_aes_gcm(key, "secret")
let pt = crypto.decrypt_aes_gcm(key, ct)

# Asymmetric
let keys = crypto.generate_rsa(2048)
let ct = crypto.rsa_encrypt(keys.public_key, "secret")
let pt = crypto.rsa_decrypt(keys.private_key, ct)

# Signatures
let sig = crypto.ed25519_sign(keys.private_key, "msg")
let ok = crypto.ed25519_verify(keys.public_key, "msg", sig)
```

```
# Key agreement
let shared = crypto.x25519_shared_secret(a.private_key, b.public_key)

# Hashing
print(crypto.hash("sha256", "hello"))
print(crypto.hmac("sha256", "key", "msg"))

# KDF
let derived = crypto.pbkdf2("password", "salt", 100000, 32)

# JWT
let token = crypto.jwt_encode({"sub": "1"}, "secret")
let decoded = crypto.jwt_decode(token, "secret")

# Security utils
crypto.constant_time_eq(a, b)
crypto.secure_zero(secret)
crypto.policy("strict")
```

## 26.13 25.13 Exercises

1. Encrypt and decrypt a message with AES-256-GCM.
2. Generate an Ed25519 keypair, sign a message, and verify it.
3. Perform X25519 key agreement between two parties and confirm shared secrets match.
4. Create a JWT, decode it, and verify tampering fails.
5. Generate a TOTP code and verify it.
6. Write `hash_password(password, salt)` using PBKDF2.
7. Write `verify_password(password, salt, expected_hash)` using PBKDF2.
8. Demonstrate `crypto.constant_time_eq` vs regular == for timing safety.

## 26.14 25.12 crypto module — advanced reference

### 26.14.1 Hashing

| Function | Purpose |
|---|---|
| `sha256/384/512`, `sha3_256/512`, `blake3`, `md5`, `sha1` | cryptographic hashes |

### 26.14.2 HMAC

| Function | Purpose |
|---|---|
| `sha256(key, data)` / `sha512(key, data)` | keyed hash |

### 26.14.3 Symmetric encryption

| Function | Purpose |
|---|---|
| `generate_key(alg)` | generate a key |
| `encrypt(alg, key, plaintext, aad="")` | encrypt |
| `decrypt(alg, key, ciphertext, aad="")` | decrypt |
| `encrypt_aes_gcm(key, plaintext, aad="")` | AES-GCM |
| `decrypt_aes_gcm(key, ciphertext, aad="")` | AES-GCM |
| `encrypt_chacha20_poly1305(...)` | ChaCha20-Poly1305 |
| `decrypt_chacha20_poly1305(...)` | ChaCha20-Poly1305 |

### 26.14.4 Asymmetric (public key)

| Function | Purpose |
|---|---|
| `generate_rsa(bits=2048)` | RSA keypair |
| `rsa_encrypt(pub, pt)` / `rsa_decrypt(priv, ct)` | RSA encrypt/decrypt |
| `rsa_sign(priv, msg)` / `rsa_verify(pub, msg, sig)` | RSA sign/verify |
| `generate_ed25519()` | Ed25519 keypair |
| `ed25519_sign(priv, msg)` / `ed25519_verify(pub, msg, sig)` | Ed25519 |
| `generate_x25519()` | X25519 keypair |
| `x25519_shared_secret(priv, pub)` | Diffie-Hellman shared secret |

### 26.14.5 Encoding

| Function | Purpose |
|---|---|
| `b64encode/decode`, `b64urlencode/decode`, `hex_encode/decode` | encodings |

### 26.14.6 JWT

| Function | Purpose |
|---|---|
| `sign(payload, key, alg="HS256")` | sign JWT |
| `verify(token, key, algorithms=["HS256"])` | verify JWT |
| `decode(token)` | decode JWT (no verify) |

### 26.14.7 KDF / passwords

| Function | Purpose |
|---|---|
| `pbkdf2(password, salt, iterations, length)` | PBKDF2 |
| `argon2id(password, salt, memory, iterations, parallelism, length)` | Argon2id |
| `argon2id_hash(password)` | hash password |
| `verify_argon2id(password, hash)` | verify password hash |

### 26.14.8 OTP

| Function | Purpose |
|---|---|
| `totp(secret, time_step, digits)` | TOTP token |
| `totp_verify(secret, token, window, time_step, digits)` | verify TOTP |
| `hotp(secret, counter, digits)` | HOTP token |
| `hotp_verify(secret, token, counter, window, digits)` | verify HOTP |
| `generate_secret()` / `bytes(n)` / `hex(n)` / `base64(n)` / `uuid4()` / `secure_token()` | randomness |

### 26.14.9 X.509

| Function | Purpose |
|---|---|
| `parse(pem)` | parse certificate |

### 26.14.10 Usage

```
import crypto
let key = crypto.generate_key("aes-256-gcm")
let ct = crypto.encrypt_aes_gcm(key, "secret message")
let pt = crypto.decrypt_aes_gcm(key, ct)
print(pt)   # secret message

let pair = crypto.generate_ed25519()
let sig = crypto.ed25519_sign(pair.private_key, "message")
print(crypto.ed25519_verify(pair.public_key, "message", sig))  # true

let token = crypto.sign(#{sub: "alice"}, "my-secret")
let payload = crypto.verify(token, "my-secret")
print(payload["sub"])   # alice

let pw_hash = crypto.argon2id_hash("hunter2")
print(crypto.verify_argon2id("hunter2", pw_hash))   # true
```

```
print(crypto.verify_argon2id("wrong", pw_hash))     # false

print(crypto.totp("JBSWY3DPEHPK3PXP"))   # 6-digit token
```

### 26.14.11 Security best practices

1. Never use MD5/SHA-1 for security; use SHA-256 or better.
2. Use Argon2id or PBKDF2 for password hashing (with salt).
3. Use AES-GCM or ChaCha20-Poly1305 for encryption (authenticated).
4. Verify JWT signatures before trusting payloads.
5. Use `crypto.secure_token`/`secrets`, not `random`, for tokens.
6. Rotate keys regularly; never hardcode secrets.
7. Constant-time comparison for secrets.

### 26.14.12 Exercises

1. Encrypt/decrypt a string with AES-GCM; verify round-trip.
2. Sign/verify a message with Ed25519.
3. Generate an RSA keypair; encrypt with public, decrypt with private.
4. Compute `x25519_shared_secret` for two keypairs; both sides match.
5. Sign a JWT and verify it; tamper with payload and confirm verification fails.
6. Hash a password with Argon2id and verify correct/wrong passwords.
7. Generate a TOTP token and verify it within the time window.
8. Compute HMAC-SHA256 of a message with a key.
9. Compare timing of constant-time vs naive comparison (conceptually).
10. Build a small “vault” CLI: encrypt a file, decrypt with passphrase.

# 27 Chapter 26: Networking Fundamentals

Zen's networking stack provides sockets, DNS, HTTP, TLS, and raw packet crafting. This chapter covers the fundamentals: IP addressing, TCP vs UDP, ports, the OSI model mapping, and how Zen's networking modules fit together.

## 27.1 26.1 Networking module map

Zen's networking is split across several modules, each at a different layer:

| Module | Layer | Purpose |
|---|---|---|
| `socket` | Transport (L4) | TCP/UDP sockets, `connect`, `bind`, `listen`, `accept`, `send`, `recv` |
| `net` | Transport (L4) | Higher-level TCP/UDP helpers (`tcp_connect`, `tcp_send`, etc.) |
| `net.dns` | Application (L7) | DNS resolution (`resolve`, `gethostbyname`) |
| `requests` | Application (L7) | HTTP client (GET/POST/etc., sessions, TLS) |
| `http` | Application (L7) | Low-level HTTP client |
| `http.server` | Application (L7) | Static file HTTP server |
| `tls` | Security (L4.5) | TLS connections with certificate pinning |
| `raw` / `net_raw` | Network (L3) | Raw IP sockets, packet crafting (`pack_ip`, `pack_tcp`) |
| `sys` | System | File descriptors, pipes, syscalls |

## 27.2 26.2 IP addressing

Zen works with IPv4 addresses (dotted-decimal strings like `"192.168.1.1"`). IPv6 support depends on the platform.

```
import socket
import dns

// Resolve a hostname to IP addresses
let addrs = dns.resolve("example.com")
for addr in addrs {
    print(addr)  # "93.184.216.34" or "2606:2800:220:1:248:1893:25c8:1946"
}

// localhost
print(dns.resolve("localhost"))  # ["127.0.0.1", "::1"]
```

### 27.2.1 Private IP ranges (RFC 1918)

| Range | Purpose |
|---|---|
| `10.0.0.0` – `10.255.255.255` | Private (class A) |
| `172.16.0.0` – `172.31.255.255` | Private (class B) |
| `192.168.0.0` – `192.168.255.255` | Private (class C) |
| `127.0.0.0` – `127.255.255.255` | Loopback (localhost) |
| `169.254.0.0` – `169.254.255.255` | Link-local (APIPA) |

```
import dns

// Localhost always resolves
print(dns.resolve("127.0.0.1"))   # ["127.0.0.1"]
print(dns.resolve("localhost"))   # ["127.0.0.1", ...]
```

## 27.3 26.3 Ports

Ports are 16-bit integers (0–65535). Ports 0–1023 are “well-known” and typically require root privileges to bind.

| Port | Service |
|---|---|
| 20 | FTP data |
| 21 | FTP control |
| 22 | SSH |
| 23 | Telnet |
| 25 | SMTP |
| 53 | DNS |
| 67/68 | DHCP |
| 80 | HTTP |
| 110 | POP3 |
| 123 | NTP |
| 143 | IMAP |
| 161/162 | SNMP |
| 389 | LDAP |
| 443 | HTTPS |
| 465 | SMTPS |
| 587 | SMTP submission |
| 636 | LDAPS |
| 993 | IMAPS |
| 995 | POP3S |
| 1433 | MSSQL |
| 3306 | MySQL |
| 3389 | RDP |
| 5432 | PostgreSQL |
| 5900 | VNC |
| 6379 | Redis |
| 27017 | MongoDB |

| Port | Service |
|---|---|
| 8080 | HTTP alternate |

```
import socket

// Bind to port 80 typically requires root
let server = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.bind(server, "0.0.0.0", 80)   # may fail without root
socket.bind(server, "0.0.0.0", 8080)  # usually OK
```

## 27.4 26.4 TCP vs UDP

| Feature | TCP | UDP |
|---|---|---|
| Connection | Connection-oriented (handshake) | Connectionless |
| Reliability | Reliable (retransmission, ACK) | Unreliable (no guarantee) |
| Ordering | Ordered | Unordered |
| Flow control | Yes | No |
| Congestion control | Yes | No |
| Header overhead | 20+ bytes | 8 bytes |
| Use case | Web, file transfer, email | DNS, streaming, VoIP, games |

```
import socket

// TCP socket (stream)
let tcp = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.connect(tcp, "example.com", 80)
socket.send(tcp, "GET / HTTP/1.0\r\n\r\n")
let resp = socket.recv(tcp, 4096)
socket.close(tcp)

// UDP socket (datagram)
let udp = socket.create(socket.AF_INET, socket.SOCK_DGRAM)
socket.send_to(udp, "query", "8.8.8.8", 53)
let [data, addr] = socket.recv_from(udp, 4096)
socket.close(udp)
```

## 27.5 26.5 The OSI model and Zen

| OSI Layer | Name | Zen module | Example |
|---|---|---|---|
| 7 | Application | `requests`, `http`, `http.server`, `dns` | HTTP GET, DNS resolve |
| 6 | Presentation | `base64`, `binascii`, `crypto` | Encoding, encryption |
| 5 | Session | `requests.Session` | HTTP sessions |
| 4 | Transport | `socket`, `net`, `tls` | TCP, UDP, TLS |
| 3 | Network | `raw`, `net_raw` | IP packets, ICMP |
| 2 | Data Link | — | (not exposed) |
| 1 | Physical | — | (not exposed) |

Zen focuses on layers 3–7. Layers 1–2 (physical, data link) are handled by the OS/kernel.

## 27.6 26.6 Sockets: file descriptors

Zen sockets use integer file descriptors (handles) into an internal resource table:

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
print(sock)   # 3 (an integer handle)
socket.connect(sock, "example.com", 80)
socket.close(sock)   # release the handle
```

Always close sockets when done to avoid resource leaks:

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
try {
    socket.connect(sock, "example.com", 80)
    socket.send(sock, "GET / HTTP/1.0\r\n\r\n")
    let resp = socket.recv(sock, 4096)
    print(resp)
} finally {
    socket.close(sock)   # always close
}
```

## 27.7 26.6 Address families

| Constant | Meaning |
|---|---|
| `socket.AF_INET` | IPv4 |
| `socket.AF_INET6` | IPv6 (platform-dependent) |
| `socket.AF_UNIX` | Unix domain sockets (platform-dependent) |

### 27.7.1 Socket types

| Constant | Meaning |
|---|---|
| `socket.SOCK_STREAM` | TCP (reliable stream) |
| `socket.SOCK_DGRAM` | UDP (unreliable datagram) |
| `socket.SOCK_RAW` | Raw IP (requires root) |

```
import socket

let tcp = socket.create(socket.AF_INET, socket.SOCK_STREAM)
let udp = socket.create(socket.AF_INET, socket.SOCK_DGRAM)
let raw = socket.create(socket.AF_INET, socket.SOCK_RAW)  # may need root
```

## 27.8 26.6 Timeouts

Socket operations can block indefinitely. Always set timeouts:

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.settimeout(sock, 5000)   # 5 second timeout

try {
    socket.connect(sock, "slow.example.com", 80)
    socket.send(sock, "GET / HTTP/1.0\r\n\r\n")
    let resp = socket.recv(sock, 4096)
    print(resp)
} catch as e {
    print(e.msg)  # timeout or connection error
} finally {
    socket.close(sock)
}
```

## 27.9 26.6 Byte order

Network byte order is big-endian. Zen's socket functions handle this internally. When crafting raw packets with `net_raw`, you must manually use big-endian:

```
import net_raw

// pack_ip and pack_tcp handle byte order internally
let pkt = net_raw.pack_ip("1.2.3.4", "5.6.7.8", 6)
```

## 27.10 26.7 Quick reference

```
import socket
import dns
import net_raw

// Resolve
let addrs = dns.resolve("example.com")

// TCP client
let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.settimeout(sock, 5000)
socket.connect(sock, addrs[0], 80)
socket.send(sock, "GET / HTTP/1.0\r\nHost: example.com\r\n\r\n")
let resp = socket.recv(sock, 4096)
socket.close(sock)

// UDP
let udp = socket.create(socket.AF_INET, socket.SOCK_DGRAM)
socket.send_to(udp, "ping", "127.0.0.1", 9999)
let [data, addr] = socket.recv_from(udp, 4096)
socket.close(udp)

// Raw IP packet
let ip_pkt = net_raw.pack_ip("1.2.3.4", "5.6.7.8", 6)
let tcp_pkt = net_raw.pack_tcp(12345, 80, "SYN")
```

## 27.11 26.8 Exercises

1. Write a function `resolve_all(host)` that prints all IP addresses for a hostname.
2. Write a TCP echo client that sends a line and prints the response.
3. Write a UDP client that sends a datagram and waits for a reply.
4. Write a port scanner for ports 1–1000 on localhost with a 100ms timeout per port.
5. Create a raw IP packet with `net_raw.pack_ip` and inspect the first 20 bytes.
6. Write a function `is_port_open(host, port, timeout)` that returns true if a TCP connect succeeds.
7. Demonstrate that `socket.close` releases the file descriptor by opening and closing multiple sockets.

## 27.12 26.12 Socket address families and types (complete reference)

### 27.12.1 Address families

| Family | Constant | Purpose |
|---|---|---|
| AF_INET | IPv4 | Internet Protocol v4 |
| AF_INET6 | IPv6 | Internet Protocol v6 |
| AF_UNIX/AF_LOCAL | Unix domain | Local IPC (file/socket path) |
| AF_PACKET | Raw packet | Low-level packet access (Linux) |

| Family | Constant | Purpose |
|---|---|---|

### 27.12.2 Socket types

| Type | Purpose | Protocol |
|---|---|---|
| SOCK_STREAM | Reliable, ordered, connection-based | TCP (usually) |
| SOCK_DGRAM | Unreliable, unordered, datagram | UDP |
| SOCK_RAW | Raw access to IP/ICMP etc | IP |
| SOCK_SEQPACKET | Sequenced packets | SCTP etc |

### 27.12.3 Protocols (IPPROTO_*)

| Protocol | Use |
|---|---|
| IPPROTO_TCP | TCP |
| IPPROTO_UDP | UDP |
| IPPROTO_ICMP | ICMP |
| IPPROTO_IP | IP |

## 27.13 26.13 Berkeley sockets API mapping

Common calls: `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `send()/sendall()`, `recv()`, `sendto()/recvfrom()`, `close()`, `setsockopt()`, `getsockname()`, `getpeername()`, `shutdown()`.

## 27.14 26.14 TCP connection lifecycle (detailed)

**Server:** 1. `socket(AF_INET, SOCK_STREAM, 0)` → sock 2. `setsockopt(SOL_SOCKET, SO_REUSEADDR, 1)` (optional but common) 3. `bind((host, port))` 4. `listen(backlog)` 5. `accept()` → (client_sock, addr) [blocks] 6. `recv/send` on client_sock 7. `close()`

**Client:** 1. `socket(AF_INET, SOCK_STREAM, 0)` 2. `connect((host, port))` 3. `send/recv` 4. `close()`

## 27.15 26.15 UDP communication

**Server:** bind, recvfrom, sendto (no listen/accept) **Client:** socket, sendto/recvfrom or connect (connected UDP)

## 27.16 26.16 Socket options (important)

| Option | Level | Purpose |
|---|---|---|
| SO_REUSEADDR | SOL_SOCKET | Allow reuse of local address |
| SO_REUSEPORT | SOL_SOCKET | Allow multiple binds to same port (load balancing) |
| SO_KEEPALIVE | SOL_SOCKET | TCP keepalive probes |
| SO_RCVBUF/SO_SNDBUF | SOL_SOCKET | Buffer sizes |
| SO_TIMEOUT | SOL_SOCKET | Timeouts |
| TCP_NODELAY | IPPROTO_TCP | Disable Nagle's algorithm |
| IP_TTL | IPPROTO_IP | Time to live |
| IP_HDRINCL | IPPROTO_IP | Include IP header in raw packets |

## 27.17 26.17 Blocking vs non-blocking

- Blocking: calls wait until ready/completed
- Non-blocking: return immediately (EAGAIN/EWOULDBLOCK)
- Use `select()`, `poll()`, or async/event loops for multiplexing

## 27.18 26.18 Byte order (network order)

Network byte order is big-endian. Use `htons()`, `ntohs()`, `htonl()`, `ntohl()` when working with raw values/ports in some APIs. In high-level APIs often handled automatically.

## 27.19 26.19 Common ports

| Port | Service |
|---|---|
| 20/21 | FTP |
| 22 | SSH |
| 23 | Telnet |
| 25 | SMTP |
| 53 | DNS |
| 67/68 | DHCP |
| 80 | HTTP |
| 110 | POP3 |
| 143 | IMAP |
| 443 | HTTPS |
| 993 | IMAPS |
| 995 | POP3S |

## 27.20 26.20 Network troubleshooting checklist

1. **Connectivity**: ping, traceroute/tracert
2. **DNS**: nslookup/dig
3. **Ports**: netstat/ss, nmap

4. **Packets**: tcpdump, wireshark
5. **Routing**: ip route/route
6. **Interfaces**: ip addr/ifconfig
7. **Firewall**: iptables/nftables/ufw

## 27.21 26.21 Lab exercises (comprehensive)

1. Draw/describe packet flow for HTTP request across all 7 OSI layers
2. Capture TCP 3-way handshake with tcpdump (theory + practice)
3. Write pseudocode for echo server using sockets
4. Compare TCP vs UDP for video streaming, DNS, file transfer
5. Explain MTU/path MTU discovery and fragmentation
6. Demonstrate ARP resolution process
7. Trace ICMP ping through layers
8. List which headers appear at each layer for TCP/IP over Ethernet
9. Explain NAT and its impact
10. Research and document difference between IPv4 and IPv6 headers

## 27.22 26.22 Further reading

- RFC 793 (TCP), RFC 768 (UDP), RFC 791 (IP)
- IEEE 802.3 (Ethernet)
- Stevens' Unix Network Programming

# 28 Chapter 27: socket Module

The `socket` module provides low-level TCP/UDP/raw sockets. It's Tier 1 of Zen's networking stack — thin wrappers over the OS socket API with integer file descriptors (handles).

## 28.1 27.1 Socket creation

```
import socket

// TCP socket (IPv4)
let tcp = socket.create(socket.AF_INET, socket.SOCK_STREAM)

// UDP socket (IPv4)
let udp = socket.create(socket.AF_INET, socket.SOCK_DGRAM)

// Raw socket (requires root on most OSes)
let raw = socket.create(socket.AF_INET, socket.SOCK_RAW)
```

### 28.1.1 Address families and socket types

| Constant | Meaning |
|---|---|
| `socket.AF_INET` | IPv4 |
| `socket.AF_INET6` | IPv6 (platform-dependent) |
| `socket.AF_UNIX` | Unix domain sockets (platform-dependent) |
| `socket.SOCK_STREAM` | TCP (reliable stream) |
| `socket.SOCK_DGRAM` | UDP (unreliable datagram) |
| `socket.SOCK_RAW` | Raw IP (requires root) |

### 28.1.2 Common combinations

| Family | Type | Protocol | Use |
|---|---|---|---|
| `AF_INET` | `SOCK_STREAM` | TCP | Web, file transfer, email |
| `AF_INET` | `SOCK_DGRAM` | UDP | DNS, streaming, games |
| `AF_INET` | `SOCK_RAW` | IP | Packet crafting, ping |
| `AF_INET6` | `SOCK_STREAM` | TCP over IPv6 | IPv6 services |
| `AF_UNIX` | `SOCK_STREAM` | Local | IPC on same machine |

## 28.2 27.2 Socket handle operations

| Function | Description |
|---|---|
| `socket.create(domain, type)` | Create socket, returns handle (int) |

| Function | Description |
|---|---|
| `socket.connect(sock, host, port)` | Connect TCP socket to remote |
| `socket.bind(sock, host, port)` | Bind socket to local address |
| `socket.listen(sock, backlog)` | Listen for incoming connections |
| `socket.accept(sock)` | Accept incoming connection, returns new socket handle |
| `socket.send(sock, data)` | Send data on connected socket |
| `socket.recv(sock, n)` | Receive up to n bytes (returns bytes) |
| `socket.recv_text(sock, n)` | Receive text |
| `socket.recv_from(sock, n)` | Receive from address (UDP), returns `{data, addr}` |
| `socket.send_to(sock, data, host, port)` | Send to address (UDP) |
| `socket.settimeout(sock, ms)` | Set timeout in milliseconds |
| `socket.close(sock)` | Close socket and release handle |
| `socket.fileno(sock)` | Get raw file descriptor |

## 28.3 27.3 TCP client

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.settimeout(sock, 5000)   // 5 second timeout

socket.connect(sock, "example.com", 80)
socket.send(sock, "GET / HTTP/1.0\r\nHost: example.com\r\n\r\n")

let response = ""
while true {
    let chunk = socket.recv(sock, 4096)
    if len(chunk) == 0 {
        break
    }
    response = response + chunk
}

print(response)
socket.close(sock)
```

### 28.3.1 recv returns bytes

`socket.recv` returns a byte list (list of ints 0–255), not a string. Use `socket.recv_text` for text, or convert:

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.connect(sock, "example.com", 80)
socket.send(sock, "GET / HTTP/1.0\r\n\r\n")

let bytes = socket.recv(sock, 1024)
// bytes is a list like [72, 84, 84, 80, ...]

let text = socket.recv_text(sock, 1024)
// text is a string

socket.close(sock)
```

To convert bytes to string manually:

```
import socket

let bs = socket.recv(sock, 1024)
let s = ""
for b in bs {
    s = s + chr(b)
}
print(s)
```

## 28.4 27.4 TCP server

```
import socket

let server = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.bind(server, "0.0.0.0", 8080)
socket.listen(server, 5)

print("Listening on 0.0.0.0:8080")

let client = socket.accept(server)
let req = socket.recv_text(client, 4096)
print("Request: " + req)

socket.send(client, "HTTP/1.0 200 OK\r\nContent-Type: text/plain\r\n\r\nHello, Zen!")
socket.close(client)
socket.close(server)
```

### 28.4.1 Accept loop

```
import socket

let server = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.bind(server, "0.0.0.0", 8080)
socket.listen(server, 5)

while true {
    let client = socket.accept(server)
    let req = socket.recv_text(client, 4096)
    socket.send(client, "HTTP/1.0 200 OK\r\n\r\npong")
    socket.close(client)
}
```

## 28.5 27.5 UDP

UDP is connectionless — no `connect`, `listen`, or `accept`. Use `send_to` and `recv_from`.

### 28.5.1 UDP client

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_DGRAM)
socket.settimeout(sock, 5000)

socket.send_to(sock, "ping", "127.0.0.1", 9999)

let [data, addr] = socket.recv_from(sock, 4096)
print("Response from " + addr.addr + ": " + data)

socket.close(sock)
```

`recv_from` returns a dict `{data: <bytes>, addr: <ip_string>}`.

### 28.5.2 UDP server

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_DGRAM)
socket.bind(sock, "0.0.0.0", 9999)

while true {
    let [data, addr] = socket.recv_from(sock, 4096)
    print("From " + addr.addr + ": " + data)
    socket.send_to(sock, "pong", addr.addr, 9999)
}
```

## 28.6 27.6 Socket options

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)

// Set timeout
socket.settimeout(sock, 5000)  // 5 seconds

// Get file descriptor (for raw operations)
let fd = socket.fileno(sock)
print(fd)  // integer

socket.close(sock)
```

## 28.7 27.7 Common patterns

### 28.7.1 Read until connection closes

```
import socket

func read_all(sock) {
    let result = ""
    while true {
        let chunk = socket.recv_text(sock, 4096)
        if len(chunk) == 0 {
            break
        }
        result = result + chunk
    }
    return result
}

let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.connect(sock, "example.com", 80)
socket.send(sock, "GET / HTTP/1.0\r\nHost: example.com\r\n\r\n")
let body = read_all(sock)
print(body)
socket.close(sock)
```

### 28.7.2 Read exactly n bytes

```
import socket

func recv_exact(sock, n) {
    let buf = []
```

```
    while len(buf) < n {
        let chunk = socket.recv(sock, n - len(buf))
        if len(chunk) == 0 {
            break
        }
        for b in chunk {
            buf.push(b)
        }
    }
    return buf
}
```

### 28.7.3 Port scanner

```
import socket

func scan_port(host, port, timeout) {
    try {
        let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
        socket.settimeout(sock, timeout)
        socket.connect(sock, host, port)
        socket.close(sock)
        return true
    } catch {
        return false
    }
}

for port in 1..1024 {
    if scan_port("127.0.0.1", port, 100) {
        print("Port " + str(port) + " is open")
    }
}
```

### 28.7.4 HTTP client (manual, no requests module)

```
import socket

func http_get(host, path) {
    let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
    socket.connect(sock, host, 80)

    let request = "GET " + path + " HTTP/1.0\r\nHost: " + host + "\r\nConnection: close
    socket.send(sock, request)

    let response = ""
```

```
    while true {
        let chunk = socket.recv_text(sock, 4096)
        if len(chunk) == 0 {
            break
        }
        response = response + chunk
    }

    socket.close(sock)
    return response
}

print(http_get("example.com", "/"))
```

## 28.8 27.8 Common pitfalls

- **Always close sockets.** Unclosed sockets leak file descriptors.
- **Set timeouts.** Without timeouts, `connect`/`recv` can block forever.
- **TCP is a stream.** `recv` may return fewer bytes than requested — loop until you have all data.
- **`recv` returns bytes**, not strings. Use `recv_text` or convert manually.
- **UDP is connectionless.** Use `send_to`/`recv_from`, not `connect`/`send`/`recv`.
- **`recv_from` returns `{data, addr}`**, not just data.
- **Raw sockets (`SOCK_RAW`) require root** on most operating systems.

## 28.9 27.8 Quick reference

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.settimeout(sock, 5000)
socket.connect(sock, "example.com", 80)
socket.send(sock, "GET / HTTP/1.0\r\n\r\n")
let data = socket.recv(sock, 4096)
let text = socket.recv_text(sock, 4096)
socket.close(sock)

// UDP
socket.send_to(sock, "data", "127.0.0.1", 9999)
let [data, addr] = socket.recv_from(sock, 4096)

// Server
socket.bind(sock, "0.0.0.0", 8080)
socket.listen(sock, 5)
let client = socket.accept(sock)
```

## 28.10 26.9 Exercises

1. Write a TCP echo client that sends a line and prints the response.
2. Write a UDP client that sends a datagram and waits for a reply.
3. Write a port scanner for ports 1–1000 on localhost.
4. Write a simple HTTP/0.9 client that fetches the root page.
5. Write a TCP echo server that handles one connection at a time.
6. Write a UDP server that echoes back datagrams.
7. Combine client and server in one script (server in a thread, client in main).

## 28.11 27.12 Advanced socket patterns

### 28.11.1 Timeouts

```
import socket
let s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(5.0)
try {
    s.connect(("example.com", 80))
} catch(e) {
    print("timeout or error")
}
```

### 28.11.2 Reuse address

```
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
```

### 28.11.3 Non-blocking

```
s.setblocking(false)
```

### 28.11.4 Sending all data

```
def sendall(sock, data) {
    let total = 0
    let bytes_data = data.encode() if typeof(data) == "string" else data
    while total < len(bytes_data) {
        let sent = sock.send(bytes_data[total:])
        if sent == 0 { raise "connection broken" }
        total = total + sent
    }
}
```

## 28.12 27.13 select/poll multiplexing concept

For handling multiple clients:

```
import select
# read_list = [server]
# readable, writable, exceptional = select.select(read_list, [], [], timeout)
```

## 28.13 27.14 Unix domain sockets

```
# server
import socket, os
let path = "/tmp/zen.sock"
try { os.unlink(path) } catch(e) {}
let s = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
s.bind(path)
s.listen(1)
```

## 28.14 27.15 SSL/TLS over sockets (preview)

Use `ssl` module or wrap socket. See Chapter 31.

## 28.15 27.16 Common errors

| Error | Cause |
|---|---|
| ConnectionRefusedError | No server listening |
| TimeoutError | connect/recv timeout |
| OSError: [Errno 98] Address already in use | Need SO_REUSEADDR |
| OSError: [Errno 32] Broken pipe | Peer closed |
| gaierror | DNS resolution failed |

## 28.16 27.17 Debugging tips

- Use `netcat`/`nc` to test servers
- Use `tcpdump -i lo port 8080` to inspect
- Check with `ss -tlnp` or `netstat`
- Log all sends/receives
- Test with curl/wget

## 28.17 27.18 Exercises

1. Echo server and client (TCP)
2. UDP echo server/client
3. Chat server handling multiple clients
4. Port scanner (educational, only scan localhost/own machines)
5. File transfer over TCP
6. HTTP request with raw socket
7. Implement timeout and retry logic

# 29 Chapter 28: HTTP Protocol

Zen's `http` module provides a low-level HTTP/1.1 client. For a higher-level experience with sessions, cookies, and JSON, use the `requests` module (Chapter 29).

## 29.1 28.1 HTTP basics

HTTP (Hypertext Transfer Protocol) is the foundation of the web. A client sends a request; a server responds.

### 29.1.1 Request anatomy

```
GET /index.html HTTP/1.1
Host: example.com
User-Agent: Zen/3.0
Accept: text/html
```

- **Method**: GET, POST, PUT, DELETE, HEAD, OPTIONS, PATCH
- **Path**: `/index.html`
- **Version**: `HTTP/1.1`
- **Headers**: `Host`, `User-Agent`, `Accept`, etc.
- **Body**: empty for GET, present for POST/PUT

### 29.1.2 Response anatomy

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 137

<html><body>Hello</body></html>
```

- **Status line**: `HTTP/1.1 200 OK`
- **Headers**: `Content-Type`, `Content-Length`, etc.
- **Body**: the actual content (may be empty)

## 29.2 28.2 The `http` module

Zen's `http` module provides low-level HTTP/1.1 client functions:

```
import http

let resp = http.get("https://example.com")
print(resp.status_code)   # 200
print(resp.body)          # response body
print(resp.headers)       # headers dict
```

### 29.2.1 http functions

| Function | Description |
|---|---|
| `http.get(url, headers?)` | GET request |
| `http.post(url, body, headers?)` | POST request |
| `http.put(url, body, headers?)` | PUT request |
| `http.delete(url, headers?)` | DELETE request |
| `http.head(url, headers?)` | HEAD request |
| `http.options(url, headers?)` | OPTIONS request |
| `http.patch(url, body, headers?)` | PATCH request |

### 29.2.2 Response object

| Field | Description |
|---|---|
| `status_code` | HTTP status code (int) |
| `reason` | Reason phrase (string) |
| `headers` | Response headers (dict) |
| `body` | Response body (string) |
| `ok` | True if 200–299 |

```
import http

let resp = http.get("https://example.com")
print(resp.status_code)   # 200
print(resp.reason)        # OK
print(resp.body)          # HTML body
print(resp.headers)       # headers dict
print(resp.ok)            # true

if resp.ok {
    print("Success!")
} else {
    print("Error: " + str(resp.status_code) + " " + resp.reason)
}
```

## 29.3 28.3 Making requests with headers

```
import http

let resp = http.get("https://api.example.com/user/1", {
    "User-Agent": "Zen/3.0",
    "Accept": "application/json",
    "Authorization": "Bearer token123"
})
```

```
print(resp.status_code)
print(resp.body)
```

## 29.4 28.4 POST with body

```
import http

let body = "name=Zen&version=3"
let resp = http.post("https://example.com/api", body, {
    "Content-Type": "application/x-www-form-urlencoded",
    "Content-Length": str(len(body))
})

print(resp.status_code)
print(resp.body)
```

## 29.5 28.5 Status codes

| Code | Name | Meaning |
|---|---|---|
| 200 | OK | Success |
| 201 | Created | Resource created |
| 204 | No Content | Success, no body |
| 301 | Moved Permanently | Redirect |
| 302 | Found | Temporary redirect |
| 304 | Not Modified | Cache hit |
| 400 | Bad Request | Client error |
| 401 | Unauthorized | Authentication required |
| 403 | Forbidden | Access denied |
| 404 | Not Found | Resource not found |
| 500 | Internal Server Error | Server error |
| 502 | Bad Gateway | Upstream error |
| 503 | Service Unavailable | Server overloaded |

```
import http

let resp = http.get("https://example.com/nonexistent")
print(resp.status_code)   # 404
print(resp.ok)            # false
```

## 29.6 28.6 HTTP vs HTTPS

Zen's `http` module supports both `http://` and `https://` URLs. HTTPS uses TLS under the hood (certificate validation via the platform root store).

```
import http

// HTTP (port 80)
let r1 = http.get("http://example.com")

// HTTPS (port 443, TLS)
let r2 = http.get("https://example.com")
```

For certificate pinning, use the `tls` module directly (Chapter 25) or `requests` with verify options (Chapter 29).

## 29.7 28.7 Common patterns

### 29.7.1 Simple GET

```
import http

let resp = http.get("https://example.com")
if resp.ok {
    print(resp.body)
} else {
    print("HTTP " + str(resp.status_code))
}
```

### 29.7.2 POST JSON

```
import http
import json

let body = json.dumps({"name": "Zen", "version": 3})
let resp = http.post("https://example.com/api", body, {
    "Content-Type": "application/json",
    "Content-Length": str(len(body))
})

print(resp.status_code)
print(resp.body)
```

### 29.7.3 Check if resource exists (HEAD)

```
import http

let resp = http.head("https://example.com/image.png")
if resp.ok {
    print("Resource exists, size: " + resp.headers["Content-Length"])
} else {
```

```
    print("Resource not found")
}
```

## 29.8 28.8 Common pitfalls

- **`http` is low-level.** No automatic redirect following, no session cookies, no connection pooling. Use `requests` for those features.
- **Always check `resp.ok`.** A non-200 response still returns a response object — it doesn't raise automatically.
- **HTTPS requires TLS support.** If Zen is built without TLS, `https://` URLs will fail.
- **`Content-Length` must match** the actual body length when sending POST/PUT.
- **Redirects are not followed** automatically by `http`. Use `requests` for redirect following.

## 29.9 28.9 Quick reference

```
import http

let resp = http.get("https://example.com", {
    "User-Agent": "Zen/3.0"
})

resp.status_code   # 200
resp.reason        # "OK"
resp.headers       # headers dict
resp.body          # response body
resp.ok            # true if 200-299

let resp2 = http.post("https://example.com/api", "data", {
    "Content-Type": "application/json"
})
```

## 29.10 28.10 Exercises

1. Write a function `fetch_title(url)` that fetches a page and extracts the `<title>` tag.
2. Write a script that checks if a list of URLs return 200 or 404.
3. Write a `head_check(url)` that returns the `Content-Length` header without downloading the body.
4. Write a POST script that sends form data (`application/x-www-form-urlencoded`).
5. Compare `http.get` vs `requests.get` — note the differences in convenience features.

## 29.11 28.12 HTTP/1.1 detailed

### 29.11.1 Request line

```
METHOD /path HTTP/1.1\r\n
```

### 29.11.2 Headers (common request)

| Header | Purpose |
|---|---|
| Host | Required in HTTP/1.1 |
| User-Agent | Client identification |
| Accept | Content types accepted |
| Accept-Language | Language preferences |
| Accept-Encoding | gzip, deflate, br |
| Authorization | Auth credentials/tokens |
| Cookie | Cookies |
| Content-Type | Body media type |
| Content-Length | Body size |
| Connection | keep-alive/close |
| Referer | Referring page |

### 29.11.3 Response headers (common)

| Header | Purpose |
|---|---|
| Server | Server software |
| Date | Response date |
| Content-Type | Media type |
| Content-Length | Size |
| Content-Encoding | Encoding |
| Cache-Control | Caching directives |
| Set-Cookie | Set cookies |
| Location | Redirect target |
| ETag | Entity tag |
| Access-Control-Allow-* | CORS |

### 29.11.4 Status codes complete categories

- 1xx Informational
- 2xx Success (200 OK, 201 Created, 202 Accepted, 204 No Content, 206 Partial Content)
- 3xx Redirection (301, 302, 303, 304, 307, 308)
- 4xx Client Error (400, 401, 403, 404, 405, 406, 409, 410, 413, 414, 415, 422, 429)
- 5xx Server Error (500, 501, 502, 503, 504, 505)

## 29.12 28.13 HTTP methods

| Method | Safe | Idempotent | Body | Purpose |
|---|---|---|---|---|
| GET | yes | yes | no | Retrieve |
| HEAD | yes | yes | no | Headers only |
| OPTIONS | yes | yes | optional | Capabilities/CORS preflight |
| POST | no | no | yes | Create/process |
| PUT | no | yes | yes | Replace |
| PATCH | no | no | yes | Partial update |
| DELETE | no | yes | optional | Delete |
| TRACE | yes | yes | no | Debug |
| CONNECT | no | no | no | Tunnel |

## 29.13 28.14 MIME types (common)

`text/html`, `text/plain`, `application/json`, `application/xml`, `application/x-www-form-urlencoded`, `multipart/form-data`, `application/octet-stream`, `image/png`, `image/jpeg`

## 29.14 28.15 Caching

Cache-Control: `max-age`, `no-cache`, `no-store`, `public`, `private`, `must-revalidate`. ETag/If-None-Match, Last-Modified/If-Modified-Since.

## 29.15 28.16 Authentication

Basic, Bearer tokens, API keys, OAuth 2.0 flows (conceptual).

## 29.16 28.17 HTTPS overview

HTTP over TLS - encrypts request/response, provides authentication via certificates.

## 29.17 28.18 Exercises

1. Parse raw HTTP request manually
2. Build minimal HTTP parser
3. Implement GET/POST handling
4. Add header parsing
5. Handle query strings
6. Implement status code responses
7. Test with curl

# 30 Chapter 29: requests Module

The `requests` module is a Python-requests-parity HTTP client. It supports GET/POST/PUT/PATCH/DELETE/HEAD/OPTIONS, sessions with cookies and headers, timeouts, redirects, JSON, TLS, and more.

## 30.1 29.1 Quick requests

```
import requests

// GET
let resp = requests.get("https://example.com")
print(resp.status_code)   // 200
print(resp.text)          // response body
print(resp.headers)       // headers dict

// GET with query params
let resp = requests.get("https://example.com/search", {"params": {"q": "zen"}})

// POST with JSON
let resp = requests.post("https://example.com/api", {
    "json": {"name": "Zen", "version": 3},
    "headers": {"Content-Type": "application/json"}
})
print(resp.status_code)

// POST with form data
let resp = requests.post("https://example.com/api", {
    "data": {"name": "Zen", "version": "3"}
})

// PUT / PATCH / DELETE / HEAD / OPTIONS
requests.put("https://example.com/api/1", {"json": {"name": "Zen"}})
requests.patch("https://example.com/api/1", {"json": {"version": 4}})
requests.delete("https://example.com/api/1")
requests.head("https://example.com")
requests.options("https://example.com")
```

### 30.1.1 Response object

| Attribute | Description |
|---|---|
| `status_code` | HTTP status code |
| `reason` | Reason phrase |
| `text` | Body as string |
| `content` | Body as bytes |
| `headers` | Headers dict |

| Attribute | Description |
|---|---|
| `ok` | True if 200–299 |
| `url` | Final URL (after redirects) |
| `elapsed` | Request time |
| `cookies` | Cookies dict |
| `json()` | Parsed JSON body |

```
import requests

let resp = requests.get("https://api.example.com/user/1")
if resp.ok {
    let user = resp.json()
    print(user["name"])
} else {
    print("error: " + str(resp.status_code))
}
```

## 30.2 29.2 Options dictionary

All request functions accept an options dict:

| Option | Description |
|---|---|
| `params` | Query parameters dict |
| `data` | Form data dict |
| `json` | JSON body dict |
| `headers` | HTTP headers dict |
| `timeout` | Timeout in seconds |
| `allow_redirects` | Follow redirects (default true) |
| `auth` | `[user, password]` list |
| `cookies` | Cookies dict |

```
import requests

let resp = requests.get("https://example.com", {
    "params": {"q": "zen", "page": 1},
    "headers": {"User-Agent": "Zen/3.0"},
    "timeout": 10,
    "allow_redirects": true
})
```

## 30.3 29.3 Sessions

`requests.Session` persists cookies, headers, and auth across requests:

```
import requests

let sess = requests.Session()
sess.headers = {"User-Agent": "Zen/3.0"}
sess.cookies = {"sessionid": "abc123"}

let r1 = requests.sess_get(sess, "https://example.com/page1")
let r2 = requests.sess_post(sess, "https://example.com/api", {
    "json": {"action": "login"}
})
requests.session_close(sess)
```

### 30.3.1 Session helpers

| Function | Description |
|---|---|
| `requests.Session()` | Create a session |
| `requests.sess_get(sess, url, opts?)` | GET with session |
| `requests.sess_post(sess, url, opts?)` | POST with session |
| `requests.sess_put(sess, url, opts?)` | PUT with session |
| `requests.sess_patch(sess, url, opts?)` | PATCH with session |
| `requests.sess_delete(sess, url, opts?)` | DELETE with session |
| `requests.sess_head(sess, url, opts?)` | HEAD with session |
| `requests.sess_options(sess, url, opts?)` | OPTIONS with session |
| `requests.session_headers(sess, opts?)` | Get/update headers |
| `requests.session_cookies(sess, opts?)` | Get/update cookies |
| `requests.session_close(sess)` | Close session |

```
import requests

let sess = requests.Session()
sess.headers = {"Authorization": "Bearer token123"}
sess.cookies = {"session": "abc123"}

let r1 = requests.sess_get(sess, "https://api.example.com/me")
print(r1.json()["username"])

requests.session_close(sess)
```

## 30.4 29.4 JSON helpers

```
import requests

// POST JSON
let resp = requests.post("https://example.com/api", {
```

```
    "json": {"name": "Zen", "version": 3}
})

// Parse response JSON
let data = resp.json()
print(data["id"])

// Raise on error status
requests.raise_for_status(resp)
```

### 30.4.1 Status codes

```
import requests

let codes = requests.codes
print(codes.ok)      // 200
print(codes.created) // 201
print(codes.not_found) // 404
print(codes.server_error) // 500
```

## 30.5 29.5 URL encoding utilities

```
import requests

let encoded = requests.utils_quote("hello world")
print(encoded)  // hello%20world

let decoded = requests.utils_unquote("hello%20world")
print(decoded)  // hello world

print(requests.default_user_agent())
```

## 30.6 29.6 Common patterns

### 30.6.1 Download a file

```
import requests
import fs

let resp = requests.get("https://example.com/file.zip")
fs.write_binary("file.zip", resp.content)
```

### 30.6.2 Upload JSON

```
import requests

let resp = requests.post("https://example.com/api", {
    "json": {"name": "Zen", "version": 3},
    "headers": {"Authorization": "Bearer token123"}
})
```

### 30.6.3 With timeout and error handling

```
import requests

let resp = requests.get("https://example.com", {
    "timeout": 30
})
if !resp.ok {
    print("HTTP " + str(resp.status_code))
} else {
    print(resp.text)
}

// Catch network errors
try {
    let resp = requests.get("https://example.com", {"timeout": 5})
    print(resp.text)
} catch as e {
    print("request error: " + e.msg)
}
```

### 30.6.4 Basic auth

```
import requests

let resp = requests.get("https://example.com", {
    "auth": ["user", "pass"]
})
```

## 30.7 29.7 Error handling

```
import requests

try {
    let resp = requests.get("https://example.com", {"timeout": 5})
    if !resp.ok {
```

```
        print("HTTP " + str(resp.status_code))
    } else {
        print(resp.text)
    }
} catch as e {
    print("request error: " + e.msg)
}
```

## 30.8 29.8 Quick reference

```
import requests

requests.get(url, opts?)
requests.post(url, opts?)
requests.put(url, opts?)
requests.patch(url, opts?)
requests.delete(url, opts?)
requests.head(url, opts?)
requests.options(url, opts?)

let sess = requests.Session()
requests.sess_get(sess, url, opts?)
requests.session_close(sess)

resp.status_code
resp.text
resp.content
resp.headers
resp.ok
resp.url
resp.json()
resp.cookies

requests.utils_quote(s)
requests.utils_unquote(s)
requests.default_user_agent()
requests.codes.ok
requests.raise_for_status(resp)
```

## 30.9 29.9 Exercises

1. Fetch a public JSON API and print a specific field.
2. POST JSON to a test endpoint and check the response.
3. Use a session to make two requests that share cookies.
4. Download an image and save it with `fs.write_binary`.
5. Add basic auth and custom headers to a request.

6. Write a function `get_json(url)` that fetches and parses JSON, returning `nil` on failure.
7. Write a retry function that retries a GET up to 3 times on failure.

## 30.10 29.10 Advanced usage patterns

### 30.10.1 Sessions (connection reuse)

```
import requests
let s = requests.Session()
s.headers.update({"User-Agent": "ZenBot/1.0"})
let r1 = s.get("https://httpbin.org/headers")
let r2 = s.get("https://httpbin.org/cookies/set?x=1")
```

### 30.10.2 Custom headers and auth

```
let r = requests.get(url, headers={"Authorization": "Bearer token"}, auth=("user","pass
```

### 30.10.3 Query parameters

```
let r = requests.get("https://httpbin.org/get", params={"q": "zen", "page": 1})
```

### 30.10.4 POST forms and JSON

```
let r = requests.post(url, data={"key": "val"})  # form
let r = requests.post(url, json={"key": "val"}) # JSON
```

### 30.10.5 File uploads

```
let files = {"file": open("doc.txt", "rb")}
let r = requests.post(url, files=files)
```

### 30.10.6 Timeouts and retries

```
let r = requests.get(url, timeout=5)
```

### 30.10.7 Streaming

```
let r = requests.get(url, stream=true)
# iterate over r.iter_content()
```

### 30.10.8 Response object details

- `r.status_code`, `r.reason`
- `r.text`, `r.content` (bytes)
- `r.json()`
- `r.headers`, `r.cookies`
- `r.url`, `r.elapsed`

### 30.10.9 SSL verification

```
let r = requests.get(url, verify=true)  # default
# let r = requests.get(url, verify=false)  # dangerous, only for testing
```

### 30.10.10 Proxies

```
let proxies = {"http": "http://proxy:8080", "https": "http://proxy:8080"}
let r = requests.get(url, proxies=proxies)
```

### 30.10.11 Redirects

```
let r = requests.get(url, allow_redirects=true)
```

## 30.11 29.11 Error handling

```
import requests
try {
    let r = requests.get("https://invalid.example", timeout=2)
    r.raise_for_status()
} catch(e) {
    print(e)
}
```

## 30.12 29.12 Best practices

- Always use timeouts
- Handle exceptions
- Check status codes
- Respect robots.txt and rate limits
- Use sessions for multiple requests
- Don’t disable verify in production

## 30.13 29.13 Exercises

1. GET request and parse JSON
2. POST JSON to API

3. Use session with cookies
4. Download file with streaming
5. Handle errors and timeouts
6. Set custom headers
7. Follow redirects and inspect history

# 31 Chapter 30: http Server Module

The `http.server` module provides a simple static file HTTP server. It's useful for serving files, directory listings, and quick local servers.

## 31.1 30.1 Starting a server

```
import http.server

// Serve current directory on port 8000
http.server.serve(".", 8000)
```

This blocks — the server runs until the process is interrupted.

## 31.2 30.2 Server options

```
import http.server

// Serve a specific directory on a specific host/port
http.server.serve("/var/www", 8080, "127.0.0.1")

// Serve once (single request) then return
let port = http.server.serve_once(".", 0)
print("served on port " + str(port))
```

### 31.2.1 `serve(root, port, host?)`

Blocks forever serving files from `root` on `port:host`.

### 31.2.2 `serve_once(root, port?)`

Serves a single request then returns the port used. Port 0 means "pick a free port."

## 31.3 30.3 What it serves

- **Files** — serves the file with appropriate `Content-Type` based on extension.
- **Directories** — generates an HTML directory listing with links.
- **MIME types** — common extensions mapped (`.html`, `.css`, `.js`, `.json`, `.png`, `.jpg`, `.txt`, etc.).
- **404** — returns a 404 page for missing files.

## 31.4 30.4 Example: serve a build directory

```
import http.server
```

```
print("Serving build/ on http://localhost:8000")
http.server.serve("build", 8000)
```

Then open `http://localhost:8000` in a browser.

## 31.5 30.5 Example: serve once and get the port

```
import http.server

// Port 0 means "pick a free port"
let port = http.server.serve_once(".", 0)
print("One-shot server on port " + str(port))
```

## 31.6 30.6 MIME type guessing

```
import http.server

print(http.server.guess_type("index.html"))   // text/html
print(http.server.guess_type("style.css"))    // text/css
print(http.server.guess_type("app.js"))       // application/javascript
print(http.server.guess_type("data.json"))    // application/json
print(http.server.guess_type("image.png"))    // image/png
```

### 31.6.1 Supported MIME types

| Extension | MIME type |
|---|---|
| `.html` | `text/html` |
| `.css` | `text/css` |
| `.js` | `application/javascript` |
| `.json` | `application/json` |
| `.txt` | `text/plain` |
| `.png` | `image/png` |
| `.jpg`, `.jpeg` | `image/jpeg` |
| `.gif` | `image/gif` |
| `.svg` | `image/svg+xml` |
| `.pdf` | `application/pdf` |
| `.mp3` | `audio/mpeg` |
| `.mp4` | `video/mp4` |
| `.zip` | `application/zip` |
| `.tar.gz` | `application/gzip` |
| `.woff2` | `font/woff2` |
| `.ico` | `image/x-icon` |
| (unknown) | `application/octet-stream` |

## 31.7 30.7 Common pitfalls

- **serve blocks.** Run it in a thread if you need to do other work.
- **Directory traversal is blocked.** `safe_join` prevents `..` escapes.
- **No HTTPS.** Use `tls` or a reverse proxy for production.
- **Single-threaded.** One request at a time.

## 31.8 30.8 Directory traversal protection

The server rejects paths containing `..` to prevent accessing files outside the root directory:

```
// Requesting GET /../../etc/passwd returns 403 or 404
```

## 31.9 30.9 Quick reference

```
import http.server

http.server.serve(root, port, host?)
let port = http.server.serve_once(root, port?)
http.server.guess_type(filename)
```

## 31.10 30.10 Exercises

1. Serve a local directory and open it in a browser.
2. Use `serve_once` to get a free port, then make a request to it.
3. Create an HTML file, serve it, and verify the `Content-Type` header.
4. Write a script that serves a directory in a thread while doing other work.
5. Test that `..` in a URL path is rejected (directory traversal protection).

## 31.11 30.10 Building a more complete server

### 31.11.1 Routing

```
# simple router pattern
fn handle(req) {
    if req.path == "/" { return "index" }
    if req.path == "/api/data" { return json.dumps({"ok": true}) }
    return "not found"
}
```

### 31.11.2 Serving static files

Read file, set Content-Type based on extension, return bytes/text.

### 31.11.3 Handling POST data

Parse form data or JSON from body based on Content-Type.

### 31.11.4 Middleware pattern

```
fn middleware(next_handler) {
    return fn(req) {
        print(req.method + " " + req.path)
        return next_handler(req)
    }
}
```

### 31.11.5 CORS headers

```
# Add headers: Access-Control-Allow-Origin, etc.
```

### 31.11.6 Error responses

Return proper status codes and JSON error bodies.

## 31.12 30.11 Production considerations

- Use proper server (gunicorn-like) not just dev server
- Handle concurrent requests (threading/async)
- Set timeouts
- Validate input
- Add logging
- Use HTTPS
- Rate limiting
- Security headers

## 31.13 30.12 Exercises

1. Multi-route server
2. JSON API endpoints (GET/POST)
3. Serve static HTML/CSS/JS
4. Form handling
5. 404 and error pages
6. Request logging middleware
7. Simple REST CRUD API

## 31.14 30.12 Worked example — a small HTTP server with routing

```
import http.server

let routes = #{}

func route(path, handler) {
    routes[path] = handler
```

```
}

func not_found(req) {
    return #(404, #{}, "Not Found")
}

func handle(req) {
    let h = routes[req.path] ?? not_found
    return h(req)
}

route("/", fn(req) {
    return #(200, #{}, "<h1>Zen HTTP Server</h1>")
})

route("/time", fn(req) {
    import datetime
    return #(200, #{}, "time=" + datetime.now())
})

route("/echo", fn(req) {
    return #(200, {"Content-Type": "text/plain"}, req.body ?? "")
})

let server = http.server.Server(port=8080, handler=handle)
print("server on http://localhost:8080")
server.start()
```

Test with curl:

```
$ curl http://localhost:8080/
<h1>Zen HTTP Server</h1>
$ curl http://localhost:8080/time
time=...
$ curl -d "hello" http://localhost:8080/echo
hello
```

## 31.15 30.13 Troubleshooting HTTP servers

| Symptom | Likely cause | Fix |
|---|---|---|
| connection refused | server not started / wrong port | start server, check port |
| port already in use | another process on 8080 | use SO_REUSE-ADDR or a different port |

| Symptom | Likely cause | Fix |
|---|---|---|
| 404 on known route | path mismatch (trailing slash) | normalize paths |
| client hangs | handler never returns | check for blocking code |
| large payload truncated | read limit | read full body / stream |

## 31.16 30.14 Exercises

1. Add a `/health` route returning `{status: "ok"}`.
2. Return JSON from a route (`Content-Type: application/json`).
3. Add request logging (method + path) for every request.
4. Handle POST body parsing for `/echo`.
5. Add a 500 error route handler for exceptions.
6. Make the port configurable via an argument/env var.

# 32 Chapter 31: HTTPS & SSL/TLS Certificates

Zen's `tls` module provides encrypted TCP connections using rustls, with optional certificate pinning. This chapter covers TLS connections, certificate pinning, and secure HTTPS practices.

## 32.1 31.1 The `tls` module

```
import tls

fd = tls.connect("example.com", 443)
tls.write(fd, "GET / HTTP/1.0\r\nHost: example.com\r\n\r\n")
let resp = tls.read(fd, 4096)
print(resp)
tls.close(fd)
```

### 32.1.1 TLS functions

| Function | Description |
|---|---|
| `tls.connect(host, port)` | Plain TLS connection, returns fd |
| `tls.connect_pinned(host, port, pins)` | TLS with certificate pinning |
| `tls.write(fd, data)` | Send data, returns nil |
| `tls.read(fd, n)` | Read up to n bytes, returns string |
| `tls.close(fd)` | Close connection, returns nil |

```
import tls

let fd = tls.connect("example.com", 443)
tls.write(fd, "GET / HTTP/1.0\r\nHost: example.com\r\nConnection: close\r\n\r\n")
let resp = tls.read(fd, 4096)
print(resp)
tls.close(fd)
```

## 32.2 31.2 Certificate pinning

Certificate pinning ensures you only connect to servers with a specific certificate, preventing man-in-the-middle attacks even if the CA is compromised.

```
import tls

let pins = ["sha256:AAAA...BBBB..."]  // SHA-256 of server's SPKI
let fd = tls.connect_pinned("example.com", 443, pins)
tls.write(fd, "GET / HTTP/1.0\r\n\r\n")
let resp = tls.read(fd, 4096)
tls.close(fd)
```

### 32.2.1 How pinning works

1. `tls.connect_pinned` connects to the server.
2. During the TLS handshake, the server presents its certificate.
3. Zen computes the SHA-256 hash of the certificate's SPKI (Subject Public Key Info).
4. If the hash matches one of the pins, the connection proceeds.
5. If no pin matches, the handshake fails.

### 32.2.2 Getting a certificate's SPKI hash

```
// You can compute this externally with openssl:
// openssl s_client -connect example.com:443 -showcerts
// Then hash the SPKI of the leaf certificate.

// Example pin format:
let pins = ["sha256:a1b2c3d4e5f6..."]
```

## 32.3 31.3 Plain TLS vs pinned TLS

| Feature | `tls.connect` | `tls.connect_pinned` |
|---|---|---|
| Verification | Platform root store (webpki-roots) | Pin SPKI hash match |
| Trust model | CA hierarchy | Explicit pins |
| Fallback | Accepts any CA-signed cert | Only pinned certs |
| Use case | General browsing | Known services (APIs, internal) |

```
import tls

// General browsing - trusts any CA-signed cert
let fd1 = tls.connect("example.com", 443)

// Known service - only accept specific cert
let pins = ["sha256:expected-spki-hash"]
let fd2 = tls.connect_pinned("api.internal.com", 443, pins)
```

## 32.4 31.4 Complete HTTPS client example

```
import tls

func https_get(host, path) {
    let fd = tls.connect(host, 443)

    let request = "GET " + path + " HTTP/1.0\r\n" +
```

```
                  "Host: " + host + "\r\n" +
                  "Connection: close\r\n\r\n"

    tls.write(fd, request)

    let response = ""
    while true {
        let chunk = tls.read(fd, 4096)
        if len(chunk) == 0 {
            break
        }
        response = response + chunk
    }

    tls.close(fd)
    return response
}

print(https_get("example.com", "/"))
```

### 32.4.1 HTTPS with pinning

```
import tls

func https_get_pinned(host, path, pins) {
    let fd = tls.connect_pinned(host, 443, pins)

    let request = "GET " + path + " HTTP/1.0\r\n" +
                  "Host: " + host + "\r\n" +
                  "Connection: close\r\n\r\n"

    tls.write(fd, request)

    let response = ""
    while true {
        let chunk = tls.read(fd, 4096)
        if len(chunk) == 0 {
            break
        }
        response = response + chunk
    }

    tls.close(fd)
    return response
}
```

```
let pins = ["sha256:AAAA...BBBB..."]
print(https_get_pinned("example.com", "/", pins))
```

## 32.5 31.5 Error handling

```
import tls

try {
    let fd = tls.connect("nonexistent.example.invalid", 443)
    tls.write(fd, "GET / HTTP/1.0\r\n\r\n")
    let resp = tls.read(fd, 4096)
    tls.close(fd)
} catch as e {
    print("TLS error: " + e.msg)
    // Could be: DNS failure, connection refused, TLS handshake failure,
    // certificate verification failure, pin mismatch
}

// Pin mismatch example
try {
    let pins = ["sha256:wrong-hash"]
    let fd = tls.connect_pinned("example.com", 443, pins)
} catch as e {
    print(e.msg)  // certificate pin mismatch
}
```

## 32.6 31.6 Quick reference

```
import tls

let fd = tls.connect("example.com", 443)
tls.write(fd, "GET / HTTP/1.0\r\n\r\n")
let resp = tls.read(fd, 4096)
tls.close(fd)

// With pinning
let pins = ["sha256:AAAA..."]
let fd2 = tls.connect_pinned("example.com", 443, pins)
tls.close(fd2)
```

## 32.7 31.7 Exercises

1. Write an HTTPS client that fetches a page and prints the status line.
2. Get a certificate's SPKI hash using openssl, then use `tls.connect_pinned` to connect only to that cert.

3. Write a function `https_get_pinned(host, path, pins)` that returns the response body.
4. Demonstrate that `tls.connect_pinned` fails with a wrong pin.
5. Write a script that connects to an HTTPS API and parses the JSON response.

## 32.8 31.12 HTTPS / SSL/TLS — deep reference

HTTPS is HTTP over TLS. It provides encryption, authentication (via certificates), and integrity.

### 32.8.1 TLS handshake (simplified)

1. ClientHello: client sends supported versions, cipher suites, random.
2. ServerHello: server picks version/cipher, sends its certificate.
3. Server proves identity (certificate signed by a trusted CA).
4. Key exchange: client and server derive shared keys.
5. Encrypted HTTP traffic begins.

### 32.8.2 Certificates

| Concept | Meaning |
|---|---|
| Certificate | binds a public key to an identity (domain/org) |
| CA | Certificate Authority that signs certificates |
| Chain | leaf → intermediate → root CA |
| Self-signed | signed by itself; not trusted by default |
| SAN | Subject Alternative Names (multiple domains) |
| Expiry | certificates expire; must be renewed |

### 32.8.3 Verifying certificates

```
import requests
# verify=true (default): validates cert against system CAs
let r = requests.get("https://example.com", verify=true)

# verify=false: skips validation — DANGEROUS, testing only
# let r = requests.get("https://self-signed.local", verify=false)
```

### 32.8.4 Common TLS errors

| Error | Cause |
|---|---|
| certificate verify failed | untrusted/self-signed/expired cert |
| hostname mismatch | cert doesn't match the domain |
| protocol version mismatch | client/server disagree on TLS version |
| handshake failure | no common cipher suite |

### 32.8.5 Best practices

1. Always `verify=true` in production.
2. Never disable verification to "fix" a cert error — fix the cert.
3. Use modern TLS (1.2+/1.3).
4. Renew certificates before expiry.
5. Use strong cipher suites; disable weak ones.

### 32.8.6 Exercises

1. Fetch an HTTPS page and print status + first 200 chars.
2. Compare `verify=true` vs `verify=false` on a self-signed site (theory).
3. Inspect a certificate's issuer, subject, expiry (using openssl).
4. Explain the TLS handshake steps in your own words.
5. List why a cert might fail verification.
6. Write a helper that fetches HTTPS with timeout and error handling.
7. Research Let's Encrypt and how to get a free cert.
8. Explain HSTS and why it matters.

## 32.9 31.12 Worked example — fetching HTTPS with verification

```
import requests

func fetch_secure(url) {
    try {
        let r = requests.get(url, verify=true, timeout=10)
        print("status=" + str(r.status_code))
        print(r.text[0:200])
    } catch as e {
        print("TLS/HTTP error: " + e.msg)
    }
}

fetch_secure("https://example.com")
```

Expected output:

```
status=200
<!doctype html>...
```

## 32.10 31.13 Troubleshooting TLS/HTTPS

| Symptom | Cause | Fix |
|---|---|---|
| certificate verify failed | self-signed/expired/untrusted cert | fix cert or add to trust store |
| hostname mismatch | cert doesn't match domain | use correct URL |

| Symptom | Cause | Fix |
|---|---|---|
| protocol mismatch | client/server TLS version differ | enable TLS 1.2/1.3 |
| handshake failure | no common cipher | update client/server |

Never disable `verify=true` in production. For testing only, `verify=false` bypasses validation (dangerous).

## 32.11 31.14 Exercises

1. Fetch https://example.com and print status + first 200 chars.
2. Compare behavior with verify=true vs verify=false on a self-signed site.
3. List reasons a certificate fails verification.
4. Write a helper `safe_get(url)` with timeout and error handling.
5. Explain the TLS handshake in your own words.

# 33 Chapter 32: dns Module

The `dns` module provides DNS resolution: forward lookups (name → address), reverse lookups (address → name), and hostname information.

## 33.1 32.1 Resolve a hostname

```
import dns

let addrs = dns.resolve("example.com")
for addr in addrs {
    print(addr)
}
// 93.184.216.34
// 2606:2800:220:1:248:1893:25c8:1946
```

`dns.resolve(host, port?)` returns a list of socket addresses.

## 33.2 32.2 `gethostbyname` — single IPv4

```
import dns

let info = dns.gethostbyname("example.com")
print(info.hostname)   # example.com
print(info.addr_list)  # ["93.184.216.34", ...]
```

## 33.3 32.3 `gethostbyname_ex` — extended info

```
import dns

let info = dns.gethostbyname_ex("example.com")
print(info.hostname)      # example.com
print(info.alias_list)    # aliases
print(info.addr_list)     # addresses
```

## 33.4 32.4 `gethostbyaddr` — reverse lookup

```
import dns

let info = dns.gethostbyaddr("93.184.216.34")
print(info.hostname)   # example.com (or similar)
print(info.alias_list) # aliases
print(info.addr_list)  # addresses
```

## 33.5 32.5 `getfqdn` — fully qualified domain name

```
import dns

let fqdn = dns.getfqdn()
print(fqdn)  # myhost.localdomain

let fqdn2 = dns.getfqdn("example.com")
print(fqdn2) # example.com
```

## 33.6 32.6 Common patterns

### 33.6.1 Check if a host resolves

```
import dns

func is_resolvable(host) {
    try {
        let addrs = dns.resolve(host)
        return len(addrs) > 0
    } catch {
        return false
    }
}

print(is_resolvable("example.com"))          # true
print(is_resolvable("nonexistent.invalid"))  # false
```

### 33.6.2 Get the first IPv4 address

```
import dns

func first_ipv4(host) {
    let addrs = dns.resolve(host)
    for addr in addrs {
        if addr.contains(".") {  // IPv4 has dots
            return addr
        }
    }
    return nil
}

print(first_ipv4("example.com"))
```

### 33.6.3 Resolve and connect

```
import dns
import socket

let addrs = dns.resolve("example.com", 443)
if len(addrs) > 0 {
    let addr = addrs[0]
    let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
    socket.connect(sock, addr, 443)
    // ... use socket ...
    socket.close(sock)
}
```

## 33.7 32.7 Quick reference

```
import dns

dns.resolve(host, port?)
dns.gethostbyname(host)
dns.gethostbyname_ex(host)
dns.gethostbyaddr(addr)
dns.getfqdn(host?)

let addrs = dns.resolve("example.com")
for addr in addrs { print(addr) }
```

## 33.8 32.8 Exercises

1. Write a function `resolve_all(host)` that prints all addresses for a host.
2. Write `is_reachable(host)` that tries DNS then a socket connect.
3. Write a script that resolves a list of hostnames and reports which fail.
4. Use `dns.getfqdn()` to print the local machine's FQDN.
5. Reverse-lookup an IP address and print the hostname.

## 33.9 32.12 DNS module — deep reference

DNS maps domain names to IP addresses (and vice versa).

### 33.9.1 DNS record types

| Type | Meaning | Example |
|---|---|---|
| A | IPv4 address | example.com → 93.184.216.34 |
| AAAA | IPv6 address | example.com → 2606:2800:220:1:248:1893:25c8:1946 |

| Type | Meaning | Example |
|---|---|---|
| CNAME | canonical name (alias) | www.example.com → example.com |
| MX | mail server | example.com → mail.example.com |
| TXT | arbitrary text | SPF, DKIM, verification |
| NS | name server | example.com → ns1.example.com |
| PTR | reverse (IP → name) | 93.184.216.34 → example.com |

### 33.9.2 Resolution process

1. Check local /etc/hosts.
2. Check OS cache / DNS cache.
3. Query configured resolver (ISP, 1.1.1.1, 8.8.8.8).
4. Resolver queries root → TLD → authoritative servers.
5. Result returned and cached (TTL seconds).

### 33.9.3 Usage

```
import dns
# forward lookup
let ips = dns.resolve("example.com")
print(ips)   # ["93.184.216.34", ...]

# specific record type
let mx = dns.resolve("example.com", "MX")
print(mx)

# reverse lookup
let names = dns.reverse("93.184.216.34")
print(names)
```

### 33.9.4 Common DNS issues

| Issue | Cause | Fix |
|---|---|---|
| NXDOMAIN | domain doesn't exist | check spelling |
| SERVFAIL | server failure | try another resolver |
| timeout | no response | check network/firewall |
| stale cache | old record cached | flush cache / lower TTL |

### 33.9.5 Exercises

1. Resolve example.com to IP and print it.

2. Look up MX records for a domain.
3. Look up TXT records (SPF/DKIM).
4. Perform a reverse lookup on an IP.
5. Explain the difference between A and AAAA records.
6. Describe what happens when you type a URL in a browser.
7. Research public DNS resolvers (1.1.1.1, 8.8.8.8, 9.9.9.9).
8. Explain DNS caching and TTL.
9. Write a small tool resolving a list of domains.
10. Describe DNS hijacking / spoofing (conceptually, ethical).

## 33.10 32.12 Worked example — DNS resolution

```
import dns

func show(host) {
    print("A records for " + host + ":")
    let ips = dns.resolve(host, "A")
    for ip in ips { print("  " + ip) }

    print("MX records for " + host + ":")
    let mx = dns.resolve(host, "MX")
    for m in mx { print("  " + str(m)) }

    print("reverse for " + ips[0] + ":")
    let names = dns.reverse(ips[0])
    for n in names { print("  " + n) }
}

show("example.com")
```

Sample output:

```
A records for example.com:
  93.184.216.34
MX records for example.com:
  ...
reverse for 93.184.216.34:
  ...
```

## 33.11 32.13 Troubleshooting DNS

| Symptom | Cause | Fix |
|---|---|---|
| NXDOMAIN | domain doesn’t exist | check spelling |
| SERVFAIL | resolver failure | try another resolver (1.1.1.1) |
| timeout | no response | check network/firewall |
| stale data | cached old record | wait for TTL / flush cache |

## 33.12 32.14 Exercises

1. Resolve example.com to its A record and print it.
2. Look up MX records for a domain.
3. Look up TXT records (SPF/DKIM).
4. Reverse-lookup an IP to its name.
5. Explain A vs AAAA records.

# 34 Chapter 33: url Module

The `url` module provides URL parsing, building, and percent-encoding: `urllib.parse` utilities.

## 34.1 33.1 Parsing URLs

```
import urllib.parse

let url = "https://user:pass@example.com:8080/path/to/page?q=1#section"
let parts = urllib.parse.urlparse(url)

print(parts.scheme)    # https
print(parts.netloc)    # user:pass@example.com:8080
print(parts.path)      # /path/to/page
print(parts.params)    # (empty)
print(parts.query)     # q=1
print(parts.fragment)  # section
print(parts.username)  # user
print(parts.password)  # pass
print(parts.hostname)  # example.com
print(parts.port)      # 8080
```

### 34.1.1 ParseResult fields

| Field | Description | Example |
|---|---|---|
| `scheme` | URL scheme | `"https"` |
| `netloc` | Network location | `"user:pass@example.com:8080"` |
| `path` | Hierarchical path | `"/path/to/page"` |
| `params` | Parameters | `""` |
| `query` | Query string | `"q=1"` |
| `fragment` | Fragment | `"section"` |
| `username` | Username | `"user"` |
| `password` | Password | `"pass"` |
| `hostname` | Hostname | `"example.com"` |
| `port` | Port | `8080` |

```
import urllib.parse

let p = urllib.parse.urlparse("https://user:pass@example.com:8080/path?q=1#frag")
print(p.scheme)     # "https"
print(p.netloc)     # "user:pass@example.com:8080"
print(p.path)       # "/path"
print(p.params)     # ""
print(p.query)      # "q=1"
```

```
print(p.fragment)   # "frag"
print(p.username)   # "user"
print(p.password)   # "pass"
print(p.hostname)   # "example.com"
print(p.port)       # 8080
```

## 34.2 23.2 Building URLs

```
import urllib.parse

let url = urllib.parse.urlunparse({
    "scheme": "https",
    "netloc": "example.com",
    "path": "/search",
    "params": "",
    "query": "q=zen",
    "fragment": ""
})
print(url)  # https://example.com/search?q=zen
```

`urlunparse` takes a dict with keys: `scheme`, `netloc`, `path`, `params`, `query`, `fragment`.

## 34.3 33.3 Query strings

```
import urllib.parse

// Encode query params
let qs = urllib.parse.urlencode({"q": "zen lang", "page": 1})
print(qs)  // q=zen+lang&page=1

// Parse query string
let params = urllib.parse.parse_qs("q=zen+lang&page=1")
print(params)  // {"q": ["zen lang"], "page": ["1"]}
```

### 34.3.1 urlencode

```
import urllib.parse

print(urllib.parse.urlencode({"q": "hello world", "page": 2}))
// q=hello+world&page=2

print(urllib.parse.urlencode({"a": 1, "b": 2, "c": 3}))
// a=1&b=2&c=3
```

### 34.3.2 parse_qs

```
import urllib.parse

print(urllib.parse.parse_qs("q=zen+lang&page=1"))
// {"q": ["zen lang"], "page": ["1"]}

print(urllib.parse.parse_qs("a=1&a=2&b=3"))
// {"a": ["1", "2"], "b": ["3"]}
```

Values are always lists (to support repeated keys).

## 34.4 33.4 Joining URLs

```
import urllib.parse

let full = urllib.parse.urljoin("https://example.com/base/", "page.html")
print(full)  // https://example.com/page.html

let full2 = urllib.parse.urljoin("https://example.com/base/", "/abs.html")
print(full2)  // https://example.com/abs.html
```

```
import urllib.parse

// Relative to base
print(urllib.parse.urljoin("https://example.com/a/b/", "c.html"))
// https://example.com/a/b/c.html

print(urllib.parse.urljoin("https://example.com/a/b/", "../c.html"))
// https://example.com/a/c.html

print(urllib.parse.urljoin("https://example.com/", "page.html"))
// https://example.com/page.html
```

## 34.5 33.5 Percent encoding

```
import urllib.parse

let encoded = urllib.parse.quote("hello world")
print(encoded)  // hello%20world

let decoded = urllib.parse.unquote("hello%20world")
print(decoded)  // hello world
```

```
import urllib.parse

print(urllib.parse.quote("hello world"))     # hello%20world
```

```
print(urllib.parse.quote("a/b?c=d&e=f"))     # a%2Fb%3Fc%3De%26f
print(urllib.parse.quote("□□"))             # %E4%B8%AD%E6%96%87

print(urllib.parse.unquote("hello%20world"))   # hello world
print(urllib.parse.unquote("%E4%B8%AD%E6%96%87"))  # □□
```

## 34.6 33.6 Splitting and joining netloc

```
import urllib.parse

let parts = urllib.parse.splituser("user:pass@example.com:8080")
print(parts)  // ["user:pass", "example.com:8080"]

let splitpass = urllib.parse.splitpasswd("user:pass@example.com:8080")
print(splitpass)  // ["user", "pass@example.com:8080"]

let port = urllib.parse.splitport("example.com:8080")
print(port)  // ["example.com", "8080"]
```

## 34.7 33.6 Common patterns

### 34.7.1 Build a search URL

```
import urllib.parse

let base = "https://example.com/search"
let qs = urllib.parse.urlencode({"q": "Zen language", "page": 2})
let url = base + "?" + qs
print(url)  // https://example.com/search?q=Zen+language&page=2
```

### 34.7.2 Extract domain from URL

```
import urllib.parse

let parts = urllib.parse.urlparse("https://sub.example.com/path")
print(parts.hostname)  // sub.example.com
```

### 34.7.3 Add a query param to existing URL

```
import urllib.parse

let url = "https://example.com/page?a=1"
let parts = urllib.parse.urlparse(url)
let params = urllib.parse.parse_qs(parts.query)
```

```
params["b"] = ["2"]
let new_qs = urllib.parse.urlencode(params)
let new_url = urllib.parse.urlunparse({
    "scheme": parts.scheme,
    "netloc": parts.netloc,
    "path": parts.path,
    "params": parts.params,
    "query": new_qs,
    "fragment": parts.fragment
})
print(new_url)
```

## 34.8 33.7 Quick reference

```
import urllib.parse

urllib.parse.urlparse(url)        // parse into parts
urllib.parse.urlunparse(parts)    // build from parts
urllib.parse.urljoin(base, ref)   // resolve relative URL
urllib.parse.urlencode(params)    // encode query string
urllib.parse.parse_qs(qs)         // parse query string
urllib.parse.quote(s)             // percent-encode
urllib.parse.unquote(s)           // percent-decode

let p = urllib.parse.urlparse("https://example.com/path?q=1")
p.scheme      // https
p.netloc      // example.com
p.path        // /path
p.query       // q=1
p.fragment    // (empty)
p.hostname    // example.com
p.port        // (none)
```

## 34.9 33.8 Exercises

1. Write `extract_domain(url)` that returns the hostname.
2. Write `add_param(url, key, value)` that adds/replaces a query parameter.
3. Write `build_api_url(base, endpoint, params)` that constructs a full API URL.
4. Write `decode_query(qs)` that returns a dict of query parameters.
5. Write `normalize_url(url)` that ensures a scheme and trailing slash.
6. Write `get_query_param(url, key)` that extracts a single query parameter value.
7. Write `merge_url_params(url, params)` that merges new params into existing URL.

## 34.10 33.12 URL module — deep reference

### 34.10.1 URL anatomy

```
https://user:pass@example.com:8080/path/to/page?q=1&x=2#section
 \__/   \___/ \_______/ \_______/ \_____/ \______/\______/
scheme  auth     host       port    path   query    fragment
```

### 34.10.2 Components

| Component | Purpose |
|---|---|
| scheme | protocol (http, https, ftp, ws) |
| host | domain or IP |
| port | TCP port (default 80/443) |
| path | resource path |
| query | key=value pairs after ? |
| fragment | anchor after # |
| username/password | basic auth (deprecated/insecure) |

### 34.10.3 Usage

```
import url
let u = url.parse("https://example.com:8080/path?q=1#sec")
print(u.scheme)    # https
print(u.host)      # example.com
print(u.port)      # 8080
print(u.path)      # /path
print(u.query)     # q=1
print(u.fragment)  # sec

let built = url.build(scheme="https", host="example.com", path="/search", query="q=zen"
print(built)   # https://example.com/search?q=zen

let joined = url.join("https://example.com/base/", "sub/page")
print(joined)  # https://example.com/sub/page

let encoded = url.quote("hello world")
print(encoded)   # hello%20world
let decoded = url.unquote("hello%20world")
print(decoded)   # hello world

let params = url.parse_qs("q=1&x=2&q=3")
print(params)   # #{q: ["1","3"], x: ["2"]}
```

### 34.10.4 Encoding rules

- `quote`: percent-encode a path segment.

- `quote_plus`: like quote but space → +.
- `unquote`/`unquote_plus`: reverse.

### 34.10.5 Exercises

1. Parse a URL and print all components.
2. Build a URL from components.
3. Join a base URL with a relative path.
4. Percent-encode/decode a search query.
5. Parse query string into a dict of lists.
6. Write `extract_domain(url)` returning just the host.
7. Write `add_query_param(url, k, v)` adding/updating a param.
8. Explain why & and = need encoding in query values.
9. Write a tiny URL shortener encoder (conceptually).
10. Compare `quote` vs `quote_plus`.

# 35 Chapter 34: Raw Sockets

The `socket` module provides low-level TCP/UDP/raw sockets, and `net_raw` provides IP/TCP packet construction. This chapter covers raw socket programming and packet crafting.

## 35.1 34.1 TCP client

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.settimeout(sock, 5000)
socket.connect(sock, "example.com", 80)

socket.send(sock, "GET / HTTP/1.0\r\nHost: example.com\r\n\r\n")
let response = socket.recv(sock, 4096)
print(socket.recv_text(sock, 4096))

socket.close(sock)
```

### 35.1.1 Socket functions

| Function | Description |
|---|---|
| `socket.create(domain, type)` | Create socket, returns handle |
| `socket.connect(sock, host, port)` | Connect TCP |
| `socket.bind(sock, host, port)` | Bind to local address |
| `socket.listen(sock, backlog)` | Listen for connections |
| `socket.accept(sock)` | Accept incoming connection |
| `socket.send(sock, data)` | Send data |
| `socket.recv(sock, n)` | Receive up to n bytes |
| `socket.recv_text(sock, n)` | Receive text |
| `socket.recv_from(sock, n)` | Receive from address (UDP) |
| `socket.send_to(sock, data, host, port)` | Send to address (UDP) |
| `socket.settimeout(sock, ms)` | Set timeout |
| `socket.close(sock)` | Close socket |
| `socket.fileno(sock)` | Get file descriptor |

### 35.1.2 Address families and socket types

| Constant | Meaning |
|---|---|
| `socket.AF_INET` | IPv4 |
| `socket.AF_INET6` | IPv6 |
| `socket.AF_UNIX` | Unix domain sockets |
| `socket.SOCK_STREAM` | TCP |

| Constant | Meaning |
|---|---|
| `socket.SOCK_DGRAM` | UDP |
| `socket.SOCK_RAW` | Raw IP (requires root) |

## 35.2 34.2 TCP client

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.settimeout(sock, 5000)
socket.connect(sock, "example.com", 80)

socket.send(sock, "GET / HTTP/1.0\r\nHost: example.com\r\n\r\n")
let response = socket.recv(sock, 4096)
print(socket.recv_text(sock, 4096))

socket.close(sock)
```

### 35.2.1 recv returns bytes

`socket.recv` returns a byte list (list of ints 0–255). Use `socket.recv_text` for strings:

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.connect(sock, "example.com", 80)
socket.send(sock, "GET / HTTP/1.0\r\n\r\n")

let bytes = socket.recv(sock, 1024)   // list of ints
let text = socket.recv_text(sock, 1024) // string

socket.close(sock)
```

## 35.3 34.3 TCP server

```
import socket

let server = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.bind(server, "0.0.0.0", 8080)
socket.listen(server, 5)

print("Listening on 0.0.0.0:8080")

let client = socket.accept(server)
let req = socket.recv_text(client, 4096)
print("Request: " + req)
```

```
socket.send(client, "HTTP/1.0 200 OK\r\n\r\nHello")
socket.close(client)
socket.close(server)
```

## 35.4 34.4 UDP

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_DGRAM)
socket.settimeout(sock, 5000)

socket.send_to(sock, "hello", "127.0.0.1", 9999)

let [data, addr] = socket.recv_from(sock, 4096)
print(data)
print(addr.addr)

socket.close(sock)
```

`recv_from` returns `{data: <bytes>, addr: <ip_string>}`.

### 35.4.1 UDP server

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_DGRAM)
socket.bind(sock, "0.0.0.0", 9999)

while true {
    let [data, addr] = socket.recv_from(sock, 4096)
    print("From " + addr.addr + ": " + data)
    socket.send_to(sock, "pong", addr.addr, 9999)
}
```

## 35.5 34.5 Raw sockets and packet crafting (net_raw)

```
import net_raw

// Build an IP packet
let ip_pkt = net_raw.pack_ip("10.0.0.1", "10.0.0.2", 6)  // proto 6 = TCP
print(len(ip_pkt))  // 20 bytes

// Build a TCP segment
let tcp_pkt = net_raw.pack_tcp(12345, 80, "SYN")
print(len(tcp_pkt))  // 20 bytes
```

### 35.5.1 `pack_ip(src, dst, proto)`

Builds a 20-byte IPv4 header: - `src`, `dst`: IP address strings - `proto`: protocol number (6 = TCP, 17 = UDP, 1 = ICMP)

### 35.5.2 `pack_tcp(sport, dport, flags)`

Builds a 20-byte TCP header: - `sport`, `dport`: port numbers - `flags`: string containing any of FIN, SYN, RST, PSH, ACK, URG

```
import net_raw

let ip = net_raw.pack_ip("1.2.3.4", "5.6.7.8", 6)
print(len(ip))   // 20

let tcp = net_raw.pack_tcp(12345, 80, "SYN")
print(len(tcp))  // 20

// IPv4 header layout:
// Bytes 0-1: Version/IHL + TOS
// Bytes 2-3: Total length
// Bytes 4-5: Identification
// Bytes 6-7: Flags/Fragment offset
// Byte 8: TTL
// Byte 9: Protocol
// Bytes 10-11: Header checksum
// Bytes 12-15: Source IP
// Bytes 16-19: Destination IP

// TCP header layout:
// Bytes 0-1: Source port
// Bytes 2-3: Dest port
// Bytes 4-7: Sequence number
// Bytes 8-11: Acknowledgment number
// Byte 12: Data offset + reserved
// Byte 13: Flags
// Bytes 14-15: Window size
// Bytes 16-17: Checksum
// Bytes 18-19: Urgent pointer
```

### 35.5.3 Raw socket flags

```
import net_raw

let syn = net_raw.pack_tcp(12345, 80, "SYN")
let syn_ack = net_raw.pack_tcp(80, 12345, "SYN,ACK")
let ack = net_raw.pack_tcp(80, 12345, "ACK")
```

```
let fin = net_raw.pack_tcp(80, 12345, "FIN,ACK")
let rst = net_raw.pack_tcp(80, 12345, "RST")
```

## 35.6 34.6 Common patterns

### 35.6.1 Port scanner

```
import socket

func scan_port(host, port, timeout) {
    try {
        let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
        socket.settimeout(sock, timeout)
        socket.connect(sock, host, port)
        socket.close(sock)
        return true
    } catch {
        return false
    }
}

for port in 1..1024 {
    if scan_port("127.0.0.1", port, 100) {
        print("Port " + str(port) + " is open")
    }
}
```

### 35.6.2 HTTP client (manual)

```
import socket

func http_get(host, path) {
    let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
    socket.connect(sock, host, 80)

    let request = "GET " + path + " HTTP/1.0\r\nHost: " + host + "\r\n\r\n"
    socket.send(sock, request)

    let response = ""
    while true {
        let chunk = socket.recv_text(sock, 4096)
        if len(chunk) == 0 {
            break
        }
        response = response + chunk
    }
```

```
    socket.close(sock)
    return response
}

print(http_get("example.com", "/"))
```

## 35.7 34.7 Common pitfalls

- **Always close sockets.** Leaks file descriptors.
- **Set timeouts.** Without timeouts, `connect`/`recv` can block forever.
- **TCP is a stream.** `recv` may return partial data; loop until you have everything.
- **`socket.recv` returns bytes.** Use `recv_text` for strings, or convert manually.
- **UDP is connectionless.** Use `send_to`/`recv_from`, not `connect`/`send`/`recv`.
- **Raw sockets need root** on most OSes for `SOCK_RAW` and packet crafting.

## 35.8 34.8 Quick reference

```
import socket
import net_raw

let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.connect(sock, "example.com", 80)
socket.send(sock, "GET / HTTP/1.0\r\n\r\n")
let data = socket.recv(sock, 4096)
socket.close(sock)

// UDP
socket.send_to(sock, "data", "127.0.0.1", 9999)
let from = socket.recv_from(sock, 4096)

// Packet crafting
let ip = net_raw.pack_ip("1.2.3.4", "5.6.7.8", 6)
let tcp = net_raw.pack_tcp(1234, 80, "SYN")
```

## 35.9 34.8 Exercises

1. Write a TCP echo client that sends a line and prints the response.
2. Write a UDP client that sends a datagram and waits for a reply.
3. Write a port scanner for ports 1–1000 on localhost.
4. Build an IP+TCP packet with `net_raw` and inspect the bytes.
5. Write a simple HTTP/0.9 client that fetches the root page.
6. Write a TCP echo server that handles one connection at a time.
7. Write a UDP server that echoes back datagrams.

## 35.10 34.12 Raw sockets — deep reference

Raw sockets (`AF_PACKET` / `SOCK_RAW`) let you construct and inspect packets at the IP layer and below. Requires privileges (root/admin).

### 35.10.1 What you can do

- Craft custom IP/TCP/UDP/ICMP headers.
- Receive all packets matching a filter (promiscuous mode).
- Implement ping (ICMP), traceroute, packet inspection.
- Learn networking by building protocols from scratch.

### 35.10.2 What you should NOT do

- Send spoofed packets on networks you don't own (illegal/unethical).
- Flood or DoS targets.
- Capture traffic on shared networks without permission.

### 35.10.3 IP header (IPv4, 20 bytes min)

| Offset | Field | Size |
|---|---|---|
| 0 | version/IHL, DSCP/ECN | 1 byte |
| 1 | total length | 2 bytes |
| 3 | identification | 2 bytes |
| 5 | flags/fragment offset | 2 bytes |
| 7 | TTL | 1 byte |
| 8 | protocol (TCP=6, UDP=17, ICMP=1) | 1 byte |
| 9 | header checksum | 2 bytes |
| 11 | source IP | 4 bytes |
| 15 | destination IP | 4 bytes |

### 35.10.4 ICMP echo (ping)

1. Build IP header + ICMP header (type 8 = echo request).
2. Compute ICMP checksum.
3. Send via raw socket.
4. Wait for ICMP type 0 (echo reply).

### 35.10.5 TCP raw (SYN scan concept)

1. Build TCP header with SYN flag.
2. Send to target port.
3. SYN-ACK = open; RST = closed.

### 35.10.6 Usage (conceptual pseudocode)

```
import socket
# requires root
let s = socket.socket(socket.AF_PACKET, socket.SOCK_RAW, socket.IPPROTO_RAW)
s.bind(("eth0", 0))
let packet = build_ip_packet(dst="93.184.216.34", proto=socket.IPPROTO_ICMP, ...)
s.send(packet)
let data, addr = s.recvfrom(65535)
```

### 35.10.7 Checksums

Internet checksum: 16-bit one's complement of the sum of 16-bit words. Implemented manually for ICMP/TCP/UDP when the OS doesn't compute them.

### 35.10.8 Exercises

1. Describe the IPv4 header fields and their sizes.
2. Explain the ICMP echo request/reply flow (ping).
3. Sketch how traceroute works (TTL expiration).
4. List raw socket privileges and risks.
5. Write pseudocode for an ICMP ping.
6. Explain the Internet checksum algorithm.
7. Describe ARP: mapping IP → MAC.
8. Explain promiscuous mode and packet capture.
9. List protocols you can build with raw sockets (ICMP, custom TCP, etc.).
10. Research ethical/legal boundaries of packet crafting.

# 36 Chapter 35: Threading Fundamentals

Zen supports OS threads via the `thread` module. Threads run concurrently (subject to the OS scheduler) and communicate through `shared` atomic values and `channel` message passing.

## 36.1 35.1 The `thread` module

```
import thread

func worker(name) {
    print("Worker " + name + " starting")
    return "done: " + name
}

let t = thread.spawn(worker("A"))
let result = thread.join(t)
print(result)  # done: A

print(thread.current())  # current thread info (nil or handle)
```

### 36.1.1 Thread functions

| Function | Description | Example |
|---|---|---|
| `thread.spawn(func)` | Start `func` in a new thread | `thread.spawn(worker)` |
| `thread.join(thread)` | Wait for thread, get return value | `thread.join(t)` |
| `thread.current()` | Current thread info | `thread.current()` |

```
import thread

func count(n) {
    let total = 0
    for i in 1..n {
        total = total + i
    }
    return total
}

let t = thread.spawn(count(100))
let result = thread.join(t)
print(result)  # 5050
```

## 36.2 35.2 Thread lifecycle

```
import thread

func slow() {
    // simulate work
    return "finished"
}

let t = thread.spawn(slow)

// Do other work in main thread...

let result = thread.join(t)  // blocks until slow() returns
print(result)
```

`thread.join` blocks the calling thread until the target thread completes, then returns the target's return value. If the target raised an error, `join` propagates it.

```
import thread

func failing() {
    throw "boom"
}

let t = thread.spawn(failing)
try {
    thread.join(t)
} catch as e {
    print(e.msg)  # boom
}
```

## 36.3 35.3 Thread-local data

Variables created inside a thread function are local to that thread:

```
import thread

func worker() {
    let local = "only in this thread"
    print(local)
}

let t = thread.spawn(worker)
thread.join(t)
// local is not accessible here
```

To share data between threads, use `shared` (atomic values) or `channel` (message passing).

## 36.4 35.3 Multiple threads

```
import thread

func worker(id) {
    print("Worker " + str(id) + " started")
    return "worker " + str(id) + " done"
}

let threads = []
for i in 1..5 {
    let t = thread.spawn(worker(i))
    threads.push(t)
}

for t in threads {
    let result = thread.join(t)
    print(result)
}
print("All workers done")
```

## 36.5 35.4 Common pitfalls

- **`thread.join` blocks.** Join all threads before your program exits, or their work may be cut short.
- **Exceptions propagate.** If a thread function raises, `thread.join` re-raises it in the joining thread.
- **No shared state by default.** Each thread has its own stack and locals. Use `shared` or `channel` to communicate.
- **Thread count.** Don't spawn more threads than necessary; OS threads are relatively heavyweight.

## 36.6 35.5 Quick reference

```
import thread

let t = thread.spawn(func() { return "result" })
let result = thread.join(t)
let cur = thread.current()
```

## 36.7 35.6 Exercises

1. Write a program that spawns 5 threads, each printing its ID, and joins them all.
2. Write `parallel_sum(nums)` that splits a list into chunks, sums each chunk in a thread, and combines results.
3. Write a thread that sleeps for 1 second then prints "done".
4. Demonstrate that `thread.join` propagates errors from the child thread.

5. Write a function `run_concurrent(funcs)` that runs a list of zero-arg functions in threads and returns their results in order.

## 36.8 Extended Content

This section provides additional depth, patterns, debugging tips, and exercises for production use.

### 36.8.1 Common pitfalls

- Race conditions
- Deadlocks (circular wait)
- Starvation
- Data races
- Forgetting to join threads

### 36.8.2 Debugging concurrent code

- Use logging with thread identifiers
- Test with different interleavings
- Consider thread sanitizers conceptually
- Design for thread safety from start

### 36.8.3 Exercises

1. Producer-consumer with channels
2. Thread pool pattern
3. Parallel map/reduce
4. Avoiding deadlocks with lock ordering
5. Thread-safe cache
6. Benchmark sequential vs parallel

## 36.9 35.12 Worked examples — threading

### 36.9.1 Example 1: Basic thread creation and join

```
import threading

func worker(name) {
    print("Thread ${name} starting")
    # simulate work
    for i in 0..3 {
        print("${name} tick ${i}")
    }
    print("Thread ${name} finished")
}
```

```
let t1 = threading.Thread(fn() { worker("A") })
let t2 = threading.Thread(fn() { worker("B") })
t1.start()
t2.start()
t1.join()
t2.join()
print("both threads done")
```

Expected output (order may vary):

```
Thread A starting
Thread B starting
A tick 0
B tick 0
...
both threads done
```

### 36.9.2 Example 2: Sharing data — the race condition

```
import threading

var shared_counter = 0

func increment(times) {
    var shared_counter = shared_counter   # read global
    for i in 0..times {
        shared_counter = shared_counter + 1   # NOT atomic!
    }
}

let t1 = threading.Thread(fn() { increment(100000) })
let t2 = threading.Thread(fn() { increment(100000) })
t1.start()
t2.start()
t1.join()
t2.join()
print(shared_counter)   # likely LESS than 200000 — race condition!
```

**Why:** read-modify-write is not atomic. Two threads read the same value, both add 1, and one update is lost.

### 36.9.3 Example 3: Fix with a mutex (see Chapter 37)

Use `mutexes.Mutex` to guard the counter (full example in 37.12).

### 36.9.4 Example 4: Daemon threads

Daemon threads are killed when the main program exits.

```
let t = threading.Thread(fn() {
    while true { print("background..."); sleep(1000) }
})
t.daemon = true
t.start()
print("main exits; daemon thread dies")
```

### 36.9.5 Example 5: Thread-local storage

Each thread gets its own copy of a value.

```
let local = threading.local()

func worker() {
    local.value = threading.current_id()
    print("thread sees value " + str(local.value))
}
```

## 36.10 35.13 Common threading bugs

1. **Race condition:** unsynchronized read-modify-write → lost updates.
2. **Deadlock:** two threads each hold a lock the other needs → both wait forever. Fix: always acquire locks in the same order.
3. **Livelock:** threads keep retrying but make no progress.
4. **Starvation:** a thread never gets CPU/lock access.
5. **Forgetting `join`:** main exits before workers finish → partial output.
6. **Modifying shared collections without a lock.**
7. **Holding a lock too long** → poor concurrency.
8. **Calling code that blocks while holding a lock.**

## 36.11 35.14 Debugging threaded programs

- Log with a thread identifier so you can tell which thread printed what.
- Reproduce races by running many iterations; races are timing-dependent.
- Use `shared` for cross-thread state rather than plain globals.
- Keep critical sections small.
- Prefer message passing (channels) over shared memory when possible.

## 36.12 35.15 Exercises

1. Create 5 threads that each print their ID 3 times; join all.
2. Write a function `run_parallel(fns)` that starts a thread per function and joins all.
3. Demonstrate the race counter example and confirm the result < expected.
4. Fix the race counter using a mutex.
5. Write a daemon thread that prints every second; let main exit.
6. Implement `sum_parallel(list)` splitting work across N threads and merging.

7. Write `thread_id()` printing `threading.current_id()`.
8. Simulate a bank account with deposit/transfer from multiple threads.
9. Show what happens if you join a thread twice (error? no-op?).
10. Benchmark a CPU-heavy task sequential vs 2 threads.

# 37 Chapter 36: Channels

Channels send values between threads. A channel is created, values are sent into it, and received from it on the other end. Channels are unbuffered — `send` blocks until a `recv` is waiting, and vice versa.

## 37.1 36.1 The `channel` module

```
import channel

let ch = channel.make()

channel.send(ch, "hello")
channel.send(ch, 42)
channel.send(ch, true)

print(channel.recv(ch))  # hello
print(channel.recv(ch))  # 42
print(channel.recv(ch))  # true
```

### 37.1.1 Channel functions

| Function | Description | Example |
|---|---|---|
| `channel.make()` | Create a new channel | `channel.make()` |
| `channel.send(ch, value)` | Send a value (blocks until received) | `channel.send(ch, 42)` |
| `channel.recv(ch)` | Receive a value (blocks until sent) | `channel.recv(ch)` |

```
import channel

let ch = channel.make()

channel.send(ch, "first")
channel.send(ch, "second")
channel.send(ch, "third")

print(channel.recv(ch))  # first
print(channel.recv(ch))  # second
print(channel.recv(ch))  # third
```

## 37.2 36.2 Blocking semantics

Channels are unbuffered and synchronous:

- `send(ch, v)` blocks until some thread calls `recv(ch)`.

- `recv(ch)` blocks until some thread calls `send(ch, v)`.
- The value is handed directly from sender to receiver.

```
import channel
import thread

let ch = channel.make()

// Sender thread
thread.spawn(fn() {
    channel.send(ch, "ping")
    print("sent")
})

// Receiver (main thread)
let msg = channel.recv(ch)
print("received: " + msg)  // received: ping
```

## 37.3 36.3 Producer-consumer

```
import channel
import thread

let ch = channel.make()

func producer() {
    for i in 1..5 {
        channel.send(ch, i)
        print("produced: " + str(i))
    }
    channel.send(ch, nil)  // sentinel: done
}

func consumer() {
    while true {
        let v = channel.recv(ch)
        if v == nil {
            break
        }
        print("consumed: " + str(v))
    }
}

let t1 = thread.spawn(producer)
let t2 = thread.spawn(consumer)
thread.join(t1)
thread.join(t2)
```

```
print("all done")
```

Output:

```
produced: 1
consumed: 1
produced: 2
consumed: 2
...
produced: 5
consumed: 5
all done
```

## 37.4 36.4 Multiple producers and consumers

```
import channel
import thread

let ch = channel.make()

func producer(id, count) {
    for i in 1..count {
        channel.send(ch, #(id, i))
    }
}

func consumer(id) {
    while true {
        let v = channel.recv(ch)
        if v == nil {
            break
        }
        print("consumer " + str(id) + " got: " + str(v[1]))
    }
}

// Start 2 producers
thread.spawn(producer(1, 3))
thread.spawn(producer(2, 3))

// Send 2 sentinels (one per consumer)
channel.send(ch, nil)
channel.send(ch, nil)

// Start 2 consumers
thread.spawn(consumer(1))
thread.spawn(consumer(2))
```

## 37.5 36.5 Channels and shared values

Use `shared` for counters, `channel` for message passing:

```
import channel
import shared
import thread

let ch = channel.make()
let counter = shared.create(0)

func worker(id) {
    for i in 1..10 {
        shared.increment(counter, 1)
        channel.send(ch, #(id, i))
    }
}

let t1 = thread.spawn(worker(1))
let t2 = thread.spawn(worker(2))

// Collect results
for i in 1..20 {
    let v = channel.recv(ch)
    print("got: " + str(v))
}

thread.join(t1)
thread.join(t2)
print("counter: " + str(shared.get(counter)))  # 20
```

## 37.6 36.6 Common pitfalls

- **`recv` blocks forever** if no value is ever sent. Always ensure a sender exists, or use a sentinel value (like `nil`) to signal completion.
- **`send` blocks forever** if no receiver is waiting.
- **Channels are unbuffered.** There's no queue — sender and receiver must rendezvous.
- **Forgetting to join threads.** The program may exit before threads finish sending.

## 37.7 36.7 Quick reference

```
import channel

let ch = channel.make()
channel.send(ch, "msg")
let v = channel.recv(ch)
```

```
// Producer-consumer pattern
channel.send(ch, nil)  // sentinel to signal completion
```

## 37.8 36.8 Exercises

1. Write a program with 1 producer and 1 consumer using a channel.
2. Write a program with 2 producers and 1 consumer.
3. Write `parallel_map(func, items)` that runs `func` on each item in a thread and sends results via a channel.
4. Write a pipeline: stage1 generates numbers, stage2 squares them, stage3 prints them. Connect stages with channels.
5. Write a program where multiple workers pull jobs from a channel channel and send results to another channel.

## 37.9 Extended Content

This section provides additional depth, patterns, debugging tips, and exercises for production use.

### 37.9.1 Common pitfalls

- Race conditions
- Deadlocks (circular wait)
- Starvation
- Data races
- Forgetting to join threads

### 37.9.2 Debugging concurrent code

- Use logging with thread identifiers
- Test with different interleavings
- Consider thread sanitizers conceptually
- Design for thread safety from start

### 37.9.3 Exercises

1. Producer-consumer with channels
2. Thread pool pattern
3. Parallel map/reduce
4. Avoiding deadlocks with lock ordering
5. Thread-safe cache
6. Benchmark sequential vs parallel

## 37.10 36.12 Worked examples — channels

Channels are the “share memory by communicating” primitive.

### 37.10.1 Example 1: Basic send/recv

```
import channels

let ch = channels.Channel()

func producer() {
    for i in 0..5 {
        ch.send(i)
        print("sent ${i}")
    }
    ch.close()
}

func consumer() {
    while true {
        let v = ch.recv()
        if v == nil { break }   # channel closed and empty
        print("received ${v}")
    }
}

let tp = threading.Thread(fn() { producer() })
let tc = threading.Thread(fn() { consumer() })
tp.start()
tc.start()
tp.join()
tc.join()
print("done")
```

### 37.10.2 Example 2: Producer-consumer with buffered channel

```
let ch = channels.Channel(buffered=true, capacity=10)
# producer can send up to `capacity` items without blocking
```

### 37.10.3 Example 3: Multiple consumers (worker pool)

```
let jobs = channels.Channel()
let results = channels.Channel()

func worker(id) {
    while true {
        let job = jobs.recv()
        if job == nil { break }
        let out = job * job
        results.send(#(id, out))
```

```
    }
}

# start 4 workers
for i in 0..4 {
    threading.Thread(fn() { worker(i) }).start()
}

# send 20 jobs
for i in 0..20 { jobs.send(i) }
jobs.close()

# collect 20 results
for i in 0..20 {
    let r = results.recv()
    print("worker ${r[0]} -> ${r[1]}")
}
results.close()
```

### 37.10.4 Example 4: select-like multiplexing

Zen may not have a built-in `select`; emulate by checking channels in a loop or using a dedicated multiplexer. Conceptually:

```
# wait on the first ready channel
```

## 37.11 36.13 Channel patterns

| Pattern | Use |
|---|---|
| One producer, one consumer | simple pipeline |
| One producer, many consumers | fan-out / work distribution |
| Many producers, one consumer | aggregation / logging |
| Request-response | send request on ch1, receive reply on ch2 |
| Pipeline | stage A → ch1 → stage B → ch2 → stage C |

## 37.12 36.14 Common channel bugs

1. **Forgetting to close** → consumers block forever on recv.
2. **Sending on a closed channel** → error.
3. **Unbuffered channel rendezvous** → sender blocks until receiver ready.
4. **Deadlock** → both sides waiting to send/recv with no progress.
5. **Not draining** → producer blocks, pipeline stalls.

## 37.13 36.15 Exercises

1. Build a channel, send 10 numbers, receive and sum them.

2. Implement producer-consumer: producer sends 1..100, consumer sums.
3. Create a worker pool of 4 workers processing a job queue.
4. Implement a request-response channel pair.
5. Build a 3-stage pipeline (parse → transform → write) with channels.
6. Demonstrate what happens when you recv from an empty closed channel.
7. Implement `fan_in(ch1, ch2)` merging two channels into one.
8. Write a broadcast: one sender, three receivers all getting each value.
9. Implement a bounded buffer with a channel.
10. Simulate a chat room: multiple clients send messages to one channel.

# 38 Chapter 37: Mutexes & Synchronization

Zen provides two synchronization primitives for concurrent programming: `shared` (atomic shared values) and `channel` (message passing between threads). Together they let you write safe multi-threaded programs without manual locks.

## 38.1 37.1 The `shared` module — atomic shared values

```
import shared

# Create a shared integer counter
let counter = shared.create(0)

# Read the current value
print(shared.get(counter))   # 0

# Set a new value
shared.set(counter, 5)
print(shared.get(counter))   # 5

# Atomic increment (returns the new value)
shared.increment(counter, 1)
print(shared.get(counter))   # 6

shared.increment(counter, 10)
print(shared.get(counter))   # 16
```

### 38.1.1 Shared functions

| Function | Description | Example |
|---|---|---|
| `shared.create(value)` | Create a shared atomic value | `shared.create(0)` |
| `shared.get(shared)` | Read current value | `shared.get(counter)` |
| `shared.set(shared, value)` | Set a new value | `shared.set(counter, 5)` |
| `shared.increment(shared, delta)` | Add delta, return new value | `shared.increment(counter, 1)` |

Shared values are safe to read and write from multiple threads — operations are atomic, so there are no torn reads or races on the value itself.

```
import shared
import thread

let counter = shared.create(0)

func worker(id) {
```

```
    for i in 1..100 {
        shared.increment(counter, 1)
    }
    print("worker " + str(id) + " done")
}

let threads = []
for i in 1..5 {
    let t = thread.spawn(worker(i))
    threads.push(t)
}

for t in threads {
    thread.join(t)
}

print(shared.get(counter))  # 500 (5 workers × 100 increments)
```

Because `shared.increment` is atomic, the final count is exactly 500 — no lost updates even with 5 threads incrementing concurrently.

## 38.2 37.2 The `channel` module — message passing

Channels send values between threads. A channel is created, values are sent into it, and received from it on the other end.

```
import channel

let ch = channel.make()

channel.send(ch, "hello")
channel.send(ch, 42)
channel.send(ch, true)

print(channel.recv(ch))  # hello
print(channel.recv(ch))  # 42
print(channel.recv(ch))  # true
```

### 38.2.1 Channel functions

| Function | Description | Example |
|---|---|---|
| `channel.make()` | Create a new channel | `channel.make()` |
| `channel.send(ch, value)` | Send a value | `channel.send(ch, 42)` |
| `channel.recv(ch)` | Receive a value (blocks) | `channel.recv(ch)` |

`recv` blocks until a value is available, making channels useful for coordinating work

between threads.

## 38.3 37.3 Threads: `thread` module

```
import thread

func worker(name) {
    print("Worker " + name + " starting")
    return "done: " + name
}

let t = thread.spawn(worker("A"))
let result = thread.join(t)
print(result)  # done: A

print(thread.current())  # nil or current thread info
```

### 38.3.1 Thread functions

| Function | Description | Example |
|---|---|---|
| `thread.spawn(func)` | Start a function in a thread | `thread.spawn(worker)` |
| `thread.join(thread)` | Wait for thread, get result | `thread.join(t)` |
| `thread.current()` | Current thread info | `thread.current()` |

## 38.4 37.4 Producer-consumer with channels

```
import channel
import thread

let ch = channel.make()

func producer() {
    for i in 1..5 {
        channel.send(ch, i)
        print("produced: " + str(i))
    }
    channel.send(ch, nil)  // sentinel: done
}

func consumer() {
    while true {
        let v = channel.recv(ch)
        if v == nil {
            break
        }
```

```
        print("consumed: " + str(v))
    }
}

let t1 = thread.spawn(producer)
let t2 = thread.spawn(consumer)
thread.join(t1)
thread.join(t2)
print("all done")
```

## 38.5 37.5 Shared counter across threads

```
import shared
import thread

let counter = shared.create(0)

func worker(id) {
    for i in 1..100 {
        shared.increment(counter, 1)
    }
    print("worker " + str(id) + " done")
}

let threads = []
for i in 1..5 {
    let t = thread.spawn(worker(i))
    threads.push(t)
}

for t in threads {
    thread.join(t)
}

print(shared.get(counter))  # 500 (5 workers × 100 increments)
```

Because `shared.increment` is atomic, the final count is exactly 500 — no lost updates even with 5 threads incrementing concurrently.

## 38.6 37.6 Common patterns

### 38.6.1 Guarded shutdown flag

```
import shared
import thread
```

```
let running = shared.create(true)

func worker() {
    while shared.get(running) {
        # do work
    }
    print("worker stopped")
}

let t = thread.spawn(worker)
// ... later ...
shared.set(running, false)
thread.join(t)
```

### 38.6.2 Work queue with channel

```
import channel
import thread

let queue = channel.make()

func dispatcher(jobs) {
    for job in jobs {
        channel.send(queue, job)
    }
    channel.send(queue, nil)  // done
}

func worker(queue) {
    while true {
        let job = channel.recv(queue)
        if job == nil { break }
        print("processing: " + str(job))
    }
}

let jobs = [1, 2, 3, 4, 5, 6, 7, 8]
let td = thread.spawn(dispatcher(jobs))
let tw1 = thread.spawn(worker(queue))
let tw2 = thread.spawn(worker(queue))
thread.join(td)
thread.join(tw1)
thread.join(tw2)
```

## 38.7 37.8 Common pitfalls

- **`recv` blocks forever** if no value is ever sent. Always ensure a sender exists, or use a sentinel value (like `nil`) to signal completion.
- **`shared` values are atomic individually**, but compound operations (read-then-write) are not atomic. Use `increment` instead of `get`+`set`+counter when you need to add.
- **`join` blocks** until the thread finishes. Join all threads before your program exits, or their work may be cut short.
- **Channels are unbuffered.** `send` blocks until a `recv` is waiting, and `recv` blocks until a `send` happens.

## 38.8 37.9 Quick reference

```
import shared
import channel
import thread

# Shared atomic values
let c = shared.create(0)
shared.get(c)
shared.set(c, 10)
shared.increment(c, 1)

# Channels
let ch = channel.make()
channel.send(ch, "msg")
channel.recv(ch)

# Threads
let t = thread.spawn(func() { return "result" })
thread.join(t)
thread.current()
```

## 38.9 37.10 Exercises

1. Write a program that spawns 5 threads, each incrementing a shared counter 1000 times, and verify the final count is 5000.
2. Use a channel to implement a producer-consumer with 2 producers and 1 consumer.
3. Write a `parallel_map(func, items)` that runs `func` on each item in a separate thread and collects results via a channel.
4. Use a shared boolean flag to gracefully stop a background thread.
5. Write a thread-safe counter class using `shared.create` and methods.

## 38.10 Extended Content

This section provides additional depth, patterns, debugging tips, and exercises for production use.

### 38.10.1 Common pitfalls

- Race conditions
- Deadlocks (circular wait)
- Starvation
- Data races
- Forgetting to join threads

### 38.10.2 Debugging concurrent code

- Use logging with thread identifiers
- Test with different interleavings
- Consider thread sanitizers conceptually
- Design for thread safety from start

### 38.10.3 Exercises

1. Producer-consumer with channels
2. Thread pool pattern
3. Parallel map/reduce
4. Avoiding deadlocks with lock ordering
5. Thread-safe cache
6. Benchmark sequential vs parallel

## 38.11 37.12 Worked examples — mutexes and locks

### 38.11.1 Example 1: Fixing the race counter with a Mutex

```
import threading
import mutexes

var counter = 0
let mu = mutexes.Mutex()

func increment(times) {
    for i in 0..times {
        mu.lock()
        counter = counter + 1
        mu.unlock()
    }
}

let t1 = threading.Thread(fn() { increment(100000) })
```

```
let t2 = threading.Thread(fn() { increment(100000) })
t1.start()
t2.start()
t1.join()
t2.join()
print(counter)   # 200000 — correct!
```

### 38.11.2 Example 2: Lock with try/finally (always unlock)

```
func safe_increment() {
    mu.lock()
    try {
        counter = counter + 1
    } finally {
        mu.unlock()
    }
}
```

### 38.11.3 Example 3: Deadlock — and how to avoid it

```
# BAD: potential deadlock
let mu1 = mutexes.Mutex()
let mu2 = mutexes.Mutex()

func thread_a() {
    mu1.lock()
    mu2.lock()   # if thread_b holds mu2 and waits for mu1 -> deadlock
    ...
    mu2.unlock()
    mu1.unlock()
}

func thread_b() {
    mu2.lock()
    mu1.lock()   # opposite order!
    ...
    mu1.unlock()
    mu2.unlock()
}
```

**Fix:** Always acquire locks in the same global order (mu1 before mu2).

```
# GOOD
func thread_b_fixed() {
    mu1.lock()   # same order as thread_a
    mu2.lock()
```

```
    ...
    mu2.unlock()
    mu1.unlock()
}
```

### 38.11.4 Example 4: Read-write pattern (conceptual)

If your build has `RWMutex`, multiple readers can hold it simultaneously, but a writer needs exclusive access. Otherwise, use a plain Mutex for all access.

## 38.12 37.13 Locking guidelines

1. **Lock order:** global, consistent order prevents deadlocks.
2. **Fine-grained locks:** lock only what's necessary; don't lock the whole data structure for a small update.
3. **Short critical sections:** minimize work inside `lock/unlock`.
4. **Never call blocking code** (sleep, network) while holding a lock.
5. **Use `finally`** to guarantee unlock even on error.
6. **Prefer channels** over shared+lock when the design allows.

## 38.13 37.14 Exercises

1. Protect a shared counter with a Mutex; verify exact final value.
2. Implement a thread-safe stack using a Mutex and a list.
3. Implement a thread-safe cache (dict + Mutex) with get/put.
4. Demonstrate deadlock with two locks in opposite order, then fix it.
5. Write `with_lock(mu, fn)` that locks, runs fn, unlocks in finally.
6. Protect a shared bank account: deposit/transfer from many threads.
7. Implement `increment_n(mu, counter, n)` using lock/unlock.
8. Show that reading without a lock can see torn/stale values (conceptually).
9. Implement a thread-safe counter class wrapping Mutex + int.
10. Benchmark locked vs unlocked (single-threaded) to measure lock cost.

# 39 Chapter 38: Thread Safety & Patterns

This chapter covers practical thread-safe patterns: worker pools, reader-writer scenarios, producer-consumer pipelines, and avoiding common concurrency bugs.

## 39.1 38.1 Worker pool pattern

A fixed number of worker threads pull jobs from a shared channel:

```
import channel
import thread

let job_queue = channel.make()
let results = channel.make()

func worker(id) {
    while true {
        let job = channel.recv(job_queue)
        if job == nil {
            break  # sentinel: shutdown
        }
        let result = job * job  # some work
        channel.send(results, [id, result])
    }
}

func dispatcher(jobs) {
    for j in jobs {
        channel.send(job_queue, j)
    }
    # Send one sentinel per worker
    for i in 1..4 {
        channel.send(job_queue, nil)
    }
}

// Start 4 workers
for i in 1..4 {
    thread.spawn(worker(i))
}

let jobs = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
thread.spawn(dispatcher(jobs))

// Collect results
let completed = 0
while completed < len(jobs) {
    let r = channel.recv(results)
```

```
    print("worker " + str(r[0]) + " -> " + str(r[1]))
    completed = completed + 1
}
print("all jobs done")
```

## 39.2 38.2 Pipeline pattern

Chain stages where each stage is a thread:

```
import channel
import thread

let stage1_out = channel.make()
let stage2_out = channel.make()

func stage1() {
    for i in 1..10 {
        channel.send(stage1_out, i * 2)
    }
    channel.send(stage1_out, nil)
}

func stage2() {
    while true {
        let v = channel.recv(stage1_out)
        if v == nil {
            channel.send(stage2_out, nil)
            break
        }
        channel.send(stage2_out, v + 10)
    }
}

func stage3() {
    while true {
        let v = channel.recv(stage2_out)
        if v == nil { break }
        print("final: " + str(v))
    }
}

thread.spawn(stage1)
thread.spawn(stage2)
thread.spawn(stage3)
```

## 39.3 38.3 Reader-writer pattern with shared flag

```
import shared
import thread

let active = shared.create(true)

func reader(id) {
    while shared.get(active) {
        # read shared resource
    }
    print("reader " + str(id) + " stopping")
}

func writer() {
    # do some writes
    shared.set(active, false)  # signal shutdown
}

let r1 = thread.spawn(reader(1))
let r2 = thread.spawn(reader(2))
let w = thread.spawn(writer)
thread.join(w)
thread.join(r1)
thread.join(r2)
```

## 39.4 38.4 Avoiding deadlocks

Deadlocks happen when two threads each wait for the other. Prevention rules:

1. **Always acquire resources in the same order.**
2. **Don’t hold a resource while waiting for another.**
3. **Use timeouts where possible.**
4. **Keep critical sections small.**

Bad (potential deadlock):

```
// Thread A: lock A, then wait for B
// Thread B: lock B, then wait for A
```

Good (consistent order):

```
// Both threads: lock A, then lock B
```

## 39.5 38.5 Avoiding race conditions

A race condition is when the outcome depends on thread timing. Prevention:

1. **Use `shared` for atomic counters** — `shared.increment` instead of `get+set`.
2. **Use channels to hand off ownership** — send a value, don’t share it.

3. **Don’t share mutable state** between threads unless protected.

Bad (race):

```
let count = 0  # not shared — each thread may see its own copy
func bad_worker() {
    for i in 1..100 {
        count = count + 1  # RACE: lost updates
    }
}
```

Good (atomic):

```
import shared
let count = shared.create(0)
func good_worker() {
    for i in 1..100 {
        shared.increment(count, 1)
    }
}
```

## 39.6 38.6 Thread-local data

Each thread has its own stack and locals. Variables created inside a thread function are not visible to other threads unless shared via `shared` or `channel`:

```
import thread

func worker() {
    let local = "only in this thread"  # thread-local
    print(local)
}

let t = thread.spawn(worker)
thread.join(t)
// print(local)  # error: local not defined here
```

## 39.7 38.7 Common patterns summary

| Pattern | Use when | Primitive |
|---|---|---|
| Worker pool | Many small tasks | channel + thread.spawn |
| Pipeline | Sequential stages | channels chained |
| Producer-consumer | One produces, another consumes | channel |
| Shared counter | Statistics, progress | shared.create/increment |
| Shutdown flag | Coordinate exit | shared.create/set/get |

## 39.8 38.8 Quick reference

```
import shared
import channel
import thread

# Worker pool
let q = channel.make()
for i in jobs { channel.send(q, i) }
for i in 1..N { channel.send(q, nil) }
for i in 1..N { thread.spawn(worker(i)) }

# Atomic counter
let c = shared.create(0)
shared.increment(c, 1)
shared.get(c)

# Join all threads
for t in threads { thread.join(t) }
```

## 39.9 38.9 Exercises

1. Write a worker pool with 4 workers processing a list of URLs (simulated).
2. Implement a 3-stage pipeline: generate → transform → print.
3. Write a `parallel_map` that runs a function on each item concurrently.
4. Use a shared flag to stop all workers when one encounters an error.
5. Write a program that counts lines across multiple "files" (lists) in parallel threads and sums the total.

## 39.10 Extended Content

This section provides additional depth, patterns, debugging tips, and exercises for production use.

### 39.10.1 Common pitfalls

- Race conditions
- Deadlocks (circular wait)
- Starvation
- Data races
- Forgetting to join threads

### 39.10.2 Debugging concurrent code

- Use logging with thread identifiers
- Test with different interleavings
- Consider thread sanitizers conceptually

- Design for thread safety from start

#### 39.10.3 Exercises

1. Producer-consumer with channels
2. Thread pool pattern
3. Parallel map/reduce
4. Avoiding deadlocks with lock ordering
5. Thread-safe cache
6. Benchmark sequential vs parallel

### 39.11 38.12 Worked examples — thread safety patterns

#### 39.11.1 Example 1: Producer-consumer with channel + mutex

```
import threading, channels, mutexes

let buffer = channels.Channel(capacity=5)
let mu = mutexes.Mutex()
var produced = 0
var consumed = 0

func producer() {
    for i in 0..20 {
        buffer.send(i)
        mu.lock(); produced = produced + 1; mu.unlock()
        print("produced ${i}")
    }
    buffer.close()
}

func consumer() {
    while true {
        let v = buffer.recv()
        if v == nil { break }
        mu.lock(); consumed = consumed + 1; mu.unlock()
        print("consumed ${v}")
    }
}

let tp = threading.Thread(fn() { producer() })
let tc = threading.Thread(fn() { consumer() })
tp.start()
tc.start()
tp.join()
tc.join()
print("produced=${produced} consumed=${consumed}")
```

### 39.11.2 Example 2: Thread-safe lazy initialization (double-checked locking)

```
import mutexes

let init_mu = mutexes.Mutex()
var instance = nil
var initialized = false

func get_instance() {
    if initialized { return instance }
    init_mu.lock()
    try {
        if !initialized {
            instance = #(value: 42)
            initialized = true
        }
    } finally {
        init_mu.unlock()
    }
    return instance
}
```

### 39.11.3 Example 3: Thread pool (fixed number of workers)

```
import threading, channels

func thread_pool(num_workers, jobs) {
    let job_ch = channels.Channel()
    let result_ch = channels.Channel()
    let done = channels.Channel()

    for w in 0..num_workers {
        threading.Thread(fn() {
            while true {
                let job = job_ch.recv()
                if job == nil { break }
                let r = job * job
                result_ch.send(r)
            }
            done.send(nil)
        }).start()
    }

    for j in jobs { job_ch.send(j) }
    job_ch.close()
    # wait for all workers to finish
    for w in 0..num_workers { done.recv() }
```

```
    result_ch.close()
    return result_ch   # caller drains results
}
```

### 39.11.4 Example 4: Parallel map

```
func parallel_map(xs, f, num_workers) {
    let results = channels.Channel(capacity=len(xs))
    let job_ch = channels.Channel()
    let done = channels.Channel()

    for w in 0..num_workers {
        threading.Thread(fn() {
            while true {
                let item = job_ch.recv()
                if item == nil { break }
                results.send(f(item))
            }
            done.send(nil)
        }).start()
    }

    for x in xs { job_ch.send(x) }
    job_ch.close()
    for w in 0..num_workers { done.recv() }
    results.close()

    let out = []
    while true {
        let r = results.recv()
        if r == nil { break }
        out.push(r)
    }
    return out
}

print(parallel_map([1,2,3,4,5], fn(x) => x * x, 2))   # [1,4,9,16,25] (order varies)
```

### 39.11.5 Example 5: Reader-writer preference

If many readers and rare writers, use a read-write lock if available; otherwise guard all access with one Mutex. Readers serialize (safe but not maximal concurrency).

## 39.12 38.13 Pattern catalog

| Pattern | When to use | Primitive |
|---|---|---|
| Guarded shared state | occasional updates | Mutex |
| Producer-consumer | pipeline stages | Channel |
| Worker pool | many independent jobs | Channel + Threads |
| Parallel map | transform a collection | Channel + Threads |
| Lazy init | one-time expensive setup | Mutex + flag |
| Request-response | client/server in-proc | two Channels |
| Pub/sub | one event, many listeners | Channel per subscriber |

## 39.13 38.14 Exercises

1. Implement a thread-safe counter class with `increment`, `decrement`, `get` using a Mutex.
2. Build a producer-consumer where the producer is faster than the consumer (use a bounded channel to apply backpressure).
3. Implement a thread pool and use it to compute squares of 1..100.
4. Write `parallel_map` and verify results match sequential map.
5. Implement double-checked lazy initialization; call from 3 threads; confirm single initialization.
6. Create a request-response channel: client sends request, server replies.
7. Implement a simple pub/sub: publisher broadcasts to N subscriber channels.
8. Build a thread-safe LRU cache (dict + Mutex + order tracking).
9. Demonstrate a race without synchronization, then fix with a Mutex.
10. Write a concurrent web fetcher (conceptually) using a thread pool.

# 40 Chapter 39: FFI Fundamentals

Zen's `ffi` module lets you call functions in native C shared libraries (`.so`, `.dll`, `.dylib`) using libffi, with true C ABI support. This chapter covers loading libraries, finding symbols, calling functions, and managing memory.

## 40.1 39.1 Loading a library

```
import ffi

// Load a shared library
let libc = ffi.load("libc.so.6")        // Linux
// let libc = ffi.load("libc.dylib")     // macOS
// let libc = ffi.load("msvcrt.dll")     // Windows

print(libc)  // library handle (integer)
```

`ffi.load(path)` returns a library handle you pass to `ffi.symbol` and `ffi.call`.

## 40.2 39.2 Finding symbols

```
import ffi

let libc = ffi.load("libc.so.6")

// Get the address of a function
let puts_addr = ffi.symbol(libc, "puts")
let printf_addr = ffi.symbol(libc, "printf")

print(puts_addr)    // integer address
print(printf_addr)  // integer address
```

`ffi.symbol(lib, name)` returns the address of `name` in `lib`.

## 40.3 39.3 Calling functions

```
import ffi

let libc = ffi.load("libc.so.6")
let puts_addr = ffi.symbol(libc, "puts")

// Call puts("hello") — returns int
let result = ffi.call(puts_addr, ["hello"], {"type": "int"})
print(result)  // number of chars printed (or 0)
```

### 40.3.1 `ffi.call(address, args, ret?)`

- `address` — function address (from `ffi.symbol`)
- `args` — list of argument values
- `ret` — return type spec (optional, defaults to `int`)

### 40.3.2 Argument types

Arguments can be: - **Integers** — passed as C `int`/`long` - **Floats** — passed as C `double` - **Strings** — passed as C `char*` (null-terminated) - **`{"type": "double", "value": 1.5}`** — typed scalar - **`{"buf": "hex..."}`** — raw byte buffer - **`{"type": "bytes", "len": n}`** — buffer of size n

### 40.3.3 Return types

| `ret` | Meaning |
|---|---|
| `nil` or omitted | `int` (default) |
| `"double"` | double |
| `"void"` | no return value |
| `"str"` | char* → string |
| `{"type": "bytes", "len": n}` | raw buffer of n bytes |

### 40.3.4 Examples

```
import ffi

let libc = ffi.load("libc.so.6")

// puts("hello") → int
let r1 = ffi.call(ffi.symbol(libc, "puts"), ["hello"])
print(r1)

// printf("value: %d\n", 42) → int
let r2 = ffi.call(ffi.symbol(libc, "printf"), ["value: %d\n", 42])
print(r2)

// sqrt(2.0) → double
let sqrt_addr = ffi.symbol(libc, "sqrt")
let r3 = ffi.call(sqrt_addr, [{"type": "double", "value": 2.0}], "double")
print(r3)  // 1.4142135623730951
```

## 40.4 39.4 Memory management

```
import ffi
```

```
// Allocate memory
let ptr = ffi.malloc(1024)
print(ptr)  // address

// Reallocate
let ptr2 = ffi.realloc(ptr, 2048)

// Free
ffi.free(ptr)

// Read / write memory
ffi.write(ptr, "hello")
let s = ffi.read(ptr, 5)
print(s)  // hello

// String conversion
let s2 = ffi.str(ptr)
print(s2)  // hello

// Set bytes at offset
ffi.set(ptr, 0, {"buf": "776f726c64"})  // "world" in hex
let s3 = ffi.read(ptr, 5)
print(s3)  // world

// Size of types
print(ffi.sizeof("int"))    # 4
print(ffi.sizeof("double")) # 8

// Alignment
print(ffi.alignof("double")) # 8

// errno
print(ffi.errno())
print(ffi.strerror(ffi.errno()))
```

### 40.4.1 Memory functions

| Function | Description |
|---|---|
| `ffi.malloc(size)` | Allocate `size` bytes |
| `ffi.realloc(ptr, size)` | Reallocate |
| `ffi.free(ptr)` | Free memory |
| `ffi.read(ptr, n)` | Read `n` bytes as string |
| `ffi.write(ptr, data)` | Write string to memory |
| `ffi.str(ptr)` | Read null-terminated string |
| `ffi.set(ptr, offset, bytes)` | Write raw bytes at offset |
| `ffi.sizeof(type)` | Size of C type |

| Function | Description |
|---|---|
| `ffi.alignof(type)` | Alignment of C type |
| `ffi.errno()` | Last errno value |
| `ffi.strerror(errno)` | Error message for errno |

## 40.5 39.5 Complete example: calling `qsort`

```
import ffi

let libc = ffi.load("libc.so.6")
let qsort_addr = ffi.symbol(libc, "qsort")

// qsort(base, nel, width, compar)
// We'd need a callback for compar — FFI doesn't support Zen closures as
// C callbacks directly. See Chapter 40 for workarounds.

print("qsort loaded at " + str(qsort_addr))
```

## 40.6 39.6 Safety warnings

 **FFI is unsafe.** Zen cannot validate: - Whether the address is a real function - Whether argument types match the C signature - Whether memory accesses are in bounds

Misuse can crash the process or corrupt memory. Only load libraries you trust, and use `ffi.allow`/`ffi.deny` to restrict which libraries can be loaded (see Chapter 40).

## 40.7 39.7 Quick reference

```
import ffi

let lib = ffi.load("libc.so.6")
let addr = ffi.symbol(lib, "puts")
let r = ffi.call(addr, ["hello"])            # int return
let r = ffi.call(addr, ["hello"], "int")     # explicit int
let r = ffi.call(addr, [4.5], "double")      # double return

let ptr = ffi.malloc(1024)
ffi.write(ptr, "data")
let s = ffi.read(ptr, 4)
ffi.free(ptr)

print(ffi.sizeof("int"))     # 4
print(ffi.alignof("double")) # 8
print(ffi.errno())
print(ffi.strerror(2))
```

## 40.8 39.8 Exercises

1. Load `libc.so.6` and call `puts` with a string.
2. Call `printf` with a format string and an integer.
3. Call `sqrt` from `libm` and print the result as a double.
4. Allocate memory with `ffi.malloc`, write a string, read it back, free it.
5. Check `ffi.errno()` after a failing operation and print `ffi.strerror`.

## 40.9 Advanced Topics and Exercises

### 40.9.1 Safety considerations

- Memory management across FFI boundary
- Type mismatches
- Calling conventions
- Null pointers
- Resource leaks

### 40.9.2 Debugging FFI

- Check symbol names (nm, objdump)
- Use cdecl/stdcall correctly
- Test with simple functions first

### 40.9.3 Exercises

1. Call simple C function from shared library
2. Pass strings and arrays
3. Handle pointers/structs
4. Manage memory allocation
5. Error handling across boundary
6. Write wrapper module

## 40.10 39.12 Worked examples — FFI fundamentals

### 40.10.1 Conceptual model

Zen's FFI lets you call C functions in shared libraries (`.so`, `.dll`, `.dylib`) and pass data across the boundary. Values are marshalled: Zen ints, floats, strings, bytes, lists, dicts map to C primitives/pointers/opaque handles.

Typical flow: 1. `load_library("libfoo.so")` → library handle 2. `get_symbol(lib, "foo_function")` → callable symbol 3. Call the symbol with Zen arguments; Zen marshals to C. 4. Receive the C return value, marshalled back to Zen. 5. Close/drop the handle when done.

### 40.10.2 Example 1: Call a simple C function (conceptual pseudocode)

Assuming a C library `libmathutils.so` with:

```c
// mathutils.c
int add(int a, int b) { return a + b; }
double scale(double v, double f) { return v * f; }
```

Zen side (exact API depends on build):

```
import ffi

let lib = ffi.load_library("libmathutils.so")
let add_fn = ffi.get_symbol(lib, "add")
let result = ffi.call(add_fn, [21, 21])   # 42
print(result)

let scale_fn = ffi.get_symbol(lib, "scale")
let scaled = ffi.call(scale_fn, [10.0, 3.5])   # 35.0
print(scaled)

ffi.close(lib)
```

### 40.10.3 Example 2: Working with strings across FFI

```
let lib = ffi.load_library("libcstring.so")
let len_fn = ffi.get_symbol(lib, "string_length")
let n = ffi.call(len_fn, ["hello"])   # 5
print(n)
```

Zen marshals a Zen string to a C `const char*`. The C side must not modify it unless the API specifies a mutable buffer (pass `bytes` or a pointer handle).

### 40.10.4 Example 3: Pointers and handles

Some APIs return opaque pointers (e.g., file handles, database connections). Zen represents these as foreign handles. You must track their lifetime and release them via a C `destroy`/`free` function.

```
let db = ffi.call(open_db_fn, ["data.db"])   # foreign handle
try {
    let row = ffi.call(query_fn, [db, "SELECT * FROM t"])
    print(row)
} finally {
    ffi.call(close_db_fn, [db])   # always release
}
```

### 40.10.5 Example 4: Callbacks (Zen function passed to C)

If the C API accepts a function pointer, Zen can pass a closure. The FFI layer creates a trampoline so C can call back into Zen.

```
let cmp = ffi.get_symbol(lib, "compare")
ffi.call(cmp, [list_a, list_b, fn(a, b) => a - b])
```

**Caution:** the Zen callback must not outlive the Zen closure's scope, and must not raise exceptions that cross the C boundary uncaught.

## 40.11 39.13 Data type mapping (typical)

| Zen type | C equivalent | Notes |
|---|---|---|
| int | int32_t / int64_t | sign depends on value |
| float | double | |
| string | const char* | NUL-terminated; Zen owns it |
| bytes | unsigned char* | raw buffer |
| list | array / pointer | marshalled copy |
| dict | not directly | serialize first (JSON/msgpack) |
| nil | NULL | |
| handle | void* | opaque pointer |

## 40.12 39.14 Safety considerations

1. **Memory ownership:** know who frees what. Zen-allocated strings passed to C must remain valid during the call; C-allocated memory returned to Zen must be freed by Zen or the C side per convention.
2. **Null pointers:** dereferencing a NULL handle crashes. Check before use.
3. **Type mismatches:** passing a float where C expects int corrupts data.
4. **Calling convention:** cdecl/stdcall must match the platform/library.
5. **Thread safety:** a C library not thread-safe must be called with a Zen Mutex held.
6. **Exceptions across boundary:** don't let a Zen throw unwind through C frames; catch at the Zen edge.

## 40.13 39.15 Debugging FFI

- `nm -D libfoo.so` to list exported symbols.
- `objdump -T libfoo.so` for dynamic symbols.
- Confirm symbol names match exactly (C name mangling).
- Start with a trivial function (add two ints) before structs/pointers.
- Log every FFI call and return value.

## 40.14 39.16 Exercises

1. Load a shared library and call a function that adds two ints.
2. Call a C function that takes a string and returns its length.
3. Pass a bytes buffer to C and read back modified bytes (if supported).
4. Open an opaque handle, use it, then close it in `finally`.
5. Pass a Zen callback to a C `qsort`-style comparison function.
6. Write a wrapper `ZenMath` module exposing `add`, `scale` via FFI.

7. Demonstrate a null-handle guard before calling C.
8. Use `nm -D` to inspect a library's exported symbols.
9. Wrap a C file-reading function with error handling.
10. Benchmark a pure-Zen function vs the same logic via FFI.

# 41 Chapter 40: FFI in Practice

This chapter covers practical FFI patterns: capability gates, calling real libraries, error handling, and safe integration.

## 41.1 40.1 FFI capability gates

 FFI can load arbitrary shared libraries and call arbitrary functions. Use `ffi.allow` and `ffi.deny` to restrict which libraries can be loaded.

```
import ffi

// Deny loading from dangerous locations
ffi.deny(["/tmp/", "/var/tmp/"])

// Only allow loading from trusted directories
ffi.allow(["/usr/lib/", "/lib/"])

// Now ffi.load("/tmp/evil.so") raises an error
// ffi.load("/usr/lib/libc.so.6") works
```

### 41.1.1 Behavior

| Allow list | Deny list | Result |
|---|---|---|
| empty | empty | All paths allowed |
| non-empty | empty | Only paths matching an allow prefix |
| empty | non-empty | All except deny prefixes |
| non-empty | non-empty | Must match allow AND not match deny |

## 41.2 40.2 Calling libc functions

```
import ffi

ffi.allow(["/usr/lib/", "/lib/"])
let libc = ffi.load("libc.so.6")

// getpid() → int
let getpid = ffi.symbol(libc, "getpid")
let pid = ffi.call(getpid, [])
print("PID: " + str(pid))

// sleep(seconds) → int (seconds remaining)
let sleep = ffi.symbol(libc, "sleep")
let remaining = ffi.call(sleep, [1])
print("slept, remaining: " + str(remaining))
```

## 41.3 40.3 Working with strings

C strings are null-terminated. `ffi.str(ptr)` reads until the first NUL:

```
import ffi

let libc = ffi.load("libc.so.6")

// gethostname(buf, len) → int
let gethostname = ffi.symbol(libc, "gethostname")
let buf = ffi.malloc(256)
ffi.call(gethostname, [buf, 256])
let hostname = ffi.str(buf)
print("Hostname: " + hostname)
ffi.free(buf)
```

## 41.4 40.4 Working with buffers

```
import ffi

let libc = ffi.load("libc.so.6")
let memcpy = ffi.symbol(libc, "memcpy")

let src = ffi.malloc(16)
ffi.write(src, "HelloWorld")

let dst = ffi.malloc(16)
ffi.call(memcpy, [dst, src, 11])
let copied = ffi.read(dst, 11)
print(copied)  // HelloWorld

ffi.free(src)
ffi.free(dst)
```

## 41.5 40.5 Error handling

```
import ffi

// ffi.load raises on failure
var lib = nil
try {
    lib = ffi.load("/nonexistent/lib.so")
} catch as e {
    print("load failed: " + e.msg)
    lib = nil
}
```

```
// ffi.symbol raises if symbol not found
if lib != nil {
    try {
        let addr = ffi.symbol(lib, "nonexistent_func")
    } catch as e {
        print("symbol not found: " + e.msg)
    }
}
```

## 41.6 40.6 Calling conventions and types

Zen's FFI uses the platform's default C calling convention (cdecl on x86, System V AMD64 ABI on x86_64). Supported argument types:

- Zen `int` → C `int/long`
- Zen `float` → C `double`
- Zen `string` → C `char*` (null-terminated)
- Typed scalars: `{"type": "double", "value": x}`
- Raw buffers: `{"buf": "hex..."}`

Return types: `int` (default), `double`, `void`, `str`, bytes.

> Zen FFI does **not** support Zen closures as C callbacks. If a C function needs a callback, you must pass a native function pointer obtained from `ffi.symbol`.

## 41.7 40.7 Complete example: libc time

```
import ffi

ffi.allow(["/usr/lib/", "/lib/"])
let libc = ffi.load("libc.so.6")

// time(NULL) → time_t (int)
let time_fn = ffi.symbol(libc, "time")
let now = ffi.call(time_fn, [nil])
print("Unix time: " + str(now))

// getpid() → int
let getpid = ffi.symbol(libc, "getpid")
let pid = ffi.call(getpid, [])
print("Process ID: " + str(pid))
```

## 41.8 40.8 Quick reference

```
import ffi
```

```
// Security
ffi.allow(["/usr/lib/"])
ffi.deny(["/tmp/"])

// Load + symbol
let lib = ffi.load("libc.so.6")
let addr = ffi.symbol(lib, "function_name")

// Call
let r = ffi.call(addr, [arg1, arg2], "int")
let r = ffi.call(addr, [4.5], "double")

// Memory
let ptr = ffi.malloc(1024)
ffi.write(ptr, "data")
let s = ffi.read(ptr, 10)
let str = ffi.str(ptr)
ffi.free(ptr)

// Types
ffi.sizeof("int")
ffi.alignof("double")
ffi.errno()
ffi.strerror(n)
```

## 41.9 40.9 Exercises

1. Load `libc`, call `getpid`, and print the current process ID.
2. Call `time(NULL)` and print the Unix timestamp.
3. Use `ffi.malloc`, `ffi.write`, `ffi.read`, `ffi.free` to copy a string.
4. Call `gethostname` and print the hostname.
5. Set `ffi.deny(["/tmp/"])` and confirm loading from `/tmp` fails.

## 41.10 Advanced Topics and Exercises

### 41.10.1 Safety considerations

- Memory management across FFI boundary
- Type mismatches
- Calling conventions
- Null pointers
- Resource leaks

### 41.10.2 Debugging FFI

- Check symbol names (nm, objdump)
- Use cdecl/stdcall correctly

- Test with simple functions first

### 41.10.3 Exercises

1. Call simple C function from shared library
2. Pass strings and arrays
3. Handle pointers/structs
4. Manage memory allocation
5. Error handling across boundary
6. Write wrapper module

## 41.11 40.12 Worked examples — FFI in practice

### 41.11.1 Example: Wrapping a small C library

C source (`cryptokey.c`):

```c
#include <stdint.h>
uint32_t hash32(const char *s) {
    uint32_t h = 2166136261u;
    while (*s) { h ^= (unsigned char)*s++; h *= 16777619u; }
    return h;
}
```

Zen wrapper (`crypto_key.z`):

```
import ffi

let lib = ffi.load_library("cryptokey.so")
let hash32_fn = ffi.get_symbol(lib, "hash32")

pub func hash32(s) {
    return ffi.call(hash32_fn, [s])
}

ffi.close(lib)
```

Usage:

```
import crypto_key
print(crypto_key.hash32("hello"))   # deterministic uint32
```

### 41.11.2 Example: Error codes from C

C functions often return -1 or set `errno`. Wrap them:

```
pub func safe_call(fn, args) {
    let r = ffi.call(fn, args)
    if r == -1 {
        let errno = ffi.call(get_errno_fn, [])
```

```
        throw "C error: errno=" + str(errno)
    }
    return r
}
```

### 41.11.3 Example: Struct layout (if supported)

If Zen FFI supports structs, you declare field offsets/sizes and use `ffi.struct_read`/`ffi.struct_writ
Otherwise, pass raw bytes and interpret with `struct.calcsize` / manual unpacking.

```
import struct
let size = struct.calcsize("<ii")   # two little-endian ints = 8 bytes
```

### 41.11.4 Example: Resource lifecycle with try/finally

```
pub func with_resource(acquire_fn, release_fn, use_fn) {
    let h = ffi.call(acquire_fn, [])
    try {
        return use_fn(h)
    } finally {
        ffi.call(release_fn, [h])
    }
}
```

## 41.12 40.13 FFI best practices

1. Wrap raw FFI calls in idiomatic Zen functions; don't expose `ffi.*` directly to users.
2. Validate all inputs before crossing the boundary.
3. Use `try/finally` for every acquired resource.
4. Document which Zen types map to which C types.
5. Keep FFI boundary small; do heavy logic in Zen, not C.
6. Test with valgrind/ASan on the C side for memory errors.

## 41.13 40.14 Exercises

1. Write a Zen module wrapping `libc`'s `puts` to print a string via C.
2. Wrap a C function computing CRC32; compare with a pure-Zen CRC32.
3. Build `with_resource` helper and use it for a DB handle.
4. Implement `errno` checking wrapper for C calls returning -1.
5. Call a C function returning a struct via raw bytes + `struct` module.
6. Create a FFI wrapper for a small compression library (zlib conceptually).
7. Write a FFI stress test: call a C function 100000 times.
8. Demonstrate catching a C error and rethrowing as a Zen error.
9. Document the C type mapping for your wrapper module.
10. Build a tiny "native addon": C + Zen working together.

## 41.14 40.12 Worked example — wrap a C hash function

C (`cryptokey.c`):

```c
uint32_t hash32(const char *s) {
    uint32_t h = 2166136261u;
    while (*s) { h ^= (unsigned char)*s++; h *= 16777619u; }
    return h;
}
```

Zen (`crypto_key.z`):

```
import ffi
let lib = ffi.load_library("cryptokey.so")
let hash32_fn = ffi.get_symbol(lib, "hash32")
pub func hash32(s) { return ffi.call(hash32_fn, [s]) }
ffi.close(lib)
```

Usage:

```
import crypto_key
print(crypto_key.hash32("hello"))   # a uint32
```

## 41.15 40.13 Troubleshooting FFI

| Symptom | Cause | Fix |
|---|---|---|
| symbol not found | wrong lib name / not exported | check `nm -D` |
| crash on call | NULL pointer / bad type | validate args |
| wrong result | calling convention mismatch | match platform |
| leak | handle never freed | release in `finally` |

## 41.16 40.14 Exercises

1. Wrap C `puts` to print via C.
2. Wrap a CRC32 C function; compare to pure-Zen CRC32.
3. Wrap with `try/finally` for resource release.
4. Check `errno` after a C call returning -1.
5. Call a C function 100000 times; benchmark.

# 42 Chapter 41: sys Module

The `sys` module provides low-level system operations: file descriptors, process control, signals, and direct syscalls. It's a thin wrapper over POSIX system calls.

 `sys` is Linux/macOS only. On Windows, `sys_stub` provides limited functionality. Use `fs`, `os`, and `process` for portable operations.

## 42.1 41.1 File descriptor operations

```
import sys

// Open a file
let fd = sys.open("data.txt", sys.O_RDONLY)
print(fd)  // file descriptor integer

// Read
let data = sys.read(fd, 1024)
print(data)

// Write
sys.write(fd, "hello")

// Seek
sys.lseek(fd, 0, sys.SEEK_SET)

// Close
sys.close(fd)
```

### 42.1.1 Open flags

| Flag | Meaning |
|---|---|
| `sys.O_RDONLY` | Read only |
| `sys.O_WRONLY` | Write only |
| `sys.O_RDWR` | Read/write |
| `sys.O_CREAT` | Create if missing |
| `sys.O_APPEND` | Append |
| `sys.O_TRUNC` | Truncate |

### 42.1.2 Seek whence

| Value | Meaning |
|---|---|
| `sys.SEEK_SET` | From start |
| `sys.SEEK_CUR` | From current position |
| `sys.SEEK_END` | From end |

## 42.2 41.2 Process control

```
import sys

// Get process IDs
print(sys.getpid())     // process ID
print(sys.getppid())    // parent process ID
print(sys.getuid())     // user ID
print(sys.geteuid())    // effective user ID
print(sys.getgid())     // group ID
print(sys.getegid())    // effective group ID

// Fork (Unix only)
let pid = sys.fork()
if pid == 0 {
    print("in child")
} else {
    print("in parent, child pid: " + str(pid))
    sys.wait(pid)
}

// Kill a process
sys.kill(pid, 15)  // SIGTERM
```

## 42.3 41.3 Pipes

```
import sys

let [read_fd, write_fd] = sys.pipe()

sys.write(write_fd, "hello through pipe")
let msg = sys.read(read_fd, 1024)
print(msg)  // hello through pipe

sys.close(read_fd)
sys.close(write_fd)
```

## 42.4 41.4 Direct syscalls

```
import sys

// Make a raw syscall by number
let result = sys.syscall(sys.SYS_write, 1, "hello\n", 6)
print(result)
```

Syscall numbers are platform-specific (see `/usr/include/asm/unistd.h` on Linux).

## 42.5 41.5 Signal handling (limited)

Zen's signal support is limited. Use `sys.kill(pid, signal)` to send signals:

| Signal | Number | Meaning |
|---|---|---|
| `sys.SIGTERM` | 15 | Terminate gracefully |
| `sys.SIGKILL` | 9 | Kill immediately |
| `sys.SIGINT` | 2 | Interrupt (Ctrl+C) |
| `sys.SIGHUP` | 1 | Hangup |

## 42.6 41.6 Common patterns

### 42.6.1 Read a file byte by byte

```
import sys

let fd = sys.open("file.txt", sys.O_RDONLY)
let chunks = []
while true {
    let chunk = sys.read(fd, 4096)
    if len(chunk) == 0 {
        break
    }
    chunks.push(chunk)
}
sys.close(fd)
let content = chunks.join("")
print(content)
```

### 42.6.2 Duplicate a file descriptor

```
import sys

let fd = sys.open("file.txt", sys.O_RDONLY)
let fd2 = sys.dup(fd)
// fd2 refers to the same file
sys.close(fd)
sys.close(fd2)
```

### 42.6.3 Create a pipe between processes

```
import sys
import process

let [rfd, wfd] = sys.pipe()
```

```
// Fork and have child write, parent read
let pid = sys.fork()
if pid == 0 {
    sys.close(rfd)
    sys.write(wfd, "from child")
    sys.close(wfd)
} else {
    sys.close(wfd)
    let msg = sys.read(rfd, 1024)
    print("parent got: " + msg)
    sys.close(rfd)
    sys.wait(pid)
}
```

## 42.7 41.7 Quick reference

```
import sys

sys.open(path, flags)          // file descriptor
sys.read(fd, n)                // read bytes
sys.write(fd, data)            // write bytes
sys.close(fd)                  // close
sys.lseek(fd, offset, whence)  // seek
sys.dup(fd)                    // duplicate fd
sys.pipe()                     // [read_fd, write_fd]
sys.getpid()                   // process ID
sys.getppid()                  // parent PID
sys.fork()                     // fork (Unix)
sys.kill(pid, sig)             // send signal
sys.wait(pid)                  // wait for child
sys.syscall(num, args...)      // raw syscall
```

## 42.8 41.8 Common pitfalls

- **sys is Unix-only.** On Windows, many functions are stubs or raise errors.
- **File descriptors are integers.** Track them yourself; Zen won't close them automatically.
- **`sys.read` may return fewer bytes** than requested. Loop to read all.
- **Forked processes share file descriptors.** Close what you don't need in each process.
- **Raw syscalls are dangerous.** Wrong arguments can crash the process.

## 42.9 41.9 Exercises

1. Open a file, read its contents, and print them using `sys`.
2. Create a pipe, write a message in one "thread", read it in another.

3. Use `sys.fork` to create a child process that prints its PID.
4. Use `sys.kill` to send SIGTERM to a child process and `sys.wait` for it.
5. Use `sys.lseek` to read the last 100 bytes of a file.

## 42.10 Advanced Topics and Exercises

### 42.10.1 Safety considerations

- Memory management across FFI boundary
- Type mismatches
- Calling conventions
- Null pointers
- Resource leaks

### 42.10.2 Debugging FFI

- Check symbol names (nm, objdump)
- Use cdecl/stdcall correctly
- Test with simple functions first

### 42.10.3 Exercises

1. Call simple C function from shared library
2. Pass strings and arrays
3. Handle pointers/structs
4. Manage memory allocation
5. Error handling across boundary
6. Write wrapper module

## 42.11 41.12 Worked examples — sys module

Typical `sys` members (exact set depends on build): `argv`, `platform`, `version`, `exit`, `environ`, `path`, `modules`.

### 42.11.1 Example 1: Read command-line arguments

```
import sys
print(sys.argv)       # ["app.z", "arg1", "arg2"]
print(len(sys.argv))  # argument count
```

### 42.11.2 Example 2: Platform detection

```
if sys.platform == "linux" {
    print("running on Linux")
} elif sys.platform == "darwin" {
    print("running on macOS")
} elif sys.platform == "windows" {
```

```
    print("running on Windows")
} else {
    print("unknown platform: " + sys.platform)
}
```

### 42.11.3 Example 3: Environment variables

```
print(sys.environ["PATH"])
let port = sys.environ["PORT"] ?? "8080"
print("port=" + port)
```

### 42.11.4 Example 4: Graceful exit

```
func main() {
    # ... work ...
    sys.exit(0)
}
```

### 42.11.5 Example 5: Inspect loaded modules

```
print(sys.modules)   # list/dict of loaded modules
```

## 42.12 41.13 Platform differences

| Area | Linux | macOS | Windows |
|---|---|---|---|
| Path separator | `/` | `/` | `\` |
| Line ending | `\n` | `\n` | `\r\n` |
| Shared lib ext | `.so` | `.dylib` | `.dll` |
| Executable ext | (none) | (none) | `.exe` |
| Case sensitivity | sensitive | insensitive | insensitive |
| Signals | SIGINT/SIGTERM | similar | Ctrl-C/console handler |

Write portable code: use `pathlib.Path` (joins correctly), avoid hardcoding `/`, test on each target.

## 42.13 41.14 Exercises

1. Print `sys.argv`, `sys.platform`, `sys.version`.
2. Write a script that behaves differently on Linux vs Windows.
3. Read `PORT` env var with a default of 8080.
4. Implement `exit_with_code(code)` calling `sys.exit`.
5. List all loaded modules at startup.
6. Write `get_env(name, default)` using `sys.environ`.

7. Detect if running under a debugger/CI via env vars.
8. Build a small CLI that prints platform info.
9. Compare `sys.path` entries before/after an import.
10. Write a portable temp-file path using `pathlib`.

## 42.14 41.12 Worked example — platform-aware script

```
import sys
import pathlib
import os

print("argv: " + str(sys.argv))
print("platform: " + sys.platform)
print("version: " + sys.version)
print("cwd: " + os.getcwd())
print("HOME: " + str(sys.environ["HOME"] ?? ""))

let p = pathlib.Path("data") / "config.txt"
print("path: " + str(p))
print("exists: " + str(p.exists()))

if sys.platform == "windows" {
    print("running on Windows")
}
```

## 42.15 41.13 Troubleshooting sys

| Symptom | Cause | Fix |
|---|---|---|
| argv empty | no args passed | pass args after script |
| wrong platform | running on unexpected OS | check sys.platform |
| env var missing | not set | use ?? default |

## 42.16 41.14 Exercises

1. Print argv, platform, version.
2. Read `PORT` env var with default 8080.
3. List loaded modules at startup.
4. Build a portable temp path with pathlib.
5. Detect Windows vs Linux behavior.

# 43 Chapter 42: pymport - Python Interoperability

`pymport` imports Python modules into Zen, letting you call Python functions, access Python objects, and use the entire Python ecosystem from Zen. This chapter covers setup, syntax, `zen py-sync`, environment management, limits, examples, and troubleshooting.

## 43.1 42.1 Why `pymport` (and not just `import`)?

Zen's `import` loads Zen modules (`.z` files or native Rust modules). `pymport` loads **CPython** modules — the entire Python ecosystem (PyPI, numpy, pandas, requests, beautifulsoup4, etc.).

```
import json          // Zen's json module (Zen or native Rust)
pymport json         // CPython's json module
```

## 43.2 42.2 Syntax and Binding

`pymport` calls are **positional-only** — no keyword arguments.

```
pymport json
pymport os
pymport requests
pymport bs4            // beautifulsoup4
pymport selenium.webdriver as webdriver
pymport urllib.request
pymport builtins
```

### 43.2.1 Calling functions

```
pymport json
let data = json.loads('{"name": "Zen", "version": 3}')
print(data["name"])     # Zen

pymport os
print(os.getcwd())
print(os.path.join("a", "b"))

pymport datetime
let now = datetime.datetime.now()
print(now.year)
print(now.isoformat())
```

### 43.2.2 Attribute access and method calls

Foreign objects (e.g. `datetime.datetime`) are opaque handles — attributes and methods work:

```
pymport datetime
let now = datetime.datetime.now()
print(now.year)           // attribute access
print(now.isoformat())    // method call
let delta = datetime.timedelta(days=7)
let later = now + delta   // operator overloading (if supported)
```

Scalar results marshal back to Zen values (int, float, string, bool, list, dict).

## 43.3 42.3 Building Zen with Python Support

`pymport` requires Zen built with the Python feature:

```
cargo build --release --features python
```

This requires `libpython` at build and run time. Without it, `pymport` reports:

```
pymport needs a linked Python bridge (this build has none). Rebuild with --
features python
```

## 43.4 42.4 Environment Setup (venv + PYTHONPATH)

You can point CPython to any Python environment via `PYTHONPATH`.

```
cd book-v2/tests
./setup.sh          # builds zen --features python, creates .venv, installs requirement

export PYTHONPATH="$(book-v2/tests/.venv/bin/python -c 'import sysconfig; print(sysconf
export ZEN_NO_PY_BOOTSTRAP=1
./target/release/zen run script.z
```

`ZEN_NO_PY_BOOTSTRAP=1` tells Zen to use that environment as-is instead of creating `.zen_venv`.

### 43.4.1 Create your own venv

```
python3 -m venv .venv
.venv/bin/pip install beautifulsoup4 mechanicalsoup selenium lxml requests
export PYTHONPATH="$(.venv/bin/python -c 'import sysconfig; print(sysconfig.get_paths()
```

### 43.4.2 Check installed packages

```
book-v2/tests/.venv/bin/pip list
```

## 43.5 42.5 Dependency Sync with `zen py-sync` and `requirements.txt`

Zen can scan your `.z` files for `pymport` statements and ensure a venv with dependencies.

### 43.5.1 Create `requirements.txt`

```
beautifulsoup4
mechanicalsoup
selenium
requests
lxml
```

> A module name is not always the pip name (e.g., `bs4` comes from `beautifulsoup4`). The scanner reports what was found; Zen does **not** guess package names.

### 43.5.2 Run sync

```
./target/release/zen py-sync
```

### 43.5.3 Behavior

1. **Scan**: recursively parses entry + transitive `.z` imports to find all `pymport` statements.
2. **Plan**: determines state (`Clean`, `NoRequirements`, `MissingVenv`, `MissingPackages`).
3. **Ensure**: creates `.zen_venv` in project root, runs `pip install -r requirements.txt` in the foreground, writes a stamp (requirements length + hash).
4. **Point**: `zen run` prepends `.zen_venv/lib/python*/site-packages` to `PYTHONPATH`.

### 43.5.4 Control flags

| Flag | Meaning |
|---|---|
| `ZEN_NO_PY_BOOTSTRAP=1` | Skip bootstrap entirely (for tests/hermetic runs). |
| (none) | If `requirements.txt` is missing and `pymport` is used, Zen reports an error. |

### 43.5.5 Example end-to-end

```
cat > requirements.txt << 'EOF'
paramiko
EOF
./target/release/zen py-sync
./target/release/zen run my_script.z  # uses .zen_venv
```

## 43.6 42.6 Using Python from Zen

### 43.6.1 Importing modules

```
pymport json        // stdlib
pymport os          // stdlib
pymport requests    // third-party (if installed)
pymport bs4         // beautifulsoup4
pymport selenium.webdriver as webdriver
pymport urllib.request
pymport builtins
```

### 43.6.2 Calling functions and accessing attributes

Foreign calls are **positional-only** in v1. Use `getattr`-style access via dot notation; `call_attr` fuses resolve+dispatch for methods.

```
pymport json
let data = json.loads('{"a": 1}')
print(data["a"])

pymport os
print(os.getcwd())
print(os.path.join("a", "b"))

pymport datetime
let now = datetime.datetime.now()
print(now.year)
print(now.isoformat())
```

### 43.6.3 Working with Python objects and binary data

Foreign objects are opaque handles:

```
pymport datetime
pymport builtins

let epoch = datetime.datetime.fromtimestamp(0)
print(epoch.year)
print(epoch.isoformat())
print(epoch.date().isoformat())

let later = datetime.datetime(2030, 1, 2, 3, 4, 5)
print(later.strftime("%Y-%m-%d %H:%M:%S"))

print(builtins.len([10, 20, 30]))
print(builtins.sum([10, 20, 30]))
print(builtins.sorted([3, 1, 2]))
```

### 43.6.4 Binary data

Python `bytes` are foreign handles. Write them through Python's own file object:

```
import fs
pymport mechanicalsoup
pymport builtins

let f = builtins.open("out.bin", "wb")
f.write(some_bytes)
f.close()
```

## 43.7 42.7 Error handling

Python exceptions become catchable Zen errors:

```
import json as zenjson
pymport json

let bad = false
try {
    json.loads("{not json}")
} catch as e {
    bad = true
    print("caught json error")
}

if !bad {
    print("expected json.loads to fail")
    exit(1)
}

// Missing modules fail at the pymport statement and are catchable too
let missing = false
try {
    pymport this_module_does_not_exist
} catch as e {
    missing = true
    print("caught import error")
}

if !missing {
    print("expected import to fail")
    exit(1)
}

print("recovered: " + str(zenjson.parse("{\"ok\": true}")["ok"]))
```

## 43.8 42.8 Limits and Non-Goals (v1)

- **No keyword arguments** in `pymport` calls.
- **No passing Zen closures into Python** (rejected by marshal).
- **No per-handle finalizers**: foreign handles remain until bridge teardown.
- **Tuples → lists**, `big int` → `f64` in some cases, non-string dict keys rejected.
- **Functions containing `PyImport` not JIT-lowered** (stay interpreted).
- **No `zen.toml [py-deps]`** yet (use `requirements.txt`).
- **Network installs** only via `zen py-sync`/preamble; tests avoid silent installs.
- **Handles are opaque** across isolates in general.

## 43.9 42.9 Working Examples (tested)

All scripts live in `book-v2/tests/web-automation/`. Five smaller, hermetic pymport tests live in `book-v2/tests/pymport/`.

### 43.9.1 Example A: `bs4` + `requests` scrape + download

See `tests/web-automation/bs4_tiktok.z` — downloads a TikTok embed MP4 via `urllib.request.urlretrieve`, parsed with BeautifulSoup from `__FRONTITY_CONNECT_STATE__`.

### 43.9.2 Example B: `mechanicalsoup` stateful scrape + download

See `tests/web-automation/ms_tiktok.z`. Uses `StatefulBrowser`, accesses `browser.page`, reuses `browser.session.get()` for CDN, writes bytes via `builtins.open(...,"wb").write()`.

### 43.9.3 Example C: Selenium automation + download

See `tests/web-automation/selenium_tiktok.z`. Launches headless Chromium, polls for `__UNIVERSAL_DATA_FOR_REHYDRATION__`, extracts stats, fetches the embed route with plain HTTP for an unsigned MP4 URL, downloads via `urllib.request.urlretrieve`.

### 43.9.4 Example D: CDP over raw TCP/WebSocket (no Selenium)

See `tests/web-automation/cdp_tiktok.z`. Launches Chromium with `pymport subprocess`, discovers targets via HTTP `/json/list`, upgrades to WebSocket, sends masked CDP frames via `sys.write_bytes(socket.fileno(sock), hex)`, parses unmasked server frames, runs `Page.navigate` + `Runtime.evaluate` to extract TikTok metadata. Cleans up with `browser.terminate()` in a `finally` block.

Run all of them:

```
cd book-v2/tests
./run-all.sh --network
```

## 43.10 42.10 Troubleshooting

| Issue | Fix |
|---|---|
| `Z0414` / “unknown native” / needs `--features python` | Rebuild with `cargo build --release --features python` and ensure libpython is available. |
| `ModuleNotFoundError` for third-party package | Install into venv, set `PYTHONPATH`, or run `zen py-sync` with `requirements.txt`. |
| Selenium can’t find chromedriver | Install `chromedriver` matching Chromium or put it on `PATH`. Use `--no-sandbox` in containers. |
| TikTok WAF blocks plain requests | Use `/embed/v2/<id>`, or Selenium/real browser. See Examples A/C. |
| Binary download corrupted | Don’t rely on string-based `fs.write_binary`; use `urllib.request.urlretrieve` or `builtins.open("wb").write()`. |
| WebSocket hang in raw sockets | Read until `Content-Length` for HTTP, handle EAGAIN on keep-alive, mask client frames, parse length (126/127). |
| `pymport` call with kwargs fails | Use positional args only in v1. |
| Bootstrap skipped unexpectedly | Check `ZEN_NO_PY_BOOTSTRAP` or that `requirements.txt` exists near entry. |

## 43.11 42.11 References

- Docs: `docs/20-pymport.md` (design/spec)
- VM bridge: `crates/zen-vm/src/pybridge.rs`, `crates/zen-vm/src/pyembed.rs`
- CLI bootstrap: `zen-cli/src/pydeps.rs`
- Tests: `crates/zen-cli/tests/pymport_e2e.rs`, `crates/zen-vm/tests/pyembed.rs`, `crates/zen-jit/tests/jit_pymport.rs`
- Scripts: `book-v2/tests/web-automation/*.z`, `book-v2/tests/pymport/*.z`

## 43.12 42.12 Quick reference

```
pymport json
pymport os
pymport requests
pymport bs4
pymport selenium.webdriver as webdriver
pymport urllib.request
pymport builtins

// Positional calls only
```

```
json.loads('{"a": 1}')
os.getcwd()
os.path.join("a", "b")
datetime.datetime.now()
builtins.len([1, 2, 3])

// Error handling
try {
    pymport nonexistent_module
} catch as e {
    print("caught import error")
}

// zen py-sync setup
// ./target/release/zen py-sync
// ./target/release/zen run script.z
```

## 43.13 42.13 Exercises

1. Write a script that imports `json`, `os`, and `datetime` via `pymport` and prints the current date.
2. Use `pymport bs4` to parse an HTML file and extract all links.
3. Use `pymport selenium.webdriver` to open a headless browser and print the page title.
4. Write `py_len(x)` that calls `builtins.len(x)` on any value.
5. Demonstrate that `pymport` of a missing module is catchable with `try`/`catch`.

## 43.14 Advanced Topics and Exercises

### 43.14.1 Safety considerations

- Memory management across FFI boundary
- Type mismatches
- Calling conventions
- Null pointers
- Resource leaks

### 43.14.2 Debugging FFI

- Check symbol names (nm, objdump)
- Use cdecl/stdcall correctly
- Test with simple functions first

### 43.14.3 Exercises

1. Call simple C function from shared library
2. Pass strings and arrays

3. Handle pointers/structs
4. Manage memory allocation
5. Error handling across boundary
6. Write wrapper module

## 43.15 42.12 Worked examples — pymport

`pymport` lets Zen import Python modules (stdlib, venv, project-local `.py`) and call Python functions, with value marshalling and chained tracebacks.

Setup:

```
# activate a venv first
source .venv/bin/activate
# or set PYTHONPATH
export PYTHONPATH="$PWD/pylibs"
```

### 43.15.1 Example 1: Import Python's math module

```
import pymport
let pymath = pymport.import("math")
print(pymath.sqrt(16))   # 4.0
print(pymath.pi)         # 3.14159...
```

### 43.15.2 Example 2: Call a Python function with Zen args

```
let pymath = pymport.import("math")
print(pymath.pow(2, 10))   # 1024.0
print(pymath.floor(3.7))   # 3.0
```

### 43.15.3 Example 3: Import a local Python module

`pylibs/hello.py`:

```
def greet(name):
    return f"Hello, {name} from Python"
```

Zen:

```
let hello = pymport.import("hello")
print(hello.greet("Zen"))   # Hello, Zen from Python
```

### 43.15.4 Example 4: Marshalling values

Zen ↔ Python conversion (typical): - Zen int → Python int - Zen float → Python float - Zen string → Python str - Zen bool → Python bool - Zen nil → Python None - Zen list → Python list - Zen dict → Python dict - Zen bytes → Python bytes

### 43.15.5 Example 5: Error chaining

When Python raises, Zen surfaces it with a chained traceback showing both Zen and Python frames:

```
Z0508: PythonError: ZeroDivisionError: division by zero
  at app.z:10
  caused by Python: ZeroDivisionError at hello.py:3
```

Catch it in Zen:

```
try {
    let bad = pymport.import("bad_module")
    bad.divide_by_zero()
} catch as e {
    print("Python error: " + e.msg)
}
```

### 43.15.6 Example 6: Passing a Zen callback to Python

Some Python APIs accept callbacks. Zen can pass a function; pymport wraps it so Python can call it back.

```
let funcs = pymport.import("my_python_module")
funcs.register_callback(fn(x) => x * 2)
```

**Caution:** the Zen closure must outlive the Python-side registration, and Python exceptions raised inside the callback propagate back to Zen.

## 43.16 42.13 pymport best practices

1. Activate the correct venv before importing Python modules.
2. Set `PYTHONPATH` for project-local `.py` files.
3. Only pass simple types across the boundary; complex objects need explicit serialization (JSON).
4. Catch Python errors at the Zen edge; don't let them propagate uncaught.
5. Don't hold the Python GIL across long Zen operations.
6. Document which Python modules your project depends on.

## 43.17 42.14 Exercises

1. Import Python `math` and compute `sqrt(2)`, `pow(2, 10)`.
2. Import Python `json` and `parse`/`dumps` a value from Zen.
3. Create `pylibs/greeter.py` with a function; call it from Zen.
4. Pass a Zen list to Python and print its length from Python.
5. Demonstrate error chaining: call a Python function that raises, catch in Zen.
6. Read a Python env var (`os.environ`) via pymport.
7. Import `datetime` from Python and format the current time.
8. Compare `pymport.import("math")` vs Zen `math` module.

9. Write a Zen function that delegates a heavy computation to Python/NumPy (conceptually).
10. Build a small hybrid app: Zen CLI + Python computation module.

# 44 Chapter 50: Zen VM and Runtime Internals

*How Zen 3.0 actually executes your code: the interpreter, the Cranelift JIT, garbage collection, memory layout, and the FFI boundary.*

This chapter is for readers who want to look under the hood. It documents the runtime architecture, value representation, execution models, and the interaction between interpreted and JIT-compiled code.

## 44.1 15.1 Runtime architecture overview

Zen 3.0 has a multi-tier execution pipeline:




- **zen-syntax** — lexer + parser; produces the AST.
- **zen-vm** — the virtual machine, bytecode interpreter, ops, builtins, FFI.
- **zen-interp** — the tree-walking / bytecode interpreter used by `zen run`.
- **zen-jit** — the Cranelift-based JIT compiler with inline caches, OSR, and deoptimization.
- **zen-cli** — the `zen` command-line front end.

A source file is lexed, parsed into an AST, lowered to bytecode, and then either interpreted directly or JIT-compiled to native code (Cranelift) for hot functions.

## 44.2 15.2 Value representation

Every Zen value at runtime is a tagged `Value`. The tag distinguishes the type and the payload holds the data (or a pointer to heap data).

### 44.2.1 Primitive (immediate) values

| Tag | Kind | Storage |
|---|---|---|
| nil | `nil`/`null` | none |
| bool | boolean | 1 bit |
| int | signed integer | i64 (i48 guaranteed range for literals) |
| float | IEEE-754 double | f64 |
| string | pointer to heap string | `String` handle |

Immediate values (nil, bool, int, float) are copied by value. Strings and all containers are heap-allocated handles (reference-counted / GC-managed).

### 44.2.2 Heap (container) values

| Kind | Zen type | Heap structure |
|---|---|---|
| list | `list` | growable array of `Value` |
| dict | `dict` | hash map string → `Value` |
| set | `set` | hash set of `Value` |
| tuple | `tuple` | fixed array of `Value` |
| bytes | `bytes` | immutable byte buffer |
| instance | class instance | field map + class pointer |
| func | function | closure + environment |
| module | module | member table |
| foreign | FFI handle | opaque pointer + finalizer |

### 44.2.3 `shared` values

`shared x = expr` wraps the value in an `Arc<Mutex<_>>`-like container so it can be safely read and written from multiple threads. Every access acquires the internal lock.

## 44.3 15.3 Memory model

Zen is garbage collected (the exact collector is build-configured). Heap objects are freed when unreachable.

### 44.3.1 Reference cycles

Cycles (e.g., object A references B and B references A) can leak unless the collector handles cycles or you break them explicitly.

```
class Node {
    var peer = nil
}
let a = new Node()
let b = new Node()
a.peer = b
b.peer = a   # cycle: a <-> b
```

To break: `a.peer = nil` when tearing down.

### 44.3.2 `let` vs `var` vs `const`

- `let` — the binding cannot be reassigned, but the referenced heap object can still be mutated (e.g., `lst.push` on a `let` list).
- `var` — the binding can be reassigned.
- `const` — compile-time/module-level constant.

```
let lst = [1, 2]
lst.push(3)     # OK: mutating the object
# lst = [4]     # error: cannot rebind a let
```

## 44.4 15.4 Execution models

### 44.4.1 Interpreter (tree-walk / bytecode)

`zen-interp` evaluates the AST or bytecode directly. Each operation dispatches on the operator and operand types at runtime. This is the default for `zen run` in many builds and is easiest to debug.

### 44.4.2 Cranelift JIT

`zen-jit` compiles hot functions to native machine code using the Cranelift code generator (the same backend used by Wasmtime).

Key JIT features: - **Monomorphic inline caches** — cache the type of a call site so repeated calls with the same types skip type dispatch. - **OSR (on-stack replacement)** — replace a running interpreted frame with a JIT-compiled frame mid-execution (and back again on deopt). - **Deoptimization** — if a guarded assumption (e.g., a type) is violated, fall back to the interpreter safely.

### 44.4.3 When is JIT used?

Hot loops and frequently called functions are JIT-compiled after a threshold of invocations. Cold code stays interpreted. You typically don't control this directly; `ZEN_DEBUG=1` may show JIT decisions in some builds.

## 44.5 15.5 Bytecode (conceptual)

Zen source is lowered to a stack/register-based bytecode. Each instruction has an opcode and operands. Common opcode categories:

- **Load**: load constant, local, global, module member
- **Store**: store to local/global
- **Call**: invoke a function (native, closure, builtin, method)
- **Return**: return a value
- **Branch**: jump, conditional jump
- **Compare**: `==`, `<`, `>`, `is`, `in`, etc.
- **Math**: `+`, `-`, `*`, `/`, `%`, `**`, bitwise
- **Container**: list/dict/set/tuple construction, index, slice
- **Exception**: throw, catch, finally setup

Example (illustrative, not exact opcode names):

```
000 LOAD_CONST 42
002 STORE_LOCAL x
004 LOAD_LOCAL x
006 LOAD_CONST 1
008 ADD
010 RETURN
```

The bytecode is what the interpreter executes and what the JIT compiles.

## 44.6 15.6 The call stack and frames

Each function call creates a **frame** containing: - the instruction pointer / program counter - local variables - the value stack (operand stack) - a reference to the enclosing scope (for closures) - the function being executed

Recursion and deep call chains grow the native stack; very deep recursion can overflow it (Z0310 `MaxRecursionDepth`, Z0317 `StackOverflow`).

## 44.7 15.7 Closures and upvalues

A closure captures variables from its enclosing scope. Captured variables become **upvalues** — heap cells shared between the closure and the scope that created it. Mutating an upvalue from inside the closure is visible outside, and vice versa.

```
func make_counter() {
    var n = 0     # n becomes an upvalue captured by the closure
    return fn() { n += 1; return n }
}
```

## 44.8 15.8 Method dispatch

Method calls (`obj.method(args)`) resolve the method through the object's class chain: 1. Look for the method on the instance's class. 2. If not found, look on parent classes (inheritance). 3. Bind `self` to the instance and invoke.

If the instance has a field holding a callable with the same name, the field may shadow the method depending on build.

## 44.9 15.9 FFI boundary (Zen ↔ C)

Zen can call C functions in shared libraries and pass data across the boundary.

### 44.9.1 Marshalling (typical)

| Zen type | C side | Direction |
|---|---|---|
| int | `int32_t/int64_t` | by value |
| float | `double` | by value |
| string | `const char*` | NUL-terminated; Zen owns lifetime |
| bytes | `unsigned char*` | raw buffer |
| nil | `NULL` | |
| handle | `void*` | opaque pointer |

### 44.9.2 Safety rules

1. Know who owns each allocation (Zen or C) and who frees it.
2. Never dereference a NULL handle.
3. Match calling conventions (cdecl/stdcall per platform).

4. Don’t let a Zen exception unwind through C frames — catch at the Zen edge.
5. Guard cross-thread FFI calls with a Mutex if the C library isn’t thread-safe.

### 44.9.3 Debugging FFI

- `nm -D libfoo.so` / `objdump -T libfoo.so` list exported symbols.
- Start with a trivial function (add two ints) before structs/pointers.
- Log every FFI call and return.

## 44.10 15.10 pymport boundary (Zen ↔ Python)

`pymport` lets Zen import Python modules and call Python functions. Values are marshalled both directions:

| Zen | Python | Zen | Python |
|---|---|---|---|
| int | int | list | list |
| float | float | dict | dict |
| string | str | nil | None |
| bool | bool | bytes | bytes |

Python exceptions are caught and re-surfaced in Zen as **chained tracebacks** (showing both Zen and Python frames), error code Z0508 `PythonError`.

## 44.11 15.11 Garbage collection tuning (conceptual)

If your build exposes GC controls, common knobs are: - heap growth factor - collection threshold (bytes allocated since last collection) - cycle collector enable/disable - write barrier on/off

Most users never touch these; the defaults are chosen for general workloads.

## 44.12 15.12 Debugging the runtime

```
ZEN_DEBUG=1 zen run app.z          # verbose runtime logging (if supported)
zen explain Z0310                  # explain MaxRecursionDepth
zen explain Z0317                  # explain StackOverflow
```

## 44.13 15.13 Exercises

1. Draw the execution pipeline from `.z` source to native code.
2. Explain why `lst.push(3)` works on a `let` binding but `lst = [4]` does not.
3. Sketch a reference cycle and show how to break it.
4. Describe what an inline cache stores and why it speeds up repeated calls.
5. Explain OSR: when does the interpreter switch to JIT code, and back?
6. What information does a stack frame hold?
7. How does a closure capture a variable (upvalue)?

8. List the FFI marshalling rules for string, int, float, handle.
9. What happens when Python raises inside a pymport call?
10. Why might deep recursion overflow the stack, and how do you avoid it?

''' print('vm internals done')

# 45 Chapter 51: Zen Type System In Depth

*A detailed look at Zen 3.0's types, type tags, coercion, equality, identity, and how types flow through the runtime.*

## 45.1 16.1 The type taxonomy

Zen 3.0 is dynamically typed: types are associated with values, not variables. A variable can hold any type at any time (unless declared `const`).

### 45.1.1 Primitive (immediate) types

| Type | Tag | Literal example | Notes |
|---|---|---|---|
| `nil` | nil | `nil`, `null` | absence of value; falsy |
| `bool` | boolean | `true`, `false` | logical values |
| `int` | integer | `42`, `0x2A`, `1_000` | arbitrary-precision in runtime; literals ≤ i48 |
| `float` | floating-point | `3.14`, `1e-10` | IEEE-754 double |
| `string` | text | `"hi"`, `'hi'`, `"""..."""` | immutable, UTF-8 |

### 45.1.2 Container (heap) types

| Type | Tag | Literal example | Notes |
|---|---|---|---|
| `list` | list | `[1, 2, 3]` | ordered, mutable, heterogeneous |
| `dict` | dict | `#{a: 1}` | string keys → any value |
| `set` | set | `%{1, 2}` | unordered unique values |
| `tuple` | tuple | `#(1, "x")` | fixed-size, immutable, heterogeneous |
| `bytes` | bytes | `bytes([1,2])` | immutable byte sequence |

### 45.1.3 Other runtime types

| Type | Example | Notes |
|---|---|---|
| `func` | `fn(x) => x` | function value / closure |
| `class` | `class C {}` | class object |
| `instance` | `new C()` | object instance |
| `module` | `import math` | loaded module |
| `foreign` | FFI handle | opaque pointer to C resource |

`typeof(x)` returns the type name as a string; `type(x)` is an alias.

## 45.2 16.2 Type tags and immediate vs heap

Every runtime `Value` carries a tag. Immediate values (nil, bool, int, float) fit inside the `Value` itself and are copied on assignment. Heap values (string, list, dict, set, tuple, bytes, instance, func, module, foreign) are pointers to heap-allocated objects and are copied by reference.

Consequence: two list literals `[1,2]` and `[1,2]` are `==` (equal by value) but not `is` (different heap objects).

```
let a = [1, 2]
let b = [1, 2]
print(a == b)   # true
print(a is b)   # false
```

## 45.3 16.3 Type introspection

| Operation | Purpose | Example |
|---|---|---|
| `typeof(x)` | type name string | `typeof(42)` → `"int"` |
| `type(x)` | alias for typeof | `type("x")` → `"string"` |
| `x is T` | identity / instance check | `x is list` |
| `x is not T` | negation | `x is not nil` |
| `typeof(x) == "int"` | string comparison | works but slower than `is` |

Prefer `is` for type checks when supported; fall back to `typeof() == "..."`.

## 45.4 16.4 Type coercion

### 45.4.1 Explicit conversion (builtins)

```
int("42")       # 42
float(3)        # 3.0
str(42)         # "42"
bool(0)         # true   (0 is truthy!)
list("ab")      # ["a","b"]
tuple([1,2])    # #(1,2)
set([1,1,2])    # {1,2}
dict([["a",1]]) # {"a":1}
```

String → number parsing failures raise `ValueError` (Z0305).

### 45.4.2 Implicit coercion in operators

| Context | Rule |
|---|---|
| + with a string | the other operand is stringified via `str()` |
| * string × int | string repetition |
| comparison == | numeric promotion: int and float compare numerically |
| truthiness (`if`, `and`, `or`) | only `false`/`nil`/`null` are falsy |

```
print("age: " + 25)     # "age: 25"
print("hi" * 3)         # "hihihi"
print(1 == 1.0)         # true
print(1 === 1.0)        # false (strict: types differ)
```

## 45.5 16.5 Equality: ==, ===, is

| Operator | Semantics | `1 == 1.0` | `[1] == [1]` | `[1] is [1]` |
|---|---|---|---|---|
| == | value equality with promotion | true | true | — |
| === | strict equality (type + value) | false | — | — |
| is | identity (same object) | true | false | false |

Container equality: - lists/tuples: order-sensitive, element-wise - dicts: order-insensitive, key/value match - sets: order-insensitive, element match

## 45.6 16.6 Numeric details

### 45.6.1 Integers

Runtime integers are arbitrary precision (big ints), so `10 ** 100` works without overflow. Integer *literals* are limited to i48 range at parse time (Z0106 `NumericLiteralOverflow` for larger literals); build large values at runtime instead.

```
let big = 10 ** 100     # fine (runtime)
# let x = 99999999999999999999   # Z0106 literal overflow
```

### 45.6.2 Floats

IEEE-754 double precision. Special values: - `inf` = `1.0 / 0.0` - `-inf` = `-1.0 / 0.0` - `nan` = `0.0 / 0.0`

IEEE rules: - `nan == nan` is `false` - `nan != nan` is `true` - comparisons with `nan` are `false` (except `!=`) - `inf > 1e308` is `true`

```
print(nan == nan)   # false
print(math.isnan(nan))   # true
print(math.isinf(inf))   # true
```

### 45.6.3 Division

- `/` true division → float (`4 / 2` is `2.0`)
- `//` floor division → int (`4 // 2` is `2`, `-7 // 2` is `-4`)
- `%` modulo (sign follows dividend in Zen)

```
print(4 / 2)    # 2.0
print(4 // 2)   # 2
print(-7 // 2)  # -4
print(-7 % 2)   # -1
```

## 45.7 16.7 String type details

Strings are immutable sequences of Unicode code points. Indexing and slicing operate on *characters*, not bytes.

```
let s = "Zën"
print(len(s))   # 3 (code points, not bytes)
print(s[1])     # "ë"
```

Convert to/from bytes explicitly:

```
let b = s.encode("utf-8")
let s2 = b.decode("utf-8")
```

## 45.8 16.8 Container type details

### 45.8.1 List

Ordered, mutable, heterogeneous. Supports indexing, slicing (copies), push, pop, insert, remove.

### 45.8.2 Dict

Unordered mapping string → value. Keys must be strings. Order-insensitive equality. Iteration yields keys; use `.items()` for pairs.

### 45.8.3 Set

Unordered collection of unique values. Order-insensitive equality.

### 45.8.4 Tuple

Fixed-size, immutable, heterogeneous. Good for fixed records and multiple return values.

## 45.9 16.9 Type stability and performance

Dynamic typing means every operation pays a type-dispatch cost. The JIT mitigates this with inline caches for hot call sites. For performance: - Keep types stable at a call

site (don’t alternate int/string in the same variable). - Avoid `typeof()` checks in hot loops. - Prefer homogeneous containers (a list of all ints) when possible.

## 45.10 16.10 Common type mistakes

1. **Assuming `0` is falsy.** It’s truthy in Zen. Use `x == nil` or `x is nil`.
2. **Confusing `==` and `is`.** Use == for value, `is` for identity.
3. **Comparing `nan` with `==`.** Always false. Use `math.isnan`.
4. **Dict keys must be strings.** `d[42]` raises TypeError; use `d["42"]`.
5. **Integer literals beyond i48.** Cause Z0106; compute at runtime.
6. **Mutable default arguments.** Use `nil` sentinel.
7. **Assuming containers copy on assignment.** `b = a` shares the object.

## 45.11 16.11 Exercises

1. Write `type_name(x)` returning a human-readable string for any value.
2. Demonstrate `nan != nan` and how to detect NaN (`x != x`).
3. Show `1 == 1.0` but `1 === 1.0` is false.
4. Prove two equal lists are not `is`-identical.
5. Write `safe_int(s)` returning nil on parse failure.
6. Demonstrate `-7 // 2` is `-4` (floor, not truncation).
7. Write `is_power_of_two(n)` using bit tricks.
8. Show `typeof` on each of: int, float, string, bool, nil, list, dict, set, tuple, func.
9. Implement `deep_equals(a, b)` handling nested containers.
10. Explain why `bool([])` is `true` in Zen.

’’’ print(‘type system done’)

# 46 Chapter 52: Zen Bytecode Reference

*A reference for the bytecode instructions Zen 3.0 source is lowered to, and how the interpreter and JIT execute them.*

This chapter is reference material. Most Zen programmers never write bytecode by hand, but understanding it helps with debugging, performance tuning, and understanding what `zen check` / `zen build` produce.

## 46.1 17.1 What bytecode is

After the lexer and parser turn source into an AST, the compiler lowers the AST to a linear sequence of bytecode instructions. Each instruction has an opcode and zero or more operands (register indices, constant pool indexes, jump targets, etc.).

The interpreter (`zen-interp`) executes these instructions directly with a value stack. The JIT (`zen-jit`) compiles frequently executed bytecode sequences to native machine code via Cranelift.

## 46.2 17.2 Instruction format (conceptual)

```
<offset> <opcode> <operand1> <operand2> ...
```

Operands may reference: - the constant pool (strings, numbers) - local variables (by index) - the value stack (implicit) - jump targets (by offset)

Example (illustrative opcode names; exact mnemonics are build-defined):

```
000 LOAD_CONST     5        # push constant 42
002 STORE_LOCAL    0         # pop into local x
004 LOAD_LOCAL     0         # push x
006 LOAD_CONST     6         # push constant 1
008 ADD                     # pop two, push sum
010 RETURN                    # return top of stack
```

## 46.3 17.3 Opcode categories

### 46.3.1 Load instructions

| Purpose | Example (conceptual) |
|---|---|
| load a constant | `LOAD_CONST <pool_index>` |
| load a local variable | `LOAD_LOCAL <index>` |
| load a global/module variable | `LOAD_GLOBAL <name>` |
| load a module member | `LOAD_MEMBER <module> <name>` |
| load `nil` / `true` / `false` | `LOAD_NIL`, `LOAD_TRUE`, `LOAD_FALSE` |

### 46.3.2 Store instructions

| Purpose | Example |
|---|---|
| store to local | `STORE_LOCAL <index>` |
| store to global | `STORE_GLOBAL <name>` |
| store object field | `STORE_FIELD <name>` |
| store list element | `STORE_INDEX` |
| store dict entry | `STORE_DICT` |

### 46.3.3 Call and return

| Purpose | Example |
|---|---|
| call a function | `CALL <arg_count>` |
| call a method | `CALL_METHOD <name> <arg_count>` |
| call a builtin | `CALL_BUILTIN <builtin_id>` |
| return a value | `RETURN` |
| return nil | `RETURN_NIL` |

### 46.3.4 Branching

| Purpose | Example |
|---|---|
| unconditional jump | `JUMP <target>` |
| jump if truthy | `JUMP_IF_TRUE <target>` |
| jump if falsy | `JUMP_IF_FALSE <target>` |
| jump if nil | `JUMP_IF_NIL <target>` |

### 46.3.5 Comparison

| Purpose | Example |
|---|---|
| equal | `EQ` (pops 2, pushes bool) |
| not equal | `NE` |
| strict equal | `SEQ` |
| less / greater / le / ge | `LT`, `GT`, `LE`, `GE` |
| identity | `IS` |
| membership | `IN` |

### 46.3.6 Arithmetic

| Purpose | Example |
|---|---|
| add | `ADD` |
| subtract | `SUB` |
| multiply | `MUL` |
| divide | `DIV` |

| Purpose | Example |
|---|---|
| floor divide | `FLOORDIV` |
| modulo | `MOD` |
| power | `POW` |
| negate | `NEG` |
| bitwise and/or/xor/not | `BAND`, `BOR`, `BXOR`, `BNOT` |
| shift left/right | `LSHIFT`, `RSHIFT` |

### 46.3.7 Container operations

| Purpose | Example |
|---|---|
| build list | `BUILD_LIST <count>` |
| build dict | `BUILD_DICT <count>` |
| build set | `BUILD_SET <count>` |
| build tuple | `BUILD_TUPLE <count>` |
| list append | `LIST_APPEND` |
| dict set | `DICT_SET` |
| subscript (index) | `BINARY_SUBSCR` |
| slice | `SLICE` |

### 46.3.8 Exception handling

| Purpose | Example |
|---|---|
| begin try | `SETUP_FINALLY / SETUP_EXCEPT <handler>` |
| end finally | `END_FINALLY` |
| throw | `RAISE` |
| re-raise | `RERAISE` |

## 46.4 17.4 Stack effect

Most instructions follow a stack discipline: they pop operands off the value stack and push results. For example:

- `ADD`: pops `b`, pops `a`, pushes `a + b`.
- `LOAD_CONST c`: pushes `c`.
- `STORE_LOCAL i`: pops value, stores in local `i`.
- `CALL n`: pops `n` arguments + the callable, pushes the return value.

The interpreter tracks stack depth to catch underflow/overflow bugs.

## 46.5 17.5 Constant pool

Strings, integers, floats, and other literals referenced by bytecode are stored in a per-function constant pool. `LOAD_CONST <index>` fetches entry `<index>`. This avoids emitting the full literal on every use.

## 46.6 17.6 How the interpreter executes

1. Fetch the instruction at the current program counter.
2. Decode opcode and operands.
3. Execute (manipulate the value stack, locals, globals).
4. Advance the PC (or jump on branch instructions).
5. Repeat until `RETURN` or an error.

Errors (Z03xx) are raised by individual ops when types are wrong, indexes are out of range, etc.

## 46.7 17.7 How the JIT compiles

For hot functions, `zen-jit`: 1. Reads the bytecode. 2. Emits Cranelift IR for each instruction, mapping stack ops to virtual registers. 3. Inserts **inline cache guards** at call sites (check the cached type; if it matches, use the fast path; otherwise bail to a slow path / deopt). 4. Compiles to native machine code. 5. On type-guard failure, **deoptimizes**: reconstruct interpreter state and continue in the interpreter.

## 46.8 17.8 Inline caches (conceptual)

A call site `obj.method()` that's executed many times with `obj` of the same type caches: - the resolved method - the type guard

On the next call, if the type still matches, it jumps directly to the cached method. This avoids repeated method-lookup work.

## 46.9 17.9 Reading bytecode (if your build exposes it)

Some builds let you dump bytecode for inspection:

```
# conceptual; exact flag depends on build
zen check --dump-bytecode app.z
```

This is useful for verifying that a hot loop compiled as expected and that no surprise type guards are present.

## 46.10 17.10 Exercises

1. Sketch the bytecode for `let x = 1 + 2; return x`.
2. What does `ADD` pop and push?
3. Why does the constant pool exist?
4. Describe how a branch instruction changes the PC.
5. Explain what an inline cache guards on.
6. What happens on deoptimization?
7. Contrast the interpreter loop with JIT-compiled native code.
8. Name three container bytecode ops.
9. Name three exception bytecode ops.
10. Why might `zen check --dump-bytecode` help optimize a hot loop?

```
''' print('bytecode done')
```

# 47 Chapter 53: Garbage Collection and Memory Management

*How Zen 3.0 reclaims memory, the collector design, reference cycles, and how to write memory-friendly code.*

## 47.1 18.1 Overview

Zen is a garbage-collected language. You do not `free` memory manually. The runtime automatically reclaims heap objects (lists, dicts, strings, instances, etc.) when they become unreachable.

The exact collector implementation is build-configured, but the semantics are consistent: unreachable memory is reclaimed; reachable memory is kept.

## 47.2 18.2 What gets collected

Heap-allocated objects: - strings, lists, dicts, sets, tuples, bytes - class instances - closures / function objects - modules - FFI foreign handles (with finalizers)

Immediate values (nil, bool, int, float) are not heap-allocated and need no collection.

## 47.3 18.3 Reachability

An object is **reachable** if it can be reached from a root: - global/module variables - local variables on the stack - currently executing frames - registers holding values - other reachable objects (transitively)

An object is **unreachable** (garbage) when no root can reach it, directly or indirectly. The collector reclaims unreachable objects.

## 47.4 18.4 Reference counting vs tracing

Zen’s collector may use reference counting, tracing (mark-and-sweep), or a hybrid, depending on build:

| Approach | How it works | Pros | Cons |
|---|---|---|---|
| Reference counting | count references; free at zero | prompt reclamation | cycles leak; per-ref overhead |
| Tracing (mark-sweep) | mark reachable from roots, sweep rest | handles cycles | pause-time; throughput cost |
| Hybrid | refcount + periodic cycle collection | prompt + cycle-safe | more complex |

## 47.5 18.5 Reference cycles (the classic leak)

Two or more objects referencing each other form a cycle. With pure reference counting, their counts never reach zero even if nothing outside references them.

```
class Node {
    var peer = nil
}
let a = new Node()
let b = new Node()
a.peer = b
b.peer = a   # cycle
# drop external refs:
a = nil
b = nil
# a and b still reference each other → leak (pure refcount)
```

With a tracing or hybrid collector, the cycle is detected and collected. With pure refcounting, break it manually:

```
a.peer = nil
b.peer = nil
```

## 47.6 18.6 `shared` and thread-shared memory

`shared x = expr` wraps the value in an `Arc<Mutex<_>>`-like container. The wrapper itself is reference counted; the inner `Mutex` protects the value for concurrent access. The wrapped value is freed when the last shared reference drops.

## 47.7 18.7 Writing memory-friendly code

1. **Drop large references when done:** `big = nil` after processing a huge list helps the collector reclaim it sooner.
2. **Avoid unbounded growth:** don't append to a global list forever; cap it or evict old entries.
3. **Break cycles in graphs:** null out back-references when tearing down object graphs.
4. **Close resources promptly:** files, sockets, FFI handles — use `with` / `finally` rather than waiting for GC.
5. **Prefer immutable values:** fewer surprise aliases means easier reasoning about lifetime.
6. **Don't micro-manage:** trust the collector; optimize only after measuring.

## 47.8 18.8 Memory leaks to watch for

| Leak | Cause | Fix |
|---|---|---|
| reference cycle | A↔B mutual refs | break one side |
| growing global cache | no eviction | size-limit / LRU |

| Leak | Cause | Fix |
|---|---|---|
| unremoved event listeners | callbacks hold refs | unsubscribe |
| open files/sockets | not closed | `with` / `finally` |
| FFI handle not freed | missing finalizer | call release in `finally` |
| closure capturing large object | closure outlives need | nil the captured ref |

## 47.9 18.9 GC and performance

Collection pauses can affect latency-sensitive code. If your build exposes GC tuning: - increase the allocation threshold to collect less often (higher throughput, larger pauses) - decrease it for more frequent, smaller pauses (better latency)

Most applications need no tuning.

## 47.10 18.10 Exercises

1. Draw a reference cycle between two objects and mark which refs to break.
2. Write a tiny object graph, drop external refs, and explain what the GC sees.
3. Show how `big = nil` helps reclaim a large list.
4. Explain why a global cache that never evicts leaks memory.
5. Write `with_resource` using `try/finally` for a file.
6. Describe the difference between reachable and unreachable.
7. Name three heap types and three immediate types.
8. Why does `shared` need both refcounting and a mutex?
9. Implement an LRU cache with eviction (conceptually).
10. What can you do if GC pauses hurt your application’s latency?

’’’ print(‘gc done’)

# 48 Chapter 54: The Cranelift JIT Compiler

*How Zen 3.0 compiles hot functions to native machine code, inline caches, on-stack replacement, and deoptimization.*

## 48.1 19.1 Overview

Zen 3.0 includes a just-in-time (JIT) compiler built on **Cranelift**, the same code generator used by Wasmtime and other projects. The JIT compiles frequently executed Zen functions to native machine code at runtime, while cold code stays interpreted.

Pipeline:

```
bytecode ⟶ Cranelift IR ⟶ native machine code ⟶ execute
```

## 48.2 19.2 Interpreter vs JIT

| Aspect | Interpreter (`zen-interp`) | JIT (`zen-jit`) |
|---|---|---|
| Startup | fast (no compile) | slower (compile cost) |
| Steady-state speed | slower (per-op dispatch) | faster (native code) |
| Best for | cold / rarely-run code | hot loops / frequent calls |
| Debugging | easier (direct mapping) | harder (native frames) |

Zen automatically decides: run in the interpreter first; after a function crosses an invocation threshold, JIT-compile it.

## 48.3 19.3 Compilation process

1. **Bytecode → Cranelift IR**: each bytecode instruction maps to one or more Cranelift intermediate-representation operations. Stack-based bytecode ops are converted to virtual-register IR.
2. **Type guards**: at call sites and type-sensitive ops, insert guards that check the operand types match expectations.
3. **Compile**: Cranelift lowers IR to target machine code (x86-64, AArch64, etc.).
4. **Install**: patch the call site to jump to the native code.

## 48.4 19.4 Inline caches

At a polymorphic call site (e.g., `obj.method()`), the types of `obj` may vary across calls. The JIT installs an **inline cache**: - On the first call, resolve the method normally (slow path) and cache the result + the observed type. - On subsequent calls, check the type guard. If the type matches the cache, jump directly to the cached method (fast path). - If the type differs, fall back to the slow path and possibly expand the cache (polymorphic inline cache) or deopt.

This makes repeated monomorphic calls nearly as fast as statically-typed code.

## 48.5 19.5 On-stack replacement (OSR)

OSR lets the VM switch between interpreted and JIT-compiled code **mid-execution**.

- **Interpreter → JIT**: a long-running loop that's interpreted gets compiled on the fly; the running frame is replaced with a JIT frame at a loop back edge, continuing from the same point.
- **JIT → interpreter (deopt)**: if a JIT assumption fails (see below), the VM reconstructs interpreter state from the native frame and resumes interpretation.

OSR means long loops benefit from JIT without restarting the function.

## 48.6 19.6 Deoptimization

JIT code is compiled under **assumptions** (guards), e.g.: - a variable always holds an int at this point - a call site always sees the same type

If a guard fails at runtime (the assumption is violated), the VM **deopts**: 1. Capture the current native machine state (registers, stack). 2. Reconstruct the equivalent interpreter frame (values, locals, PC). 3. Transfer control to the interpreter at the corresponding bytecode point. 4. Continue interpreting (possibly re-JIT later if it gets hot again).

Deopt guarantees correctness even when speculation was wrong.

## 48.7 19.7 When does JIT help?

JIT wins when: - a function is called many times - a loop iterates many times - types at call sites are stable (monomorphic)

JIT does **not** help (and may hurt) when: - code runs once (compile cost outweighs benefit) - types change constantly (guards always fail → deopt thrashing)

## 48.8 19.8 Observing JIT behavior

Some builds support debug logging:

```
ZEN_DEBUG=1 zen run app.z   # may print JIT/compile/deopt events (build-dependent)
```

There's typically no user-facing flag to force or disable JIT; it's automatic. `zen explain` covers runtime errors (Z03xx), not JIT internals.

## 48.9 19.9 JIT and FFI / pymport

JIT-compiled code that calls out to C (FFI) or Python (pymport) uses the same marshalling as interpreted code. The JIT emits the call sequence; the runtime handles crossing the boundary. A deopt during an FFI/pymport call must reconstruct state at the boundary safely.

## 48.10 19.10 Exercises

1. Contrast interpreter startup vs steady-state speed with JIT.
2. What does an inline cache store at a call site?
3. When does OSR trigger (interpreter → JIT)?
4. What triggers deoptimization (JIT → interpreter)?
5. Name two situations where JIT helps and two where it doesn’t.
6. Sketch the bytecode→IR→native pipeline.
7. Why must deopt reconstruct interpreter state?
8. How do type guards keep JIT code correct?
9. What does `ZEN_DEBUG=1` potentially show?
10. Does JIT change the semantics of your program? (No — only performance.)

’’’ print(‘jit done’)

# 49 Chapter 43: CLI Tools

Zen is excellent for building command-line applications. This chapter shows how to parse arguments, read input, write output, and build useful CLI tools.

## 49.1 43.1 Argument parsing: `argparser`

```
import argparser

let parser = argparser.ArgumentParser("mycli")
parser.add_argument("input", help="input file")
parser.add_argument("--output", "-o", help="output file")
parser.add_argument("--verbose", "-v", action="store_true", help="verbose mode")

let args = parser.parse_args()

print(args.input)
print(args.output)
print(args.verbose)
```

### 49.1.1 ArgumentParser methods

| Method | Description |
|---|---|
| `ArgumentParser(prog?)` | Create a parser |
| `add_argument(name, opts?)` | Add an argument |
| `parse_args()` | Parse `sys.argv` |

### 49.1.2 Argument options

| Option | Description |
|---|---|
| `help` | Help text |
| `action` | `"store_true"`, `"store_false"`, `"count"` |
| `default` | Default value |
| `type` | `"int"`, `"float"`, `"str"` |
| `choices` | Allowed values |
| `required` | Boolean |

### 49.1.3 Example

```
import argparser

let parser = argparser.ArgumentParser("wc")
parser.add_argument("file", help="file to count")
```

```
parser.add_argument("--lines", "-l", action="store_true", help="count lines")
parser.add_argument("--words", "-w", action="store_true", help="count words")
parser.add_argument("--bytes", "-c", action="store_true", help="count bytes")

let args = parser.parse_args()

let content = fs.read(args.file)
let lines = len(content.split("\n"))
let words = len(content.split())
let bytes = len(content)

if args.lines {
    print(lines)
}
if args.words {
    print(words)
}
if args.bytes {
    print(bytes)
}
```

## 49.2 43.2 Reading stdin

```
import fs

// Read all stdin
let input = fs.read("/dev/stdin")
print(input)

// Or line by line
let lines = input.split("\n")
for line in lines {
    print(line)
}
```

## 49.3 43.3 Writing output

```
import fs

// stdout
print("hello")

// stderr
fs.write("/dev/stderr", "error message\n")

// To a file
```

```
fs.write("output.txt", "result")
```

## 49.4 43.4 Exit codes

```
import sys

// Success
exit(0)

// Failure
exit(1)

// Specific error
exit(2)
```

`exit(code)` terminates the process with the given code.

## 49.5 43.5 Complete example: a `cat` clone

```
import fs
import sys
import argparser

let parser = argparser.ArgumentParser("zcat")
parser.add_argument("files", nargs="*")
let args = parser.parse_args()

let files = args.files
if len(files) == 0 {
    // Read stdin
    print(fs.read("/dev/stdin"))
} else {
    for f in files {
        try {
            print(fs.read(f))
        } catch as e {
            fs.write("/dev/stderr", "zcat: " + f + ": " + e.msg + "\n")
            exit(1)
        }
    }
}
```

## 49.6 43.6 Complete example: a `grep` clone

```
import fs
import argparser
import re

let parser = argparser.ArgumentParser("zgrep")
parser.add_argument("pattern", help="search pattern")
parser.add_argument("file", help="file to search")
parser.add_argument("-i", action="store_true", help="ignore case")
let args = parser.parse_args()

let pattern = args.pattern
let flags = args.i ? "i" : ""

let content = fs.read(args.file)
let lines = content.split("\n")

for (i, line) in enumerate(lines) {
    if re.search(pattern, line, flags) != nil {
        print(str(i + 1) + ": " + line)
    }
}
```

## 49.7 43.7 Complete example: a JSON pretty-printer

```
import fs
import json
import argparser

let parser = argparser.ArgumentParser("zjson")
parser.add_argument("file", help="JSON file")
parser.add_argument("--indent", "-i", type="int", default=2, help="indent spaces")
let args = parser.parse_args()

let text = fs.read(args.file)
let data = json.loads(text)

// Zen's json.dumps doesn't support indent in this version
// Output compact JSON
print(json.dumps(data))
```

## 49.8 43.8 Making scripts executable

```
#!/usr/bin/env zen
print("Hello from a script!")
```

```
chmod +x mycli.z
./mycli.z
```

## 49.9 43.9 Common pitfalls

- **`argparser` reads from `sys.argv`.** Don't call `parse_args` twice.
- **`nargs="*"`** collects remaining positional args into a list.
- **`exit(code)` terminates immediately** — use `try`/`finally` for cleanup before exiting if needed.
- **`fs.read("/dev/stdin")`** reads all stdin at once; for large input, read in chunks.

## 49.10 43.10 Quick reference

```
import argparser
import fs
import sys
import exit

let parser = argparser.ArgumentParser("prog")
parser.add_argument("pos", help="positional")
parser.add_argument("--opt", "-o", help="optional")
parser.add_argument("--flag", "-f", action="store_true")
let args = parser.parse_args()

print(args.pos)
print(args.opt)
print(args.flag)

fs.write("/dev/stderr", "error\n")
exit(0)
```

## 49.11 43.11 Exercises

1. Write a `wc` clone that counts lines, words, and bytes.
2. Write a `head` clone that prints the first N lines.
3. Write a `cp` clone that copies one file to another.
4. Write a `find` clone that lists files matching a pattern.
5. Write a CLI config tool that reads/writes a JSON config file.

## 49.12 Extended Project Ideas and Implementation Notes

### 49.12.1 Design checklist

- Requirements and scope
- Error handling
- Logging
- Configuration

- Testing
- Documentation
- Security considerations

### 49.12.2 Exercises

1. Extend the project with new features
2. Add tests
3. Improve error messages
4. Add help/usage
5. Make it configurable
6. Package for distribution

## 49.13 43.12 Full tutorial — building a CLI todo manager

### 49.13.1 Requirements

- Add, list, mark done, delete tasks
- Store tasks in a JSON file
- Accept subcommands: `add`, `list`, `done`, `delete`
- Print usage when no args or `--help`

### 49.13.2 Implementation (`todo.z`)

```
import sys
import json
import os
import pathlib

let TASKS_FILE = pathlib.Path("tasks.json")

func load_tasks() {
    if !TASKS_FILE.exists() { return [] }
    try {
        return json.load(str(TASKS_FILE))
    } catch as e {
        print("warn: could not load tasks: " + e.msg)
        return []
    }
}

func save_tasks(tasks) {
    json.save(str(TASKS_FILE), tasks)
}

func cmd_add(args) {
    if len(args) < 1 { print("usage: todo add <task>"); return }
```

```
    let tasks = load_tasks()
    let task = #{id: len(tasks) + 1, title: args[0], done: false}
    tasks.push(task)
    save_tasks(tasks)
    print("added: " + args[0])
}

func cmd_list(_) {
    let tasks = load_tasks()
    if len(tasks) == 0 { print("no tasks"); return }
    for t in tasks {
        let mark = if t["done"] { "[x]" } else { "[ ]" }
        print("${mark} ${t["id"]}. ${t["title"]}")
    }
}

func cmd_done(args) {
    if len(args) < 1 { print("usage: todo done <id>"); return }
    let id = int(args[0])
    let tasks = load_tasks()
    var found = false
    for t in tasks {
        if t["id"] == id { t["done"] = true; found = true }
    }
    if found { save_tasks(tasks); print("marked ${id} done") }
    else { print("no task with id ${id}") }
}

func cmd_delete(args) {
    if len(args) < 1 { print("usage: todo delete <id>"); return }
    let id = int(args[0])
    let tasks = load_tasks()
    let filtered = []
    for t in tasks { if t["id"] != id { filtered.push(t) } }
    save_tasks(filtered)
    print("deleted ${id}")
}

func main(argv) {
    if len(argv) < 2 {
        print("Todo Manager")
        print("usage: todo <add|list|done|delete> [args]")
        return 0
    }
    let cmd = argv[1]
    let rest = []
    for i in 2..len(argv) { rest.push(argv[i]) }
```

```
    if cmd == "add" { cmd_add(rest) }
    elif cmd == "list" { cmd_list(rest) }
    elif cmd == "done" { cmd_done(rest) }
    elif cmd == "delete" { cmd_delete(rest) }
    elif cmd == "--help" || cmd == "help" {
        print("usage: todo <add|list|done|delete> [args]")
    } else {
        print("unknown command: " + cmd)
    }
    return 0
}

if __name__ == "__main__" {
    sys.exit(main(sys.argv))
}
```

### 49.13.3 Sample session

```
$ zen run todo.z add "Learn Zen"
added: Learn Zen
$ zen run todo.z add "Build a CLI"
added: Build a CLI
$ zen run todo.z list
[ ] 1. Learn Zen
[ ] 2. Build a CLI
$ zen run todo.z done 1
marked 1 done
$ zen run todo.z list
[x] 1. Learn Zen
[ ] 2. Build a CLI
$ zen run todo.z delete 2
deleted 2
```

### 49.13.4 Design notes

- Tasks are dicts in a list, serialized to JSON.
- IDs are simple (len+1); not robust to deletion gaps but fine for a demo.
- `save_tasks` rewrites the whole file each time (simple, safe).
- Errors are caught so a corrupt JSON file doesn't crash the app.

## 49.14 43.13 CLI design checklist

1. **Parse args** from `sys.argv` (first element is the program).
2. **Dispatch** on subcommand.
3. **Validate** argument count and types before use.
4. **Print usage** on `--help`, missing args, or unknown command.

5. **Return exit codes:** 0 success, non-zero on error.
6. **Persist** state to a file (JSON/text) if stateful.
7. **Handle errors** gracefully (missing file, bad input).
8. **Document** each subcommand.

## 49.15 43.14 Exercises

1. Add a `priority` field to tasks and sort `list` by priority.
2. Add `clear` command to remove all tasks.
3. Add `--file` option to choose a different storage path.
4. Add `edit <id> <new title>` command.
5. Validate that `done`/`delete` IDs exist before operating.
6. Add timestamps (`created_at`, `completed_at`) to tasks.
7. Print overdue tasks (conceptually, with due dates).
8. Write `todo count` showing number of pending/done.
9. Add tab-completion hints (usage text only).
10. Package the CLI with `zen build todo.z -o todo`.

# 50 Chapter 44: Web Scraping & Automation

Web scraping and automation let you extract data from websites, download media, and control browsers programmatically. This chapter covers Zen's HTTP client, **pymport** (Python interoperability), browser automation with Selenium, and driving Chrome over **raw TCP/WebSocket sockets with the Chrome DevTools Protocol (CDP)**.

Every example here is runnable. The complete scripts, setup script, and test runner live in `book-v2/tests/web-automation/`; the pymport fundamentals are in Chapter 42.

> **Legal and ethical note:** Only scrape sites you are permitted to scrape. Check `robots.txt`, Terms of Service, and applicable laws. The examples target publicly accessible TikTok pages for educational purposes; sites may rate-limit, require authentication, or block automated access.

## 50.1 44.1 Setup

From the repository root:

```
cd book-v2/tests
./setup.sh
```

`setup.sh` does three things: 1. builds Zen with the Python bridge (`cargo build --release --features python`); 2. creates `.venv/` beside the tests; 3. installs `requirements.txt`.

Then, in the shell you use for the examples:

```
cd book-v2/tests/web-automation
export PYTHONPATH="$(../.venv/bin/python -c 'import sysconfig; print(sysconfig.get_path
export ZEN_NO_PY_BOOTSTRAP=1
ZEN=../../../target/release/zen
```

Requirements: - Zen built with `--features python` - Python 3.9+ with `venv` - `chromium` for Selenium and CDP - `chromedriver` matching Chromium for Selenium - Network access

Alternatively, let Zen manage a project environment: this directory has its own `requirements.txt`, so `zen py-sync` creates `.zen_venv/` and `zen run` uses it automatically. See Chapter 42.

> **pymport calls are positional-only.** No keyword arguments in v1. Pass `Options()` to `webdriver.Chrome(opts)` positionally, and pass `Popen` arguments positionally.

## 50.2 44.2 Run the whole suite

```
cd book-v2/tests
./run-all.sh              # hermetic pymport tests
./run-all.sh --network    # also runs the four examples below
```

The runner validates that each downloaded file is an MP4 (`ftyp` box) before reporting success.

## 50.3 44.3 1) Scrape + Download with `requests` + `bs4` (pymport)

`bs4` parses HTML. TikTok's embed page (`/embed/v2/<id>`) is accessible over plain HTTP and embeds video metadata in `__FRONTITY_CONNECT_STATE__`. The MP4 CDN URL is in `videoData.itemInfos.video.urls[0]`; `urllib.request.urlretrieve` writes it without re-encoding.

`bs4_tiktok.z`:

```
import requests
import fs
pymport bs4
pymport json
pymport urllib.request
pymport socket

socket.setdefaulttimeout(45)

let UA = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gec
let VIDEO = "6718335390845095173"
let url = "https://www.tiktok.com/embed/v2/" + VIDEO

let _ = fs.mkdirs("out")
let out = "out/tiktok_bs4.mp4"

// TikTok's edge intermittently answers 503 to non-browser clients. Retry
// with backoff instead of failing the example on the first response.
let fetch = fn(u) {
    let attempt = 0
    let resp = null
    while attempt < 6 {
        resp = requests.get(u, {
            "headers": {
                "User-Agent": UA,
                "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=
                "Accept-Language": "en-US,en;q=0.9",
                "Referer": "https://www.tiktok.com/"
            },
            "timeout": 40
        })
        if resp["status_code"] == 200 {
            return resp
        }
        print("http " + str(resp["status_code"]) + " (attempt " + str(attempt + 1) + ")
        time.sleep(3 * (attempt + 1))
```

```
        attempt += 1
    }
    return resp
}

print("fetching " + url)
let resp = fetch(url)
print("status: " + str(resp["status_code"]) + " bytes: " + str(len(resp["text"])))
if resp["status_code"] != 200 {
    print("giving up: TikTok kept returning " + str(resp["status_code"]))
    exit(1)
}

let soup = bs4.BeautifulSoup(resp["text"], "html.parser")
let tag = soup.find("script", {"id": "__FRONTITY_CONNECT_STATE__"})
if tag == null {
    print("no state script")
    exit(1)
}

let data = json.loads(tag.string)
let page = data["source"]["data"]["/embed/v2/" + VIDEO]
let item = page["videoData"]["itemInfos"]
let author = page["videoData"]["authorInfos"]
let music = page["videoData"]["musicInfos"]
let video_url = item["video"]["urls"][0]

print("id: " + item["id"])
print("desc: " + item["text"])
print("author: @" + author["uniqueId"] + " (" + author["nickName"] + ")")
print("music: " + music["musicName"])
print("plays: " + str(item["playCount"]))
print("likes: " + str(item["diggCount"]))
print("comments: " + str(item["commentCount"]))
print("shares: " + str(item["shareCount"]))
print("mp4: " + video_url[0:80] + "...")

urllib.request.urlretrieve(video_url, out)
print("downloaded: " + out + " (" + str(fs.size(out)) + " bytes)")
```

Run:

```
$ZEN run bs4_tiktok.z
file out/tiktok_bs4.mp4
```

Expected: metadata for `@scout2015`, a `downloaded: out/tiktok_bs4.mp4 (3224978 bytes)` line, and `ISO Media, MP4 Base Media v1`.

## 50.4 44.4 2) Stateful Scraping + Download with `mechanicalsoup` (pymport)

`MechanicalSoup` wraps `requests` and BeautifulSoup in a stateful browser. `browser.open()` keeps cookies/history, `browser.page` is the current BS4 soup, and `browser.session` can be reused for the CDN fetch. Python `bytes` are written through Python's own file object because native `fs.write_binary` is string-oriented.

`ms_tiktok.z`:

```
import fs
import time
pymport mechanicalsoup
pymport json
pymport builtins
pymport socket

socket.setdefaulttimeout(45)

let UA = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gec
let VIDEO = "6718335390845095173"
let embed = "https://www.tiktok.com/embed/v2/" + VIDEO

let _ = fs.mkdirs("out")
let out = "out/tiktok_ms.mp4"

let browser = mechanicalsoup.StatefulBrowser()
browser.session.headers.update({
    "User-Agent": UA,
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
    "Accept-Language": "en-US,en;q=0.9",
    "Referer": "https://www.tiktok.com/"
})

let resp = browser.open(embed)
let attempt = 0
while resp.status_code != 200 && attempt < 6 {
    print("http " + str(resp.status_code) + " (attempt " + str(attempt + 1) + "), retry
    time.sleep(3 * (attempt + 1))
    resp = browser.open(embed)
    attempt += 1
}

print("opened: " + browser.get_url())
print("status: " + str(resp.status_code) + " bytes: " + str(len(resp.text)))
if resp.status_code != 200 {
    print("giving up: TikTok kept returning " + str(resp.status_code))
    exit(1)
}
```

```
let soup = browser.page
let tag = soup.find("script", {"id": "__FRONTITY_CONNECT_STATE__"})
if tag == null {
    print("no state script")
    exit(1)
}

let data = json.loads(tag.string)
let page = data["source"]["data"]["/embed/v2/" + VIDEO]
let item = page["videoData"]["itemInfos"]
let author = page["videoData"]["authorInfos"]
let video_url = item["video"]["urls"][0]

print("id: " + item["id"])
print("author: @" + author["uniqueId"] + " (" + author["nickName"] + ")")
print("plays: " + str(item["playCount"]) + " likes: " + str(item["diggCount"]))

let bin = browser.session.get(video_url)
print("cdn status: " + str(bin.status_code) + " bytes: " + str(len(bin.content)))
if bin.status_code != 200 {
    print("giving up: CDN returned " + str(bin.status_code))
    exit(1)
}

let f = builtins.open(out, "wb")
f.write(bin.content)
f.close()
print("downloaded: " + out + " (" + str(fs.size(out)) + " bytes)")
```

Run:

```
$ZEN run ms_tiktok.z
```

## 50.5 44.5 3) Browser Automation with Selenium (pymport)

Headless Chromium solves JavaScript challenges and renders pages like a user. Selenium navigates to the real TikTok video page, waits for `__UNIVERSAL_DATA_FOR_REHYDRATION__`, and extracts the full `itemStruct` (author, stats, description). The video page's `playAddr` is signed and rejected by the CDN outside the player, so the script fetches the embed route with plain HTTP for an unsigned MP4 URL.

`selenium_tiktok.z`:

```
import fs
import time
import requests
pymport selenium.webdriver as webdriver
pymport selenium.webdriver.chrome.options as options
```

```
pymport bs4
pymport json
pymport urllib.request
pymport socket

socket.setdefaulttimeout(45)

let VIDEO = "6718335390845095173"
let UA = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gec
let url = "https://www.tiktok.com/@scout2015/video/" + VIDEO

let _ = fs.mkdirs("out")
let out = "out/tiktok_selenium.mp4"

// Quit the browser and stop the chromedriver service. The embedded
// interpreter exits immediately, so this must happen before exit().
let close_browser = fn(d) {
    d.quit()
    try {
        d.service.stop()
    } catch {
        # already stopped
    }
}

let opts = options.Options()
opts.add_argument("--headless=new")
opts.add_argument("--no-sandbox")
opts.add_argument("--disable-dev-shm-usage")
opts.add_argument("--disable-gpu")
opts.add_argument("--window-size=1280,900")
opts.add_argument("--mute-audio")

print("launching chromium...")
let driver = webdriver.Chrome(opts)
driver.set_page_load_timeout(60)

print("navigating to " + url)
try {
    driver.get(url)
} catch as e {
    print("navigation warning: " + str(e))
}

// The page renders its data after the WAF/JS boot; poll for the payload.
let src = ""
let tries = 0
```

```
while tries < 15 {
    src = driver.page_source
    if src.contains("__UNIVERSAL_DATA_FOR_REHYDRATION__") {
        break
    }
    time.sleep(2)
    tries += 1
}
print("title: " + driver.title)
print("page bytes: " + str(len(src)) + " (polls: " + str(tries) + ")")

let soup = bs4.BeautifulSoup(src, "html.parser")
let tag = soup.find("script", {"id": "__UNIVERSAL_DATA_FOR_REHYDRATION__"})
if tag == null {
    print("no rehydration payload - TikTok served a challenge")
    close_browser(driver)
    exit(1)
}

let data = json.loads(tag.string)
let item = data["__DEFAULT_SCOPE__"]["webapp.video-detail"]["itemInfo"]["itemStruct"]
let stats = item["stats"]
let author = item["author"]

print("id: " + item["id"])
print("desc: " + item["desc"])
print("author: @" + author["uniqueId"] + " (" + author["nickname"] + ")")
print("plays: " + str(stats["playCount"]) + " likes: " + str(stats["diggCount"]))
print("comments: " + str(stats["commentCount"]) + " shares: " + str(stats["shareCount"]

print("fetching embed route for an unsigned mp4 url...")
let embed = "https://www.tiktok.com/embed/v2/" + VIDEO
let embed_headers = {
    "User-Agent": UA,
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
    "Accept-Language": "en-US,en;q=0.9",
    "Referer": "https://www.tiktok.com/"
}
let eresp = null
let eretry = 0
while eretry < 6 {
    eresp = requests.get(embed, {"headers": embed_headers, "timeout": 40})
    if eresp["status_code"] == 200 {
        break
    }
    print("embed http " + str(eresp["status_code"]) + " (attempt " + str(eretry + 1) +
    time.sleep(3 * (eretry + 1))
```

```
    eretry += 1
}
print("embed status: " + str(eresp["status_code"]))
if eresp["status_code"] != 200 {
    close_browser(driver)
    exit(1)
}
let esoup = bs4.BeautifulSoup(eresp["text"], "html.parser")
let etag = esoup.find("script", {"id": "__FRONTITY_CONNECT_STATE__"})
if etag == null {
    print("no embed state script")
    close_browser(driver)
    exit(1)
}
let edata = json.loads(etag.string)
let epage = edata["source"]["data"]["/embed/v2/" + VIDEO]
let mp4 = epage["videoData"]["itemInfos"]["video"]["urls"][0]
print("mp4: " + mp4[0:80] + "...")

urllib.request.urlretrieve(mp4, out)
print("downloaded: " + out + " (" + str(fs.size(out)) + " bytes)")

close_browser(driver)
time.sleep(1)
print("browser closed")
```

Run (Chromium startup can take 30-40s):

```
timeout 300 $ZEN run selenium_tiktok.z
```

Chromedriver must match the Chromium major version. On Debian/Ubuntu: `apt install chromium-driver`.

## 50.6 44.6 4) Low-Level Control: CDP over raw TCP + WebSocket sockets

This example uses no automation library. Zen launches Chromium with `--remote-debugging-port`, discovers the page target over HTTP (`/json/list`), upgrades to a WebSocket, sends masked CDP frames through `sys.write_bytes` on the socket fd, and parses unmasked server frames byte by byte.

Key protocol details: - **HTTP**: parse headers and stop at `Content-Length`. Chrome may keep the connection open despite `Connection: close`; waiting for EOF produces a timeout (`EAGAIN`). - **WebSocket**: client frames must be masked. Extended lengths are 126 + 16-bit or 127 + 64-bit. Server frames are not masked. - **Binary writes**: `socket.send` takes a string and would UTF-8 re-encode masked bytes. `sys.write_bytes(fd, hex)` writes raw bytes, so the frame is built as a byte list, hex-encoded, and written to `socket.fileno(sock)`.

cdp_tiktok.z helpers:

```
// CDP over raw sockets: drive headless Chromium from Zen with no libraries.
import socket
import sys
import json
import time
import fs
pymport subprocess
import base64
import bytes

let PORT = 9223
let PROFILE = "out/cdp-profile"
let VIDEO = "6718335390845095173"
let VIDEO_PAGE = "https://www.tiktok.com/@scout2015/video/" + VIDEO

let HEX = "0123456789abcdef"

let to_hex = fn(bs) {
    let out = ""
    for b in bs {
        out += HEX[b >> 4] + HEX[b & 15]
    }
    return out
}

let ws_send = fn(sock, text) {
    let payload = []
    for ch in list(text) {
        for b in bytes.utf8_bytes(ch.ord()) {
            payload.push(b)
        }
    }
    let mask = [0x37, 0xA1, 0x0C, 0xF2]
    let masked = []
    let i = 0
    while i < len(payload) {
        masked.push(payload[i] ^ mask[i % 4])
        i += 1
    }
    let frame = [0x81]
    let n = len(masked)
    if n < 126 {
        frame.push(0x80 | n)
    } elif n < 65536 {
        frame.push(0x80 | 126)
        frame.push((n >> 8) & 255)
```

```
        frame.push(n & 255)
    } else {
        frame.push(0x80 | 127)
        let k = 7
        while k >= 0 {
            frame.push((n >> (8 * k)) & 255)
            k -= 1
        }
    }
    for b in mask {
        frame.push(b)
    }
    for b in masked {
        frame.push(b)
    }
    sys.write_bytes(socket.fileno(sock), to_hex(frame))
}

let recv_exact = fn(sock, n) {
    let out = []
    while len(out) < n {
        let chunk = list(socket.recv(sock, n - len(out)))
        if len(chunk) == 0 {
            throw "websocket closed"
        }
        for b in chunk {
            out.push(b)
        }
    }
    return out
}

let ws_read = fn(sock) {
    let h = recv_exact(sock, 2)
    let opcode = h[0] & 0x0F
    let masked = (h[1] & 0x80) != 0
    let n = h[1] & 0x7F
    if n == 126 {
        let e = recv_exact(sock, 2)
        n = e[0] * 256 + e[1]
    } elif n == 127 {
        let e = recv_exact(sock, 8)
        n = 0
        for b in e {
            n = n * 256 + b
        }
    }
```

```
    let mask = []
    if masked {
        mask = recv_exact(sock, 4)
    }
    let data = recv_exact(sock, n)
    if masked {
        let i = 0
        while i < len(data) {
            data[i] = data[i] ^ mask[i % 4]
            i += 1
        }
    }
    return {"opcode": opcode, "data": data}
}

let cdp_call = fn(sock, id, method, params) {
    ws_send(sock, json.stringify({"id": id, "method": method, "params": params}))
    let pending = []
    while true {
        let f = ws_read(sock)
        if f["opcode"] == 8 {
            throw "websocket closed by peer"
        }
        if f["opcode"] == 9 {
            continue
        }
        if f["opcode"] == 1 or f["opcode"] == 0 {
            for b in f["data"] {
                pending.push(b)
            }
            let text = bytes.str_from_bytes(pending)
            try {
                let msg = json.parse(text)
                if msg.get("id", 0) == id {
                    return msg
                }
                pending = []
            } catch {
                # fragmented frame: keep collecting
            }
        }
    }
}
```

The full script then launches Chromium with `pymport subprocess`, performs the HTTP discovery and WebSocket upgrade, and polls `Runtime.evaluate`:

```
let _ = fs.mkdirs("out")
let _ = fs.rmtree(PROFILE)
let cmd = "chromium --headless=new --no-sandbox --disable-gpu --disable-dev-shm-usage "
cmd += "--remote-debugging-port=" + str(PORT) + " "
cmd += "--user-data-dir=" + PROFILE + " about:blank"
let argv = cmd.split(" ")
let browser = subprocess.Popen(argv, -1, null, null, null, -3)
print("chromium pid " + str(browser.pid) + " on port " + str(PORT))

try {
    # ... HTTP discovery, WS upgrade ...
    let nav = cdp_call(ws, 1, "Page.navigate", {"url": VIDEO_PAGE})
    let reply = cdp_call(ws, 100, "Runtime.evaluate", {"expression": expr, "returnByVal
    # expr returns id, author, plays, likes, comments, desc
} finally {
    browser.terminate()
}
```

See the complete, tested script: tests/web-automation/cdp_tiktok.z.

Run:

```
timeout 180 $ZEN run cdp_tiktok.z
```

## 50.7 44.7 Tips and Gotchas

- **WAF challenges**: plain `requests` to TikTok video/profile pages may return a WAF challenge. Use `/embed/v2/<id>` or a real browser (Selenium). The embed route is reliable for the MP4 URL.
- **Binary downloads**: `fs.write_binary` expects a string and has UTF-8/string constraints. Prefer `urllib.request.urlretrieve` or `builtins.open(..., "wb").write(py_bytes)`.
- **pymport limits**: positional-only calls (no kwargs). `webdriver.Chrome(opts)` and `Popen(argv, -1, null, null, null, -3)` are positional.
- **Environment bootstrap**: use `book-v2/tests/setup.sh`, or `zen py-sync` with a local `requirements.txt`.
- **Chrome sandbox**: in containers, add `--no-sandbox` and `--disable-dev-shm-usage`.
- **Sockets**: `socket.recv` returns `bytes`; use `list(bytes)` to index. `socket.recv_text` returns a string and is lossy for binary. For raw binary protocols, write through `sys.write_bytes(fd, hex)`.
- **JSON**: the pure-Zen `std/json.z` stringifies numbers with `str()`; nested dict access in CDP responses needs explicit keys.
- **Cleanup**: Selenium can leave `chromedriver` alive when the embedded interpreter exits; the example calls `driver.service.stop()`. For a hard-killed CDP run, remove leftovers with `pkill -x chromium`.
- **Hangs**: `MechanicalSoup`, `urllib.request.urlretrieve`, and `requests` do not impose a default timeout. Set `socket.setdefaulttimeout(45)` (via `pympor socket`)

or pass an explicit timeout where supported.

## 50.8 44.8 Verified Artifacts

Tested against real TikTok on 2026-10-23:

- `out/tiktok_bs4.mp4` — ISO Media, MP4 Base Media v1 (3,224,978 bytes)
- `out/tiktok_ms.mp4` — ISO Media, MP4 Base Media v1 (3,224,978 bytes)
- `out/tiktok_selenium.mp4` — ISO Media, MP4 Base Media v1 (3,224,978 bytes)
- CDP over raw sockets extracted metadata from `@scout2015/video/6718335390845095173`: 159600 plays, 35200 likes, 5486 comments.

## 50.9 44.9 Appendix: Tested Scripts

Everything in this chapter is in the repository:

- `tests/web-automation/bs4_tiktok.z` — requests + bs4 + urllib download
- `tests/web-automation/ms_tiktok.z` — MechanicalSoup + `browser.session` + `builtins` binary write
- `tests/web-automation/selenium_tiktok.z` — Selenium headless + embed route
- `tests/web-automation/cdp_tiktok.z` — raw TCP + WebSocket CDP
- `tests/web-automation/requirements.txt`
- `tests/setup.sh` — build + venv + install
- `tests/run-all.sh` — run and validate

Quick start:

```
git clone https://github.com/3CN-O4D/zen.git
cd zen/book-v2/tests
./setup.sh
./run-all.sh --network
```

See Chapter 42: pymport for setup, `zen py-sync`, and troubleshooting.

## 50.10 Extended Project Ideas and Implementation Notes

### 50.10.1 Design checklist

- Requirements and scope
- Error handling
- Logging
- Configuration
- Testing
- Documentation
- Security considerations

### 50.10.2 Exercises

1. Extend the project with new features
2. Add tests

3. Improve error messages
4. Add help/usage
5. Make it configurable
6. Package for distribution

## 50.11 44.12 Full tutorial — simple web scraper

### 50.11.1 Goal

Fetch a page, extract links, and save results to a file.

### 50.11.2 Implementation (`scrape.z`)

```
import requests
import re
import json
import pathlib

func fetch(url) {
    try {
        let r = requests.get(url, timeout=10)
        r.raise_for_status()
        return r.text
    } catch as e {
        print("fetch failed for " + url + ": " + e.msg)
        return nil
    }
}

func extract_links(html, base) {
    let links = []
    # naive regex for href="..." (real HTML needs a proper parser)
    for m in re.findall(r'href="([^"]+)"', html) {
        # resolve relative URLs conceptually
        links.push(m)
    }
    return links
}

func main(argv) {
    if len(argv) < 2 {
        print("usage: scrape <url> [output.json]")
        return 1
    }
    let url = argv[1]
    let out = argv[2] ?? "links.json"
    print("fetching " + url + "...")
```

```
    let html = fetch(url)
    if html == nil { return 1 }
    let links = extract_links(html, url)
    print("found " + str(len(links)) + " links")
    json.save(out, links)
    print("saved to " + out)
    return 0
}

if __name__ == "__main__" {
    sys.exit(main(sys.argv))
}
```

### 50.11.3 Sample run

```
$ zen run scrape.z https://example.com links.json
fetching https://example.com...
found 3 links
saved to links.json
```

### 50.11.4 Ethical scraping

- Respect `robots.txt`.
- Add delays between requests (rate limiting).
- Identify yourself via a `User-Agent` header.
- Don’t scrape sites that forbid it.
- Only scrape data you’re allowed to use.

## 50.12 44.13 Requests usage recap

```
import requests
let r = requests.get("https://example.com", timeout=10)
print(r.status_code)
print(r.text)
let data = r.json()   # if JSON
```

POST JSON:

```
let r = requests.post("https://api.example.com/submit",
                      json={"name": "Zen", "value": 42})
```

## 50.13 44.14 Exercises

1. Scrape all `<h1>` headings from a page (regex-based).
2. Add a `User-Agent` header to requests.
3. Save page title and first paragraph instead of links.

4. Scrape multiple URLs from a list, one after another.
5. Add a delay (sleep) between requests to be polite.
6. Write results as JSON with url + links per entry.
7. Handle 404/500 responses gracefully.
8. Add `--output` flag for the destination file.
9. Extract links matching a specific domain only.
10. Build a tiny “link checker” that verifies each link returns 200.

# 51 Chapter 45: REST API Client

This chapter shows how to build a REST API client in Zen using `requests`, `json`, and `argparser`. It covers authentication, pagination, rate limiting, error handling, and building a reusable client class.

## 51.1 45.1 Basic GET request

```
import requests
import json

let resp = requests.get("https://api.example.com/users/1")
if resp.ok {
    let user = resp.json()
    print(user["name"])
} else {
    print("HTTP " + str(resp.status_code))
}
```

## 51.2 45.2 POST with JSON body

```
import requests
import json

let body = json.dumps({"name": "Zen", "version": 3})
let resp = requests.post("https://example.com/api/users", body, {
    "headers": {"Content-Type": "application/json"}
})
print(resp.status_code)
print(resp.text)
```

## 51.3 45.3 Reusable API client

```
import requests
import json

class ApiClient {
    let base_url
    let headers
    let session

    func init(base_url, headers = {}) {
        this.base_url = base_url
        this.headers = headers
        this.session = requests.Session()
        this.session.headers = headers
```

```
    }

    func get(path, params = {}) {
        let url = this.base_url + path
        let resp = requests.sess_get(this.session, url, {"params": params})
        return resp
    }

    func post(path, data) {
        let url = this.base_url + path
        let body = json.dumps(data)
        let resp = requests.sess_post(this.session, url, {
            "json": body,
            "headers": {"Content-Type": "application/json"}
        })
        return resp
    }

    func put(path, data) {
        let url = this.base_url + path
        let body = json.dumps(data)
        let resp = requests.sess_put(this.session, url, {
            "json": body,
            "headers": {"Content-Type": "application/json"}
        })
        return resp
    }

    func delete(path) {
        let url = this.base_url + path
        let resp = requests.sess_delete(this.session, url)
        return resp
    }

    func close() {
        requests.session_close(this.session)
    }
}
```

### 51.3.1 Usage

```
let client = new ApiClient("https://api.example.com", {
    "Authorization": "Bearer token123",
    "User-Agent": "ZenClient/1.0"
})
```

```
let resp = client.get("/users/1")
if resp.ok {
    print(resp.json()["name"])
}

let created = client.post("/users", {"name": "New User"})
print(created.status_code)

client.close()
```

## 51.4 45.4 Authentication

### 51.4.1 Bearer token

```
let client = new ApiClient("https://api.example.com", {
    "Authorization": "Bearer " + token
})
```

### 51.4.2 Basic auth

```
import requests

let resp = requests.get("https://example.com/api", {
    "auth": ["user", "pass"]
})
```

### 51.4.3 API key header

```
let client = new ApiClient("https://api.example.com", {
    "X-API-Key": "your-api-key"
})
```

## 51.5 45.5 Pagination

Many APIs paginate results. Here's a pattern for following `next` links:

```
import requests

func get_all_pages(url) {
    let all_items = []
    let next_url = url
    while next_url != nil {
        let resp = requests.get(next_url)
        if !resp.ok {
            break
        }
```

```
        let data = resp.json()
        for item in data["items"] {
            all_items.push(item)
        }
        next_url = data["next"]  // null when no more pages
    }
    return all_items
}

let items = get_all_pages("https://api.example.com/items?page=1")
print(len(items))
```

### 51.5.1 Offset-based pagination

```
func get_all_offset(base_url, page_size = 50) {
    let all_items = []
    let offset = 0
    while true {
        let resp = requests.get(base_url, {
            "params": {"offset": offset, "limit": page_size}
        })
        if !resp.ok {
            break
        }
        let data = resp.json()
        let items = data["items"]
        for item in items {
            all_items.push(item)
        }
        if len(items) < page_size {
            break
        }
        offset = offset + page_size
    }
    return all_items
}
```

## 51.6 45.6 Rate limiting

Respect `RateLimit` headers and retry on 429:

```
import requests
import time

func get_with_retry(url, max_retries = 3) {
    let attempt = 0
    while attempt < max_retries {
```

```
        let resp = requests.get(url)
        if resp.ok {
            return resp
        }
        if resp.status_code == 429 {
            let retry_after = resp.headers["Retry-After"]
            let wait = retry_after != nil ? int(retry_after) : 1
            print("Rate limited, waiting " + str(wait) + "s")
            time.sleep(wait)
            attempt = attempt + 1
        } else {
            break
        }
    }
    return nil
}

let resp = get_with_retry("https://api.example.com/data")
if resp != nil {
    print(resp.text)
}
```

## 51.7 45.7 Error handling

```
import requests

func api_get(client, path) {
    let resp = client.get(path)
    if resp.status_code == 404 {
        return nil  // not found
    }
    if !resp.ok {
        throw "API error: " + str(resp.status_code) + " " + resp.reason
    }
    return resp.json()
}

try {
    let user = api_get(client, "/users/999")
    if user == nil {
        print("User not found")
    } else {
        print(user["name"])
    }
} catch as e {
    print("API call failed: " + e.msg)
```

```
}
```

## 51.8 45.8 Complete CLI example

```
import requests
import json
import argparser
import fs

let parser = argparser.ArgumentParser("api-client")
parser.add_argument("url", help="API URL")
parser.add_argument("--method", "-m", default="GET", help="HTTP method")
parser.add_argument("--data", "-d", help="JSON body")
parser.add_argument("--header", "-H", action="append", help="Header (key:value)")
let args = parser.parse_args()

let headers = {}
if args.header != nil {
    for h in args.header {
        let parts = h.split(":", 1)
        if len(parts) == 2 {
            headers[parts[0]] = parts[1]
        }
    }
}

let body = args.data != nil ? args.data : nil

let resp = requests.get(args.url, {"headers": headers})
// ... extend for other methods

print(resp.status_code)
print(resp.text)
```

## 51.9 45.9 Common pitfalls

- **Always check `resp.ok`.** A non-200 response still returns a response object.
- **JSON bodies must be strings.** Use `json.dumps()` to serialize.
- **Sessions persist cookies and headers.** Close them with `requests.session_close()`.
- **Rate limits.** Respect `Retry-After` and back off on 429.
- **Timeouts.** Always pass a `timeout` option to avoid hanging.

## 51.10 45.10 Quick reference

```
import requests
import json
```

```
// GET
let resp = requests.get("https://api.example.com/items", {
    "params": {"page": 1},
    "headers": {"Authorization": "Bearer token"}
})

// POST JSON
let resp = requests.post("https://example.com/api", {
    "json": json.dumps({"name": "Zen"}),
    "headers": {"Content-Type": "application/json"}
})

// Session
let sess = requests.Session()
sess.headers = {"Authorization": "Bearer token"}
let resp = requests.sess_get(sess, "https://example.com/api")
requests.session_close(sess)

// Status
resp.status_code
resp.ok
resp.text
resp.json()
resp.headers
```

## 51.11 45.11 Exercises

1. Write a REST client for a public API (e.g., JSONPlaceholder) that lists posts.
2. Add pagination support to fetch all pages.
3. Add retry logic with exponential backoff.
4. Write a CLI that takes a URL and method and prints the response.
5. Build a small config client that reads/writes JSON config via REST.

## 51.12 Extended Project Ideas and Implementation Notes

### 51.12.1 Design checklist

- Requirements and scope
- Error handling
- Logging
- Configuration
- Testing
- Documentation
- Security considerations

### 51.12.2 Exercises

1. Extend the project with new features
2. Add tests
3. Improve error messages
4. Add help/usage
5. Make it configurable
6. Package for distribution

## 51.13 45.12 Full tutorial — REST API client

### 51.13.1 Goal

A small client for a JSON REST API with GET/POST/PUT/DELETE and auth.

### 51.13.2 Implementation (`api.z`)

```
import requests
import json

class ApiClient {
    var base_url = ""
    var token = nil

    func init(base_url, token = nil) {
        self.base_url = base_url
        self.token = token
    }

    func headers() {
        let h = #{}
        h["Content-Type"] = "application/json"
        if self.token != nil { h["Authorization"] = "Bearer " + self.token }
        return h
    }

    func request(method, path, body = nil) {
        let url = self.base_url + path
        try {
            let r = nil
            if method == "GET" { r = requests.get(url, headers=self.headers(), timeout=
            elif method == "POST" { r = requests.post(url, headers=self.headers(), jso
            elif method == "PUT" { r = requests.put(url, headers=self.headers(), json=b
            elif method == "DELETE" { r = requests.delete(url, headers=self.headers(),
            else { throw "unsupported method: " + method }
            r.raise_for_status()
            let ct = r.headers["Content-Type"] ?? ""
            if ct.contains("json") { return r.json() }
```

```
            return r.text
        } catch as e {
            throw "API request failed (" + method + " " + path + "): " + e.msg
        }
    }

    func get(path) { return self.request("GET", path) }
    func post(path, body) { return self.request("POST", path, body) }
    func put(path, body) { return self.request("PUT", path, body) }
    func delete(path) { return self.request("DELETE", path) }
}

func main(argv) {
    let client = new ApiClient("https://api.example.com", "my-token")
    try {
        let users = client.get("/users")
        print("users: " + json.stringify(users))
        let created = client.post("/users", #{name: "Zen User"})
        print("created: " + json.stringify(created))
    } catch as e {
        print(e.msg)
        return 1
    }
    return 0
}

if __name__ == "__main__" { sys.exit(main(sys.argv)) }
```

### 51.13.3 Key design points

- `base_url` + path composition.
- Bearer token auth header.
- Central error handling with context (method + path).
- Content-type check before `r.json()`.
- Timeout on every request (never hang forever).

## 51.14 45.13 REST client checklist

1. **Base URL** stored once; paths appended.
2. **Auth header** (Bearer/API key) added to every request.
3. **Timeouts** on all requests.
4. **`raise_for_status`** or manual status checks.
5. **Parse JSON only when Content-Type is JSON.**
6. **Wrap errors** with method + path context.
7. **Retry** transient failures (optional).
8. **Pagination** support (optional: follow `Link` header or `page` param).

## 51.15 45.14 Exercises

1. Add `patch` method to the client.
2. Add automatic retry (3 attempts) on 5xx errors.
3. Add pagination: follow `next` URL until none.
4. Add query-parameter support to `get`.
5. Add file upload (`files=` param) support.
6. Add a `--verbose` flag printing each request URL.
7. Implement OAuth2 bearer token refresh (conceptually).
8. Write tests against a local mock server.
9. Add rate-limit handling (respect `Retry-After`).
10. Turn the client into a reusable module `api_client.z`.

# 52 Chapter 46: File Processing Tools

This chapter shows how to process files in Zen: reading/writing text and binary, CSV, JSON, line-by-line processing, temporary files, and directory walking.

## 52.1 46.1 Reading text files

```
import fs

// Read entire file
let content = fs.read("data.txt")
print(content)

// Read line by line
let lines = fs.read("data.txt").split("\n")
for line in lines {
    if len(line) > 0 {
        print(line)
    }
}
```

## 52.2 46.2 Writing text files

```
import fs

// Write entire file (overwrites)
fs.write("output.txt", "Line 1\nLine 2\nLine 3\n")

// Append (read-modify-write)
let existing = ""
if fs.exists("log.txt") {
    existing = fs.read("log.txt")
}
fs.write("log.txt", existing + "new entry\n")
```

## 52.3 46.3 Binary files

```
import fs

// Read binary
let data = fs.read_binary("image.png")
print(len(data))   // byte count

// Write binary
fs.write_binary("copy.png", data)
```

```
// Copy a file
fs.copy("source.txt", "destination.txt")
```

## 52.4 46.4 CSV processing (manual)

```
import fs

func read_csv(path) {
    let lines = fs.read(path).split("\n")
    if len(lines) == 0 {
        return []
    }
    let header = lines[0].split(",")
    let rows = []
    for i in 1..len(lines) {
        if lines[i] == "" {
            continue
        }
        let values = lines[i].split(",")
        let row = {}
        for j in 0..len(header) {
            row[header[j]] = values[j]
        }
        rows.push(row)
    }
    return rows
}

let data = read_csv("data.csv")
for row in data {
    print(row["name"])
}
```

## 52.5 46.5 JSON files

```
import fs
import json

// Read JSON
let text = fs.read("config.json")
let config = json.loads(text)
print(config["theme"])

// Write JSON
let data = {"name": "Zen", "version": 3}
fs.write("config.json", json.dumps(data))
```

## 52.6 46.6 Line processing pipeline

```
import fs

func process_lines(path, func) {
    let lines = fs.read(path).split("\n")
    let results = []
    for line in lines {
        if len(line) > 0 {
            results.push(func(line))
        }
    }
    return results
}

let upper_lines = process_lines("data.txt", fn(line) {
    return line.upper()
})
for line in upper_lines {
    print(line)
}
```

## 52.7 46.7 Temporary files

```
import tempfile
import fs

let tmp = tempfile.mktemp()
fs.write(tmp, "temporary data")
let content = fs.read(tmp)
fs.remove(tmp)

// Or use a directory
let dir = tempfile.mkdtemp()
// ... use dir ...
fs.rmtree(dir)
```

## 52.8 46.8 Directory walking

```
import fs

func walk(dir, func) {
    for name in fs.list(dir) {
        let path = fs.join(dir, name)
```

```
        if fs.is_dir(path) {
            walk(path, func)
        } else {
            func(path)
        }
    }
}

walk(".", fn(path) {
    print(path)
})
```

### 52.9 46.9 File info

```
import fs

print(fs.exists("file.txt"))      # true/false
print(fs.is_file("file.txt"))     # true/false
print(fs.is_dir("dir"))           # true/false
print(fs.size("file.txt"))        // byte count
print(fs.join("a", "b", "c"))     // a/b/c
print(fs.basename("/a/b/c.txt"))  // c.txt
print(fs.dirname("/a/b/c.txt"))   // /a/b
```

### 52.10 46.10 Quick reference

```
import fs
import json
import tempfile

fs.read("file.txt")           # string
fs.read_binary("file.bin")    # bytes
fs.write("file.txt", "data")
fs.write_binary("file.bin", bytes)
fs.exists("file.txt")
fs.is_file("file.txt")
fs.is_dir("dir")
fs.size("file.txt")
fs.mkdirs("a/b/c")
fs.remove("file.txt")
fs.rmtree("dir")
fs.copy("src", "dst")
fs.move("old", "new")
fs.join("a", "b")
fs.basename("/a/b/c.txt")
fs.dirname("/a/b/c.txt")
```

```
fs.list(".")                  # directory entries

let tmp = tempfile.mktemp()
fs.write(tmp, "data")
fs.remove(tmp)

let dir = tempfile.mkdtemp()
fs.rmtree(dir)
```

## 52.11 46.11 Exercises

1. Write `file_size_human(path)` that returns a human-readable file size.
2. Write a script that copies all `.txt` files from one directory to another.
3. Write `grep_file(pattern, path)` that prints lines containing `pattern`.
4. Write a file tailer that reads the last `n` lines of a file.
5. Write `csv_to_json(path)` that converts a CSV file to JSON.
6. Write `count_lines(path)` that counts non-empty lines in a file.
7. Write `merge_files(paths, output)` that concatenates multiple files into one.

## 52.12 Extended Project Ideas and Implementation Notes

### 52.12.1 Design checklist

- Requirements and scope
- Error handling
- Logging
- Configuration
- Testing
- Documentation
- Security considerations

### 52.12.2 Exercises

1. Extend the project with new features
2. Add tests
3. Improve error messages
4. Add help/usage
5. Make it configurable
6. Package for distribution

## 52.13 46.12 Full tutorial — file processing pipeline

### 52.13.1 Goal

Read a CSV-like file, transform each row, write results to another file.

### 52.13.2 Implementation (`process.z`)

```
import os
import pathlib

func read_lines(path) {
    let lines = []
    with open(path, "r") as f {
        for line in f {
            lines.push(line)
        }
    }
    return lines
}

func parse_csv_line(line) {
    return line.strip().split(",")
}

func transform(row) {
    # row: ["name", "age", "city"]
    if len(row) < 3 { return nil }
    return #{name: row[0], age: int(row[1]), city: row[2]}
}

func format_record(r) {
    return r["name"] + "," + str(r["age"]) + "," + r["city"] + "\n"
}

func process(input_path, output_path) {
    let lines = read_lines(input_path)
    let results = []
    for line in lines {
        let row = parse_csv_line(line)
        let r = transform(row)
        if r != nil { results.push(format_record(r)) }
    }
    with open(output_path, "w") as f {
        for r in results { f.write(r) }
    }
    print("processed " + str(len(results)) + " records -> " + output_path)
}

func main(argv) {
    if len(argv) < 3 {
        print("usage: process <input.csv> <output.csv>")
        return 1
```

```
    }
    process(argv[1], argv[2])
    return 0
}

if __name__ == "__main__" { sys.exit(main(sys.argv)) }
```

#### 52.13.3 Sample data (`input.csv`)

```
Ada,36,London
Alan,42,Manchester
Grace,85,New York
```

#### 52.13.4 Sample run

```
$ zen run process.z input.csv output.csv
processed 3 records -> output.csv
```

#### 52.13.5 Design notes

- `with` ensures the file closes even on error.
- `transform` returns `nil` on malformed rows; the pipeline skips them.
- Output written only after all rows transform successfully (simple).

### 52.14 46.13 File processing patterns

| Pattern | Use |
|---|---|
| Read → transform → write | ETL pipeline |
| Stream line-by-line | large files (low memory) |
| Chunked binary copy | large binary files |
| Filter + aggregate | reports/summaries |
| Parse-validate-load | data import |

### 52.15 46.14 Exercises

1. Add a header row detection (skip first line).
2. Support a delimiter flag (`,`, `\t`, `;`).
3. Filter rows where `age > 40`.
4. Sort output by age.
5. Write a summary line: total, min, max, average age.
6. Handle missing input file gracefully.
7. Write output incrementally (streaming) instead of collecting all.
8. Support `.tsv` input automatically by extension.
9. Add a `--dry-run` flag that prints without writing.
10. Process all `.csv` files in a directory.

# 53 Chapter 47: Network Security Tools

*Note: This chapter is for educational and ethical purposes only. Always get proper authorization before testing network security.*

## 53.1 47.1 Port Scanner

```
import socket

func scan_port(host, port, timeout) {
    try {
        let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
        socket.settimeout(sock, timeout)
        socket.connect(sock, host, port)
        socket.close(sock)
        return true
    } catch {
        return false
    }
}

func scan_ports(host, start_port, end_port) {
    print("Scanning " + host + " from port " + str(start_port) + " to " + str(end_port)
    let open_ports = []
    let port = start_port
    while port <= end_port {
        if scan_port(host, port, 500) {
            print("Port " + str(port) + " is OPEN")
            open_ports.push(port)
        }
        port = port + 1
    }
    return open_ports
}

// Usage (only scan hosts you own or have permission to scan)
// let open = scan_ports("127.0.0.1", 1, 1024)
```

## 53.2 44. Subdomain Finder

```
import dns
import requests

func find_subdomains(domain, wordlist) {
    let found = []
    for sub in wordlist {
```

```
        let hostname = sub + "." + domain
        try {
            let ips = dns.resolve(hostname)
            print("Found: " + hostname + " -> " + ips[0])
            found.push({"subdomain": hostname, "ips": ips})
        } catch {
            # Not found
        }
    }
    return found
}

let wordlist = ["www", "mail", "ftp", "api", "admin", "test", "dev"]
// find_subdomains("example.com", wordlist)
```

### 53.3 44. HTTP Header Checker

```
import requests

func check_headers(url) {
    try {
        let resp = requests.get(url)
        print("Security Headers for " + url)
        print("=" * 40)

        let security_headers = [
            "Strict-Transport-Security",
            "Content-Security-Policy",
            "X-Frame-Options",
            "X-Content-Type-Options",
            "Referrer-Policy"
        ]

        for h in security_headers {
            let val = resp.headers[h]
            if val != nil {
                print(h + ": " + val)
            } else {
                print(h + ": MISSING")
            }
        }
    } catch as e {
        print("Error: " + e.msg)
    }
}
```

```
// check_headers("https://example.com")
```

## 53.4 44.5 Port Scan Report

```
import socket
import fs

func scan_and_report(host, start, end, output_file) {
    let open_ports = []
    for port in start..end {
        try {
            let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
            socket.settimeout(sock, 300)
            socket.connect(sock, host, port)
            socket.close(sock)
            open_ports.push(port)
            print("OPEN: " + str(port))
        } catch {
            # closed or filtered
        }
    }

    let report = "# Scan Report for " + host + "\n"
    report = report + "Open ports: " + str(len(open_ports)) + "\n"
    for p in open_ports {
        report = report + "  - " + str(p) + "\n"
    }

    fs.write(output_file, report)
    print("Report written to " + output_file)
}

// scan_and_report("127.0.0.1", 1, 1000, "scan_report.txt")
```

## 53.5 44.6 HTTP Header Analysis

```
import requests

func analyze_security(url) {
    try {
        let resp = requests.get(url, {"timeout": 10})

        print("URL: " + url)
        print("Status: " + str(resp.status_code))
        print("")
```

```
        let headers = [
            "Strict-Transport-Security",
            "Content-Security-Policy",
            "X-Frame-Options",
            "X-Content-Type-Options",
            "X-XSS-Protection",
            "Referrer-Policy",
            "Permissions-Policy"
        ]

        for h in headers {
            let val = resp.headers[h]
            if val == nil {
                print("[MISSING] " + h)
            } else {
                print("[OK] " + h + ": " + val)
            }
        }

        // Check for server header disclosure
        let server = resp.headers["Server"]
        if server != nil {
            print("[INFO] Server header: " + server)
        }

    } catch as e {
        print("Error: " + e.msg)
    }
}

// analyze_security("https://example.com")
```

## 53.6 44.7 DNS Reconnaissance

```
import dns

func dns_recon(domain) {
    print("DNS Recon for " + domain)
    print("")

    // A record
    try {
        let addrs = dns.resolve(domain)
        print("A records:")
        for a in addrs {
            print("  " + a)
```

```
        }
    } catch {
        print("No A records found")
    }

    // MX record (if supported)
    // ...

    print("")
    print("FQDN: " + dns.getfqdn(domain))
}

// dns_recon("example.com")
```

## 53.7 44.8 Common pitfalls

- **Only scan hosts you own or have explicit written permission to test.**
- **Rate-limit your requests** to avoid overwhelming the target.
- **TLS certificate errors** may occur with self-signed certs.
- **Some WAFs block automated scanning.**

## 53.8 44.9 Quick reference

```
import socket
import dns
import requests

// Port scan
socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.connect(sock, host, port)
socket.close(sock)

// DNS resolve
dns.resolve(domain)

// HTTP headers
requests.get(url)
resp.headers[header_name]
```

## 53.9 44.10 Exercises

1. Write a port scanner that scans common ports (21, 22, 25, 53, 80, 443, 3306, 3389).
2. Write a subdomain enumerator with a custom wordlist.
3. Write a security header checker for a list of URLs.
4. Write a DNS reconnaissance script that gathers A, MX, and NS records.

5. Write a scan report generator that saves results to a file.

## 53.10 Extended Project Ideas and Implementation Notes

### 53.10.1 Design checklist

- Requirements and scope
- Error handling
- Logging
- Configuration
- Testing
- Documentation
- Security considerations

### 53.10.2 Exercises

1. Extend the project with new features
2. Add tests
3. Improve error messages
4. Add help/usage
5. Make it configurable
6. Package for distribution

## 53.11 47.12 Full tutorial — hash utility (ethical security tool)

### 53.11.1 Goal

Compute and compare file hashes (SHA-256) to verify integrity.

### 53.11.2 Implementation (`hashutil.z`)

```
import hashlib
import sys
import pathlib

func hash_file(path) {
    let b = pathlib.Path(path)
    if !b.exists() { throw "file not found: " + path }
    # read in chunks for large files
    let data = []
    with open(path, "rb") as f {
        while true {
            let chunk = f.read(8192)
            if chunk == nil || len(chunk) == 0 { break }
            data.push(chunk)
        }
    }
    # note: hashlib.sha256 expects a string in this build; for real binary
```

```
    # hashing, feed bytes appropriately per stdlib API.
    let joined = data.join("")   # if bytes join supported
    return hashlib.sha256(joined)
}

func main(argv) {
    if len(argv) < 2 {
        print("usage: hashutil <file> [expected_hash]")
        return 1
    }
    let path = argv[1]
    let expected = argv[2] ?? nil
    try {
        let actual = hash_file(path)
        print("SHA-256(" + path + ") = " + actual)
        if expected != nil {
            if actual == expected {
                print("MATCH: file integrity verified")
                return 0
            } else {
                print("MISMATCH: file may be corrupted or tampered")
                return 1
            }
        }
    } catch as e {
        print("error: " + e.msg)
        return 1
    }
    return 0
}

if __name__ == "__main__" { sys.exit(main(sys.argv)) }
```

### 53.11.3 Sample run

```
$ zen run hashutil.z document.pdf
SHA-256(document.pdf) = a3f5...c9
$ zen run hashutil.z document.pdf a3f5...c9
SHA-256(document.pdf) = a3f5...c9
MATCH: file integrity verified
```

### 53.11.4 Ethical use

- Only hash files you own or have permission to analyze.
- Hashing verifies integrity, not “hacking.”
- Don’t use MD5/SHA-1 for security; prefer SHA-256.

## 53.12 47.13 Security tool principles

1. **Authorization:** only operate on systems/data you're permitted to test.
2. **Integrity over secrecy:** hashing confirms a file hasn't changed.
3. **Use strong algorithms:** SHA-256+, Argon2id, AES-GCM.
4. **Never log secrets:** avoid printing keys/passwords/tokens.
5. **Constant-time comparison** for secrets (avoid timing leaks).
6. **Input validation:** sanitize file paths, URLs, and user input.

## 53.13 47.14 Exercises

1. Add MD5 and SHA-1 options (note: not for security).
2. Compare two files and report if identical.
3. Hash all files in a directory and print a manifest.
4. Verify a file against a manifest file (hash per line).
5. Add a `--quiet` flag printing only MATCH/MISMATCH.
6. Implement constant-time string compare (conceptually).
7. Hash a string input (not file) with SHA-256.
8. Add `pbkdf2`-based password hashing (from crypto module).
9. Create a "file changed" watcher: hash, wait, re-hash, report diff.
10. Document ethical boundaries in a README for the tool.

# 54 Chapter 48: Capstone Projects

This chapter presents complete, end-to-end projects that combine multiple Zen features: pymport, web scraping, file processing, security tools, and CLI building.

## 54.1 48.1 Capstone 1: Web Metadata Archiver

A tool that fetches a webpage, extracts metadata (title, description, images, links), and saves it as a JSON file.

```
import requests
import bs4
import json
import fs
import argparser
import sys

func archive_metadata(url, output_file) {
    let resp = requests.get(url, {
        "headers": {"User-Agent": "Zen-Archiver/1.0"},
        "timeout": 30
    })

    if !resp.ok {
        throw "Failed to fetch " + url + ": HTTP " + str(resp.status_code)
    }

    let soup = bs4.BeautifulSoup(resp["text"], "html.parser")

    let metadata = {
        "url": url,
        "title": "",
        "description": "",
        "images": [],
        "links": []
    }

    // Title
    let title_tag = soup.find("title")
    if title_tag != null {
        metadata["title"] = title_tag.string
    }

    // Meta description
    let desc_tag = soup.find("meta", {"name": "description"})
    if desc_tag != null {
        metadata["description"] = desc_tag.get("content", "")
    }
```

```
    // Images
    let img_tags = soup.find_all("img")
    for img in img_tags {
        let src = img.get("src", "")
        if len(src) > 0 {
            metadata["images"].push(src)
        }
    }

    // Links
    let link_tags = soup.find_all("a")
    for link in link_tags {
        let href = link.get("href", "")
        if len(href) > 0 {
            metadata["links"].push(href)
        }
    }

    fs.write(output_file, json.dumps(metadata, {"indent": 2}))
    print("Metadata saved to " + output_file)
    print("Title: " + metadata["title"])
    print("Images: " + str(len(metadata["images"])))
    print("Links: " + str(len(metadata["links"])))
}

let parser = argparser.ArgumentParser("archive-metadata")
parser.add_argument("url", help="URL to archive")
parser.add_argument("--output", "-o", default="metadata.json", help="Output file")
let args = parser.parse_args()

try {
    archive_metadata(args.url, args.output)
} catch as e {
    fs.write("/dev/stderr", "Error: " + e.msg + "\n")
    sys.exit(1)
}
```

## 54.2 48.2 Capstone 2: Password Strength Checker

A tool that checks password strength using entropy calculation and common password lists.

```
import fs
import math
import random
import argparser
```

```
func password_strength(password) {
    let score = 0
    let reasons = []

    let length = len(password)
    if length >= 8 {
        score = score + 1
    } else {
        reasons.push("Password too short (min 8)")
    }

    let has_lower = false
    let has_upper = false
    let has_digit = false
    let has_special = false

    for ch in password {
        let code = ord(ch)
        if code >= 97 and code <= 122 {
            has_lower = true
        } elif code >= 65 and code <= 90 {
            has_upper = true
        } elif code >= 48 and code <= 57 {
            has_digit = true
        } else {
            has_special = true
        }
    }

    if has_lower { score = score + 1 } else { reasons.push("Add lowercase") }
    if has_upper { score = score + 1 } else { reasons.push("Add uppercase") }
    if has_digit { score = score + 1 } else { reasons.push("Add digit") }
    if has_special { score = score + 1 } else { reasons.push("Add special char") }

    let strength = ["Very Weak", "Weak", "Fair", "Good", "Strong"]
    let level = strength[min(score, 4)]

    return {"score": score, "level": level, "reasons": reasons}
}

func generate_password(length) {
    let chars = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#$%^&*
    let pw = []
    for i in 1..length {
        pw.push(random.choice(chars))
    }
    return "".join(pw)
```

```
}

let parser = argparser.ArgumentParser("password-checker")
parser.add_argument("password", nargs="?", help="Password to check")
parser.add_argument("--generate", "-g", type="int", help="Generate random password of l
let args = parser.parse_args()

if args.generate != nil {
    let pw = generate_password(args.generate)
    let strength = password_strength(pw)
    print("Generated: " + pw)
    print("Strength: " + strength["level"] + " (" + str(strength["score"]) + "/5)")
} elif args.password != nil {
    let strength = password_strength(args.password)
    print("Password: " + args.password)
    print("Strength: " + strength["level"] + " (" + str(strength["score"]) + "/5)")
    if len(strength["reasons"]) > 0 {
        print("Suggestions:")
        for r in strength["reasons"] {
            print("  - " + r)
        }
    }
} else {
    parser.print_help()
}
```

## 54.3 48.3 Capstone 3: Log Analyzer

A tool that parses web server logs and produces summary statistics.

```
import fs
import re
import argparser

func analyze_log(path) {
    let content = fs.read(path)
    let lines = content.split("\n")

    let total = 0
    let status_codes = {}
    let ips = {}
    let paths = {}

    let log_re = re.compile("^(\\S+) \\S+ \\S+ \\[.*\\] \"(\\S+) (\\S+) \\S+\" (\\d{3})

    for line in lines {
        if len(line) == 0 {
```

```
            continue
        }
        total = total + 1

        let m = log_re.search(line)
        if m != nil {
            let ip = m.group(1)
            let method = m.group(2)
            let path = m.group(3)
            let status = m.group(4)

            ips[ip] = (ips.get(ip, 0) + 1)
            paths[path] = (paths.get(path, 0) + 1)
            status_codes[status] = (status_codes.get(status, 0) + 1)
        }
    }

    return {
        "total": total,
        "status_codes": status_codes,
        "top_ips": sort_by_value(ips, 10),
        "top_paths": sort_by_value(paths, 10)
    }
}

func sort_by_value(dict, n) {
    let items = []
    for k in dict.keys() {
        items.push([k, dict[k]])
    }
    // Simple bubble sort by value descending
    for i in 0..len(items) {
        for j in 0..(len(items) - i - 1) {
            if items[j][1] < items[j + 1][1] {
                let tmp = items[j]
                items[j] = items[j + 1]
                items[j + 1] = tmp
            }
        }
    }
    let result = []
    for i in 0..min(n, len(items)) {
        result.push(items[i])
    }
    return result
}
```

```
let parser = argparser.ArgumentParser("log-analyzer")
parser.add_argument("file", help="Log file to analyze")
let args = parser.parse_args()

try {
    let result = analyze_log(args.file)
    print("Log Analysis Report")
    print("=" * 40)
    print("Total requests: " + str(result["total"]))
    print("")
    print("Status codes:")
    for code in result["status_codes"].keys() {
        print("  " + code + ": " + str(result["status_codes"][code]))
    }
    print("")
    print("Top IPs:")
    for item in result["top_ips"] {
        print("  " + item[0] + ": " + str(item[1]))
    }
    print("")
    print("Top paths:")
    for item in result["top_paths"] {
        print("  " + item[0] + ": " + str(item[1]))
    }
} catch as e {
    fs.write("/dev/stderr", "Error: " + e.msg + "\n")
    sys.exit(1)
}
```

## 54.4 48.4 Capstone 4: Site Uptime Monitor

A tool that checks multiple URLs and reports which are down.

```
import requests
import fs
import argparser
import time

func check_url(url, timeout) {
    try {
        let resp = requests.get(url, {"timeout": timeout})
        return {
            "url": url,
            "status": resp.status_code,
            "ok": resp.ok,
            "error": nil
        }
```

```
    } catch as e {
        return {
            "url": url,
            "status": nil,
            "ok": false,
            "error": e.msg
        }
    }
}

func monitor(urls, timeout) {
    let results = []
    for url in urls {
        let result = check_url(url, timeout)
        results.push(result)
        let status = result["ok"] ? "UP" : "DOWN"
        print(url + ": " + status + (result["error"] != nil ? " (" + result["error"] +
    }
    return results
}

let parser = argparser.ArgumentParser("uptime-monitor")
parser.add_argument("urls", nargs="*", help="URLs to monitor")
parser.add_argument("--timeout", "-t", type="int", default=10, help="Timeout in seconds
let args = parser.parse_args()

if len(args.urls) == 0 {
    print("No URLs to monitor")
    sys.exit(1)
}

let results = monitor(args.urls, args.timeout)

let up_count = 0
for r in results {
    if r["ok"] {
        up_count = up_count + 1
    }
}
print("")
print("Summary: " + str(up_count) + "/" + str(len(results)) + " sites up")

// Save report
let report = json.dumps(results, {"indent": 2})
fs.write("uptime_report.json", report)
print("Report saved to uptime_report.json")
```

## 54.5 48.5 Running the Capstones

```
# Metadata archiver
cd book-v2/tests
export PYTHONPATH="$(../.venv/bin/python -c 'import sysconfig; print(sysconfig.get_path
export ZEN_NO_PY_BOOTSTRAP=1
ZEN=../../../target/release/zen

# Note: capstone scripts are examples; adapt paths as needed
# $ZEN run projects/capstone/archive_metadata.z https://example.com --output meta.json
```

## 54.6 48.6 Quick reference

```
# Web metadata archiver: requests + bs4 + json + fs
# Password checker: math + random + argparser + fs
# Log analyzer: fs + re + argparser
# Uptime monitor: requests + fs + argparser + json
```

## 54.7 48.7 Exercises

1. Extend the metadata archiver to also extract <h1> tags and meta keywords.
2. Add rate limiting to the uptime monitor (wait between checks).
3. Add email/Slack notifications to the uptime monitor when a site goes down.
4. Extend the password checker to check against a common-passwords list.
5. Add HTML report generation to the log analyzer.

## 54.8 Extended Project Ideas and Implementation Notes

### 54.8.1 Design checklist

- Requirements and scope
- Error handling
- Logging
- Configuration
- Testing
- Documentation
- Security considerations

### 54.8.2 Exercises

1. Extend the project with new features
2. Add tests
3. Improve error messages
4. Add help/usage
5. Make it configurable
6. Package for distribution

## 54.9 48.12 Capstone — personal network toolkit

### 54.9.1 Goal

Combine networking, CLI, file, and crypto into one tool: `pkit <command>` with subcommands: - `get <url>` — fetch a page (HTTP) - `hash <file>` — SHA-256 of a file - `scan <host> <port>` — check if a TCP port is open - `totp <secret>` — generate a TOTP token - `genkey` — generate a secure random token

### 54.9.2 Implementation (`pkit.z`)

```
import sys
import requests
import hashlib
import socket
import crypto

func cmd_get(args) {
    if len(args) < 1 { print("usage: pkit get <url>"); return 1 }
    try {
        let r = requests.get(args[0], timeout=10)
        print("status=" + str(r.status_code))
        print(r.text[0:500])
    } catch as e {
        print("error: " + e.msg)
        return 1
    }
    return 0
}

func cmd_hash(args) {
    if len(args) < 1 { print("usage: pkit hash <file>"); return 1 }
    try {
        print(hashlib.sha256(args[0]))
    } catch as e {
        print("error: " + e.msg)
        return 1
    }
    return 0
}

func cmd_scan(args) {
    if len(args) < 2 { print("usage: pkit scan <host> <port>"); return 1 }
    let host = args[0]
    let port = int(args[1])
    let s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(2.0)
    try {
```

```
        s.connect((host, port))
        print(host + ":" + str(port) + " OPEN")
    } catch as e {
        print(host + ":" + str(port) + " closed/unreachable (" + e.msg + ")")
    } finally {
        s.close()
    }
    return 0
}

func cmd_totp(args) {
    if len(args) < 1 { print("usage: pkit totp <secret>"); return 1 }
    print(crypto.totp(args[0]))
    return 0
}

func cmd_genkey(_) {
    print(crypto.secure_token())
    return 0
}

func main(argv) {
    if len(argv) < 2 {
        print("pkit - personal network toolkit")
        print("commands: get <url> | hash <file> | scan <host> <port> | totp <secret> |
        return 1
    }
    let cmd = argv[1]
    let rest = []
    for i in 2..len(argv) { rest.push(argv[i]) }
    if cmd == "get" { return cmd_get(rest) }
    elif cmd == "hash" { return cmd_hash(rest) }
    elif cmd == "scan" { return cmd_scan(rest) }
    elif cmd == "totp" { return cmd_totp(rest) }
    elif cmd == "genkey" { return cmd_genkey(rest) }
    else { print("unknown command: " + cmd); return 1 }
}

if __name__ == "__main__" { sys.exit(main(sys.argv)) }
```

### 54.9.3 Sample session

```
$ zen run pkit.z genkey
aB3xK9mP2vR5wL8n
$ zen run pkit.z hash pkit.z
e3b0c442...  (SHA-256)
```

```
$ zen run pkit.z scan localhost 80
localhost:80 closed/unreachable (ConnectionRefusedError)
$ zen run pkit.z totp JBSWY3DPEHPK3PXP
742193
$ zen run pkit.z get https://example.com
status=200
<!doctype html>...
```

### 54.9.4 Extending the capstone

- Add `post <url> <json>` for POST requests.
- Add `resolve <host>` for DNS lookup.
- Add `--timeout` flag globally.
- Add config file for default settings.
- Add colors/progress indicators.
- Add tests for each subcommand.

## 54.10 48.13 Capstone lessons

1. **Compose** modules (requests, socket, hashlib, crypto, sys).
2. **Dispatch** on subcommands cleanly.
3. **Handle errors** per subcommand without crashing the whole tool.
4. **Use timeouts** on all network calls.
5. **Keep each subcommand small** and focused.
6. **Document** usage in the `main` help text.

## 54.11 48.14 Exercises

1. Add `post` subcommand sending JSON.
2. Add `resolve <host>` using the dns module.
3. Add a global `--timeout N` flag.
4. Add `ports <host> <start>-<end>` scanning a range.
5. Add `--output <file>` to save results.
6. Add colors to output (ANSI escape codes).
7. Write tests verifying `hash` output matches `hashlib.sha256`.
8. Add `scan` concurrent port scanning (threading).
9. Create a config file `pkit.toml` with defaults.
10. Build and distribute with `zen build pkit.z -o pkit`.

# 55 Appendix A: Built-in Functions

Zen's built-in functions are always available without importing. They cover type conversion, iteration, introspection, and basic I/O.

## 55.1 A.1 Type conversion

| Function | Description | Example | Result |
|---|---|---|---|
| `int(x)` | Convert to integer | `int("42")` | `42` |
| `float(x)` | Convert to float | `float("3.14")` | `3.14` |
| `str(x)` | Convert to string | `str(42)` | `"42"` |
| `bool(x)` | Convert to boolean | `bool(1)` | `true` |
| `list(x)` | Convert to list | `list("abc")` | `["a", "b", "c"]` |
| `tuple(x)` | Convert to tuple | `tuple([1, 2])` | `#(1, 2)` |
| `set(x)` | Convert to set | `set([1, 1, 2])` | `{1, 2}` |
| `dict(x)` | Convert from pairs | `dict([["a", 1]])` | `{"a": 1}` |

```
print(int("42"))      # 42
print(float("3.14"))  # 3.14
print(str(42))        # "42"
print(bool(0))        # false
print(list("abc"))    # ["a", "b", "c"]
print(tuple([1, 2]))  # #(1, 2)
print(set([1, 1, 2])) # {1, 2}
print(dict([["a", 1]])) # {"a": 1}
```

String-to-number parsing failures raise `ValueError`:

```
try {
    int("not a number")
} catch as e {
    print(e.msg)
}
```

## 55.2 A.2 Iteration

| Function | Description | Example |
|---|---|---|
| `len(x)` | Length | `len("abc")` → `3` |
| `range(stop)` | Range 0..stop-1 | `range(5)` → `0..4` |
| `range(start, stop)` | Range start..stop-1 | `range(2, 5)` → `2..4` |
| `range(start, stop, step)` | Range with step | `range(0, 10, 2)` → `0..8` |
| `enumerate(iter)` | Index + value | `enumerate(["a","b"])` → `[[0,"a"],[1,"b"]]` |

| Function | Description | Example |
|---|---|---|
| `zip(*iterables)` | Aggregate | `zip([1,2], ["a","b"])` → `[[1,"a"],[2,"b"]]` |
| `sorted(iter)` | Sorted list | `sorted([3,1,2])` → `[1,2,3]` |
| `reversed(iter)` | Reversed | `reversed([1,2,3])` → `[3,2,1]` |
| `any(iter)` | Any true | `any([false, true])` → `true` |
| `all(iter)` | All true | `all([true, true])` → `true` |
| `sum(iter)` | Sum | `sum([1,2,3])` → `6` |
| `min(iter)` | Minimum | `min([3,1,2])` → `1` |
| `max(iter)` | Maximum | `max([3,1,2])` → `3` |
| `map(func, iter)` | Map | `map(lambda x: x*2, [1,2])` → `[2,4]` |
| `filter(func, iter)` | Filter | `filter(lambda x: x>1, [1,2,3])` → `[2,3]` |

```
print(len("hello"))   # 5
print(list(range(5))) # [0, 1, 2, 3, 4]
print(list(range(2, 5))) # [2, 3, 4]
print(list(range(0, 10, 2))) # [0, 2, 4, 6, 8]
print(list(enumerate(["a","b"]))) # [[0,"a"],[1,"b"]]
print(list(zip([1,2], ["a","b"]))) # [[1,"a"],[2,"b"]]
print(sorted([3,1,2])) # [1, 2, 3]
print(reversed([1,2,3])) # [3, 2, 1]
print(any([false, true])) # true
print(all([true, true])) # true
print(sum([1,2,3]))   # 6
print(min([3,1,2]))   # 1
print(max([3,1,2]))   # 3
print(list(map(lambda x: x*2, [1,2]))) # [2, 4]
print(list(filter(lambda x: x>1, [1,2,3]))) # [2, 3]
```

## 55.3 A.3 Introspection

| Function | Description | Example |
|---|---|---|
| `typeof(x)` | Type name | `typeof(42)` → `"int"` |
| `callable(x)` | Is callable | `callable(print)` → `true` |
| `id(x)` | Object identity | `id(42)` |

```
print(typeof(42))     # "int"
print(typeof("hi"))   # "string"
print(callable(print)) # true
print(callable(42))   # false
print(id(42))         # some integer
```

## 55.4 A.4 I/O

| Function | Description | Example |
|---|---|---|
| `print(*args)` | Print to stdout | `print("hello", 42)` |
| `input(prompt?)` | Read stdin | `input("name: ")` |
| `exit(code?)` | Exit process | `exit(0)` |

```
print("hello", 42)     # hello 42
// let name = input("name: ")
exit(0)
```

## 55.5 A.5 Truthiness

Everything is truthy except `nil` and `false`:

```
print(bool(nil))    # false
print(bool(false))  # false
print(bool(0))      # true   (unlike Python)
print(bool(""))     # true   (unlike Python)
print(bool([]))     # true   (unlike Python)
print(bool({}))     # true   (unlike Python)
print(bool(42))     # true
print(bool("hello")) # true
print(bool([1, 2])) # true
```

Use `bool(x)` to coerce explicitly.

## 55.6 A.6 Quick reference

```
# Type conversion
int("42")      # 42
float("3.14")  # 3.14
str(42)        # "42"
bool(0)        # false
list("abc")    # ["a", "b", "c"]
tuple([1, 2])  # #(1, 2)
set([1, 1, 2]) # {1, 2}
dict([["a", 1]]) # {"a": 1}
```

```
# Iteration
len("hello")     # 5
list(range(5))   # [0, 1, 2, 3, 4]
list(enumerate(["a","b"])) # [[0,"a"],[1,"b"]]
list(zip([1,2], ["a","b"])) # [[1,"a"],[2,"b"]]
sorted([3,1,2])  # [1, 2, 3]
reversed([1,2,3]) # [3, 2, 1]
any([false, true]) # true
all([true, true]) # true
sum([1,2,3])     # 6
min([3,1,2])     # 1
max([3,1,2])     # 3

# Introspection
typeof(42)     # "int"
callable(print) # true
id(42)         # some integer

# I/O
print("hello", 42)
exit(0)
```

## 55.7 A.7 Exercises

1. Write `safe_int(s, default=0)` that parses an int or returns default.
2. Write `chunk_list(lst, n)` that splits a list into chunks of size n using range and slicing.
3. Write `interleave(a, b)` using zip.
4. Write `all_unique(lst)` using set and len.
5. Write `flatten_dict(d)` that flattens a nested dict using recursion and dict methods.

## 55.8 A.12 Complete built-in function reference

All built-ins are available without importing.

| Function | Category | Purpose | Example | Result |
|---|---|---|---|---|
| `print(*vals)` | I/O | print values to stdout | `print("hi", 42)` | hi 42 |
| `str(x)` | conversion | to string | `str(42)` | `"42"` |
| `int(x)` | conversion | to integer | `int("42")` | `42` |
| `float(x)` | conversion | to float | `float("3.14")` | `3.14` |
| `bool(x)` | conversion | to boolean | `bool(0)` | `false` |
| `list(x)` | conversion | to list | `list("ab")` | `["a","b"]` |
| `tuple(x)` | conversion | to tuple | `tuple([1,2])` | `#(1,2)` |
| `set(x)` | conversion | to set | `set([1,1,2])` | `{1,2}` |

| Function | Category | Purpose | Example | Result |
|---|---|---|---|---|
| `dict(x)` | conversion | from pairs | `dict([["a",1]])` | `{"a":1}` |
| `len(x)` | iteration | length | `len("abc")` | `3` |
| `range(...)` | iteration | range | `range(5)` | `0..4` |
| `enumerate(it)` | iteration | index+value | `enumerate(["a","b"])` | `[[0,"a"],[1,"b"]]` |
| `zip(...)` | iteration | aggregate | `zip([1,2],["a","b"])` | `[[1,"a"],[2,"b"]]` |
| `sorted(x)` | iteration | sorted list | `sorted([3,1,2])` | `[1,2,3]` |
| `reversed(x)` | iteration | reversed | `reversed([1,2,3])` | `[3,2,1]` |
| `any(it)` | iteration | any true | `any([f,t])` | `true` |
| `all(it)` | iteration | all true | `all([t,t])` | `true` |
| `sum(it)` | math | sum | `sum([1,2,3])` | `6` |
| `min(it/a,b)` | math | minimum | `min([3,1,2])` | `1` |
| `max(it/a,b)` | math | maximum | `max([3,1,2])` | `3` |
| `abs(x)` | math | absolute value | `abs(-5)` | `5` |
| `typeof(x)` | introspection | type name | `typeof(42)` | `"int"` |
| `type(x)` | introspection | type (alias) | `type("x")` | `"string"` |
| `chr(n)` | string | int to char | `chr(65)` | `"A"` |
| `ord(s)` | string | char to int | `ord("A")` | `65` |
| `exit(n)` | system | exit process | `exit(0)` | exits |
| `open(path, mode)` | I/O | open file | `open("f.txt")` | file handle |
| `assert(cond, msg?)` | debugging | assert condition | `assert x > 0` | raises if false |
| `repr(x)` | debugging | developer repr | `repr("hi")` | `"\"hi\""` |

### 55.8.1 Type conversion details

```
print(int("42"))      # 42
print(float("3.14"))  # 3.14
print(str(42))        # "42"
print(bool(0))        # false
print(list("abc"))    # ["a","b","c"]
print(tuple([1, 2]))  # #(1, 2)
print(set([1, 1, 2])) # {1, 2}
print(dict([["a", 1]])) # {"a": 1}
```

Parsing failures raise `ValueError` (Z0305):

```
try { int("not a number") } catch as e { print(e.msg) }
```

### 55.8.2 Truthiness reference

Falsy: `false`, `nil`, `null`. Truthy: everything else — including `0`, `0.0`, `""`, `[]`, `#{}`, `%{}`, `#()`.

```
print(bool(0))      # true
print(bool(""))     # true
```

```
print(bool([]))     # true
print(bool(false))  # false
print(bool(nil))    # false
```

### 55.8.3 `ord/chr`

```
print(ord("A"))   # 65
print(chr(65))    # "A"
```

### 55.8.4 `assert`

```
assert x > 0, "x must be positive"
```

### 55.8.5 `exit`

```
exit(0)   # success
exit(1)   # failure
```

## 55.9 A.13 Built-in constants

| Constant | Value |
|---|---|
| `true` | boolean true |
| `false` | boolean false |
| `nil` / `null` | absence of value |

## 55.10 A.14 Exercises

1. Predict the result of `bool([])`, `bool("")`, `bool(0)` then verify.
2. Write `safe_int(s)` returning nil on parse failure.
3. Demonstrate `chr`/`ord` round-trip for several characters.
4. Use `enumerate` to print lines with line numbers.
5. Use `zip` to pair two lists into a dict.
6. Compare `min`/`max` on a list vs two scalars.
7. Write `assert_positive(x)` using `assert`.
8. Demonstrate `reversed` on a list and a string.
9. Write `all_even(xs)` using `all`.
10. Show `type`/`typeof` on int, float, string, list, dict, nil, func.

# 56 Appendix B: Error Codes Reference

Zen errors carry codes in the format `Z####` where the first digit indicates the error band:

- `Z01xx` — Lexer errors
- `Z02xx` — Parser errors
- `Z03xx` — Compile-time errors
- `Z04xx` — Runtime errors
- `Z05xx` — CLI/tool errors

Use `zen explain <code>` to see the full explanation for any code.

## 56.1 B.1 Lexer errors (Z01xx)

| Code | Meaning |
|---|---|
| `Z0101` | Invalid character in source |
| `Z0102` | Unterminated string literal |
| `Z0103` | Unterminated comment |
| `Z0104` | Invalid number literal |
| `Z0105` | Invalid escape sequence |

## 56.2 B.2 Parser errors (Z02xx)

| Code | Meaning |
|---|---|
| `Z0201` | Unexpected token / missing `)` |
| `Z0202` | Expected expression |
| `Z0203` | Invalid expression |
| `Z0204` | Expected `:` or `=>` (match arm) |
| `Z0205` | Duplicate key in dict literal |
| `Z0206` | Invalid pattern in match |
| `Z0207` | Unmatched delimiter |

## 56.3 B.3 Compile-time errors (Z03xx)

| Code | Meaning |
|---|---|
| `Z0301` | Undefined variable |
| `Z0302` | Duplicate definition |
| `Z0303` | Import not found |
| `Z0304` | Circular import |
| `Z0305` | Type mismatch in declaration |
| `Z0306` | Invalid `return` outside function |
| `Z0307` | `break`/`continue` outside loop |
| `Z0308` | Undefined function |

| Code | Meaning |
|---|---|
| `Z0309` | Invalid import path |
| `Z0310` | Duplicate import |
| `Z0324` | Module not found |
| `Z0325` | Cannot read file |

## 56.4 B.4 Runtime errors (Z04xx)

| Code | Meaning |
|---|---|
| `Z0401` | Type mismatch |
| `Z0402` | Index out of bounds |
| `Z0403` | Key not found |
| `Z0404` | Division by zero |
| `Z0405` | Value error |
| `Z0406` | Attribute error |
| `Z0407` | Native error |
| `Z0408` | Argument count |
| `Z0409` | Handle error |
| `Z0410` | I/O mismatch |
| `Z0411` | Native module error (general) |
| `Z0412` | Handle error |
| `Z0413` | PyImport error |
| `Z0414` | Python import failed |

## 56.5 B.5 CLI/tool errors (Z05xx)

| Code | Meaning |
|---|---|
| `Z0501` | Usage error |
| `Z0502` | Unknown command |
| `Z0503` | Build error |
| `Z0504` | Lint error |

## 56.6 Using `zen explain`

```
zen explain Z0401
zen explain Z0301
zen explain Z0501
zen explain --list   # list all codes
```

Each explanation includes: - The error title - A detailed description - A `= help:` suggestion with a code example

## 56.7 Looking up an error

When a script fails, Zen prints:

```
zen: error: TypeError: expected number, got string[Z0401]
  --> script.z:5:5
   |
5 | print("caught: " + e)
   |     ^^^^^^^^^^^^^^^^
   |
   = help: check operand types with `typeof(x)` and convert explicitly
   = note: run `zen explain Z0401` for detail
```

Run `zen explain Z0401` for the full explanation.

## 56.8 Common runtime errors and fixes

### 56.8.1 Z0401 — Type mismatch

```
# Wrong
print(42 + "hello")

# Right
print(str(42) + "hello")
```

### 56.8.2 Z0402 — Index out of bounds

```
# Wrong
let a = [1, 2, 3]
print(a[5])

# Right
let a = [1, 2, 3]
if 5 < len(a) {
    print(a[5])
}
```

### 56.8.3 Z0403 — Key not found

```
# Wrong
let d = {"a": 1}
print(d["b"])

# Right
let d = {"a": 1}
print(d.get("b", 0))
```

### 56.8.4 Z0404 — Division by zero

```
# Wrong
let x = 1 / 0

# Right
if b != 0 {
    let x = a / b
}
```

## 56.9 Exercises

1. Write a script that intentionally triggers each error type and catches it.
2. Use `zen explain` on three different error codes and summarize them.
3. Write `safe_div(a, b)` that returns `nil` on division by zero.
4. Write `safe_index(lst, i, default)` that returns `default` on index error.

## 56.10 B.12 Error code reference Z01xx – Z05xx (exhaustive)

Zen's coded diagnostics use the prefix `Z` followed by three digits. Use `zen explain Z####` to see the full explanation, common causes, and a fix.

### 56.10.1 Z01xx — Lexer / lexical errors

| Code | Name | Meaning | Common cause | Fix |
|---|---|---|---|---|
| Z0101 | InvalidCharacter | A character not allowed in source | stray control char, smart quotes | remove/replace the character |
| Z0102 | UnterminatedString | String literal never closed | missing closing quote | add closing quote |
| Z0103 | InvalidNumber | Malformed numeric literal | `1.2.3`, `0x`, trailing dot | use valid literal form |
| Z0104 | InvalidEscape | Bad escape in string | `\q` instead of `\n` | use a valid escape |
| Z0105 | InvalidIdentifier | Identifier starts with digit or bad char | `1abc`, `foo-bar` | rename identifier |
| Z0106 | NumericLiteralOverflow | Integer literal beyond i48 range | `99999999999999999999` | use a smaller literal or big-int at runtime |

| Code | Name | Meaning | Common cause | Fix |
|---|---|---|---|---|
| Z0107 | MultiDotRejected | `1.2.3` style version number | `1.2.3` | not a number; use string |
| Z0108 | BadComment | malformed block comment | unclosed `/* */` | close the block comment |

### 56.10.2 Z02xx — Parser / syntax errors

| Code | Name | Meaning | Common cause | Fix |
|---|---|---|---|---|
| Z0201 | UnexpectedToken | token where parser didn't expect it | missing operator, extra keyword | correct syntax |
| Z0202 | MissingSemicolon | statement separator required | newline after non-continuable token | add `;` or restructure |
| Z0203 | UnbalancedBraces | `{`/`}` or `[`/`]` or `(`/`)` mismatch | missing closing brace | add the closing token |
| Z0204 | UnexpectedEOF | source ended mid-construct | unclosed block, string, or call | complete the construct |
| Z0205 | InvalidStatement | statement not allowed here | `return` outside function | move into a function |
| Z0206 | DuplicateParameter | same parameter name twice | `func f(a, a) {}` | rename parameters |
| Z0207 | InvalidDefault | default value not allowed | `func f(a = 1 + 2) {}` (complex default) | use a literal/simple default |
| Z0208 | InvalidMatchArm | bad match arm syntax | missing `=>`/`->`/`:` | add arm separator |
| Z0209 | InvalidImport | malformed import statement | `import` with no module | write valid import |
| Z0210 | Redefinition | redefining `let`/const | `let x = 1; let x = 2` | use `var` or rename |

### 56.10.3 Z03xx — Runtime errors

| Code | Name | Meaning | Common cause | Fix |
|---|---|---|---|---|
| Z0301 | TypeError | operation on wrong type | `"a" + 1`, `nil.x` | check types before operating |
| Z0302 | IndexError | index out of range | `xs[10]` on 3-element list | check `len` first |
| Z0303 | KeyError | missing dict key | `d["missing"]` | use `in` or `??` |
| Z0304 | DivideByZero | division by zero | `x / 0` | check divisor != 0 |
| Z0305 | ValueError | bad value for operation | `int("abc")`, `sqrt(-1)` | validate input |
| Z0306 | AttributeError | object has no attribute | `obj.missing` | define the attribute/method |
| Z0307 | NotCallable | calling a non-function | `x = 5; x()` | call a function value |
| Z0308 | ArgumentCount | wrong number of arguments | `f(1, 2)` when `f` takes 1 | match arity |
| Z0309 | UnboundVariable | variable used before assignment | `print(y)` when `y` unset | initialize first |
| Z0310 | MaxRecursionDepth | recursion too deep | infinite recursion | convert to iteration or raise limit |
| Z0311 | OverflowError | arithmetic overflow | huge int/float result | use bounded math |
| Z0312 | DomainError | math domain error | `log(0)`, `sqrt(-1)` | check domain |
| Z0313 | RangeError | value out of allowed range | index negative on some ops | clamp value |
| Z0314 | EncodingError | can't encode/decode | bad UTF-8 bytes | use valid encoding / errors param |
| Z0315 | ImmutableError | mutating immutable value | `s[0] = "X"` on string | build a new value |

| Code | Name | Meaning | Common cause | Fix |
|---|---|---|---|---|
| Z0316 | ConcurrentModification | collection changed during iteration | removing while looping | iterate over a copy |
| Z0317 | StackOverflow | native stack exhausted | deep recursion | iterative approach |

### 56.10.4 Z04xx — I/O / system errors

| Code | Name | Meaning | Common cause | Fix |
|---|---|---|---|---|
| Z0401 | FileNotFound | file does not exist | wrong path | check path, create file |
| Z0402 | PermissionDenied | insufficient rights | read-only file, root needed | fix permissions |
| Z0403 | FileExists | file already exists (create-only) | `os.mkdir` on existing dir | use `exist_ok=true` |
| Z0404 | IsADirectory | expected file, got directory | opening a dir as file | use a file path |
| Z0405 | NotADirectory | expected directory, got file | `os.listdir("file.txt")` | use a directory |
| Z0406 | DirectoryNotEmpty | can’t remove non-empty dir | `os.rmdir` on populated dir | remove contents first |
| Z0407 | BrokenPipe | write to closed pipe/socket | peer closed connection | handle disconnection |
| Z0408 | ConnectionRefused | nothing listening on port | server not started | start server / check port |
| Z0409 | ConnectionReset | connection reset by peer | peer aborted | retry / handle error |
| Z0410 | Timeout | operation timed out | network delay, no response | increase timeout / retry |
| Z0411 | NetworkUnreachable | network not available | offline, wrong interface | check connectivity |
| Z0412 | HostUnreachable | host not reachable | wrong IP, host down | check host |

| Code | Name | Meaning | Common cause | Fix |
|---|---|---|---|---|
| Z0413 | AddressInUse | bind to occupied port | two servers same port | use SO_REUSEADDR / different port |
| Z0414 | DiskFull | no space left | filesystem full | free space |
| Z0415 | TooManyOpenFiles | file descriptor limit | leaking file handles | close files with `with` |
| Z0416 | InvalidArgument | invalid syscall argument | bad flag, bad path | check argument |
| Z0417 | SignalInterrupted | syscall interrupted by signal | EINTR | retry the operation |

### 56.10.5 Z05xx — FFI / interoperability errors

| Code | Name | Meaning | Common cause | Fix |
|---|---|---|---|---|
| Z0501 | SymbolNotFound | C function symbol missing | wrong library name, not exported | check library, export symbol |
| Z0502 | HandleInvalid | invalid FFI handle | handle closed, never opened | track handle lifetime |
| Z0503 | CallingConvention | wrong calling convention | cdecl vs stdcall mismatch | match platform convention |
| Z0504 | TypeMismatch | FFI argument type wrong | passing int where pointer expected | fix argument types |
| Z0505 | MemoryAccess | invalid memory read/write | null pointer, freed memory | check pointers |
| Z0506 | PymportNotFound | Python module not found | module not installed, wrong path | install module, set PYTHONPATH |
| Z0507 | PymportMarshal | can’t marshal value to Python | unsupported Zen type | convert to supported type |

| Code | Name | Meaning | Common cause | Fix |
|---|---|---|---|---|
| Z0508 | PythonError | Python raised an exception | error inside Python call | inspect chained traceback |
| Z0509 | GILContention | pymport GIL issue | deadlock with Python lock | avoid nested GIL acquisition |
| Z0510 | ForeignTypeUnsupported | foreign type not supported | struct/union not mapped | use bytes/opaque handle |

## 56.11 B.13 How to read a Zen error

A Zen error message looks like:

```
Z0301: TypeError: cannot add string and int
  at main.z:42:10
```

Parts: - `Z0301` — the error code; run `zen explain Z0301`. - `TypeError` — the error category/name. - `cannot add string and int` — human-readable message. - `main.z:42:10` — file, line, column where it occurred.

Use `zen explain` for deeper context:

```
zen explain Z0301
zen explain Z0408
```

## 56.12 B.14 Error handling patterns

### 56.12.1 Catch and rethrow with context

```
try {
    load_config(path)
} catch as e {
    throw "load_config failed: " + e.msg
}
```

### 56.12.2 Catch specific types first

```
try {
    risky()
} catch [FileNotFound] as e {
    print("missing file")
} catch as e {
```

```
    print("other error: " + e.msg)
}
```

### 56.12.3 Result-style (no exceptions)

```
fn result_style() -> Dict {
    try {
        let v = fallible()
        return #{ok: true, value: v}
    } catch as e {
        return #{ok: false, error: e.msg}
    }
}
```

### 56.12.4 Finally for cleanup

```
let f = open(path)
try {
    return f.read()
} finally {
    f.close()
}
```

## 56.13 B.15 Exercises

1. Run `zen explain Z0301`, `Z0408`, `Z0506` and record the explanations.
2. Write a function that deliberately triggers each Z03xx error and catches it.
3. Build a table mapping error code → suggested fix for your codebase.
4. Demonstrate `finally` running after `return`.
5. Implement a `retry` that catches Z0410 (Timeout) and retries.
6. Write a safe wrapper around `int()` returning nil on Z0305.
7. Show what happens when you access a missing dict key (Z0303).
8. Demonstrate division by zero (Z0304) and guard against it.
9. Write a program that handles FileNotFound (Z0401) gracefully.
10. Create an error lookup tool: input a code, print name + meaning.

# 57 Appendix C: Zen CLI Reference

Exhaustive reference for the `zen` command-line interface (Zen 3.0).

## 57.1 C.1 Commands

| Command | Purpose | Example |
|---|---|---|
| `zen` / `zen repl` | Start interactive REPL | `zen` |
| `zen run <file>` | Execute a Zen source file | `zen run app.z` |
| `zen -m <module>` | Run a module as `__main__` | `zen -m http.server` |
| `zen -e <code>` | Execute inline code | `zen -e 'print(1+1)'` |
| `zen check <file>` | Type-check/compile without running | `zen check lib.z` |
| `zen lint <file>` | Warn on suspicious patterns | `zen lint app.z` |
| `zen explain <code>` | Explain an error code | `zen explain Z0308` |
| `zen build <file>` | Bundle into a standalone executable | `zen build app.z -o app` |
| `zen test` | Run tests in the project | `zen test` |
| `zen fmt` | Format source files | `zen fmt .` |
| `zen help` | Show help | `zen help` |
| `zen --version` | Print version | `zen --version` |

### 57.1.1 REPL

Start with no arguments or `zen repl`. Features: - Enter expressions and statements; results are printed. - Multi-line blocks are supported (the REPL detects open braces). - Type `:help`, `:quit`, `:exit` to leave (build-dependent). - Use `print()` for output; the last expression's value is also shown.

### 57.1.2 `zen run`

Executes a `.z` file as a script. `__name__` is set to `"__main__"`. Exit code is 0 on success, non-zero on error.

```
zen run hello.z
zen run ./scripts/build.z
```

### 57.1.3 `zen -m <module>`

Runs a module as the main program (calls its `main(argv)` if defined). Module resolution follows the normal import rules.

```
zen -m http.server
zen -m myproject.cli
```

### 57.1.4 `zen -e <code>`

Executes inline Zen code. Useful for quick calculations.

```
zen -e 'print(2 ** 10)'
zen -e 'import math; print(math.pi)'
```

### 57.1.5 `zen check <file>`

Compiles/type-checks without executing. Reports syntax and type errors.

```
zen check library.z
```

### 57.1.6 `zen lint <file>`

Static analysis: unused variables, dead code, suspicious patterns.

```
zen lint app.z
zen lint .
```

### 57.1.7 `zen explain <code>`

Prints the meaning, common causes, and fix for an error code.

```
zen explain Z0301
zen explain Z0408
zen explain Z0506
```

### 57.1.8 `zen build <file>`

Bundles a program and its dependencies into a single standalone executable.

```
zen build app.z -o myapp
./myapp
```

### 57.1.9 `zen test`

Discovers and runs tests in the current project. Test files typically match `test_*.z` or `*_test.z`, or functions named `test_*`.

```
zen test
zen test tests/
```

### 57.1.10 `zen fmt`

Formats Zen source files to a canonical style.

```
zen fmt file.z
zen fmt src/
```

## 57.2 C.2 Exit codes

| Exit code | Meaning |
|---|---|
| 0 | success |
| 1 | general error |
| 2 | usage / command-line error |
| 127 | command not found |
| 130 | interrupted by signal (Ctrl-C) |
| N | program's `exit(N)` value |

## 57.3 C.3 Environment variables

| Variable | Purpose | Example |
|---|---|---|
| `ZEN_PATH` | Module search paths (colon-separated) | `ZEN_PATH="$HOME/zen/lib"` |
| `ZEN_DEBUG` | Enable debug/verbose logging | `ZEN_DEBUG=1` |
| `PYTHONPATH` | Python module resolution for pymport | `PYTHONPATH=/opt/pylibs` |
| `VIRTUAL_ENV` | Python venv used by pymport | `VIRTUAL_ENV=.venv` |
| `HOME` | user home (used for config/cache) | inherited |
| `PATH` | executable search | inherited |

### 57.3.1 ZEN_PATH examples

```
# add a local modules directory
export ZEN_PATH="$PWD/lib:$HOME/zen/modules"
zen run app.z

# multiple directories
export ZEN_PATH="/opt/zen/stdlib:/usr/local/zen/lib"
```

### 57.3.2 ZEN_DEBUG

When set, the VM prints additional diagnostic info (bytecode, JIT decisions, GC events, depending on build).

```
ZEN_DEBUG=1 zen run app.z
```

## 57.4 C.4 Configuration files

Zen does not require a config file, but projects may use: - `zen.toml` / `zen.json` — project settings (build-dependent) - `.zenrc` — REPL preferences - `.gitignore` — exclude `target/`, `__pycache__/`, `.zen/`

## 57.5 C.5 Common workflows

```
# 1. Create and run a script
echo 'print("hello")' > hello.z
zen run hello.z

# 2. Type-check before running
zen check hello.z && zen run hello.z

# 3. Quick calculation
zen -e 'print(100 * 1.05)'

# 4. Explain an error
zen explain Z0304

# 5. Format and lint
zen fmt src/
zen lint src/

# 6. Run tests
zen test

# 7. Build a binary
zen build app.z -o app
./app

# 8. Run with debug logging
ZEN_DEBUG=1 zen run app.z

# 9. Use a custom module path
ZEN_PATH="$PWD/lib" zen run app.z
```

## 57.6 C.6 REPL commands (if supported)

| Command | Purpose |
|---|---|
| `:help` | show REPL help |
| `:quit` / `:exit` | leave the REPL |
| `:load <file>` | load a file into the REPL |
| `:vars` | list current variables |
| `:clear` | clear the screen |

## 57.7 C.7 Exercises

1. Print the Zen version: `zen --version`.
2. Start the REPL, evaluate `2 + 2`, then exit.
3. Write `hello.z` and run it with `zen run`.

4. Type-check a file with `zen check`.
5. Explain an error code with `zen explain Z0304`.
6. Run an inline calculation with `zen -e`.
7. Format a file with `zen fmt`.
8. Lint a file with `zen lint`.
9. Set `ZEN_PATH` and import a module from another directory.
10. Build a script into a standalone executable with `zen build`.

''' print('appendix c done')

# 58 Appendix D: Standard Library Index (exhaustive)

A complete index of Zen 3.0 standard library modules and their principal members. "→" indicates the category of each member.

## 58.1 D.1 Module list

| Module | Purpose |
|---|---|
| `math` | Mathematical functions and constants |
| `random` | Pseudorandom numbers (not for secrets) |
| `string` | String utilities and constants |
| `collections` | `Counter`, `chain`, `flatten` |
| `json` | JSON parse/serialize |
| `time` / `datetime` | Time and date operations |
| `pathlib` / `os` | File system operations |
| `re` | Regular expressions |
| `base64` / `binascii` | Base64 and hex encoding |
| `hashlib` | Cryptographic hashes (MD5, SHA, BLAKE2) |
| `crypto` | Full cryptography (hash, HMAC, symmetric, asymmetric, JWT, KDF, OTP, X.509) |
| `socket` | Low-level socket programming |
| `http` | HTTP protocol helpers |
| `requests` | High-level HTTP client |
| `http.server` | Building HTTP servers |
| `dns` | Domain name resolution |
| `url` | URL parsing and manipulation |
| `raw sockets` | Packet-level networking (AF_PACKET) |
| `threading` | Threads and synchronization |
| `channels` | Inter-thread communication |
| `mutexes` / locks | Synchronization primitives |
| `ffi` | Foreign Function Interface (C) |
| `sys` | System information and operations |
| `pymport` | Python interoperability |
| `logging` | Logging facility |
| `functools` | `reduce`, `partial`, `lru_cache`, `cmp_to_key` |
| `itertools` | `enumerate`, `zip`, `chain`, `product`, `permutations`, `combinations` |
| `statistics` | `mean`, `median`, `mode`, `variance`, `stdev`, `linear_regression` |
| `operator` | `add`, `sub`, `mul`, `itemgetter`, `attrgetter`, `methodcaller` |
| `struct` | `calcsize` (binary pack/unpack helpers) |
| `tempfile` | `mkdtemp`, `mkstemp`, `dir` |
| `uuid` | UUID generation (`uuid4`, `uuid1`, `uuid3`, `uuid5`) |
| `glob` | pathname pattern expansion |
| `shutil` | `copy`, `copy2`, `copytree`, `move`, `rmtree`, `which` |

| Module | Purpose |
|---|---|
| `crunch` | random data generation |
| `nmap` | network scanning helpers |
| `ffmpeg` | media processing helpers |

## 58.2 D.2 math module

Constants: `pi`, `e`, `tau`, `inf`, `nan`.

Functions: `abs`, `sqrt`, `pow`, `exp`, `log`, `log2`, `log10`, `floor`, `ceil`, `trunc`, `round`, `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `atan2`, `degrees`, `radians`, `gcd`, `lcm`, `factorial`, `comb`, `perm`, `remainder`, `copysign`, `min`, `max`, `fsum`, `prod`, `hypot`, `isnan`, `isfinite`, `isinf`, `modf`, `frexp`, `ldexp`.

## 58.3 D.3 random module

Functions: `seed`, `random`, `randint`, `randrange`, `choice`, `choices`, `sample`, `shuffle`, `uniform`, `hex`.

 NOT cryptographically secure. Use `crypto` module for secrets/tokens.

## 58.4 D.4 string module

Constants: `digits`, `hexdigits`, `octdigits`, `ascii_letters`, `ascii_lowercase`, `ascii_uppercase`, `punctuation`, `whitespace`, `printable`.

Functions: `upper`, `lower`, `title`, `capitalize`, `swapcase`, `strip`, `lstrip`, `rstrip`, `split`, `join`, `replace`, `count`, `find`, `startswith`, `endswith`, `isdigit`, `isalpha`, `isalnum`, `isspace`, `islower`, `isupper`.

## 58.5 D.5 collections module

Functions: `Counter`, `chain`, `flatten`.

## 58.6 D.6 json module

Functions: `parse`, `stringify/encode`, `pretty`, `load`, `save`.

JSON ↔ Zen mapping: object↔dict, array↔list, string↔string, number↔int/float, boolean↔bool, null↔nil.

## 58.7 D.7 time / datetime module

Functions: `time`, `sleep`, `now`, `today`, `strftime`, `strptime`, `timestamp`, `fromtimestamp`, `timedelta`.

## 58.8 D.8 pathlib / os module

pathlib: `Path`, `/` join, `exists`, `is_file`, `is_dir`, `name`, `stem`, `suffix`, `parent`, `resolve`, `mkdir`, `glob`.

os: `getcwd`, `chdir`, `listdir`, `makedirs`, `remove`, `rmdir`, `rename`, `stat`, `environ`.

## 58.9 D.9 re module

Functions: `match`, `search`, `findall`, `sub`, `split`, `compile`.

Patterns: `\d` `\D` `\w` `\W` `\s` `\S` `.` `^` `$` `*` `+` `?` `{n,m}` `(...)` `|` `\b`.

## 58.10 D.10 base64 / binascii module

base64: `b64encode`, `b64decode`, `url_encode`, `url_decode`. binascii: `hexlify`, `unhexlify`.

## 58.11 D.11 hashlib module

Hashes: `md5`, `sha1`, `sha224`, `sha256`, `sha384`, `sha512`, `sha3_256`, `sha3_512`, `blake2b`, `blake2s`, `pbkdf2_hmac`.

## 58.12 D.12 crypto module

Hashing: `sha256/384/512`, `sha3_256/512`, `blake3`, `md5`, `sha1`. HMAC: `sha256(key,data)`, `sha512(key,data)`. Symmetric: `generate_key`, `encrypt`, `decrypt`, `encrypt_aes_gcm`, `decrypt_aes_gcm`, `encrypt_chacha20_poly1305`, `decrypt_chacha20_poly1305`. Asymmetric: `generate_rsa`, `rsa_encrypt/decrypt`, `rsa_sign/verify`, `generate_ed25519`, `ed25519_sign/verify`, `generate_x25519`, `x25519_shared_secret`. Encoding: `b64encode/decode`, `b64urlencode/decode`, `hex_encode/decode`. JWT: `sign`, `verify`, `decode`. KDF/passwords: `pbkdf2`, `argon2id`, `argon2id_hash`, `verify_argon2id`, `hkdf`. OTP: `totp`, `totp_verify`, `hotp`, `hotp_verify`, `generate_secret`. Random: `bytes(n)`, `hex(n)`, `base64(n)`, `uuid4()`, `secure_token()`, `int(min, max)`. X.509: `parse(pem)`.

## 58.13 D.13 networking modules

socket: `socket`, `connect`, `bind`, `listen`, `accept`, `send`, `recv`, `sendto`, `recvfrom`, `close`, `setsockopt`, `settimeout`, `setblocking`. http: request/response parsing, status codes, headers. requests: `get`, `post`, `put`, `patch`, `delete`, `head`, `options`, `Session`. http.server: request handler, routing, static files, middleware. dns: `resolve`, `reverse`, record types A/AAAA/CNAME/MX/TXT/NS/PTR. url: `parse`, `build`, `join`, `quote`, `unquote`, `parse_qs`. raw sockets: `AF_PACKET`, `SOCK_RAW`, packet crafting.

## 58.14 D.14 concurrency modules

threading: `Thread`, `spawn`, `join`, `start`. channels: `Channel`, `send`, `recv`, `close`. mutexes: `Mutex`, `Lock`, `acquire`, `release`. patterns: thread-safe caches, producer-consumer, thread pools.

## 58.15 D.15 FFI / system / interop modules

ffi: `load_library`, `get_symbol`, `call`, `Pointer`, `Handle`. sys: `argv`, `platform`, `version`, `exit`, `environ`, `getsizeof` (if present). pymport: `import` Python modules, marsalling values, chained tracebacks.

## 58.16 D.16 Module → chapter mapping

| Chapter | Module(s) |
|---|---|
| 15 | math |
| 16 | random |
| 17 | string |
| 18 | collections |
| 19 | json |
| 20 | time / datetime |
| 21 | pathlib / os |
| 22 | re |
| 23 | base64 / binascii |
| 24 | hashlib |
| 25 | crypto (complete) |
| 26 | networking fundamentals |
| 27 | socket |
| 28 | http |
| 29 | requests |
| 30 | http.server |
| 31 | HTTPS / SSL / TLS |
| 32 | dns |
| 33 | url |
| 34 | raw sockets |
| 35 | threading |
| 36 | channels |
| 37 | mutexes / locks |
| 38 | thread safety patterns |
| 39 | ffi fundamentals |
| 40 | ffi in practice |
| 41 | sys |
| 42 | pymport |

## 58.17 D.17 Exercises

1. List the hash functions available in `hashlib` and their digest sizes.
2. Name three modules you must NOT use for cryptographic secrets (random, math, etc.) and say which to use instead (`crypto`).
3. For the URL `https://a.b:8080/p?q=1#f`, name each component.
4. List the DNS record types and what each resolves.
5. Name the exit codes used by the `zen` CLI.
6. For `crypto`, name two authenticated-encryption algorithms supported.

7. List the concurrency primitives and their purpose.
8. For `pymport`, name two error codes in the Z05xx range.
9. Write a one-line summary of what each of modules math/random/string does.
10. Build your own “cheat sheet” mapping module → most-used 3 functions.

’’’ print(‘appendix d done’)

# 59 Appendix E: Quick Reference Card

Zen 3.0 — the essentials on one page (expanded).

## 59.1 E.1 Syntax at a glance

```
# Comments
# line comment        // line comment
/* block comment */

# Declarations
let x = 10            # immutable binding
var y = 20            # mutable binding
const MAX = 100       # constant
shared counter = 0    # thread-shared mutable

# Functions
func add(a, b) { return a + b }
let add2 = fn(a, b) => a + b
let add3 = (a, b) => a + b

# Defaults
func greet(name, greeting = "Hi") { return greeting + ", " + name }

# Variadic
func sum_all(...nums) { var t = 0; for n in nums { t += n }; return t }

# Classes
class Point {
    var x = 0.0
    var y = 0.0
    func init(x, y) { self.x = x; self.y = y }
    func dist() { return (self.x**2 + self.y**2) ** 0.5 }
    func __str__() { return "Point(${x},${y})" }
}
let p = new Point(3, 4)

# Modules
import math
import math as m
from math import sin, pi
pub func exported() {}   # visible outside module
```

## 59.2 E.2 Literals

| Type | Literal | Example |
|---|---|---|
| nil | `nil` / `null` | `let x = nil` |
| bool | `true` / `false` | `let ok = true` |
| int | `42`, `0xFF`, `0b1010`, `0o755`, `1_000` | `let n = 0x2A` |
| float | `3.14`, `1.5e-10` | `let f = 2.5` |
| string | `"hi ${name}"`, `'raw'`, `"""triple"""`, `` `backtick` `` | `let s = "Zen"` |
| bytes | `bytes([1,2,0xFF])` | `let b = bytes([0x41])` |
| list | `[a, b, c]` | `let l = [1,2,3]` |
| dict | `#{k: v}` | `let d = #{a:1}` |
| set | `%{a, b}` | `let s = %{1,2}` |
| tuple | `#(a, b)` | `let t = #(1,"x")` |

## 59.3 E.3 Operators (precedence high→low)

```
() [] . ?.          postfix / grouping
**                  power (right)
- ! not ~ typeof    unary (right)
* / %              multiplicative
+ -                additive
<< >>              shifts
&                  bitwise and
^                  bitwise xor
|                  bitwise or
is is not === !== == != < > <= >= in not in   comparison
and &&             logical and
or ||              logical or
??                 nullish coalescing
cond ? a : b / a if c else b   ternary (right)
= += -= *= ...     assignment (right)
```

Truthiness: ONLY `false`, `nil`, `null` are falsy. Everything else (including `0`, `0.0`, `""`, `[]`, `#{}`, `%{}`, `#()`) is truthy.

## 59.4 E.4 Control flow

```
if c { } elif c { } else { }
while c { }
for x in iter { }          # for k, v in d.items()
for i in 0..5 { }          # exclusive end
for i in 0->5 { }          # inclusive end
break / continue
return [expr]
throw expr
try { } catch [T] as e { } finally { }
match v { pat => expr, _ => expr }
```

```
when { c => expr, _ => expr }
```

## 59.5 E.5 Built-ins (selected)

`print`, `str`, `int`, `float`, `bool`, `list`, `tuple`, `set`, `dict`, `len`, `range`, `enumerate`, `zip`, `sorted`, `reversed`, `any`, `all`, `sum`, `min`, `max`, `abs`, `typeof`, `type`, `chr`, `ord`, `open`, `exit`, `assert`, `repr`.

## 59.6 E.6 String methods

`upper lower title capitalize swapcase strip lstrip rsplit split splitlines join replace count find rfind startswith endswith contains ljust rjust center zfill isdigit isalpha isalnum isspace islower isupper`.

## 59.7 E.7 List methods

`push pop insert remove clear map filter reduce each join index count`.

## 59.8 E.8 Dict methods

`keys values items get pop clear update`.

Iterating a dict yields keys; use `.items()` for key/value.

## 59.9 E.9 Socket basics

```
import socket
let s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(("host", 80))
s.send(b"GET / HTTP/1.1\r\nHost: host\r\n\r\n")
let data = s.recv(4096)
s.close()
```

Options: `SO_REUSEADDR`, `SO_KEEPALIVE`, `TCP_NODELAY`, `SO_RCVBUF`, `SO_SNDBUF`, `IP_TTL`, `IP_HDRINCL`.

## 59.10 E.10 Error codes (Z-ranges)

- Z01xx lexer | Z02xx parser | Z03xx runtime | Z04xx I/O/system | Z05xx FFI/interop

Run: `zen explain Z####`

## 59.11 E.11 CLI commands

`zen` `zen run <f>` `zen -m <mod>` `zen -e <code>` `zen check <f>` `zen lint <f>` `zen explain <code>` `zen build <f>` `zen test` `zen fmt` `zen help` `zen --version`

Env: `ZEN_PATH`, `ZEN_DEBUG`, `PYTHONPATH`, `VIRTUAL_ENV`.

## 59.12 E.12 Truthiness cheat

```
value        bool(value)
----         -----------
false        false
nil / null   false
everything else   true   (0, 0.0, "", [], #{}, %{}, #() all true)
```

## 59.13 E.13 Equality cheat

| Operator | Meaning | `1 == 1.0` | `[1] == [1]` | `[1] is [1]` |
|---|---|---|---|---|
| `==` | value equality | true | true | — |
| `===` | strict equality | false | — | — |
| `is` | identity | true | false | false |

## 59.14 E.14 Common gotchas (one-liners)

- `0` and `""` are **truthy** in Zen.
- Strings are **immutable**: `s[0] = "X"` errors.
- `b = a` shares a list; use `list(a)` to copy (shallow).
- `and`/`or` return **booleans**, not operands; use `??` for defaults.
- `-2 ** 2` is `-4`; use `(-2) ** 2` for `4`.
- Dict keys must be **strings**.
- `s[0:100]` clamps; `s[100]` errors.
- `4 / 2` is `2.0`; `4 // 2` is `2`.
- Modules are **cached**; re-import returns the cached module.
- `pub` exports; without it, symbols are file-private.

## 59.15 E.15 Exercises

1. Without running, state the truthiness of `0`, `""`, `[]`, `false`, `nil`.
2. Write `clamp(x, lo, hi)` in one line using ternary.
3. Show `is` vs `==` for two equal lists.
4. Write `safe_div(a, b)` returning nil on `b == 0`.
5. Convert `"1,2,3"` to `[1, 2, 3]` using split + map/int.
6. Reverse a string without `.reverse()`.
7. Count word frequencies in a sentence using a dict.
8. Merge two dicts into a third.
9. Write `is_prime(n)` and test up to 100.
10. Memorize the Z-code ranges: Z01xx/Z02xx/Z03xx/Z04xx/Z05xx.

''' print('appendix e done')

# 60 Appendix F: Zen 3.0 Complete Language Reference

An exhaustive, compact reference of every builtin, keyword, operator, flag, environment variable, error code, and standard library member. Use this appendix when you need to look something up quickly.

## 60.1 F.1 Keywords and reserved words

```
let const var global shared
func def function fn lambda
if elif else while for in break continue return
try catch except finally throw
import from as class new extends inherit super
pub match when
true false nil null
is typeof and or not is not
```

## 60.2 F.2 Literal forms

| Type | Form | Example |
|---|---|---|
| nil | `nil` / `null` | `let x = nil` |
| bool | `true` / `false` | `let ok = true` |
| int decimal | digits, `_` allowed | `1_000_000` |
| int hex | `0x` / `0X` | `0xFF` |
| int binary | `0b` / `0B` | `0b1010` |
| int octal | `0o` / `0O` | `0o755` |
| float | digits `.` digits, exponent | `3.14`, `1.5e-10` |
| string double | `"...${expr}..."` (interpolated) | `"hi ${name}"` |
| string single | `'...'` (no interpolation) | `'raw'` |
| string triple | `"""..."""` / `'''...'''` | multiline |
| string backtick | `` `...` `` (raw, no interpolation) | `` `C:\path` `` |
| bytes | `bytes([1,2,0xFF])` | `bytes([0x41])` |
| list | `[a, b, c]` | `[1,2,3]` |
| dict | `#{k: v}` | `#{a:1}` |
| set | `%{a, b}` | `%{1,2}` |
| tuple | `#(a, b)` | `#(1,"x")` |

## 60.3 F.3 Operator precedence (high → low)

| Level | Operators | Associativity |
|---|---|---|
| 15 | `() [] . ?.` | left |

| Level | Operators | Associativity |
|---|---|---|
| 14 | ** | right |
| 13 | unary - ! not ~ typeof | right |
| 12 | * / % | left |
| 11 | + - | left |
| 10 | << >> | left |
| 9 | & | left |
| 8 | ^ | left |
| 7 | \| | left |
| 6 | is is not === !== == != < > <= >= in not in | left |
| 5 | and && | left |
| 4 | or \|\| | left |
| 3 | ?? | left |
| 2 | ?: / if expr else expr | right |
| 1 | = += -= … | right |

## 60.4 F.4 Truthiness

Falsy: `false`, `nil`, `null`. Truthy: everything else, including `0`, `0.0`, `""`, `[]`, `#{}`, `%{}`, `#()`.

## 60.5 F.5 Type coercion matrix

| From | int | float | string | bool |
|---|---|---|---|---|
| int | identity | widen | decimal digits | 0→false, else true |
| float | truncate | identity | repr | 0.0→false, else true |
| string | parse (or error) | parse (or error) | identity | empty→false? (Zen: true) |
| bool | true→1, false→0 | true→1.0, false→0.0 | “true”/“false” | identity |
| nil | error | error | “nil” | false |
| list/dict/set/tuple | error | error | repr | empty→true in Zen |

## 60.6 F.6 Built-in functions (complete)

| Function | Purpose |
|---|---|
| `print` | print to stdout |
| `str` | to string |
| `int` | to int |
| `float` | to float |

| Function | Purpose |
|---|---|
| `bool` | to bool |
| `list` | to list |
| `tuple` | to tuple |
| `set` | to set |
| `dict` | to dict |
| `len` | length |
| `range` | range |
| `enumerate` | index+value |
| `zip` | aggregate |
| `sorted` | sorted list |
| `reversed` | reversed |
| `any` | any true |
| `all` | all true |
| `sum` | sum |
| `min` / `max` | min/max |
| `abs` | absolute value |
| `typeof` / `type` | type name |
| `chr` | int → char |
| `ord` | char → int |
| `open` | open file |
| `exit` | exit process |
| `assert` | assert condition |
| `repr` | developer representation |

## 60.7 F.7 Statement forms

| Form | Meaning |
|---|---|
| `let x = e` | immutable binding |
| `var x = e` | mutable binding |
| `const x = e` | constant |
| `shared x = e` | thread-shared mutable |
| `x = e` | assignment |
| `x += e` etc | compound assignment |
| `x++` / `x--` | increment/decrement |
| `if c { }` | conditional |
| `while c { }` | loop |
| `for x in i { }` | iterate |
| `break` / `continue` | loop control |
| `return [e]` | return from function |
| `throw e` | raise error |
| `try { } catch [T] as e { } finally { }` | error handling |
| `match v { p => e, ... }` | pattern match |
| `when { c => e, ... }` | cond chain |
| `import m` / `from m import n` | imports |

## 60.8 F.8 Zen CLI commands

| Command | Purpose |
|---|---|
| `zen` / `zen repl` | REPL |
| `zen run <file>` | execute script |
| `zen -m <module>` | run module as main |
| `zen -e <code>` | inline code |
| `zen check <file>` | type-check only |
| `zen lint <file>` | static analysis |
| `zen explain <code>` | explain error code |
| `zen build <file>` | bundle to executable |
| `zen test` | run tests |
| `zen fmt` | format sources |
| `zen help` | help |
| `zen --version` | version |

Exit codes: 0 success, 1 general, 2 usage, 127 not found, 130 signal.

## 60.9 F.9 Environment variables

| Variable | Purpose |
|---|---|
| `ZEN_PATH` | module search paths (colon-separated) |
| `ZEN_DEBUG` | enable debug logging |
| `PYTHONPATH` | Python module resolution for pymport |
| `VIRTUAL_ENV` | Python venv for pymport |

## 60.10 F.10 Error codes (exhaustive Z01xx–Z05xx)

### 60.10.1 Z01xx Lexer

- Z0101 InvalidCharacter
- Z0102 UnterminatedString
- Z0103 InvalidNumber
- Z0104 InvalidEscape
- Z0105 InvalidIdentifier
- Z0106 NumericLiteralOverflow
- Z0107 MultiDotRejected
- Z0108 BadComment

### 60.10.2 Z02xx Parser

- Z0201 UnexpectedToken
- Z0202 MissingSemicolon
- Z0203 UnbalancedBraces
- Z0204 UnexpectedEOF

- Z0205 InvalidStatement
- Z0206 DuplicateParameter
- Z0207 InvalidDefault
- Z0208 InvalidMatchArm
- Z0209 InvalidImport
- Z0210 Redefinition

### 60.10.3 Z03xx Runtime

- Z0301 TypeError
- Z0302 IndexError
- Z0303 KeyError
- Z0304 DivideByZero
- Z0305 ValueError
- Z0306 AttributeError
- Z0307 NotCallable
- Z0308 ArgumentCount
- Z0309 UnboundVariable
- Z0310 MaxRecursionDepth
- Z0311 OverflowError
- Z0312 DomainError
- Z0313 RangeError
- Z0314 EncodingError
- Z0315 ImmutableError
- Z0316 ConcurrentModification
- Z0317 StackOverflow

### 60.10.4 Z04xx I/O / system

- Z0401 FileNotFound
- Z0402 PermissionDenied
- Z0403 FileExists
- Z0404 IsADirectory
- Z0405 NotADirectory
- Z0406 DirectoryNotEmpty
- Z0407 BrokenPipe
- Z0408 ConnectionRefused
- Z0409 ConnectionReset
- Z0410 Timeout
- Z0411 NetworkUnreachable
- Z0412 HostUnreachable
- Z0413 AddressInUse
- Z0414 DiskFull
- Z0415 TooManyOpenFiles
- Z0416 InvalidArgument
- Z0417 SignalInterrupted

### 60.10.5 Z05xx FFI / interop

- Z0501 SymbolNotFound
- Z0502 HandleInvalid
- Z0503 CallingConvention
- Z0504 TypeMismatch
- Z0505 MemoryAccess
- Z0506 PymportNotFound
- Z0507 PymportMarshal
- Z0508 PythonError
- Z0509 GILContention
- Z0510 ForeignTypeUnsupported

Run `zen explain Z####` for details on any code.

## 60.11 F.11 Standard library — module → principal members

math: `pi e tau inf nan abs sqrt pow exp log log2 log10 floor ceil trunc round sin cos tan asin acos atan atan2 degrees radians gcd lcm factorial comb perm remainder copysign min max fsum prod hypot isnan isfinite isinf modf frexp ldexp`

random: `seed random randint randrange choice choices sample shuffle uniform hex`

string: `upper lower title capitalize swapcase strip lstrip rstrip split splitlines join replace count find rfind startswith endswith contains ljust rjust center zfill isdigit isalpha isalnum isspace islower isupper` constants: `digits hexdigits octdigits ascii_letters ascii_lowercase ascii_uppercase punctuation whitespace printable`

collections: `Counter chain flatten`

json: `parse stringify encode pretty load save`

time/datetime: `time sleep now today strftime strptime timestamp fromtimestamp timedelta`

pathlib: `Path / exists is_file is_dir name stem suffix parent resolve mkdir glob` os: `getcwd chdir listdir makedirs remove rmdir rename stat environ`

re: `match search findall sub split compile`

base64: `b64encode b64decode url_encode url_decode` binascii: `hexlify unhexlify`

hashlib: `md5 sha1 sha224 sha256 sha384 sha512 sha3_256 sha3_512 blake2b blake2s pbkdf2_hmac`

crypto: `sha256/384/512 sha3_256/512 blake3 md5 sha1 sha256(key,data) sha512(key,data) generate_key encrypt decrypt encrypt_aes_gcm decrypt_aes_gcm encrypt_chacha20_poly1305 decrypt_chacha20_poly1305 generate_rsa rsa_encrypt rsa_decrypt rsa_sign rsa_verify generate_ed25519 ed25519_sign ed25519_verify generate_x25519 x25519_shared_secret b64encode b64decode b64urlencode b64urldecode hex_encode`

hex_decode sign verify decode pbkdf2 argon2id argon2id_hash verify_argon2id hkdf totp totp_verify hotp hotp_verify generate_secret bytes hex base64 uuid4 secure_token int parse

socket: socket connect bind listen accept send recv sendto recvfrom close setsockopt settimeout setblocking SO_REUSEADDR SO_KEEPALIVE TCP_NODELAY SO_RCVBUF SO_SNDBUF IP_TTL IP_HDRINCL AF_INET AF_INET6 AF_UNIX AF_PACKET SOCK_STREAM SOCK_DGRAM SOCK_RAW IPPROTO_TCP IPPROTO_UDP IPPROTO_ICMP IPPROTO_IP

http/requests: get post put patch delete head options Session raise_for_status

http.server: request handler routing static files middleware

dns: resolve reverse records A AAAA CNAME MX TXT NS PTR

url: parse build join quote unquote parse_qs

threading: Thread start join daemon channels: Channel send recv close buffered capacity mutexes: Mutex lock unlock

ffi: load_library get_symbol call close Pointer Handle sys: argv platform version exit environ path modules pymport: import Python modules with marshalling and chained tracebacks

logging: DEBUG INFO WARNING ERROR CRITICAL NOTSET functools: reduce partial wraps lru_cache cmp_to_key itertools: enumerate zip chain repeat product permutations combinations accumulate take drop range statistics: mean median mode multimode variance stdev pvariance pstdev quantiles linear_regression operator: add sub mul truediv floordiv mod divmod pow neg pos abs inv invert lshift rshift and_ or_ xor truth is_ is_not eq ne lt le gt ge contains itemgetter attrgetter methodcaller struct: calcsize tempfile: dir mkdtemp mkstemp uuid: NAMESPACE_DNS NAMESPACE_URL NAMESPACE_OID NAMESPACE_X500 UUID uuid4 uuid3 uuid5 uuid1 uuid1_fields glob: glob iglob escape shutil: copy copy2 copyfile copytree move rmtree which crunch: generate nmap: scanning helpers ffmpeg: media helpers

## 60.12 F.12 Socket option reference

| Option | Level | Purpose |
|---|---|---|
| SO_REUSEADDR | SOL_SOCKET | reuse local address |
| SO_REUSEPORT | SOL_SOCKET | multiple binds to same port |
| SO_KEEPALIVE | SOL_SOCKET | TCP keepalive probes |
| SO_RCVBUF | SOL_SOCKET | receive buffer size |
| SO_SNDBUF | SOL_SOCKET | send buffer size |
| SO_TIMEOUT | SOL_SOCKET | operation timeout |
| TCP_NODELAY | IPPROTO_TCP | disable Nagle’s algorithm |
| IP_TTL | IPPROTO_IP | time-to-live |
| IP_HDRINCL | IPPROTO_IP | include IP header (raw) |

## 60.13 F.13 Common ports

20/21 FTP, 22 SSH, 23 Telnet, 25 SMTP, 53 DNS, 67/68 DHCP, 80 HTTP, 110 POP3, 143 IMAP, 443 HTTPS, 993 IMAPS, 995 POP3S.

## 60.14 F.14 HTTP status codes

1xx Informational | 2xx Success (200 201 202 204 206) | 3xx Redirection (301 302 303 304 307 308) | 4xx Client Error (400 401 403 404 405 406 409 410 413 414 415 422 429) | 5xx Server Error (500 501 502 503 504 505)

## 60.15 F.15 HTTP methods

| Method | Safe | Idempotent | Body |
|---|---|---|---|
| GET | yes | yes | no |
| HEAD | yes | yes | no |
| OPTIONS | yes | yes | optional |
| POST | no | no | yes |
| PUT | no | yes | yes |
| PATCH | no | no | yes |
| DELETE | no | yes | optional |
| TRACE | yes | yes | no |
| CONNECT | no | no | no |

## 60.16 F.16 DNS record types

A (IPv4), AAAA (IPv6), CNAME (alias), MX (mail), TXT (text), NS (nameserver), PTR (reverse IP→name).

## 60.17 F.17 URL components

scheme :// [user:pass@] host [:port] /path [?query] [#fragment]

## 60.18 F.18 IPv4 header (20 bytes min)

ver/IHL DSCP/ECN | total length | identification | flags/frag | TTL | proto | checksum | src IP (4) | dst IP (4)

## 60.19 F.19 TCP connection state machine

CLOSED → SYN_SENT → SYN_RECEIVED → ESTABLISHED → FIN_WAIT_1 → FIN_WAIT_2 → TIME_WAIT → CLOSED (normal close); also RST transitions at any error.

## 60.20 F.20 Coercion / truthiness quick examples

```
print(1 == 1.0)     # true  (numeric promotion)
print(1 === 1.0)    # false (strict: different types)
print(bool(0))      # true  (0 is truthy in Zen)
print(bool(""))     # true  (empty string truthy)
print(bool(nil))    # false
print(nil ?? "d")   # "d"
print(0 ?? "d")     # 0     (0 is not nil)
```

## 60.21 F.21 Exercises (reference drill)

1. Look up `zen explain Z0304` and summarize the fix.
2. Name the three falsy values in Zen.
3. List the HTTP methods that are idempotent.
4. Name the socket option to disable Nagle's algorithm.
5. List the DNS record types.
6. Name the exit code for "command not found" (127).
7. Name the environment variable for module search paths.
8. List the falsy values that are nonetheless truthy in Zen (0, "", []...).
9. Name the function to parse JSON: `json.parse`.
10. Name the crypto function for time-based one-time passwords: `crypto.totp`.

''' print('appendix f done')